neTCLib

v0.3.0

Generated by Doxygen 1.9.8

1 Namespace Index 1

1.1 Namespace List . . . 1

2 Hierarchical Index 3

2.1 Class Hierarchy . . . 3

3 Class Index 5

3.1 Class List . . . 5

4 File Index 7

4.1 File List . . . 7

5 Namespace Documentation 9

5.1 Foam Namespace Reference . . . 9

5.1.1 Typedef Documentation . . . 10

5.1.1.1 blottnerEucken2TGasHThermoPhysics . . . 10

5.1.1.2 blottnerEucken2TGasHThermoPhysicsHReaction . . . 11

5.1.1.3 blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie . . . 11

5.1.1.4 heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie

5.1.1.5 heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGass

5.1.2 Function Documentation . . . 11

5.1.2.1 addThermoPhysicsThermo() [1/6] . . . 11

5.1.2.2 addThermoPhysicsThermo() [2/6] . . . 11

5.1.2.3 addThermoPhysicsThermo() [3/6] . . . 11

5.1.2.4 addThermoPhysicsThermo() [4/6] . . . 12

5.1.2.5 addThermoPhysicsThermo() [5/6] . . . 12

5.1.2.6 addThermoPhysicsThermo() [6/6] . . . 12

5.1.2.7 defineRunTimeSelectionTable() [1/6] . . . 12

5.1.2.8 defineRunTimeSelectionTable() [2/6] . . . 12

5.1.2.9 defineRunTimeSelectionTable() [3/6] . . . 12

5.1.2.10 defineRunTimeSelectionTable() [4/6] . . . 12

5.1.2.11 defineRunTimeSelectionTable() [5/6] . . . 13

5.1.2.12 defineRunTimeSelectionTable() [6/6] . . . 13

5.1.2.13 defineTemplateTypeNameAndDebugWithName() [1/2] . . . 13

5.1.2.14 defineTemplateTypeNameAndDebugWithName() [2/2] . . . 13

5.1.2.15 makeThermos() [1/2] . . . 13

5.1.2.16 makeThermos() [2/2] . . . 13

5.1.2.17 operator∗() [1/2] . . . 14

5.1.2.18 operator∗() [2/2] . . . 14

5.1.2.19 operator+() [1/2] . . . 14

5.1.2.20 operator+() [2/2] . . . 14

5.1.2.21 operator<<() [1/2] . . . 14

5.1.2.22 operator<<() [2/2] . . . 15

5.1.2.23 operator==() . . . 15

5.2 Foam::species Namespace Reference . . . 15
5.2.1 Function Documentation . . . 15
5.2.1.1 operator∗() . . . 15
5.2.1.2 operator+() . . . 15
5.2.1.3 operator<<() . . . 16
5.2.1.4 operator==() . . . 16

6 Class Documentation **17**
6.1 Foam::basic2TThermo Class Reference . . . 17
6.1.1 Detailed Description . . . 24
6.1.2 Constructor & Destructor Documentation . . . 24
6.1.2.1 basic2TThermo() [1/8] . . . 24
6.1.2.2 basic2TThermo() [2/8] . . . 24
6.1.2.3 basic2TThermo() [3/8] . . . 24
6.1.2.4 basic2TThermo() [4/8] . . . 24
6.1.2.5 ∼basic2TThermo() [1/2] . . . 24
6.1.2.6 basic2TThermo() [5/8] . . . 25
6.1.2.7 basic2TThermo() [6/8] . . . 25
6.1.2.8 basic2TThermo() [7/8] . . . 25
6.1.2.9 basic2TThermo() [8/8] . . . 25
6.1.2.10 ∼basic2TThermo() [2/2] . . . 25
6.1.3 Member Function Documentation . . . 25
6.1.3.1 alpha() [1/4] . . . 25
6.1.3.2 alpha() [2/4] . . . 25
6.1.3.3 alpha() [3/4] . . . 25
6.1.3.4 alpha() [4/4] . . . 26
6.1.3.5 alphaEff() [1/4] . . . 26
6.1.3.6 alphaEff() [2/4] . . . 26
6.1.3.7 alphaEff() [3/4] . . . 26
6.1.3.8 alphaEff() [4/4] . . . 26
6.1.3.9 alphaheTR() [1/4] . . . 26
6.1.3.10 alphaheTR() [2/4] . . . 26
6.1.3.11 alphaheTR() [3/4] . . . 26
6.1.3.12 alphaheTR() [4/4] . . . 26
6.1.3.13 correct() [1/2] . . . 27
6.1.3.14 correct() [2/2] . . . 27
6.1.3.15 CpByCpvTR() [1/4] . . . 27
6.1.3.16 CpByCpvTR() [2/4] . . . 27
6.1.3.17 CpByCpvTR() [3/4] . . . 27
6.1.3.18 CpByCpvTR() [4/4] . . . 27
6.1.3.19 CpTR() [1/6] . . . 27
6.1.3.20 CpTR() [2/6] . . . 27

6.1.3.21 CpTR() [3/6] 27
6.1.3.22 CpTR() [4/6] 28
6.1.3.23 CpTR() [5/6] 28
6.1.3.24 CpTR() [6/6] 28
6.1.3.25 CpvTR() [1/4] 28
6.1.3.26 CpvTR() [2/4] 28
6.1.3.27 CpvTR() [3/4] 28
6.1.3.28 CpvTR() [4/4] 28
6.1.3.29 CvR() [1/4] 28
6.1.3.30 CvR() [2/4] 29
6.1.3.31 CvR() [3/4] 29
6.1.3.32 CvR() [4/4] 29
6.1.3.33 CvT() [1/4] 29
6.1.3.34 CvT() [2/4] 29
6.1.3.35 CvT() [3/4] 29
6.1.3.36 CvT() [4/4] 29
6.1.3.37 CvVib() [1/4] 29
6.1.3.38 CvVib() [2/4] 30
6.1.3.39 CvVib() [3/4] 30
6.1.3.40 CvVib() [4/4] 30
6.1.3.41 declareRunTimeSelectionTable() [1/4] 30
6.1.3.42 declareRunTimeSelectionTable() [2/4] 30
6.1.3.43 declareRunTimeSelectionTable() [3/4] 30
6.1.3.44 declareRunTimeSelectionTable() [4/4] 31
6.1.3.45 dpdt() [1/2] 31
6.1.3.46 dpdt() [2/2] 31
6.1.3.47 gamma() [1/4] 31
6.1.3.48 gamma() [2/4] 31
6.1.3.49 gamma() [3/4] 31
6.1.3.50 gamma() [4/4] 31
6.1.3.51 hc() [1/2] 32
6.1.3.52 hc() [2/2] 32
6.1.3.53 he() [1/10] 32
6.1.3.54 he() [2/10] 32
6.1.3.55 he() [3/10] 32
6.1.3.56 he() [4/10] 32
6.1.3.57 he() [5/10] 32
6.1.3.58 he() [6/10] 32
6.1.3.59 he() [7/10] 32
6.1.3.60 he() [8/10] 33
6.1.3.61 he() [9/10] 33
6.1.3.62 he() [10/10] 33

6.1.3.63 heBoundaryBaseTypes() [1/2] . . . 33
6.1.3.64 heBoundaryBaseTypes() [2/2] . . . 33
6.1.3.65 heBoundaryTypes() [1/2] . . . 33
6.1.3.66 heBoundaryTypes() [2/2] . . . 33
6.1.3.67 incompressible() [1/2] . . . 33
6.1.3.68 incompressible() [2/2] . . . 34
6.1.3.69 isochoric() [1/2] . . . 34
6.1.3.70 isochoric() [2/2] . . . 34
6.1.3.71 kappaEff() [1/4] . . . 34
6.1.3.72 kappaEff() [2/4] . . . 34
6.1.3.73 kappaEff() [3/4] . . . 34
6.1.3.74 kappaEff() [4/4] . . . 34
6.1.3.75 kappaTR() [1/4] . . . 34
6.1.3.76 kappaTR() [2/4] . . . 34
6.1.3.77 kappaTR() [3/4] . . . 35
6.1.3.78 kappaTR() [4/4] . . . 35
6.1.3.79 kappaVib() [1/4] . . . 35
6.1.3.80 kappaVib() [2/4] . . . 35
6.1.3.81 kappaVib() [3/4] . . . 35
6.1.3.82 kappaVib() [4/4] . . . 35
6.1.3.83 lookupThermo() [1/2] . . . 35
6.1.3.84 lookupThermo() [2/2] . . . 35
6.1.3.85 New() [1/11] . . . 36
6.1.3.86 New() [2/11] . . . 36
6.1.3.87 New() [3/11] . . . 36
6.1.3.88 New() [4/11] . . . 36
6.1.3.89 New() [5/11] . . . 36
6.1.3.90 New() [6/11] . . . 37
6.1.3.91 New() [7/11] . . . 37
6.1.3.92 New() [8/11] . . . 37
6.1.3.93 New() [9/11] . . . 38
6.1.3.94 New() [10/11] . . . 38
6.1.3.95 New() [11/11] . . . 38
6.1.3.96 p() [1/4] . . . 38
6.1.3.97 p() [2/4] . . . 38
6.1.3.98 p() [3/4] . . . 38
6.1.3.99 p() [4/4] . . . 38
6.1.3.100 phasePropertyName() [1/4] . . . 39
6.1.3.101 phasePropertyName() [2/4] . . . 39
6.1.3.102 phasePropertyName() [3/4] . . . 39
6.1.3.103 phasePropertyName() [4/4] . . . 40
6.1.3.104 phaseScopedName() [1/2] . . . 40

6.1.3.105 phaseScopedName() [2/2] 41
6.1.3.106 printThermoNames() [1/2] 41
6.1.3.107 printThermoNames() [2/2] 41
6.1.3.108 read() [1/2] 42
6.1.3.109 read() [2/2] 42
6.1.3.110 rho() [1/4] 42
6.1.3.111 rho() [2/4] 42
6.1.3.112 rho() [3/4] 42
6.1.3.113 rho() [4/4] 42
6.1.3.114 rhoEoS() [1/2] 42
6.1.3.115 rhoEoS() [2/2] 43
6.1.3.116 splitThermoName() [1/2] 43
6.1.3.117 splitThermoName() [2/2] 43
6.1.3.118 TEH_Vib() [1/4] 43
6.1.3.119 TEH_Vib() [2/4] 43
6.1.3.120 TEH_Vib() [3/4] 44
6.1.3.121 TEH_Vib() [4/4] 44
6.1.3.122 THE_TR() [1/4] 44
6.1.3.123 THE_TR() [2/4] 44
6.1.3.124 THE_TR() [3/4] 44
6.1.3.125 THE_TR() [4/4] 44
6.1.3.126 thermoName() [1/2] 45
6.1.3.127 thermoName() [2/2] 45
6.1.3.128 TTR() [1/4] 45
6.1.3.129 TTR() [2/4] 45
6.1.3.130 TTR() [3/4] 45
6.1.3.131 TTR() [4/4] 45
6.1.3.132 TVib() [1/4] 45
6.1.3.133 TVib() [2/4] 46
6.1.3.134 TVib() [3/4] 46
6.1.3.135 TVib() [4/4] 46
6.1.3.136 updateTTR() [1/2] 46
6.1.3.137 updateTTR() [2/2] 46
6.1.3.138 updateTVib() [1/2] 46
6.1.3.139 updateTVib() [2/2] 46
6.1.3.140 validate() [1/8] 47
6.1.3.141 validate() [2/8] 47
6.1.3.142 validate() [3/8] 47
6.1.3.143 validate() [4/8] 47
6.1.3.144 validate() [5/8] 47
6.1.3.145 validate() [6/8] 47
6.1.3.146 validate() [7/8] 48

6.1.3.147 validate() [8/8] . . . 48
6.1.3.148 W() [1/2] . . . 48
6.1.3.149 W() [2/2] . . . 48
6.1.4 Member Data Documentation . . . 48
6.1.4.1 alpha_ . . . 48
6.1.4.2 dictName . . . 48
6.1.4.3 dpdt_ . . . 49
6.1.4.4 p_ . . . 49
6.1.4.5 phaseName_ . . . 49
6.1.4.6 pOwner_ . . . 49
6.1.4.7 TOwnerTR_ . . . 49
6.1.4.8 TOwnerVib_ . . . 49
6.1.4.9 TTR_ . . . 50
6.1.4.10 TVib_ . . . 50
6.2 Foam::blottnerEucken2TTransport< Thermo2T > Class Template Reference . . . 50
6.2.1 Detailed Description . . . 53
6.2.2 Constructor & Destructor Documentation . . . 53
6.2.2.1 blottnerEucken2TTransport() [1/8] . . . 53
6.2.2.2 blottnerEucken2TTransport() [2/8] . . . 54
6.2.2.3 blottnerEucken2TTransport() [3/8] . . . 54
6.2.2.4 blottnerEucken2TTransport() [4/8] . . . 54
6.2.2.5 blottnerEucken2TTransport() [5/8] . . . 54
6.2.2.6 blottnerEucken2TTransport() [6/8] . . . 54
6.2.2.7 blottnerEucken2TTransport() [7/8] . . . 54
6.2.2.8 blottnerEucken2TTransport() [8/8] . . . 55
6.2.3 Member Function Documentation . . . 55
6.2.3.1 alphah() [1/2] . . . 55
6.2.3.2 alphah() [2/2] . . . 55
6.2.3.3 clone() [1/2] . . . 55
6.2.3.4 clone() [2/2] . . . 56
6.2.3.5 kappaTR() [1/2] . . . 56
6.2.3.6 kappaTR() [2/2] . . . 56
6.2.3.7 kappaVib() [1/2] . . . 57
6.2.3.8 kappaVib() [2/2] . . . 57
6.2.3.9 mu() [1/2] . . . 57
6.2.3.10 mu() [2/2] . . . 57
6.2.3.11 New() [1/2] . . . 57
6.2.3.12 New() [2/2] . . . 58
6.2.3.13 operator*=() [1/2] . . . 58
6.2.3.14 operator*=() [2/2] . . . 58
6.2.3.15 operator+=() [1/2] . . . 58
6.2.3.16 operator+=() [2/2] . . . 58

6.2.3.17 typeName() [1/2] . . . 58
6.2.3.18 typeName() [2/2] . . . 58
6.2.3.19 write() [1/2] . . . 59
6.2.3.20 write() [2/2] . . . 59
6.2.4 Friends And Related Symbol Documentation . . . 59
6.2.4.1 operator∗ [1/2] . . . 59
6.2.4.2 operator∗ [2/2] . . . 59
6.2.4.3 operator+ [1/2] . . . 59
6.2.4.4 operator+ [2/2] . . . 59
6.2.4.5 operator<< [1/2] . . . 60
6.2.4.6 operator<< [2/2] . . . 60
6.3 Foam::fluid2TThermo Class Reference . . . 60
6.3.1 Detailed Description . . . 67
6.3.2 Constructor & Destructor Documentation . . . 68
6.3.2.1 fluid2TThermo() [1/6] . . . 68
6.3.2.2 fluid2TThermo() [2/6] . . . 68
6.3.2.3 fluid2TThermo() [3/6] . . . 68
6.3.2.4 ∼fluid2TThermo() [1/2] . . . 68
6.3.2.5 fluid2TThermo() [4/6] . . . 68
6.3.2.6 fluid2TThermo() [5/6] . . . 68
6.3.2.7 fluid2TThermo() [6/6] . . . 69
6.3.2.8 ∼fluid2TThermo() [2/2] . . . 69
6.3.3 Member Function Documentation . . . 69
6.3.3.1 correctRho() [1/4] . . . 69
6.3.3.2 correctRho() [2/4] . . . 69
6.3.3.3 correctRho() [3/4] . . . 69
6.3.3.4 correctRho() [4/4] . . . 69
6.3.3.5 declareRunTimeSelectionTable() [1/4] . . . 70
6.3.3.6 declareRunTimeSelectionTable() [2/4] . . . 70
6.3.3.7 declareRunTimeSelectionTable() [3/4] . . . 70
6.3.3.8 declareRunTimeSelectionTable() [4/4] . . . 70
6.3.3.9 mu() [1/4] . . . 70
6.3.3.10 mu() [2/4] . . . 70
6.3.3.11 mu() [3/4] . . . 71
6.3.3.12 mu() [4/4] . . . 71
6.3.3.13 New() [1/4] . . . 71
6.3.3.14 New() [2/4] . . . 71
6.3.3.15 New() [3/4] . . . 72
6.3.3.16 New() [4/4] . . . 72
6.3.3.17 nu() [1/4] . . . 72
6.3.3.18 nu() [2/4] . . . 72
6.3.3.19 nu() [3/4] . . . 72

6.3.3.20 nu() [4/4] . . . 72
6.3.3.21 psi() [1/2] . . . 73
6.3.3.22 psi() [2/2] . . . 73
6.4 Foam::he2TThermo< Basic2TThermo, MixtureType > Class Template Reference . . . 73
6.4.1 Detailed Description . . . 77
6.4.2 Constructor & Destructor Documentation . . . 78
6.4.2.1 he2TThermo() [1/6] . . . 78
6.4.2.2 he2TThermo() [2/6] . . . 78
6.4.2.3 he2TThermo() [3/6] . . . 78
6.4.2.4 ~he2TThermo() [1/2] . . . 78
6.4.2.5 he2TThermo() [4/6] . . . 78
6.4.2.6 he2TThermo() [5/6] . . . 79
6.4.2.7 he2TThermo() [6/6] . . . 79
6.4.2.8 ~he2TThermo() [2/2] . . . 79
6.4.3 Member Function Documentation . . . 79
6.4.3.1 alphaEff() [1/4] . . . 79
6.4.3.2 alphaEff() [2/4] . . . 79
6.4.3.3 alphaEff() [3/4] . . . 79
6.4.3.4 alphaEff() [4/4] . . . 80
6.4.3.5 alphaheTR() [1/4] . . . 80
6.4.3.6 alphaheTR() [2/4] . . . 80
6.4.3.7 alphaheTR() [3/4] . . . 80
6.4.3.8 alphaheTR() [4/4] . . . 80
6.4.3.9 composition() [1/4] . . . 80
6.4.3.10 composition() [2/4] . . . 80
6.4.3.11 composition() [3/4] . . . 81
6.4.3.12 composition() [4/4] . . . 81
6.4.3.13 CpByCpvTR() [1/4] . . . 81
6.4.3.14 CpByCpvTR() [2/4] . . . 81
6.4.3.15 CpByCpvTR() [3/4] . . . 81
6.4.3.16 CpByCpvTR() [4/4] . . . 81
6.4.3.17 CpTR() [1/6] . . . 82
6.4.3.18 CpTR() [2/6] . . . 82
6.4.3.19 CpTR() [3/6] . . . 82
6.4.3.20 CpTR() [4/6] . . . 82
6.4.3.21 CpTR() [5/6] . . . 82
6.4.3.22 CpTR() [6/6] . . . 82
6.4.3.23 CpvTR() [1/4] . . . 83
6.4.3.24 CpvTR() [2/4] . . . 83
6.4.3.25 CpvTR() [3/4] . . . 83
6.4.3.26 CpvTR() [4/4] . . . 83
6.4.3.27 CvR() [1/4] . . . 83

6.4.3.28 CvR() [2/4] 84
6.4.3.29 CvR() [3/4] 84
6.4.3.30 CvR() [4/4] 84
6.4.3.31 CvT() [1/4] 84
6.4.3.32 CvT() [2/4] 84
6.4.3.33 CvT() [3/4] 85
6.4.3.34 CvT() [4/4] 85
6.4.3.35 CvVib() [1/4] 85
6.4.3.36 CvVib() [2/4] 85
6.4.3.37 CvVib() [3/4] 86
6.4.3.38 CvVib() [4/4] 86
6.4.3.39 gamma() [1/4] 86
6.4.3.40 gamma() [2/4] 86
6.4.3.41 gamma() [3/4] 86
6.4.3.42 gamma() [4/4] 87
6.4.3.43 hc() [1/2] 87
6.4.3.44 hc() [2/2] 87
6.4.3.45 he() [1/10] 87
6.4.3.46 he() [2/10] 87
6.4.3.47 he() [3/10] 87
6.4.3.48 he() [4/10] 88
6.4.3.49 he() [5/10] 88
6.4.3.50 he() [6/10] 88
6.4.3.51 he() [7/10] 88
6.4.3.52 he() [8/10] 88
6.4.3.53 he() [9/10] 89
6.4.3.54 he() [10/10] 89
6.4.3.55 heBoundaryCorrection() [1/2] 89
6.4.3.56 heBoundaryCorrection() [2/2] 89
6.4.3.57 incompressible() [1/2] 89
6.4.3.58 incompressible() [2/2] 89
6.4.3.59 isochoric() [1/2] 90
6.4.3.60 isochoric() [2/2] 90
6.4.3.61 kappaEff() [1/4] 90
6.4.3.62 kappaEff() [2/4] 90
6.4.3.63 kappaEff() [3/4] 91
6.4.3.64 kappaEff() [4/4] 91
6.4.3.65 kappaTR() [1/4] 91
6.4.3.66 kappaTR() [2/4] 92
6.4.3.67 kappaTR() [3/4] 92
6.4.3.68 kappaTR() [4/4] 92
6.4.3.69 kappaVib() [1/4] 92

6.4.3.70 kappaVib() [2/4] . . . 93
6.4.3.71 kappaVib() [3/4] . . . 93
6.4.3.72 kappaVib() [4/4] . . . 93
6.4.3.73 read() [1/2] . . . 93
6.4.3.74 read() [2/2] . . . 93
6.4.3.75 rhoEoS() [1/2] . . . 94
6.4.3.76 rhoEoS() [2/2] . . . 94
6.4.3.77 TEH_Vib() [1/4] . . . 94
6.4.3.78 TEH_Vib() [2/4] . . . 94
6.4.3.79 TEH_Vib() [3/4] . . . 94
6.4.3.80 TEH_Vib() [4/4] . . . 95
6.4.3.81 THE_TR() [1/4] . . . 95
6.4.3.82 THE_TR() [2/4] . . . 95
6.4.3.83 THE_TR() [3/4] . . . 95
6.4.3.84 THE_TR() [4/4] . . . 95
6.4.3.85 thermoName() [1/2] . . . 96
6.4.3.86 thermoName() [2/2] . . . 96
6.4.3.87 W() [1/2] . . . 96
6.4.3.88 W() [2/2] . . . 96
6.4.4 Member Data Documentation . . . 96
6.4.4.1 he_ . . . 96
6.5 Foam::heNe2TThermo< BasicNe2TThermo, MixtureType > Class Template Reference . . . 97
6.5.1 Detailed Description . . . 101
6.5.2 Constructor & Destructor Documentation . . . 101
6.5.2.1 heNe2TThermo() [1/4] . . . 101
6.5.2.2 heNe2TThermo() [2/4] . . . 101
6.5.2.3 ~heNe2TThermo() [1/2] . . . 102
6.5.2.4 heNe2TThermo() [3/4] . . . 102
6.5.2.5 heNe2TThermo() [4/4] . . . 102
6.5.2.6 ~heNe2TThermo() [2/2] . . . 102
6.5.3 Member Function Documentation . . . 102
6.5.3.1 correct() [1/2] . . . 102
6.5.3.2 correct() [2/2] . . . 102
6.6 Foam::janaf2TThermo< EquationOfState > Class Template Reference . . . 103
6.6.1 Detailed Description . . . 106
6.6.2 Member Typedef Documentation . . . 106
6.6.2.1 coeffArray [1/2] . . . 106
6.6.2.2 coeffArray [2/2] . . . 106
6.6.3 Constructor & Destructor Documentation . . . 106
6.6.3.1 janaf2TThermo() [1/6] . . . 106
6.6.3.2 janaf2TThermo() [2/6] . . . 106
6.6.3.3 janaf2TThermo() [3/6] . . . 107

6.6.3.4 janaf2TThermo() [4/6] . . . 107
6.6.3.5 janaf2TThermo() [5/6] . . . 107
6.6.3.6 janaf2TThermo() [6/6] . . . 107
6.6.4 Member Function Documentation . . . 107
6.6.4.1 CpTR() [1/2] . . . 107
6.6.4.2 CpTR() [2/2] . . . 108
6.6.4.3 dCpdT() [1/2] . . . 108
6.6.4.4 dCpdT() [2/2] . . . 108
6.6.4.5 Gstd() [1/2] . . . 108
6.6.4.6 Gstd() [2/2] . . . 108
6.6.4.7 Ha() [1/2] . . . 108
6.6.4.8 Ha() [2/2] . . . 109
6.6.4.9 Hc() [1/2] . . . 109
6.6.4.10 Hc() [2/2] . . . 109
6.6.4.11 highCpCoeffs() [1/2] . . . 109
6.6.4.12 highCpCoeffs() [2/2] . . . 109
6.6.4.13 Hs() [1/2] . . . 109
6.6.4.14 Hs() [2/2] . . . 109
6.6.4.15 limit() [1/2] . . . 110
6.6.4.16 limit() [2/2] . . . 110
6.6.4.17 lowCpCoeffs() [1/2] . . . 110
6.6.4.18 lowCpCoeffs() [2/2] . . . 110
6.6.4.19 operator+=() [1/2] . . . 110
6.6.4.20 operator+=() [2/2] . . . 110
6.6.4.21 S() [1/2] . . . 110
6.6.4.22 S() [2/2] . . . 111
6.6.4.23 Tcommon() [1/2] . . . 111
6.6.4.24 Tcommon() [2/2] . . . 111
6.6.4.25 ThetaVib() [1/2] . . . 111
6.6.4.26 ThetaVib() [2/2] . . . 111
6.6.4.27 Thigh() [1/2] . . . 111
6.6.4.28 Thigh() [2/2] . . . 111
6.6.4.29 Tlow() [1/2] . . . 112
6.6.4.30 Tlow() [2/2] . . . 112
6.6.4.31 typeName() [1/2] . . . 112
6.6.4.32 typeName() [2/2] . . . 112
6.6.4.33 write() [1/2] . . . 112
6.6.4.34 write() [2/2] . . . 112
6.6.5 Friends And Related Symbol Documentation . . . 112
6.6.5.1 operator* [1/2] . . . 112
6.6.5.2 operator* [2/2] . . . 113
6.6.5.3 operator+ [1/2] . . . 113

6.6.5.4 operator+ [2/2] . . . 113
6.6.5.5 operator<< [1/2] . . . 113
6.6.5.6 operator<< [2/2] . . . 113
6.6.5.7 operator== [1/2] . . . 113
6.6.5.8 operator== [2/2] . . . 113
6.6.6 Member Data Documentation . . . 114
6.6.6.1 nCoeffs_ . . . 114
6.7 Foam::ne2TThermo Class Reference . . . 114
6.7.1 Detailed Description . . . 122
6.7.2 Constructor & Destructor Documentation . . . 122
6.7.2.1 ne2TThermo() [1/6] . . . 122
6.7.2.2 ne2TThermo() [2/6] . . . 122
6.7.2.3 ne2TThermo() [3/6] . . . 122
6.7.2.4 ~ne2TThermo() [1/2] . . . 123
6.7.2.5 ne2TThermo() [4/6] . . . 123
6.7.2.6 ne2TThermo() [5/6] . . . 123
6.7.2.7 ne2TThermo() [6/6] . . . 123
6.7.2.8 ~ne2TThermo() [2/2] . . . 123
6.7.3 Member Function Documentation . . . 123
6.7.3.1 correctRho() [1/4] . . . 123
6.7.3.2 correctRho() [2/4] . . . 123
6.7.3.3 correctRho() [3/4] . . . 124
6.7.3.4 correctRho() [4/4] . . . 124
6.7.3.5 declareRunTimeSelectionTable() [1/4] . . . 124
6.7.3.6 declareRunTimeSelectionTable() [2/4] . . . 124
6.7.3.7 declareRunTimeSelectionTable() [3/4] . . . 124
6.7.3.8 declareRunTimeSelectionTable() [4/4] . . . 124
6.7.3.9 mu() [1/4] . . . 125
6.7.3.10 mu() [2/4] . . . 125
6.7.3.11 mu() [3/4] . . . 125
6.7.3.12 mu() [4/4] . . . 125
6.7.3.13 New() [1/4] . . . 125
6.7.3.14 New() [2/4] . . . 126
6.7.3.15 New() [3/4] . . . 126
6.7.3.16 New() [4/4] . . . 126
6.7.3.17 psi() [1/2] . . . 126
6.7.3.18 psi() [2/2] . . . 126
6.7.3.19 rho() [1/4] . . . 127
6.7.3.20 rho() [2/4] . . . 127
6.7.3.21 rho() [3/4] . . . 127
6.7.3.22 rho() [4/4] . . . 127
6.7.4 Member Data Documentation . . . 127

6.7.4.1 mu_ . . . 127
6.7.4.2 psi_ . . . 127
6.8 Foam::sensibleEnthalpy2T< Thermo2T > Class Template Reference . . . 128
6.8.1 Detailed Description . . . 129
6.8.2 Constructor & Destructor Documentation . . . 129
6.8.2.1 sensibleEnthalpy2T() [1/2] . . . 129
6.8.2.2 sensibleEnthalpy2T() [2/2] . . . 129
6.8.3 Member Function Documentation . . . 129
6.8.3.1 CpByCpvTR() [1/2] . . . 129
6.8.3.2 CpByCpvTR() [2/2] . . . 129
6.8.3.3 CpvTR() [1/2] . . . 130
6.8.3.4 CpvTR() [2/2] . . . 130
6.8.3.5 energyName() [1/2] . . . 130
6.8.3.6 energyName() [2/2] . . . 130
6.8.3.7 HE() [1/2] . . . 130
6.8.3.8 HE() [2/2] . . . 130
6.8.3.9 TEH_Vib() [1/2] . . . 131
6.8.3.10 TEH_Vib() [2/2] . . . 131
6.8.3.11 THE_TR() [1/2] . . . 131
6.8.3.12 THE_TR() [2/2] . . . 131
6.8.3.13 typeName() [1/2] . . . 131
6.8.3.14 typeName() [2/2] . . . 132
6.9 Foam::species::thermo2T< Thermo2T, Type > Class Template Reference . . . 132
6.9.1 Detailed Description . . . 135
6.9.2 Member Typedef Documentation . . . 136
6.9.2.1 thermoType [1/2] . . . 136
6.9.2.2 thermoType [2/2] . . . 136
6.9.3 Constructor & Destructor Documentation . . . 136
6.9.3.1 thermo2T() [1/6] . . . 136
6.9.3.2 thermo2T() [2/6] . . . 136
6.9.3.3 thermo2T() [3/6] . . . 136
6.9.3.4 thermo2T() [4/6] . . . 136
6.9.3.5 thermo2T() [5/6] . . . 137
6.9.3.6 thermo2T() [6/6] . . . 137
6.9.4 Member Function Documentation . . . 137
6.9.4.1 A() [1/2] . . . 137
6.9.4.2 a() [1/2] . . . 137
6.9.4.3 A() [2/2] . . . 137
6.9.4.4 a() [2/2] . . . 137
6.9.4.5 CpByCpvTR() [1/2] . . . 138
6.9.4.6 CpByCpvTR() [2/2] . . . 138
6.9.4.7 cpTR() [1/2] . . . 138

6.9.4.8 cpTR() [2/2] . . . 138
6.9.4.9 CpvTR() [1/2] . . . 139
6.9.4.10 CpvTR() [2/2] . . . 139
6.9.4.11 cvR() [1/2] . . . 139
6.9.4.12 cvR() [2/2] . . . 139
6.9.4.13 cvT() [1/2] . . . 140
6.9.4.14 cvT() [2/2] . . . 140
6.9.4.15 cvVib() [1/2] . . . 140
6.9.4.16 cvVib() [2/2] . . . 141
6.9.4.17 dcpdT() [1/2] . . . 141
6.9.4.18 dcpdT() [2/2] . . . 141
6.9.4.19 dKcdTbyKc() [1/2] . . . 141
6.9.4.20 dKcdTbyKc() [2/2] . . . 141
6.9.4.21 eaT() [1/2] . . . 142
6.9.4.22 eaT() [2/2] . . . 142
6.9.4.23 esR() [1/2] . . . 142
6.9.4.24 esR() [2/2] . . . 143
6.9.4.25 esT() [1/2] . . . 143
6.9.4.26 esT() [2/2] . . . 143
6.9.4.27 esVib() [1/2] . . . 143
6.9.4.28 esVib() [2/2] . . . 144
6.9.4.29 G() [1/2] . . . 144
6.9.4.30 g() [1/2] . . . 144
6.9.4.31 G() [2/2] . . . 144
6.9.4.32 g() [2/2] . . . 145
6.9.4.33 gamma() [1/2] . . . 145
6.9.4.34 gamma() [2/2] . . . 145
6.9.4.35 ha() [1/2] . . . 145
6.9.4.36 ha() [2/2] . . . 145
6.9.4.37 hc() [1/2] . . . 146
6.9.4.38 hc() [2/2] . . . 146
6.9.4.39 HE() [1/2] . . . 146
6.9.4.40 he() [1/2] . . . 146
6.9.4.41 HE() [2/2] . . . 146
6.9.4.42 he() [2/2] . . . 146
6.9.4.43 heName() [1/2] . . . 146
6.9.4.44 heName() [2/2] . . . 147
6.9.4.45 hs() [1/2] . . . 147
6.9.4.46 hs() [2/2] . . . 147
6.9.4.47 K() [1/2] . . . 147
6.9.4.48 K() [2/2] . . . 147
6.9.4.49 Kc() [1/2] . . . 147

6.9.4.50 Kc() [2/2] . . . 148
6.9.4.51 Kn() [1/2] . . . 148
6.9.4.52 Kn() [2/2] . . . 148
6.9.4.53 Kp() [1/2] . . . 148
6.9.4.54 Kp() [2/2] . . . 148
6.9.4.55 Kx() [1/2] . . . 148
6.9.4.56 Kx() [2/2] . . . 149
6.9.4.57 operator∗=() [1/2] . . . 149
6.9.4.58 operator∗=() [2/2] . . . 149
6.9.4.59 operator+=() [1/2] . . . 149
6.9.4.60 operator+=() [2/2] . . . 149
6.9.4.61 s() [1/2] . . . 149
6.9.4.62 s() [2/2] . . . 149
6.9.4.63 TEa_TR() [1/2] . . . 150
6.9.4.64 TEa_TR() [2/2] . . . 150
6.9.4.65 TEH_Vib() [1/2] . . . 150
6.9.4.66 TEH_Vib() [2/2] . . . 150
6.9.4.67 TEs_TR() [1/2] . . . 150
6.9.4.68 TEs_TR() [2/2] . . . 151
6.9.4.69 TEs_Vib() [1/2] . . . 151
6.9.4.70 TEs_Vib() [2/2] . . . 151
6.9.4.71 THa_TR() [1/2] . . . 151
6.9.4.72 THa_TR() [2/2] . . . 151
6.9.4.73 THE_TR() [1/2] . . . 152
6.9.4.74 THE_TR() [2/2] . . . 152
6.9.4.75 THs_TR() [1/2] . . . 152
6.9.4.76 THs_TR() [2/2] . . . 152
6.9.4.77 typeName() [1/2] . . . 152
6.9.4.78 typeName() [2/2] . . . 153
6.9.4.79 write() [1/2] . . . 154
6.9.4.80 write() [2/2] . . . 154
6.9.5 Friends And Related Symbol Documentation . . . 154
6.9.5.1 operator [1/2] . . . 154
6.9.5.2 operator [2/2] . . . 154
6.9.5.3 operator∗ [1/2] . . . 154
6.9.5.4 operator∗ [2/2] . . . 154
6.9.5.5 operator+ [1/2] . . . 155
6.9.5.6 operator+ [2/2] . . . 155
6.9.5.7 operator== [1/2] . . . 155
6.9.5.8 operator== [2/2] . . . 155
6.10 Foam::typeDecider< OneT, TwoT > Class Template Reference . . . 155
6.10.1 Detailed Description . . . 156

6.10.2 Constructor & Destructor Documentation . . . 156
6.10.2.1 typeDecider() . . . 156
6.10.3 Member Function Documentation . . . 157
6.10.3.1 construct() . . . 157
6.10.4 Member Data Documentation . . . 157
6.10.4.1 tempModel . . . 157
6.10.4.2 thermoDict . . . 157
6.10.4.3 tt . . . 157

7 File Documentation **159**

7.1 basic2TThermo/basic2TThermo.C File Reference . . . 159
7.2 basic2TThermo/basic2TThermo.C . . . 160
7.3 lnInclude/basic2TThermo.C File Reference . . . 167
7.4 lnInclude/basic2TThermo.C . . . 168
7.5 basic2TThermo/basic2TThermo.H File Reference . . . 176
7.6 basic2TThermo/basic2TThermo.H . . . 177
7.7 lnInclude/basic2TThermo.H File Reference . . . 185
7.8 lnInclude/basic2TThermo.H . . . 186
7.9 basic2TThermo/basic2TThermoTemplates.C File Reference . . . 194
7.10 basic2TThermo/basic2TThermoTemplates.C . . . 194
7.11 lnInclude/basic2TThermoTemplates.C File Reference . . . 197
7.12 lnInclude/basic2TThermoTemplates.C . . . 197
7.13 fluid2TThermo/fluid2TThermo.C File Reference . . . 200
7.14 fluid2TThermo/fluid2TThermo.C . . . 200
7.15 lnInclude/fluid2TThermo.C File Reference . . . 202
7.16 lnInclude/fluid2TThermo.C . . . 202
7.17 fluid2TThermo/fluid2TThermo.H File Reference . . . 204
7.18 fluid2TThermo/fluid2TThermo.H . . . 205
7.19 lnInclude/fluid2TThermo.H File Reference . . . 207
7.20 lnInclude/fluid2TThermo.H . . . 208
7.21 he2TThermo/he2TThermo.C File Reference . . . 210
7.22 he2TThermo/he2TThermo.C . . . 210
7.23 lnInclude/he2TThermo.C File Reference . . . 223
7.24 lnInclude/he2TThermo.C . . . 224
7.25 he2TThermo/he2TThermo.H File Reference . . . 237
7.26 he2TThermo/he2TThermo.H . . . 237
7.27 lnInclude/he2TThermo.H File Reference . . . 243
7.28 lnInclude/he2TThermo.H . . . 243
7.29 lnInclude/heNe2TThermo.C File Reference . . . 249
7.30 lnInclude/heNe2TThermo.C . . . 249
7.31 ne2TThermo/heNe2TThermo.C File Reference . . . 252
7.32 ne2TThermo/heNe2TThermo.C . . . 252

7.33 lnInclude/heNe2TThermo.H File Reference . . . 255
7.34 lnInclude/heNe2TThermo.H . . . 256
7.35 ne2TThermo/heNe2TThermo.H File Reference . . . 258
7.36 ne2TThermo/heNe2TThermo.H . . . 259
7.37 fluid2TThermo/make2TThermo.H File Reference . . . 261
7.37.1 Macro Definition Documentation . . . 261
7.37.1.1 addThermoPhysicsThermo . . . 261
7.37.1.2 defineThermoPhysicsThermo . . . 262
7.37.1.3 makeThermo . . . 262
7.37.1.4 makeThermoPhysicsThermo . . . 262
7.37.1.5 makeThermoPhysicsThermos . . . 263
7.37.1.6 makeThermos . . . 263
7.37.1.7 typedefThermoPhysics . . . 263
7.38 fluid2TThermo/make2TThermo.H . . . 264
7.39 lnInclude/make2TThermo.H File Reference . . . 266
7.39.1 Macro Definition Documentation . . . 266
7.39.1.1 addThermoPhysicsThermo . . . 266
7.39.1.2 defineThermoPhysicsThermo . . . 267
7.39.1.3 makeThermo . . . 267
7.39.1.4 makeThermoPhysicsThermo . . . 268
7.39.1.5 makeThermoPhysicsThermos . . . 268
7.39.1.6 makeThermos . . . 268
7.39.1.7 typedefThermoPhysics . . . 269
7.40 lnInclude/make2TThermo.H . . . 269
7.41 lnInclude/ne2TThermo.C File Reference . . . 271
7.42 lnInclude/ne2TThermo.C . . . 271
7.43 ne2TThermo/ne2TThermo.C File Reference . . . 274
7.44 ne2TThermo/ne2TThermo.C . . . 274
7.45 lnInclude/ne2TThermo.H File Reference . . . 277
7.46 lnInclude/ne2TThermo.H . . . 278
7.47 ne2TThermo/ne2TThermo.H File Reference . . . 280
7.48 ne2TThermo/ne2TThermo.H . . . 281
7.49 lnInclude/ne2TThermos.C File Reference . . . 283
7.50 lnInclude/ne2TThermos.C . . . 284
7.51 ne2TThermo/ne2TThermos.C File Reference . . . 285
7.52 ne2TThermo/ne2TThermos.C . . . 286
7.53 lnInclude/ne2TZoneThermos.C File Reference . . . 287
7.54 lnInclude/ne2TZoneThermos.C . . . 288
7.55 ne2TThermo/ne2TZoneThermos.C File Reference . . . 289
7.56 ne2TThermo/ne2TZoneThermos.C . . . 289
7.57 lnInclude/blottnerEucken2TTransport.C File Reference . . . 290
7.58 lnInclude/blottnerEucken2TTransport.C . . . 291

7.59 transport/blottnerEucken2T/blottnerEucken2TTransport.C File Reference . . . 292
7.60 transport/blottnerEucken2T/blottnerEucken2TTransport.C . . . 293
7.61 lnInclude/blottnerEucken2TTransport.H File Reference . . . 295
7.62 lnInclude/blottnerEucken2TTransport.H . . . 296
7.63 transport/blottnerEucken2T/blottnerEucken2TTransport.H File Reference . . . 299
7.64 transport/blottnerEucken2T/blottnerEucken2TTransport.H . . . 300
7.65 lnInclude/blottnerEucken2TTransportI.H File Reference . . . 303
7.66 lnInclude/blottnerEucken2TTransportI.H . . . 303
7.67 transport/blottnerEucken2T/blottnerEucken2TTransportI.H File Reference . . . 306
7.68 transport/blottnerEucken2T/blottnerEucken2TTransportI.H . . . 307
7.69 lnInclude/EtoHthermo2T.H File Reference . . . 310
7.69.1 Function Documentation . . . 310
7.69.1.1 CpTR() . . . 310
7.69.1.2 HaTR() . . . 311
7.69.1.3 HsTR() . . . 311
7.70 lnInclude/EtoHthermo2T.H . . . 312
7.71 thermo/thermo2T/EtoHthermo2T.H File Reference . . . 312
7.71.1 Function Documentation . . . 312
7.71.1.1 CpTR() . . . 312
7.71.1.2 HaTR() . . . 313
7.71.1.3 HsTR() . . . 313
7.72 thermo/thermo2T/EtoHthermo2T.H . . . 313
7.73 lnInclude/HtoEthermo2T.H File Reference . . . 314
7.73.1 Function Documentation . . . 314
7.73.1.1 CvR() . . . 314
7.73.1.2 CvT() . . . 315
7.73.1.3 CvVib() . . . 316
7.73.1.4 EaT() . . . 317
7.73.1.5 EsR() . . . 317
7.73.1.6 EsT() . . . 318
7.73.1.7 EsVib() . . . 318
7.74 lnInclude/HtoEthermo2T.H . . . 319
7.75 thermo/thermo2T/HtoEthermo2T.H File Reference . . . 320
7.75.1 Function Documentation . . . 320
7.75.1.1 CvR() . . . 320
7.75.1.2 CvT() . . . 320
7.75.1.3 CvVib() . . . 321
7.75.1.4 EaT() . . . 321
7.75.1.5 EsR() . . . 321
7.75.1.6 EsT() . . . 321
7.75.1.7 EsVib() . . . 322
7.76 thermo/thermo2T/HtoEthermo2T.H . . . 322

7.77 lnInclude/janaf2TThermo.C File Reference . . . 323
7.78 lnInclude/janaf2TThermo.C . . . 324
7.79 thermo/janaf2T/janaf2TThermo.C File Reference . . . 325
7.80 thermo/janaf2T/janaf2TThermo.C . . . 326
7.81 lnInclude/janaf2TThermo.H File Reference . . . 327
7.82 lnInclude/janaf2TThermo.H . . . 328
7.83 thermo/janaf2T/janaf2TThermo.H File Reference . . . 332
7.84 thermo/janaf2T/janaf2TThermo.H . . . 333
7.85 lnInclude/janaf2TThermoI.H File Reference . . . 336
7.86 lnInclude/janaf2TThermoI.H . . . 337
7.87 thermo/janaf2T/janaf2TThermoI.H File Reference . . . 343
7.88 thermo/janaf2T/janaf2TThermoI.H . . . 344
7.89 include/reaction2TTypes.H File Reference . . . 349
7.90 include/reaction2TTypes.H . . . 350
7.91 lnInclude/reaction2TTypes.H File Reference . . . 350
7.92 lnInclude/reaction2TTypes.H . . . 351
7.93 lnInclude/sensibleEnthalpy2T.H File Reference . . . 352
7.94 lnInclude/sensibleEnthalpy2T.H . . . 352
7.95 thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H File Reference . . . 354
7.96 thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H . . . 355
7.97 lnInclude/thermo2T.C File Reference . . . 357
7.98 lnInclude/thermo2T.C . . . 357
7.99 thermo/thermo2T/thermo2T.C File Reference . . . 358
7.100 thermo/thermo2T/thermo2T.C . . . 358
7.101 lnInclude/thermo2T.H File Reference . . . 359
7.102 lnInclude/thermo2T.H . . . 361
7.103 thermo/thermo2T/thermo2T.H File Reference . . . 366
7.104 thermo/thermo2T/thermo2T.H . . . 367
7.105 lnInclude/thermo2TI.H File Reference . . . 372
7.106 lnInclude/thermo2TI.H . . . 373
7.107 thermo/thermo2T/thermo2TI.H File Reference . . . 381
7.108 thermo/thermo2T/thermo2TI.H . . . 381
7.109 include/thermoPhysics2TTypes.H File Reference . . . 389
7.110 include/thermoPhysics2TTypes.H . . . 390
7.111 lnInclude/thermoPhysics2TTypes.H File Reference . . . 391
7.112 lnInclude/thermoPhysics2TTypes.H . . . 392
7.113 typeDecider.C File Reference . . . 393
7.114 typeDecider.C . . . 393
7.115 typeDecider.H File Reference . . . 394
7.116 typeDecider.H . . . 395
7.117 typeDeciderI.H File Reference . . . 396
7.118 typeDeciderI.H . . . 396

7.119 types.C File Reference 397
7.120 types.C 397

Index **399**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Foam 9
Foam::species 15

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Basic2TThermo
- Foam::he2TThermo< Basic2TThermo, MixtureType > . . . 73
- Foam::he2TThermo< Basic2TThermo, MixtureType > . . . 73

BasicNe2TThermo
- Foam::he2TThermo< BasicNe2TThermo, MixtureType > . . . 73
  - Foam::heNe2TThermo< BasicNe2TThermo, MixtureType > . . . 97
  - Foam::heNe2TThermo< BasicNe2TThermo, MixtureType > . . . 97
- Foam::he2TThermo< BasicNe2TThermo, MixtureType > . . . 73

compressibleTransportModel
- Foam::fluid2TThermo . . . 60
  - Foam::ne2TThermo . . . 114
  - Foam::ne2TThermo . . . 114
- Foam::fluid2TThermo . . . 60

EquationOfState
- Foam::janaf2TThermo< EquationOfState > . . . 103
- Foam::janaf2TThermo< EquationOfState > . . . 103

IOdictionary
- Foam::basic2TThermo . . . 17
  - Foam::fluid2TThermo . . . 60
  - Foam::fluid2TThermo . . . 60
- Foam::basic2TThermo . . . 17

MixtureType
- Foam::he2TThermo< BasicNe2TThermo, MixtureType > . . . 73
- Foam::he2TThermo< BasicNe2TThermo, MixtureType > . . . 73
- Foam::he2TThermo< Basic2TThermo, MixtureType > . . . 73
- Foam::he2TThermo< Basic2TThermo, MixtureType > . . . 73

Foam::sensibleEnthalpy2T< Thermo2T > . . . 128

Thermo2T
- Foam::blottnerEucken2TTransport< Thermo2T > . . . 50
- Foam::blottnerEucken2TTransport< Thermo2T > . . . 50
- Foam::species::thermo2T< Thermo2T, Type > . . . 132
- Foam::species::thermo2T< Thermo2T, Type > . . . 132

Type
- Foam::species::thermo2T< Thermo2T, Type > . . . 132
- Foam::species::thermo2T< Thermo2T, Type > . . . 132

Foam::typeDecider< OneT, TwoT > . . . 155

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Foam::basic2TThermo 17
Foam::blottnerEucken2TTransport< Thermo2T > 50
Foam::fluid2TThermo 60
Foam::he2TThermo< Basic2TThermo, MixtureType > 73
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType > 97
Foam::janaf2TThermo< EquationOfState > 103
Foam::ne2TThermo 114
Foam::sensibleEnthalpy2T< Thermo2T > 128
Foam::species::thermo2T< Thermo2T, Type > 132
Foam::typeDecider< OneT, TwoT > 155

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

basic2TThermo/basic2TThermo.C . . . 159
lnInclude/basic2TThermo.C . . . 167
basic2TThermo/basic2TThermo.H . . . 176
lnInclude/basic2TThermo.H . . . 185
basic2TThermo/basic2TThermoTemplates.C . . . 194
lnInclude/basic2TThermoTemplates.C . . . 197
fluid2TThermo/fluid2TThermo.C . . . 200
lnInclude/fluid2TThermo.C . . . 202
fluid2TThermo/fluid2TThermo.H . . . 204
lnInclude/fluid2TThermo.H . . . 207
he2TThermo/he2TThermo.C . . . 210
lnInclude/he2TThermo.C . . . 223
he2TThermo/he2TThermo.H . . . 237
lnInclude/he2TThermo.H . . . 243
lnInclude/heNe2TThermo.C . . . 249
ne2TThermo/heNe2TThermo.C . . . 252
lnInclude/heNe2TThermo.H . . . 255
ne2TThermo/heNe2TThermo.H . . . 258
fluid2TThermo/make2TThermo.H . . . 261
lnInclude/make2TThermo.H . . . 266
lnInclude/ne2TThermo.C . . . 271
ne2TThermo/ne2TThermo.C . . . 274
lnInclude/ne2TThermo.H . . . 277
ne2TThermo/ne2TThermo.H . . . 280
lnInclude/ne2TThermos.C . . . 283
ne2TThermo/ne2TThermos.C . . . 285
lnInclude/ne2TZoneThermos.C . . . 287
ne2TThermo/ne2TZoneThermos.C . . . 289
lnInclude/blottnerEucken2TTransport.C . . . 290
transport/blottnerEucken2T/blottnerEucken2TTransport.C . . . 292
lnInclude/blottnerEucken2TTransport.H . . . 295
transport/blottnerEucken2T/blottnerEucken2TTransport.H . . . 299
lnInclude/blottnerEucken2TTransportI.H . . . 303
transport/blottnerEucken2T/blottnerEucken2TTransportI.H . . . 306
lnInclude/EtoHthermo2T.H . . . 310

thermo/thermo2T/EtoHthermo2T.H . . . 312
lnInclude/HtoEthermo2T.H . . . 314
thermo/thermo2T/HtoEthermo2T.H . . . 320
lnInclude/janaf2TThermo.C . . . 323
thermo/janaf2T/janaf2TThermo.C . . . 325
lnInclude/janaf2TThermo.H . . . 327
thermo/janaf2T/janaf2TThermo.H . . . 332
lnInclude/janaf2TThermoI.H . . . 336
thermo/janaf2T/janaf2TThermoI.H . . . 343
include/reaction2TTypes.H . . . 349
lnInclude/reaction2TTypes.H . . . 350
lnInclude/sensibleEnthalpy2T.H . . . 352
thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H . . . 354
lnInclude/thermo2T.C . . . 357
thermo/thermo2T/thermo2T.C . . . 358
lnInclude/thermo2T.H . . . 359
thermo/thermo2T/thermo2T.H . . . 366
lnInclude/thermo2TI.H . . . 372
thermo/thermo2T/thermo2TI.H . . . 381
include/thermoPhysics2TTypes.H . . . 389
lnInclude/thermoPhysics2TTypes.H . . . 391
typeDecider.C . . . 393
typeDecider.H . . . 394
typeDeciderI.H . . . 396
types.C . . . 397

Chapter 5

Namespace Documentation

5.1 Foam Namespace Reference

Namespaces

- namespace species

Classes

- class basic2TThermo
- class blottnerEucken2TTransport
- class fluid2TThermo
- class he2TThermo
- class heNe2TThermo
- class janaf2TThermo
- class ne2TThermo
- class sensibleEnthalpy2T
- class typeDecider

Typedefs

- typedef blottnerEucken2TTransport< species::thermo2T< janaf2TThermo< perfectGas< specie > >, sensibleEnthalpy2T > > blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
- typedef heNe2TThermo< ne2TThermo, pureMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGa > > heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
- typedef heNe2TThermo< ne2TThermo, pureZoneMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperf > > heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
- typedef Reaction< blottnerEucken2TGasHThermoPhysics > blottnerEucken2TGasHThermoPhysicsHReaction
- typedef blottnerEucken2TTransport< species::thermo2T< janaf2TThermo< perfectGas< specie > >, sensibleEnthalpy2T > > blottnerEucken2TGasHThermoPhysics

Functions

- defineRunTimeSelectionTable (basic2TThermo, fvMesh)
- defineRunTimeSelectionTable (basic2TThermo, fvMeshDictPhase)
- defineRunTimeSelectionTable (fluid2TThermo, fvMesh)
- defineRunTimeSelectionTable (fluid2TThermo, fvMeshDictPhase)
- defineRunTimeSelectionTable (ne2TThermo, fvMesh)
- defineRunTimeSelectionTable (ne2TThermo, fvMeshDictPhase)
- defineTemplateTypeNameAndDebugWithName (heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjan Ne2TThermo""<"+pureMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie >::typeName()+">").c_str(), 0)
- addThermoPhysicsThermo (basic2TThermo, heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2
- addThermoPhysicsThermo (fluid2TThermo, heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2T
- addThermoPhysicsThermo (ne2TThermo, heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TT
- defineTemplateTypeNameAndDebugWithName (heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2 Ne2TThermo""<"+pureZoneMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie >::typeName()+">").c_str(), 0)
- addThermoPhysicsThermo (basic2TThermo, heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tja
- addThermoPhysicsThermo (fluid2TThermo, heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjan
- addThermoPhysicsThermo (ne2TThermo, heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjana
- makeThermos (ne2TThermo, heNe2TThermo, pureMixture, blottnerEucken2TTransport, sensibleEnthalpy2T, janaf2TThermo, perfectGas, specie)
- makeThermos (ne2TThermo, heNe2TThermo, pureZoneMixture, blottnerEucken2TTransport, sensibleEnthalpy2T, janaf2TThermo, perfectGas, specie)
- template<class Thermo2T > blottnerEucken2TTransport< Thermo2T > operator+ (const blottnerEucken2TTransport< Thermo2T > &, const blottnerEucken2TTransport< Thermo2T > &)
- template<class Thermo2T > blottnerEucken2TTransport< Thermo2T > operator* (const scalar, const blottnerEucken2TTransport< Thermo2T > &)
- template<class Thermo2T > Ostream & operator<< (Ostream &, const blottnerEucken2TTransport< Thermo2T > &)
- template<class EquationOfState > janaf2TThermo< EquationOfState > operator+ (const janaf2TThermo< EquationOfState > &, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState > janaf2TThermo< EquationOfState > operator* (const scalar, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState > janaf2TThermo< EquationOfState > operator== (const janaf2TThermo< EquationOfState > &, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState > Ostream & operator<< (Ostream &, const janaf2TThermo< EquationOfState > &)

5.1.1 Typedef Documentation

5.1.1.1 blottnerEucken2TGasHThermoPhysics

```
typedef blottnerEucken2TTransport< species::thermo2T< janaf2TThermo< perfectGas< specie > >,
sensibleEnthalpy2T > > Foam::blottnerEucken2TGasHThermoPhysics
```

Definition at line 72 of file include/thermoPhysics2TTypes.H.

5.1.1.2 blottnerEucken2TGasHThermoPhysicsHReaction

```
typedef Reaction< blottnerEucken2TGasHThermoPhysics > Foam::blottnerEucken2TGasHThermoPhysicsHReaction
```

Definition at line 50 of file include/reaction2TTypes.H.

5.1.1.3 blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie

```
typedef blottnerEucken2TTransport< species::thermo2T< janaf2TThermo< perfectGas< specie > >,
sensibleEnthalpy2T > > Foam::blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
```

Definition at line 63 of file lnInclude/ne2TThermos.C.

5.1.1.4 heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie

```
typedef heNe2TThermo< ne2TThermo , pureMixture < blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperf
> > Foam::heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
```

Definition at line 63 of file lnInclude/ne2TThermos.C.

5.1.1.5 heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie

```
typedef heNe2TThermo< ne2TThermo , pureZoneMixture < blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermo
> > Foam::heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
```

Definition at line 65 of file lnInclude/ne2TZoneThermos.C.

5.1.2 Function Documentation

5.1.2.1 addThermoPhysicsThermo() [1/6]

```
Foam::addThermoPhysicsThermo (
            basic2TThermo ,
            heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
)
```

5.1.2.2 addThermoPhysicsThermo() [2/6]

```
Foam::addThermoPhysicsThermo (
            basic2TThermo ,
            heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspec
)
```

5.1.2.3 addThermoPhysicsThermo() [3/6]

```
Foam::addThermoPhysicsThermo (
            fluid2TThermo ,
            heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
)
```

5.1.2.4 addThermoPhysicsThermo() [4/6]

```
Foam::addThermoPhysicsThermo (
            fluid2TThermo ,
            heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspec
)
```

5.1.2.5 addThermoPhysicsThermo() [5/6]

```
Foam::addThermoPhysicsThermo (
            ne2TThermo ,
            heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
)
```

5.1.2.6 addThermoPhysicsThermo() [6/6]

```
Foam::addThermoPhysicsThermo (
            ne2TThermo ,
            heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspec
)
```

5.1.2.7 defineRunTimeSelectionTable() [1/6]

```
Foam::defineRunTimeSelectionTable (
            basic2TThermo ,
            fvMesh  )
```

5.1.2.8 defineRunTimeSelectionTable() [2/6]

```
Foam::defineRunTimeSelectionTable (
            basic2TThermo ,
            fvMeshDictPhase  )
```

5.1.2.9 defineRunTimeSelectionTable() [3/6]

```
Foam::defineRunTimeSelectionTable (
            fluid2TThermo ,
            fvMesh  )
```

5.1.2.10 defineRunTimeSelectionTable() [4/6]

```
Foam::defineRunTimeSelectionTable (
            fluid2TThermo ,
            fvMeshDictPhase  )
```

5.1.2.11 defineRunTimeSelectionTable() [5/6]

```
Foam::defineRunTimeSelectionTable (
            ne2TThermo ,
            fvMesh  )
```

5.1.2.12 defineRunTimeSelectionTable() [6/6]

```
Foam::defineRunTimeSelectionTable (
            ne2TThermo ,
            fvMeshDictPhase  )
```

5.1.2.13 defineTemplateTypeNameAndDebugWithName() [1/2]

```
Foam::defineTemplateTypeNameAndDebugWithName (
            heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
,
            ("heNe2TThermo""<"+pureMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectG
>::typeName()+">").c_str() ,
            0  )
```

5.1.2.14 defineTemplateTypeNameAndDebugWithName() [2/2]

```
Foam::defineTemplateTypeNameAndDebugWithName (
            heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspec
,
            ("heNe2TThermo""<"+pureZoneMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperf
>::typeName()+">").c_str() ,
            0  )
```

5.1.2.15 makeThermos() [1/2]

```
Foam::makeThermos (
            ne2TThermo ,
            heNe2TThermo ,
            pureMixture ,
            blottnerEucken2TTransport ,
            sensibleEnthalpy2T ,
            janaf2TThermo ,
            perfectGas ,
            specie  )
```

5.1.2.16 makeThermos() [2/2]

```
Foam::makeThermos (
            ne2TThermo ,
            heNe2TThermo ,
            pureZoneMixture ,
            blottnerEucken2TTransport ,
            sensibleEnthalpy2T ,
            janaf2TThermo ,
            perfectGas ,
            specie  )
```

5.1.2.17 operator*() [1/2]

```
template<class Thermo2T >
blottnerEucken2TTransport< Thermo2T > Foam::operator* (
            const scalar ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [inline]
```

5.1.2.18 operator*() [2/2]

```
template<class EquationOfState >
janaf2TThermo< EquationOfState > Foam::operator* (
            const scalar ,
            const janaf2TThermo< EquationOfState > &  )  [inline]
```

5.1.2.19 operator+() [1/2]

```
template<class Thermo2T >
blottnerEucken2TTransport< Thermo2T > Foam::operator+ (
            const blottnerEucken2TTransport< Thermo2T > & ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [inline]
```

5.1.2.20 operator+() [2/2]

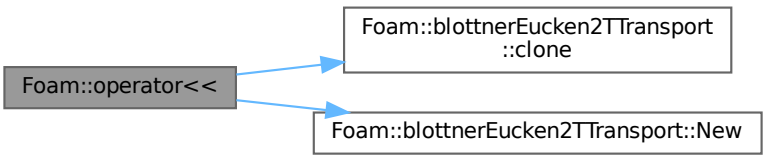
```
template<class EquationOfState >
janaf2TThermo< EquationOfState > Foam::operator+ (
            const janaf2TThermo< EquationOfState > & ,
            const janaf2TThermo< EquationOfState > &  )  [inline]
```

5.1.2.21 operator<<() [1/2]

```
template<class Thermo2T >
Ostream & Foam::operator<< (
            Ostream & ,
            const blottnerEucken2TTransport< Thermo2T > &  )
```

References Foam::blottnerEucken2TTransport< Thermo2T >::clone(), and Foam::blottnerEucken2TTransport< Thermo2T >::New().

Here is the call graph for this function:



5.1.2.22 operator<<() [2/2]

```
template<class EquationOfState >
Ostream & Foam::operator<< (
            Ostream & ,
            const janaf2TThermo< EquationOfState > &  )
```

5.1.2.23 operator==()

```
template<class EquationOfState >
janaf2TThermo< EquationOfState > Foam::operator== (
            const janaf2TThermo< EquationOfState > & ,
            const janaf2TThermo< EquationOfState > &  )  [inline]
```

5.2 Foam::species Namespace Reference

Classes

- class thermo2T

Functions

- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > operator+ (const thermo2T< Thermo2T, Type > &, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > operator* (const scalar, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > operator== (const thermo2T< Thermo2T, Type > &, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  Ostream & operator<< (Ostream &, const thermo2T< Thermo2T, Type > &)

5.2.1 Function Documentation

5.2.1.1 operator*()

```
template<class Thermo2T , template< class > class Type>
thermo2T< Thermo2T, Type > Foam::species::operator* (
            const scalar ,
            const thermo2T< Thermo2T, Type > &  )  [inline]
```

5.2.1.2 operator+()

```
template<class Thermo2T , template< class > class Type>
thermo2T< Thermo2T, Type > Foam::species::operator+ (
            const thermo2T< Thermo2T, Type > & ,
            const thermo2T< Thermo2T, Type > &  )  [inline]
```

5.2.1.3 operator<<()

```
template<class Thermo2T , template< class > class Type>
Ostream & Foam::species::operator<< (
            Ostream & ,
            const thermo2T< Thermo2T, Type > &  )
```

5.2.1.4 operator==()

```
template<class Thermo2T , template< class > class Type>
thermo2T< Thermo2T, Type > Foam::species::operator== (
            const thermo2T< Thermo2T, Type > & ,
            const thermo2T< Thermo2T, Type > &  )  [inline]
```

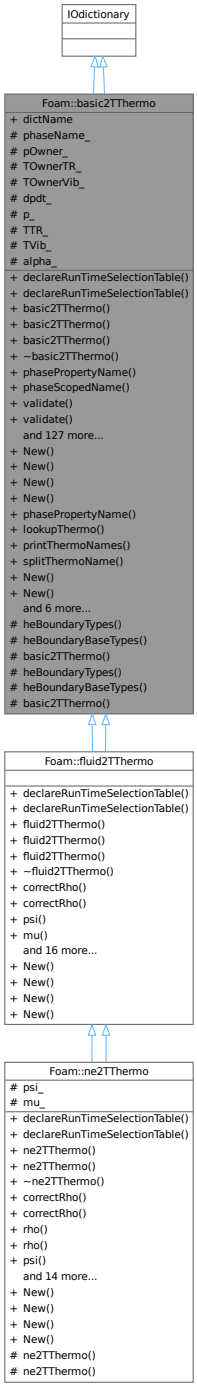
Chapter 6

Class Documentation

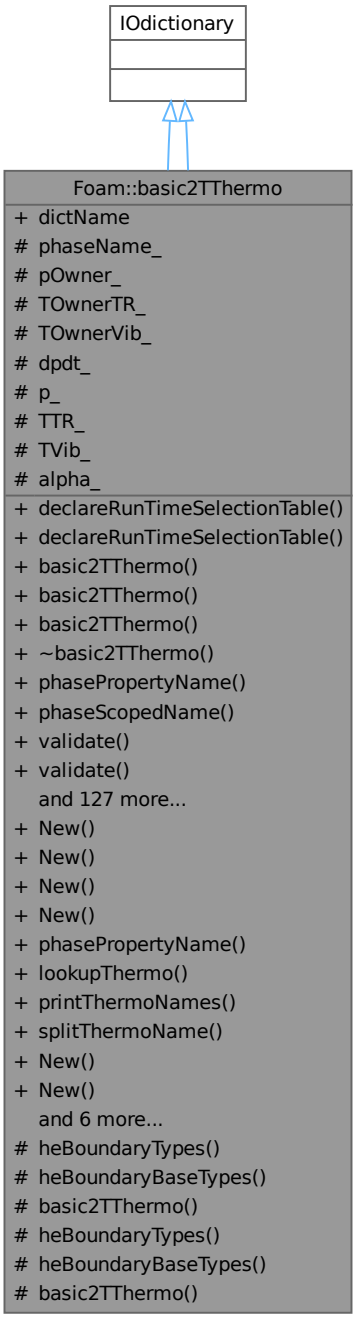
6.1 Foam::basic2TThermo Class Reference

```
#include <basic2TThermo.H>
```

Inheritance diagram for Foam::basic2TThermo:



Collaboration diagram for Foam::basic2TThermo:



Public Member Functions

- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- basic2TThermo (const fvMesh &, const word &phaseName)

- basic2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- basic2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~basic2TThermo ()
- word phasePropertyName (const word &name) const
- word phaseScopedName (const std::string &scope, const word &name) const
- void validate (const string &app, const word &) const
- void validate (const string &app, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &, const word &) const
- virtual void correct ()=0
- virtual word thermoName () const =0
- virtual bool incompressible () const =0
- virtual bool isochoric () const =0
- bool dpdt () const noexcept
- bool updateTTR () const noexcept
- bool updateTVib () const noexcept
- virtual volScalarField & p ()
- virtual const volScalarField & p () const
- virtual tmp< volScalarField > rho () const =0
- virtual tmp< scalarField > rho (const label patchi) const =0
- virtual volScalarField & he ()=0
- virtual const volScalarField & he () const =0
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > hc () const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const =0
- virtual const volScalarField & TTR () const
- virtual const volScalarField & TVib () const
- virtual volScalarField & TTR ()
- virtual volScalarField & TVib ()
- virtual tmp< volScalarField > CpTR () const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > CvT () const =0
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvR () const =0
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvVib () const =0
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const =0
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > gamma () const =0
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpvTR () const =0

- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpByCpvTR () const =0
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > W () const =0
- virtual const volScalarField & alpha () const
- virtual const scalarField & alpha (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const =0
- virtual tmp< scalarField > kappaTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaVib () const =0
- virtual tmp< scalarField > kappaVib (const label patchi) const =0
- virtual tmp< volScalarField > alphaheTR () const =0
- virtual tmp< scalarField > alphaheTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const =0
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const =0
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const =0
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const =0
- virtual bool read ()
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- basic2TThermo (const fvMesh &, const word &phaseName)
- basic2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- basic2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~basic2TThermo ()
- word phasePropertyName (const word &name) const
- word phaseScopedName (const std::string &scope, const word &name) const
- void validate (const string &app, const word &) const
- void validate (const string &app, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &, const word &) const
- virtual void correct ()=0
- virtual word thermoName () const =0
- virtual bool incompressible () const =0
- virtual bool isochoric () const =0
- bool dpdt () const noexcept
- bool updateTTR () const noexcept
- bool updateTVib () const noexcept
- virtual volScalarField & p ()
- virtual const volScalarField & p () const
- virtual tmp< volScalarField > rho () const =0
- virtual tmp< scalarField > rho (const label patchi) const =0
- virtual volScalarField & he ()=0
- virtual const volScalarField & he () const =0
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > hc () const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const =0

- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const =0
- virtual const volScalarField & TTR () const
- virtual const volScalarField & TVib () const
- virtual volScalarField & TTR ()
- virtual volScalarField & TVib ()
- virtual tmp< volScalarField > CpTR () const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > CvT () const =0
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvR () const =0
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvVib () const =0
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const =0
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > gamma () const =0
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpvTR () const =0
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpByCpvTR () const =0
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > W () const =0
- virtual const volScalarField & alpha () const
- virtual const scalarField & alpha (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const =0
- virtual tmp< scalarField > kappaTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaVib () const =0
- virtual tmp< scalarField > kappaVib (const label patchi) const =0
- virtual tmp< volScalarField > alphaheTR () const =0
- virtual tmp< scalarField > alphaheTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const =0
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const =0
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const =0
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const =0
- virtual bool read ()
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const word &phaseName)
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const dictionary &dict, const word &phaseName)
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const word &phaseName, const word &dictName)

Static Public Member Functions

- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const dictionary &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< basic2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static word phasePropertyName (const word &name, const word &phaseName)
- static const basic2TThermo & lookupThermo (const fvPatchScalarField &pf)
- static Ostream & printThermoNames (Ostream &os, const wordList &cmptNames, const wordList &thermoNames)
- static wordList splitThermoName (const std::string &thermoName, const int nExpectedCmpts)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const dictionary &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< basic2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static word phasePropertyName (const word &name, const word &phaseName)
- static const basic2TThermo & lookupThermo (const fvPatchScalarField &pf)
- static Ostream & printThermoNames (Ostream &os, const wordList &cmptNames, const wordList &thermoNames)
- static wordList splitThermoName (const std::string &thermoName, const int nExpectedCmpts)

Static Public Attributes

- static const word dictName

Protected Member Functions

- wordList heBoundaryTypes ()
- wordList heBoundaryBaseTypes ()
- basic2TThermo (const basic2TThermo &)=delete
- wordList heBoundaryTypes ()
- wordList heBoundaryBaseTypes ()
- basic2TThermo (const basic2TThermo &)=delete

Protected Attributes

- const word & phaseName_
- bool pOwner_
- bool TOwnerTR_
- bool TOwnerVib_
- bool dpdt_
- volScalarField & p_
- volScalarField & TTR_
- volScalarField & TVib_
- volScalarField alpha_

6.1.1 Detailed Description

Definition at line 64 of file basic2TThermo/basic2TThermo.H.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 basic2TThermo() [1/8]

```
Foam::basic2TThermo::basic2TThermo (
            const basic2TThermo &  )  [protected], [delete]
```

6.1.2.2 basic2TThermo() [2/8]

```
Foam::basic2TThermo::basic2TThermo (
            const fvMesh & mesh,
            const word & phaseName )
```

Definition at line 275 of file basic2TThermo/basic2TThermo.C.

References TOwnerTR_, and TOwnerVib_.

6.1.2.3 basic2TThermo() [3/8]

```
Foam::basic2TThermo::basic2TThermo (
            const fvMesh & mesh,
            const dictionary & dict,
            const word & phaseName )
```

Definition at line 325 of file basic2TThermo/basic2TThermo.C.

References TOwnerTR_, and TOwnerVib_.

6.1.2.4 basic2TThermo() [4/8]

```
Foam::basic2TThermo::basic2TThermo (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictionaryName )
```

Definition at line 377 of file basic2TThermo/basic2TThermo.C.

References alpha_, TOwnerTR_, and TTR_.

6.1.2.5 ~basic2TThermo() [1/2]

```
Foam::basic2TThermo::~basic2TThermo ( )  [virtual]
```

Definition at line 452 of file basic2TThermo/basic2TThermo.C.

6.1.2.6 basic2TThermo() [5/8]

```
Foam::basic2TThermo::basic2TThermo (
            const basic2TThermo &  )  [protected], [delete]
```

6.1.2.7 basic2TThermo() [6/8]

```
Foam::basic2TThermo::basic2TThermo (
            const fvMesh & ,
            const word & phaseName )
```

6.1.2.8 basic2TThermo() [7/8]

```
Foam::basic2TThermo::basic2TThermo (
            const fvMesh & ,
            const dictionary & ,
            const word & phaseName )
```

6.1.2.9 basic2TThermo() [8/8]

```
Foam::basic2TThermo::basic2TThermo (
            const fvMesh & ,
            const word & phaseName,
            const word & dictionaryName )
```

6.1.2.10 ~basic2TThermo() [2/2]

```
virtual Foam::basic2TThermo::~basic2TThermo ( )  [virtual]
```

6.1.3 Member Function Documentation

6.1.3.1 alpha() [1/4]

```
const Foam::volScalarField & Foam::basic2TThermo::alpha ( ) const  [virtual]
```

Definition at line 662 of file basic2TThermo/basic2TThermo.C.

6.1.3.2 alpha() [2/4]

```
virtual const volScalarField & Foam::basic2TThermo::alpha ( ) const  [virtual]
```

6.1.3.3 alpha() [3/4]

```
const Foam::scalarField & Foam::basic2TThermo::alpha (
            const label patchi ) const  [virtual]
```

Definition at line 668 of file basic2TThermo/basic2TThermo.C.

6.1.3.4 alpha() [4/4]

```
virtual const scalarField & Foam::basic2TThermo::alpha (
            const label patchi ) const  [virtual]
```

6.1.3.5 alphaEff() [1/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::alphaEff (
            const scalarField & alphat,
            const label patchi ) const  [pure virtual]
```

6.1.3.6 alphaEff() [2/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::alphaEff (
            const scalarField & alphat,
            const label patchi ) const  [pure virtual]
```

6.1.3.7 alphaEff() [3/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::alphaEff (
            const volScalarField & alphat ) const  [pure virtual]
```

6.1.3.8 alphaEff() [4/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::alphaEff (
            const volScalarField & alphat ) const  [pure virtual]
```

6.1.3.9 alphaheTR() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::alphaheTR ( ) const  [pure virtual]
```

6.1.3.10 alphaheTR() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::alphaheTR ( ) const  [pure virtual]
```

6.1.3.11 alphaheTR() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::alphaheTR (
            const label patchi ) const  [pure virtual]
```

6.1.3.12 alphaheTR() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::alphaheTR (
            const label patchi ) const  [pure virtual]
```

6.1.3.13 correct() [1/2]

```
virtual void Foam::basic2TThermo::correct ( )  [pure virtual]
```

6.1.3.14 correct() [2/2]

```
virtual void Foam::basic2TThermo::correct ( )  [pure virtual]
```

6.1.3.15 CpByCpvTR() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CpByCpvTR ( ) const  [pure virtual]
```

6.1.3.16 CpByCpvTR() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CpByCpvTR ( ) const  [pure virtual]
```

6.1.3.17 CpByCpvTR() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpByCpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.18 CpByCpvTR() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpByCpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.19 CpTR() [1/6]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CpTR ( ) const  [pure virtual]
```

6.1.3.20 CpTR() [2/6]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CpTR ( ) const  [pure virtual]
```

6.1.3.21 CpTR() [3/6]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.22 CpTR() [4/6]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.23 CpTR() [5/6]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.24 CpTR() [6/6]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.25 CpvTR() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CpvTR ( ) const  [pure virtual]
```

6.1.3.26 CpvTR() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CpvTR ( ) const  [pure virtual]
```

6.1.3.27 CpvTR() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.28 CpvTR() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.29 CvR() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CvR ( ) const  [pure virtual]
```

6.1.3.30 CvR() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CvR ( ) const  [pure virtual]
```

6.1.3.31 CvR() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CvR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.32 CvR() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CvR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.33 CvT() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CvT ( ) const  [pure virtual]
```

6.1.3.34 CvT() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CvT ( ) const  [pure virtual]
```

6.1.3.35 CvT() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CvT (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.36 CvT() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CvT (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.37 CvVib() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CvVib ( ) const  [pure virtual]
```

6.1.3.38 CvVib() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::CvVib ( ) const  [pure virtual]
```

6.1.3.39 CvVib() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CvVib (
            const scalarField & p,
            const scalarField & TTR,
            const scalarField & Tvib,
            const scalar ThetaVib,
            const label patchi ) const  [pure virtual]
```

6.1.3.40 CvVib() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::CvVib (
            const scalarField & p,
            const scalarField & TTR,
            const scalarField & Tvib,
            const scalar ThetaVib,
            const label patchi ) const  [pure virtual]
```

6.1.3.41 declareRunTimeSelectionTable() [1/4]

```
Foam::basic2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            basic2TThermo ,
            fvMesh ,
            (const fvMesh &mesh, const word &phaseName) ,
            (mesh, phaseName)  )
```

6.1.3.42 declareRunTimeSelectionTable() [2/4]

```
Foam::basic2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            basic2TThermo ,
            fvMesh ,
            (const fvMesh &mesh, const word &phaseName) ,
            (mesh, phaseName)  )
```

6.1.3.43 declareRunTimeSelectionTable() [3/4]

```
Foam::basic2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            basic2TThermo ,
            fvMeshDictPhase ,
            (const fvMesh &mesh, const word &phaseName, const word &dictName) ,
            (mesh, phaseName, dictName)  )
```

6.1.3.44 declareRunTimeSelectionTable() [4/4]

```
Foam::basic2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            basic2TThermo ,
            fvMeshDictPhase ,
            (const fvMesh &mesh, const word &phaseName, const word &dictName) ,
            (mesh, phaseName, dictName)  )
```

6.1.3.45 dpdt() [1/2]

```
bool Foam::basic2TThermo::dpdt ( ) const  [inline], [noexcept]
```

Definition at line 364 of file basic2TThermo/basic2TThermo.H.

References dpdt_.

6.1.3.46 dpdt() [2/2]

```
bool Foam::basic2TThermo::dpdt ( ) const  [inline], [noexcept]
```

Definition at line 364 of file lnInclude/basic2TThermo.H.

References dpdt_.

6.1.3.47 gamma() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::gamma ( ) const  [pure virtual]
```

6.1.3.48 gamma() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::gamma ( ) const  [pure virtual]
```

6.1.3.49 gamma() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::gamma (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.50 gamma() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::gamma (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.51 hc() [1/2]

```
virtual tmp< volScalarField > Foam::basic2TThermo::hc ( ) const  [pure virtual]
```

6.1.3.52 hc() [2/2]

```
virtual tmp< volScalarField > Foam::basic2TThermo::hc ( ) const  [pure virtual]
```

6.1.3.53 he() [1/10]

```
virtual const volScalarField & Foam::basic2TThermo::he ( ) const  [pure virtual]
```

6.1.3.54 he() [2/10]

```
virtual const volScalarField & Foam::basic2TThermo::he ( ) const  [pure virtual]
```

6.1.3.55 he() [3/10]

```
virtual volScalarField & Foam::basic2TThermo::he ( )  [pure virtual]
```

6.1.3.56 he() [4/10]

```
virtual volScalarField & Foam::basic2TThermo::he ( )  [pure virtual]
```

6.1.3.57 he() [5/10]

```
virtual tmp< scalarField > Foam::basic2TThermo::he (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.58 he() [6/10]

```
virtual tmp< scalarField > Foam::basic2TThermo::he (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [pure virtual]
```

6.1.3.59 he() [7/10]

```
virtual tmp< scalarField > Foam::basic2TThermo::he (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.60 he() [8/10]

```
virtual tmp< scalarField > Foam::basic2TThermo::he (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.61 he() [9/10]

```
virtual tmp< volScalarField > Foam::basic2TThermo::he (
            const volScalarField & p,
            const volScalarField & TTR ) const  [pure virtual]
```

6.1.3.62 he() [10/10]

```
virtual tmp< volScalarField > Foam::basic2TThermo::he (
            const volScalarField & p,
            const volScalarField & TTR ) const  [pure virtual]
```

6.1.3.63 heBoundaryBaseTypes() [1/2]

```
Foam::wordList Foam::basic2TThermo::heBoundaryBaseTypes ( )  [protected]
```

Definition at line 164 of file basic2TThermo/basic2TThermo.C.

6.1.3.64 heBoundaryBaseTypes() [2/2]

```
wordList Foam::basic2TThermo::heBoundaryBaseTypes ( )  [protected]
```

6.1.3.65 heBoundaryTypes() [1/2]

```
Foam::wordList Foam::basic2TThermo::heBoundaryTypes ( )  [protected]
```

Definition at line 198 of file basic2TThermo/basic2TThermo.C.

6.1.3.66 heBoundaryTypes() [2/2]

```
wordList Foam::basic2TThermo::heBoundaryTypes ( )  [protected]
```

6.1.3.67 incompressible() [1/2]

```
virtual bool Foam::basic2TThermo::incompressible ( ) const  [pure virtual]
```

6.1.3.68 incompressible() [2/2]

```
virtual bool Foam::basic2TThermo::incompressible ( ) const  [pure virtual]
```

6.1.3.69 isochoric() [1/2]

```
virtual bool Foam::basic2TThermo::isochoric ( ) const  [pure virtual]
```

6.1.3.70 isochoric() [2/2]

```
virtual bool Foam::basic2TThermo::isochoric ( ) const  [pure virtual]
```

6.1.3.71 kappaEff() [1/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::kappaEff (
            const scalarField & alphat,
            const label patchi ) const  [pure virtual]
```

6.1.3.72 kappaEff() [2/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::kappaEff (
            const scalarField & alphat,
            const label patchi ) const  [pure virtual]
```

6.1.3.73 kappaEff() [3/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::kappaEff (
            const volScalarField &  ) const  [pure virtual]
```

6.1.3.74 kappaEff() [4/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::kappaEff (
            const volScalarField &  ) const  [pure virtual]
```

6.1.3.75 kappaTR() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::kappaTR ( ) const  [pure virtual]
```

6.1.3.76 kappaTR() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::kappaTR ( ) const  [pure virtual]
```

6.1.3.77 kappaTR() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::kappaTR (
            const label patchi ) const  [pure virtual]
```

6.1.3.78 kappaTR() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::kappaTR (
            const label patchi ) const  [pure virtual]
```

6.1.3.79 kappaVib() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::kappaVib ( ) const  [pure virtual]
```

6.1.3.80 kappaVib() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::kappaVib ( ) const  [pure virtual]
```

6.1.3.81 kappaVib() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::kappaVib (
            const label patchi ) const  [pure virtual]
```

6.1.3.82 kappaVib() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::kappaVib (
            const label patchi ) const  [pure virtual]
```

6.1.3.83 lookupThermo() [1/2]

```
const Foam::basic2TThermo & Foam::basic2TThermo::lookupThermo (
            const fvPatchScalarField & pf )  [static]
```

Definition at line 473 of file basic2TThermo/basic2TThermo.C.

6.1.3.84 lookupThermo() [2/2]

```
static const basic2TThermo & Foam::basic2TThermo::lookupThermo (
            const fvPatchScalarField & pf )  [static]
```

6.1.3.85 New() [1/11]

```
template<class Thermo >
static autoPtr< Thermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const dictionary & ,
            const word & phaseName = word::null )  [static]
```

6.1.3.86 New() [2/11]

```
template<class Thermo >
static autoPtr< Thermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const dictionary & ,
            const word & phaseName = word::null )  [static]
```

6.1.3.87 New() [3/11]

```
template<class Thermo >
static autoPtr< Thermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const word & phaseName,
            const word & dictName )  [static]
```

6.1.3.88 New() [4/11]

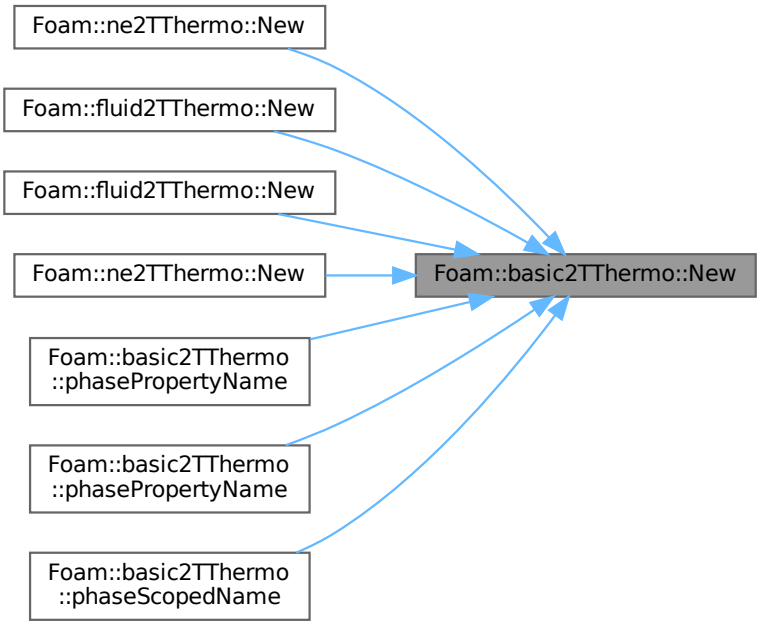
```
template<class Thermo >
static autoPtr< Thermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const word & phaseName,
            const word & dictName )  [static]
```

6.1.3.89 New() [5/11]

```
template<class Thermo >
static autoPtr< Thermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const word & phaseName = word::null )  [static]
```

Referenced by Foam::ne2TThermo::New(), Foam::fluid2TThermo::New(), Foam::fluid2TThermo::New(), Foam::ne2TThermo::New(), phasePropertyName(), phasePropertyName(), and phaseScopedName().

Here is the caller graph for this function:



6.1.3.90 New() [6/11]

```
Foam::autoPtr< Foam::basic2TThermo > Foam::basic2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName = word::null )  [static]
```

Definition at line 440 of file basic2TThermo/basic2TThermo.C.

6.1.3.91 New() [7/11]

```
template<class Thermo >
static autoPtr< Thermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const word & phaseName = word::null )  [static]
```

6.1.3.92 New() [8/11]

```
static autoPtr< basic2TThermo > Foam::basic2TThermo::New (
            const fvMesh & ,
            const word & phaseName = word::null )  [static]
```

6.1.3.93 New() [9/11]

```
template<class TThermo >
Foam::autoPtr< TThermo > Foam::basic2TThermo::New (
            const fvMesh & mesh,
            const dictionary & dict,
            const word & phaseName )
```

Definition at line 161 of file basic2TThermo/basic2TThermoTemplates.C.

6.1.3.94 New() [10/11]

```
template<class TThermo >
Foam::autoPtr< TThermo > Foam::basic2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName )
```

Definition at line 131 of file basic2TThermo/basic2TThermoTemplates.C.

6.1.3.95 New() [11/11]

```
template<class TThermo >
Foam::autoPtr< TThermo > Foam::basic2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictName )
```

Definition at line 179 of file basic2TThermo/basic2TThermoTemplates.C.

6.1.3.96 p() [1/4]

```
Foam::volScalarField & Foam::basic2TThermo::p ( )  [virtual]
```

Definition at line 627 of file basic2TThermo/basic2TThermo.C.

6.1.3.97 p() [2/4]

```
virtual volScalarField & Foam::basic2TThermo::p ( )  [virtual]
```

6.1.3.98 p() [3/4]

```
const Foam::volScalarField & Foam::basic2TThermo::p ( ) const  [virtual]
```

Definition at line 633 of file basic2TThermo/basic2TThermo.C.

6.1.3.99 p() [4/4]

```
virtual const volScalarField & Foam::basic2TThermo::p ( ) const  [virtual]
```

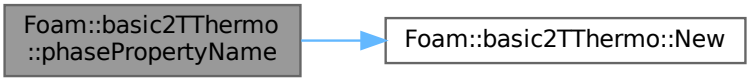
6.1.3.100 phasePropertyName() [1/4]

```
word Foam::basic2TThermo::phasePropertyName (
            const word & name ) const  [inline]
```

Definition at line 275 of file basic2TThermo/basic2TThermo.H.

References New(), and phaseName_.

Here is the call graph for this function:



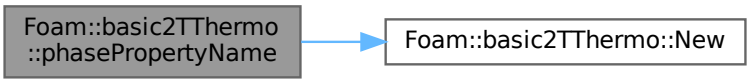
6.1.3.101 phasePropertyName() [2/4]

```
word Foam::basic2TThermo::phasePropertyName (
            const word & name ) const  [inline]
```

Definition at line 275 of file InInclude/basic2TThermo.H.

References New(), and phaseName_.

Here is the call graph for this function:



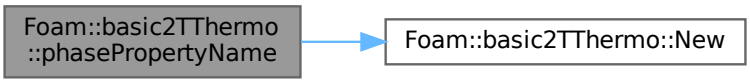
6.1.3.102 phasePropertyName() [3/4]

```
static word Foam::basic2TThermo::phasePropertyName (
            const word & name,
            const word & phaseName )  [inline], [static]
```

Definition at line 268 of file basic2TThermo/basic2TThermo.H.

References New().

Here is the call graph for this function:



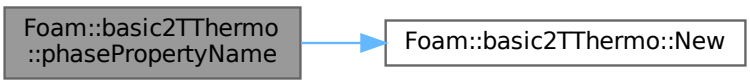
6.1.3.103 phasePropertyName() [4/4]

```
static word Foam::basic2TThermo::phasePropertyName (
            const word & name,
            const word & phaseName )  [inline], [static]
```

Definition at line 268 of file InInclude/basic2TThermo.H.

References New().

Here is the call graph for this function:



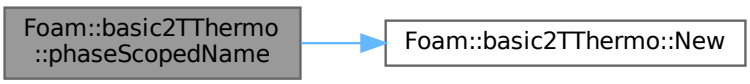
6.1.3.104 phaseScopedName() [1/2]

```
word Foam::basic2TThermo::phaseScopedName (
            const std::string & scope,
            const word & name ) const  [inline]
```

Definition at line 282 of file basic2TThermo/basic2TThermo.H.

References New(), and phaseName_.

Here is the call graph for this function:



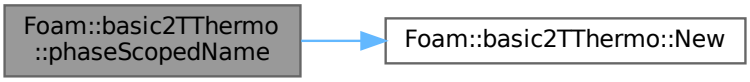
6.1.3.105 phaseScopedName() [2/2]

```
word Foam::basic2TThermo::phaseScopedName (
            const std::string & scope,
            const word & name ) const  [inline]
```

Definition at line 282 of file InInclude/basic2TThermo.H.

References New(), and phaseName_.

Here is the call graph for this function:



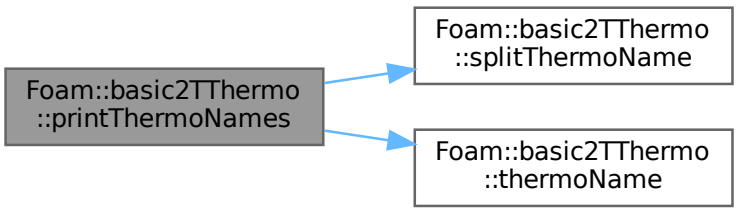
6.1.3.106 printThermoNames() [1/2]

```
Foam::Ostream & Foam::basic2TThermo::printThermoNames (
            Ostream & os,
            const wordList & cmptNames,
            const wordList & thermoNames )  [static]
```

Definition at line 74 of file basic2TThermo/basic2TThermo.C.

References splitThermoName(), and thermoName().

Here is the call graph for this function:



6.1.3.107 printThermoNames() [2/2]

```
static Ostream & Foam::basic2TThermo::printThermoNames (
            Ostream & os,
            const wordList & cmptNames,
            const wordList & thermoNames )  [static]
```

6.1.3.108 read() [1/2]

```
bool Foam::basic2TThermo::read ( )  [virtual]
```

Definition at line 674 of file basic2TThermo/basic2TThermo.C.

6.1.3.109 read() [2/2]

```
virtual bool Foam::basic2TThermo::read ( )  [virtual]
```

6.1.3.110 rho() [1/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::rho ( ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.1.3.111 rho() [2/4]

```
virtual tmp< volScalarField > Foam::basic2TThermo::rho ( ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.1.3.112 rho() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::rho (
            const label patchi ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.1.3.113 rho() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::rho (
            const label patchi ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.1.3.114 rhoEoS() [1/2]

```
virtual tmp< scalarField > Foam::basic2TThermo::rhoEoS (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.115 rhoEoS() [2/2]

```
virtual tmp< scalarField > Foam::basic2TThermo::rhoEoS (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [pure virtual]
```

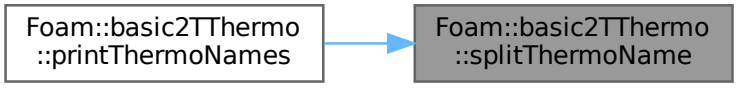
6.1.3.116 splitThermoName() [1/2]

```
Foam::wordList Foam::basic2TThermo::splitThermoName (
            const std::string & thermoName,
            const int nExpectedCmpts )  [static]
```

Definition at line 596 of file basic2TThermo/basic2TThermo.C.

Referenced by printThermoNames().

Here is the caller graph for this function:



6.1.3.117 splitThermoName() [2/2]

```
static wordList Foam::basic2TThermo::splitThermoName (
            const std::string & thermoName,
            const int nExpectedCmpts )  [static]
```

6.1.3.118 TEH_Vib() [1/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::TEH_Vib (
            const scalarField & esVib,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const label patchi ) const  [pure virtual]
```

6.1.3.119 TEH_Vib() [2/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::TEH_Vib (
            const scalarField & esVib,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const label patchi ) const  [pure virtual]
```

6.1.3.120 TEH_Vib() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::TEH_Vib (
            const scalarField & esVib,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.121 TEH_Vib() [4/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::TEH_Vib (
            const scalarField & esVib,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.122 THE_TR() [1/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const label patchi ) const  [pure virtual]
```

6.1.3.123 THE_TR() [2/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const label patchi ) const  [pure virtual]
```

6.1.3.124 THE_TR() [3/4]

```
virtual tmp< scalarField > Foam::basic2TThermo::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.125 THE_TR() [4/4]

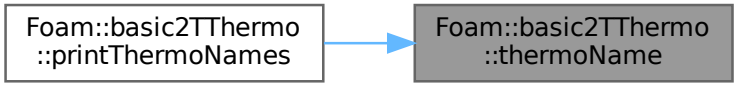
```
virtual tmp< scalarField > Foam::basic2TThermo::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const labelList & cells ) const  [pure virtual]
```

6.1.3.126 thermoName() [1/2]

```
virtual word Foam::basic2TThermo::thermoName ( ) const  [pure virtual]
```

Referenced by printThermoNames().

Here is the caller graph for this function:



6.1.3.127 thermoName() [2/2]

```
virtual word Foam::basic2TThermo::thermoName ( ) const  [pure virtual]
```

6.1.3.128 TTR() [1/4]

```
Foam::volScalarField & Foam::basic2TThermo::TTR ( )  [virtual]
```

Definition at line 645 of file basic2TThermo/basic2TThermo.C.

6.1.3.129 TTR() [2/4]

```
virtual volScalarField & Foam::basic2TThermo::TTR ( )  [virtual]
```

6.1.3.130 TTR() [3/4]

```
const Foam::volScalarField & Foam::basic2TThermo::TTR ( ) const  [virtual]
```

Definition at line 639 of file basic2TThermo/basic2TThermo.C.

6.1.3.131 TTR() [4/4]

```
virtual const volScalarField & Foam::basic2TThermo::TTR ( ) const  [virtual]
```

6.1.3.132 TVib() [1/4]

```
Foam::volScalarField & Foam::basic2TThermo::TVib ( )  [virtual]
```

Definition at line 656 of file basic2TThermo/basic2TThermo.C.

6.1.3.133 TVib() [2/4]

```
virtual volScalarField & Foam::basic2TThermo::TVib ( )  [virtual]
```

6.1.3.134 TVib() [3/4]

```
const Foam::volScalarField & Foam::basic2TThermo::TVib ( ) const  [virtual]
```

Definition at line 650 of file basic2TThermo/basic2TThermo.C.

6.1.3.135 TVib() [4/4]

```
virtual const volScalarField & Foam::basic2TThermo::TVib ( ) const  [virtual]
```

6.1.3.136 updateTTR() [1/2]

```
bool Foam::basic2TThermo::updateTTR ( ) const  [inline], [noexcept]
```

Definition at line 370 of file basic2TThermo/basic2TThermo.H.

References TOwnerTR_.

6.1.3.137 updateTTR() [2/2]

```
bool Foam::basic2TThermo::updateTTR ( ) const  [inline], [noexcept]
```

Definition at line 370 of file InInclude/basic2TThermo.H.

References TOwnerTR_.

6.1.3.138 updateTVib() [1/2]

```
bool Foam::basic2TThermo::updateTVib ( ) const  [inline], [noexcept]
```

Definition at line 376 of file basic2TThermo/basic2TThermo.H.

References TOwnerVib_.

6.1.3.139 updateTVib() [2/2]

```
bool Foam::basic2TThermo::updateTVib ( ) const  [inline], [noexcept]
```

Definition at line 376 of file InInclude/basic2TThermo.H.

References TOwnerVib_.

6.1.3.140 validate() [1/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & a ) const
```

Definition at line 502 of file basic2TThermo/basic2TThermo.C.

6.1.3.141 validate() [2/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word &  ) const
```

6.1.3.142 validate() [3/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & a,
            const word & b ) const
```

Definition at line 517 of file basic2TThermo/basic2TThermo.C.

6.1.3.143 validate() [4/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & ,
            const word &  ) const
```

6.1.3.144 validate() [5/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & a,
            const word & b,
            const word & c ) const
```

Definition at line 540 of file basic2TThermo/basic2TThermo.C.

6.1.3.145 validate() [6/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & ,
            const word & ,
            const word &  ) const
```

6.1.3.146 validate() [7/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & a,
            const word & b,
            const word & c,
            const word & d ) const
```

Definition at line 566 of file basic2TThermo/basic2TThermo.C.

6.1.3.147 validate() [8/8]

```
void Foam::basic2TThermo::validate (
            const string & app,
            const word & ,
            const word & ,
            const word & ,
            const word &  ) const
```

6.1.3.148 W() [1/2]

```
virtual tmp< volScalarField > Foam::basic2TThermo::W ( ) const  [pure virtual]
```

6.1.3.149 W() [2/2]

```
virtual tmp< volScalarField > Foam::basic2TThermo::W ( ) const  [pure virtual]
```

6.1.4 Member Data Documentation

6.1.4.1 alpha_

```
volScalarField Foam::basic2TThermo::alpha_  [protected]
```

Definition at line 149 of file basic2TThermo/basic2TThermo.H.

Referenced by basic2TThermo().

6.1.4.2 dictName

```
static const word Foam::basic2TThermo::dictName  [static]
```

Definition at line 264 of file basic2TThermo/basic2TThermo.H.

6.1.4.3 dpdt_

```
bool Foam::basic2TThermo::dpdt_ [protected]
```

Definition at line 134 of file basic2TThermo/basic2TThermo.H.

Referenced by dpdt().

6.1.4.4 p_

```
volScalarField & Foam::basic2TThermo::p_ [protected]
```

Definition at line 140 of file basic2TThermo/basic2TThermo.H.

6.1.4.5 phaseName_

```
const word & Foam::basic2TThermo::phaseName_ [protected]
```

Definition at line 122 of file basic2TThermo/basic2TThermo.H.

Referenced by phasePropertyName(), and phaseScopedName().

6.1.4.6 pOwner_

```
bool Foam::basic2TThermo::pOwner_ [protected]
```

Definition at line 125 of file basic2TThermo/basic2TThermo.H.

6.1.4.7 TOwnerTR_

```
bool Foam::basic2TThermo::TOwnerTR_ [protected]
```

Definition at line 128 of file basic2TThermo/basic2TThermo.H.

Referenced by basic2TThermo(), basic2TThermo(), basic2TThermo(), and updateTTR().

6.1.4.8 TOwnerVib_

```
bool Foam::basic2TThermo::TOwnerVib_ [protected]
```

Definition at line 131 of file basic2TThermo/basic2TThermo.H.

Referenced by basic2TThermo(), basic2TThermo(), and updateTVib().

6.1.4.9 TTR_

```
volScalarField & Foam::basic2TThermo::TTR_ [protected]
```

Definition at line 143 of file basic2TThermo/basic2TThermo.H.

Referenced by basic2TThermo().

6.1.4.10 TVib_

```
volScalarField & Foam::basic2TThermo::TVib_ [protected]
```

Definition at line 146 of file basic2TThermo/basic2TThermo.H.

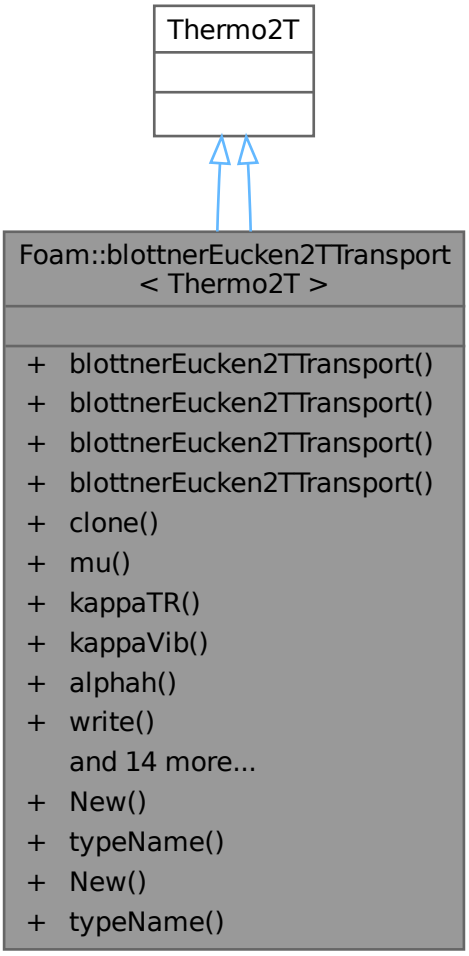
The documentation for this class was generated from the following files:

- basic2TThermo/basic2TThermo.H
- lnInclude/basic2TThermo.H
- basic2TThermo/basic2TThermo.C
- basic2TThermo/basic2TThermoTemplates.C
- lnInclude/basic2TThermo.C
- lnInclude/basic2TThermoTemplates.C

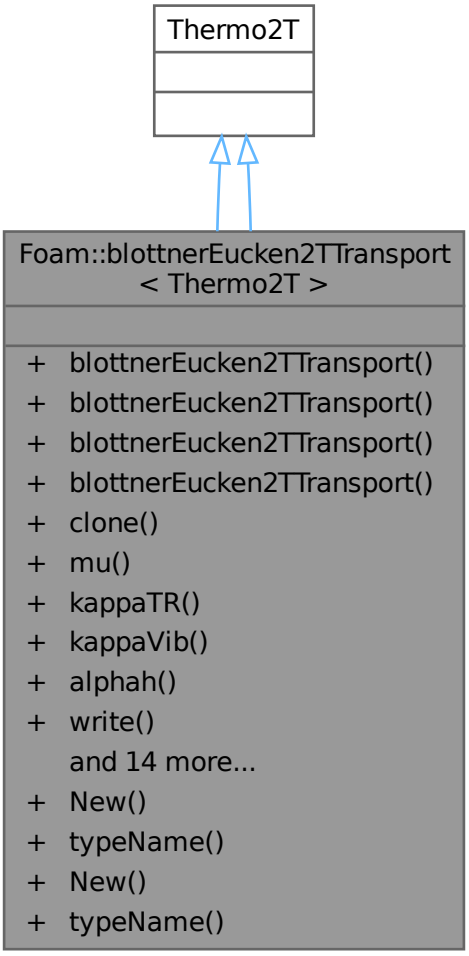
6.2 Foam::blottnerEucken2TTransport< Thermo2T > Class Template Reference

```
#include <blottnerEucken2TTransport.H>
```

Inheritance diagram for Foam::blottnerEucken2TTransport< Thermo2T >:



Collaboration diagram for Foam::blottnerEucken2TTransport< Thermo2T >:



Public Member Functions

- blottnerEucken2TTransport (const Thermo2T &t, const scalar AB, const scalar BB, const scalar CB)
- blottnerEucken2TTransport (const word &, const blottnerEucken2TTransport &)
- blottnerEucken2TTransport (const dictionary &dict)
- blottnerEucken2TTransport (const Thermo2T &t, const dictionary &dict)
- autoPtr< blottnerEucken2TTransport > clone () const
- scalar mu (const scalar TTR) const
- scalar kappaTR (const scalar p, const scalar TTR) const
- scalar kappaVib (const scalar TTR) const
- scalar alphah (const scalar p, const scalar TTR) const
- void write (Ostream &os) const
- void operator+= (const blottnerEucken2TTransport &)
- void operator*= (const scalar)
- blottnerEucken2TTransport (const Thermo2T &t, const scalar AB, const scalar BB, const scalar CB)
- blottnerEucken2TTransport (const word &, const blottnerEucken2TTransport &)

- blottnerEucken2TTransport (const dictionary &dict)
- blottnerEucken2TTransport (const Thermo2T &t, const dictionary &dict)
- autoPtr< blottnerEucken2TTransport > clone () const
- scalar mu (const scalar TTR) const
- scalar kappaTR (const scalar p, const scalar TTR) const
- scalar kappaVib (const scalar TTR) const
- scalar alphah (const scalar p, const scalar TTR) const
- void write (Ostream &os) const
- void operator+= (const blottnerEucken2TTransport &)
- void operator*= (const scalar)

Static Public Member Functions

- static autoPtr< blottnerEucken2TTransport > New (const dictionary &dict)
- static word typeName ()
- static autoPtr< blottnerEucken2TTransport > New (const dictionary &dict)
- static word typeName ()

Friends

- blottnerEucken2TTransport operator+ (const blottnerEucken2TTransport &, const blottnerEucken2TTransport &)
- blottnerEucken2TTransport operator* (const scalar, const blottnerEucken2TTransport &)
- Ostream & operator<< (Ostream &, const blottnerEucken2TTransport &)
- blottnerEucken2TTransport operator+ (const blottnerEucken2TTransport &, const blottnerEucken2TTransport &)
- blottnerEucken2TTransport operator* (const scalar, const blottnerEucken2TTransport &)
- Ostream & operator<< (Ostream &, const blottnerEucken2TTransport &)

6.2.1 Detailed Description

template<class Thermo2T>
class Foam::blottnerEucken2TTransport< Thermo2T >

Definition at line 99 of file transport/blottnerEucken2T/blottnerEucken2TTransport.H.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 blottnerEucken2TTransport() [1/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const Thermo2T & t,
            const scalar AB,
            const scalar BB,
            const scalar CB )  [inline]
```

Definition at line 36 of file InInclude/blottnerEucken2TTransportI.H.

6.2.2.2 blottnerEucken2TTransport() [2/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const word & name,
            const blottnerEucken2TTransport< Thermo2T > & st )  [inline]
```

Definition at line 52 of file lnInclude/blottnerEucken2TTransportI.H.

6.2.2.3 blottnerEucken2TTransport() [3/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const dictionary & dict )  [explicit]
```

Definition at line 47 of file lnInclude/blottnerEucken2TTransport.C.

6.2.2.4 blottnerEucken2TTransport() [4/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const Thermo2T & t,
            const dictionary & dict )
```

Definition at line 57 of file lnInclude/blottnerEucken2TTransport.C.

6.2.2.5 blottnerEucken2TTransport() [5/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const Thermo2T & t,
            const scalar AB,
            const scalar BB,
            const scalar CB )  [inline]
```

6.2.2.6 blottnerEucken2TTransport() [6/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const word & ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [inline]
```

6.2.2.7 blottnerEucken2TTransport() [7/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const dictionary & dict )  [explicit]
```

6.2.2.8 blottnerEucken2TTransport() [8/8]

```
template<class Thermo2T >
Foam::blottnerEucken2TTransport< Thermo2T >::blottnerEucken2TTransport (
            const Thermo2T & t,
            const dictionary & dict )
```

6.2.3 Member Function Documentation

6.2.3.1 alphah() [1/2]

```
template<class Thermo2T >
Foam::scalar Foam::blottnerEucken2TTransport< Thermo2T >::alphah (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 122 of file InInclude/blottnerEucken2TTransportI.H.

References CpTR().

Here is the call graph for this function:

6.2.3.2 alphah() [2/2]

```
template<class Thermo2T >
scalar Foam::blottnerEucken2TTransport< Thermo2T >::alphah (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.2.3.3 clone() [1/2]

```
template<class Thermo2T >
Foam::autoPtr< Foam::blottnerEucken2TTransport< Thermo2T > > Foam::blottnerEucken2TTransport<
Thermo2T >::clone ( ) const  [inline]
```

Definition at line 67 of file InInclude/blottnerEucken2TTransportI.H.

Referenced by Foam::operator<<().

Here is the caller graph for this function:

6.2.3.4 clone() [2/2]

```
template<class Thermo2T >
autoPtr< blottnerEucken2TTransport > Foam::blottnerEucken2TTransport< Thermo2T >::clone ( )
const  [inline]
```

6.2.3.5 kappaTR() [1/2]

```
template<class Thermo2T >
Foam::scalar Foam::blottnerEucken2TTransport< Thermo2T >::kappaTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 98 of file InInclude/blottnerEucken2TTransportI.H.

References CvT().

Here is the call graph for this function:

6.2.3.6 kappaTR() [2/2]

```
template<class Thermo2T >
scalar Foam::blottnerEucken2TTransport< Thermo2T >::kappaTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.2.3.7 kappaVib() [1/2]

```
template<class Thermo2T >
Foam::scalar Foam::blottnerEucken2TTransport< Thermo2T >::kappaVib (
            const scalar TTR ) const  [inline]
```

Definition at line 111 of file InInclude/blottnerEucken2TTransportI.H.

6.2.3.8 kappaVib() [2/2]

```
template<class Thermo2T >
scalar Foam::blottnerEucken2TTransport< Thermo2T >::kappaVib (
            const scalar TTR ) const  [inline]
```

6.2.3.9 mu() [1/2]

```
template<class Thermo2T >
Foam::scalar Foam::blottnerEucken2TTransport< Thermo2T >::mu (
            const scalar TTR ) const  [inline]
```

Definition at line 87 of file InInclude/blottnerEucken2TTransportI.H.

6.2.3.10 mu() [2/2]

```
template<class Thermo2T >
scalar Foam::blottnerEucken2TTransport< Thermo2T >::mu (
            const scalar TTR ) const  [inline]
```

6.2.3.11 New() [1/2]

```
template<class Thermo2T >
Foam::autoPtr< Foam::blottnerEucken2TTransport< Thermo2T > > Foam::blottnerEucken2TTransport<
Thermo2T >::New (
            const dictionary & dict )  [inline], [static]
```

Definition at line 75 of file InInclude/blottnerEucken2TTransportI.H.

Referenced by Foam::operator<<().

Here is the caller graph for this function:

6.2.3.12 New() [2/2]

```
template<class Thermo2T >
static autoPtr< blottnerEucken2TTransport > Foam::blottnerEucken2TTransport< Thermo2T >::New
(
            const dictionary & dict )  [inline], [static]
```

6.2.3.13 operator*=() [1/2]

```
template<class Thermo2T >
void Foam::blottnerEucken2TTransport< Thermo2T >::operator*= (
            const scalar s )  [inline]
```

Definition at line 158 of file InInclude/blottnerEucken2TTransportI.H.

6.2.3.14 operator*=() [2/2]

```
template<class Thermo2T >
void Foam::blottnerEucken2TTransport< Thermo2T >::operator*= (
            const scalar  )  [inline]
```

6.2.3.15 operator+=() [1/2]

```
template<class Thermo2T >
void Foam::blottnerEucken2TTransport< Thermo2T >::operator+= (
            const blottnerEucken2TTransport< Thermo2T > & st )  [inline]
```

Definition at line 136 of file InInclude/blottnerEucken2TTransportI.H.

6.2.3.16 operator+=() [2/2]

```
template<class Thermo2T >
void Foam::blottnerEucken2TTransport< Thermo2T >::operator+= (
            const blottnerEucken2TTransport< Thermo2T > &  )  [inline]
```

6.2.3.17 typeName() [1/2]

```
template<class Thermo2T >
static word Foam::blottnerEucken2TTransport< Thermo2T >::typeName ( )  [inline], [static]
```

Definition at line 150 of file InInclude/blottnerEucken2TTransport.H.

6.2.3.18 typeName() [2/2]

```
template<class Thermo2T >
static word Foam::blottnerEucken2TTransport< Thermo2T >::typeName ( )  [inline], [static]
```

Definition at line 150 of file transport/blottnerEucken2T/blottnerEucken2TTransport.H.

6.2.3.19 write() [1/2]

```
template<class Thermo2T >
void Foam::blottnerEucken2TTransport< Thermo2T >::write (
            Ostream & os ) const
```

Definition at line 73 of file InInclude/blottnerEucken2TTransport.C.

6.2.3.20 write() [2/2]

```
template<class Thermo2T >
void Foam::blottnerEucken2TTransport< Thermo2T >::write (
            Ostream & os ) const
```

6.2.4 Friends And Related Symbol Documentation

6.2.4.1 operator* [1/2]

```
template<class Thermo2T >
blottnerEucken2TTransport operator* (
            const scalar ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [friend]
```

6.2.4.2 operator* [2/2]

```
template<class Thermo2T >
blottnerEucken2TTransport operator* (
            const scalar ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [friend]
```

6.2.4.3 operator+ [1/2]

```
template<class Thermo2T >
blottnerEucken2TTransport operator+ (
            const blottnerEucken2TTransport< Thermo2T > & ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [friend]
```

6.2.4.4 operator+ [2/2]

```
template<class Thermo2T >
blottnerEucken2TTransport operator+ (
            const blottnerEucken2TTransport< Thermo2T > & ,
            const blottnerEucken2TTransport< Thermo2T > &  )  [friend]
```

6.2.4.5 operator<< [1/2]

```
template<class Thermo2T >
Ostream & operator<< (
            Ostream & ,
            const blottnerEucken2TTransport< Thermo2T > & ) [friend]
```

6.2.4.6 operator<< [2/2]

```
template<class Thermo2T >
Ostream & operator<< (
            Ostream & ,
            const blottnerEucken2TTransport< Thermo2T > & ) [friend]
```

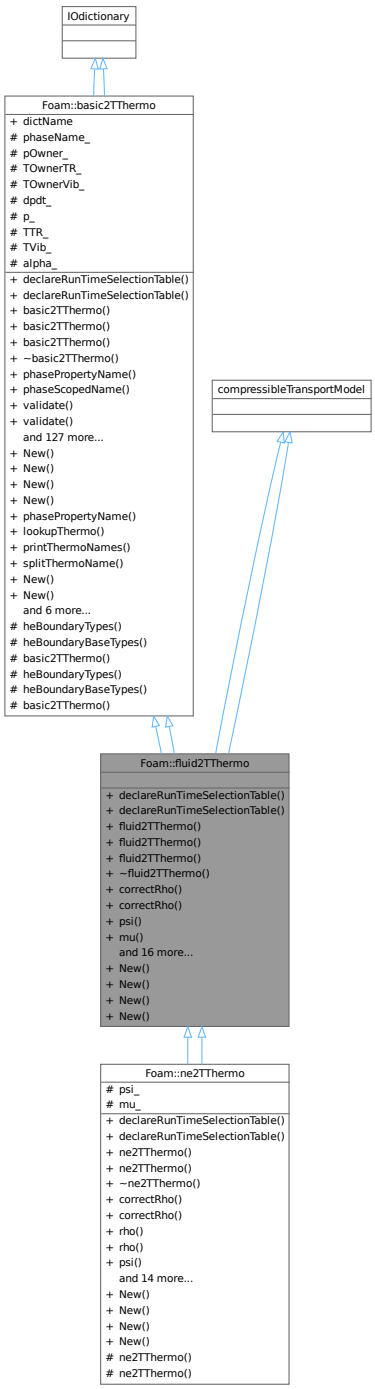
The documentation for this class was generated from the following files:

- lnInclude/blottnerEucken2TTransport.H
- transport/blottnerEucken2T/blottnerEucken2TTransport.H
- lnInclude/blottnerEucken2TTransport.C
- lnInclude/blottnerEucken2TTransportI.H
- transport/blottnerEucken2T/blottnerEucken2TTransport.C
- transport/blottnerEucken2T/blottnerEucken2TTransportI.H

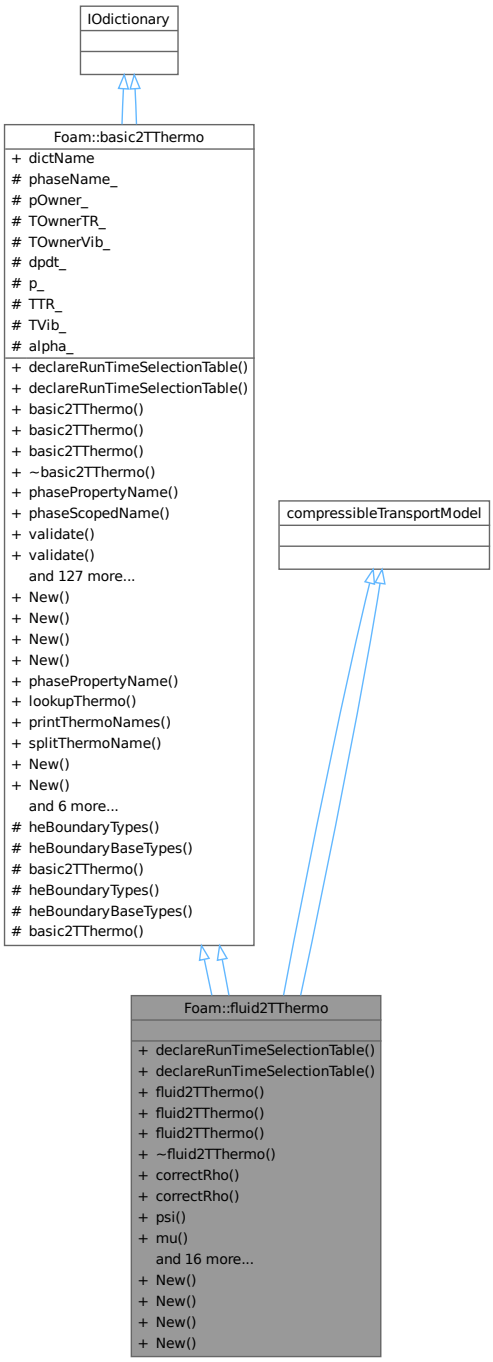
6.3 Foam::fluid2TThermo Class Reference

```
#include <fluid2TThermo.H>
```

Inheritance diagram for Foam::fluid2TThermo:



Collaboration diagram for Foam::fluid2TThermo:



Public Member Functions

- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- fluid2TThermo (const fvMesh &, const word &phaseName)

- fluid2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~fluid2TThermo ()
- virtual void correctRho (const volScalarField &deltaRho, const dimensionedScalar &rhoMin, const dimensionedScalar &rhoMax)=0
- virtual void correctRho (const volScalarField &deltaRho)=0
- virtual const volScalarField & psi () const =0
- virtual tmp< volScalarField > mu () const =0
- virtual tmp< scalarField > mu (const label patchi) const =0
- virtual tmp< volScalarField > nu () const
- virtual tmp< scalarField > nu (const label patchi) const
- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- fluid2TThermo (const fvMesh &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~fluid2TThermo ()
- virtual void correctRho (const volScalarField &deltaRho, const dimensionedScalar &rhoMin, const dimensionedScalar &rhoMax)=0
- virtual void correctRho (const volScalarField &deltaRho)=0
- virtual const volScalarField & psi () const =0
- virtual tmp< volScalarField > mu () const =0
- virtual tmp< scalarField > mu (const label patchi) const =0
- virtual tmp< volScalarField > nu () const
- virtual tmp< scalarField > nu (const label patchi) const

Public Member Functions inherited from Foam::basic2TThermo

- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- basic2TThermo (const fvMesh &, const word &phaseName)
- basic2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- basic2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~basic2TThermo ()
- word phasePropertyName (const word &name) const
- word phaseScopedName (const std::string &scope, const word &name) const
- void validate (const string &app, const word &) const
- void validate (const string &app, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &, const word &) const
- virtual void correct ()=0
- virtual word thermoName () const =0
- virtual bool incompressible () const =0
- virtual bool isochoric () const =0
- bool dpdt () const noexcept
- bool updateTTR () const noexcept
- bool updateTVib () const noexcept
- virtual volScalarField & p ()
- virtual const volScalarField & p () const
- virtual tmp< volScalarField > rho () const =0

- virtual tmp< scalarField > rho (const label patchi) const =0
- virtual volScalarField & he ()=0
- virtual const volScalarField & he () const =0
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > hc () const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const =0
- virtual const volScalarField & TTR () const
- virtual const volScalarField & TVib () const
- virtual volScalarField & TTR ()
- virtual volScalarField & TVib ()
- virtual tmp< volScalarField > CpTR () const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > CvT () const =0
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvR () const =0
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvVib () const =0
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const =0
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > gamma () const =0
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpvTR () const =0
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpByCpvTR () const =0
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > W () const =0
- virtual const volScalarField & alpha () const
- virtual const scalarField & alpha (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const =0
- virtual tmp< scalarField > kappaTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaVib () const =0
- virtual tmp< scalarField > kappaVib (const label patchi) const =0
- virtual tmp< volScalarField > alphaheTR () const =0
- virtual tmp< scalarField > alphaheTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const =0
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const =0
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const =0
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const =0
- virtual bool read ()

- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- basic2TThermo (const fvMesh &, const word &phaseName)
- basic2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- basic2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~basic2TThermo ()
- word phasePropertyName (const word &name) const
- word phaseScopedName (const std::string &scope, const word &name) const
- void validate (const string &app, const word &) const
- void validate (const string &app, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &, const word &) const
- virtual void correct ()=0
- virtual word thermoName () const =0
- virtual bool incompressible () const =0
- virtual bool isochoric () const =0
- bool dpdt () const noexcept
- bool updateTTR () const noexcept
- bool updateTVib () const noexcept
- virtual volScalarField & p ()
- virtual const volScalarField & p () const
- virtual tmp< volScalarField > rho () const =0
- virtual tmp< scalarField > rho (const label patchi) const =0
- virtual volScalarField & he ()=0
- virtual const volScalarField & he () const =0
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > hc () const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const =0
- virtual const volScalarField & TTR () const
- virtual const volScalarField & TVib () const
- virtual volScalarField & TTR ()
- virtual volScalarField & TVib ()
- virtual tmp< volScalarField > CpTR () const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > CvT () const =0
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvR () const =0
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvVib () const =0
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const =0

- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > gamma () const =0
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpvTR () const =0
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpByCpvTR () const =0
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > W () const =0
- virtual const volScalarField & alpha () const
- virtual const scalarField & alpha (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const =0
- virtual tmp< scalarField > kappaTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaVib () const =0
- virtual tmp< scalarField > kappaVib (const label patchi) const =0
- virtual tmp< volScalarField > alphaheTR () const =0
- virtual tmp< scalarField > alphaheTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const =0
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const =0
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const =0
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const =0
- virtual bool read ()
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const word &phaseName)
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const dictionary &dict, const word &phaseName)
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const word &phaseName, const word &dictName)

Static Public Member Functions

- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName, const word &dictName)

Static Public Member Functions inherited from Foam::basic2TThermo

- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const dictionary &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< basic2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static word phasePropertyName (const word &name, const word &phaseName)
- static const basic2TThermo & lookupThermo (const fvPatchScalarField &pf)
- static Ostream & printThermoNames (Ostream &os, const wordList &cmptNames, const wordList &thermoNames)
- static wordList splitThermoName (const std::string &thermoName, const int nExpectedCmpts)

- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const dictionary &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< basic2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static word phasePropertyName (const word &name, const word &phaseName)
- static const basic2TThermo & lookupThermo (const fvPatchScalarField &pf)
- static Ostream & printThermoNames (Ostream &os, const wordList &cmptNames, const wordList &thermoNames)
- static wordList splitThermoName (const std::string &thermoName, const int nExpectedCmpts)

Additional Inherited Members

Static Public Attributes inherited from Foam::basic2TThermo

- static const word dictName

Protected Member Functions inherited from Foam::basic2TThermo

- wordList heBoundaryTypes ()
- wordList heBoundaryBaseTypes ()
- basic2TThermo (const basic2TThermo &)=delete
- wordList heBoundaryTypes ()
- wordList heBoundaryBaseTypes ()
- basic2TThermo (const basic2TThermo &)=delete

Protected Attributes inherited from Foam::basic2TThermo

- const word & phaseName_
- bool pOwner_
- bool TOwnerTR_
- bool TOwnerVib_
- bool dpdt_
- volScalarField & p_
- volScalarField & TTR_
- volScalarField & TVib_
- volScalarField alpha_

6.3.1 Detailed Description

Definition at line 53 of file fluid2TThermo/fluid2TThermo.H.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 fluid2TThermo() [1/6]

```
Foam::fluid2TThermo::fluid2TThermo (
            const fvMesh & mesh,
            const word & phaseName )
```

Definition at line 43 of file fluid2TThermo/fluid2TThermo.C.

6.3.2.2 fluid2TThermo() [2/6]

```
Foam::fluid2TThermo::fluid2TThermo (
            const fvMesh & mesh,
            const dictionary & dict,
            const word & phaseName )
```

Definition at line 50 of file fluid2TThermo/fluid2TThermo.C.

6.3.2.3 fluid2TThermo() [3/6]

```
Foam::fluid2TThermo::fluid2TThermo (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictionaryName )
```

Definition at line 61 of file fluid2TThermo/fluid2TThermo.C.

6.3.2.4 ~fluid2TThermo() [1/2]

```
Foam::fluid2TThermo::~fluid2TThermo ( )  [virtual]
```

Definition at line 97 of file fluid2TThermo/fluid2TThermo.C.

6.3.2.5 fluid2TThermo() [4/6]

```
Foam::fluid2TThermo::fluid2TThermo (
            const fvMesh & ,
            const word & phaseName )
```

6.3.2.6 fluid2TThermo() [5/6]

```
Foam::fluid2TThermo::fluid2TThermo (
            const fvMesh & ,
            const dictionary & ,
            const word & phaseName )
```

6.3.2.7 fluid2TThermo() [6/6]

```
Foam::fluid2TThermo::fluid2TThermo (
            const fvMesh & ,
            const word & phaseName,
            const word & dictionaryName )
```

6.3.2.8 ~fluid2TThermo() [2/2]

```
virtual Foam::fluid2TThermo::~fluid2TThermo ( )  [virtual]
```

6.3.3 Member Function Documentation

6.3.3.1 correctRho() [1/4]

```
virtual void Foam::fluid2TThermo::correctRho (
            const volScalarField & deltaRho )  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.2 correctRho() [2/4]

```
virtual void Foam::fluid2TThermo::correctRho (
            const volScalarField & deltaRho )  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.3 correctRho() [3/4]

```
virtual void Foam::fluid2TThermo::correctRho (
            const volScalarField & deltaRho,
            const dimensionedScalar & rhoMin,
            const dimensionedScalar & rhoMax )  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.4 correctRho() [4/4]

```
virtual void Foam::fluid2TThermo::correctRho (
            const volScalarField & deltaRho,
            const dimensionedScalar & rhoMin,
            const dimensionedScalar & rhoMax )  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.5 declareRunTimeSelectionTable() [1/4]

```
Foam::fluid2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            fluid2TThermo ,
            fvMesh ,
            (const fvMesh &mesh, const word &phaseName) ,
            (mesh, phaseName)  )
```

6.3.3.6 declareRunTimeSelectionTable() [2/4]

```
Foam::fluid2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            fluid2TThermo ,
            fvMesh ,
            (const fvMesh &mesh, const word &phaseName) ,
            (mesh, phaseName)  )
```

6.3.3.7 declareRunTimeSelectionTable() [3/4]

```
Foam::fluid2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            fluid2TThermo ,
            fvMeshDictPhase ,
            (const fvMesh &mesh, const word &phaseName, const word &dictName) ,
            (mesh, phaseName, dictName)  )
```

6.3.3.8 declareRunTimeSelectionTable() [4/4]

```
Foam::fluid2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            fluid2TThermo ,
            fvMeshDictPhase ,
            (const fvMesh &mesh, const word &phaseName, const word &dictName) ,
            (mesh, phaseName, dictName)  )
```

6.3.3.9 mu() [1/4]

```
virtual tmp< volScalarField > Foam::fluid2TThermo::mu ( ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.10 mu() [2/4]

```
virtual tmp< volScalarField > Foam::fluid2TThermo::mu ( ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.11 mu() [3/4]

```
virtual tmp< scalarField > Foam::fluid2TThermo::mu (
            const label patchi ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.12 mu() [4/4]

```
virtual tmp< scalarField > Foam::fluid2TThermo::mu (
            const label patchi ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

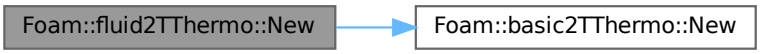
6.3.3.13 New() [1/4]

```
Foam::autoPtr< Foam::fluid2TThermo > Foam::fluid2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictName )  [static]
```

Definition at line 84 of file fluid2TThermo/fluid2TThermo.C.

References Foam::basic2TThermo::New().

Here is the call graph for this function:



6.3.3.14 New() [2/4]

```
static autoPtr< fluid2TThermo > Foam::fluid2TThermo::New (
            const fvMesh & ,
            const word & phaseName,
            const word & dictName )  [static]
```

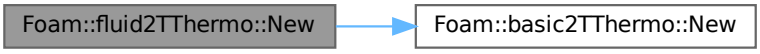
6.3.3.15 New() [3/4]

```
Foam::autoPtr< Foam::fluid2TThermo > Foam::fluid2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName = word::null )  [static]
```

Definition at line 74 of file fluid2TThermo/fluid2TThermo.C.

References Foam::basic2TThermo::New().

Here is the call graph for this function:



6.3.3.16 New() [4/4]

```
static autoPtr< fluid2TThermo > Foam::fluid2TThermo::New (
            const fvMesh & ,
            const word & phaseName = word::null )  [static]
```

6.3.3.17 nu() [1/4]

```
Foam::tmp< Foam::volScalarField > Foam::fluid2TThermo::nu ( ) const  [virtual]
```

Definition at line 103 of file fluid2TThermo/fluid2TThermo.C.

6.3.3.18 nu() [2/4]

```
virtual tmp< volScalarField > Foam::fluid2TThermo::nu ( ) const  [virtual]
```

6.3.3.19 nu() [3/4]

```
Foam::tmp< Foam::scalarField > Foam::fluid2TThermo::nu (
            const label patchi ) const  [virtual]
```

Definition at line 109 of file fluid2TThermo/fluid2TThermo.C.

6.3.3.20 nu() [4/4]

```
virtual tmp< scalarField > Foam::fluid2TThermo::nu (
            const label patchi ) const  [virtual]
```

6.3.3.21 psi() [1/2]

```
virtual const volScalarField & Foam::fluid2TThermo::psi ( ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

6.3.3.22 psi() [2/2]

```
virtual const volScalarField & Foam::fluid2TThermo::psi ( ) const  [pure virtual]
```

Implemented in Foam::ne2TThermo, and Foam::ne2TThermo.

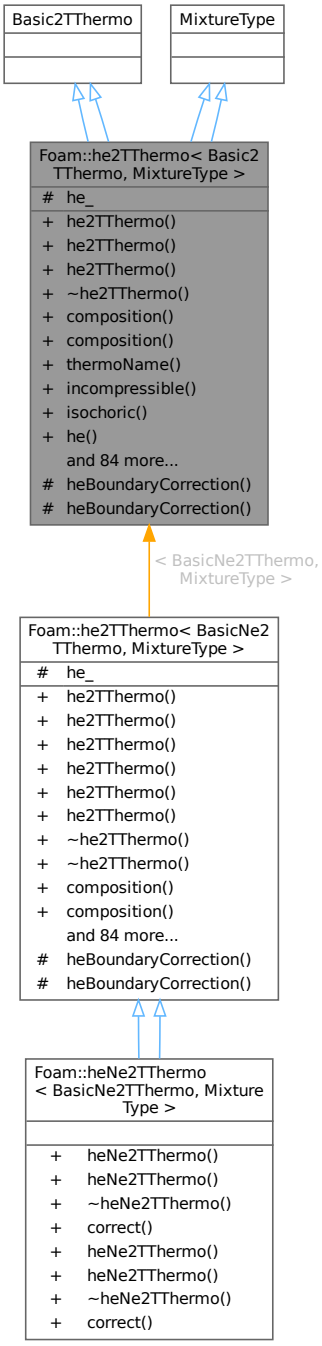
The documentation for this class was generated from the following files:

- fluid2TThermo/fluid2TThermo.H
- lnInclude/fluid2TThermo.H
- fluid2TThermo/fluid2TThermo.C
- lnInclude/fluid2TThermo.C

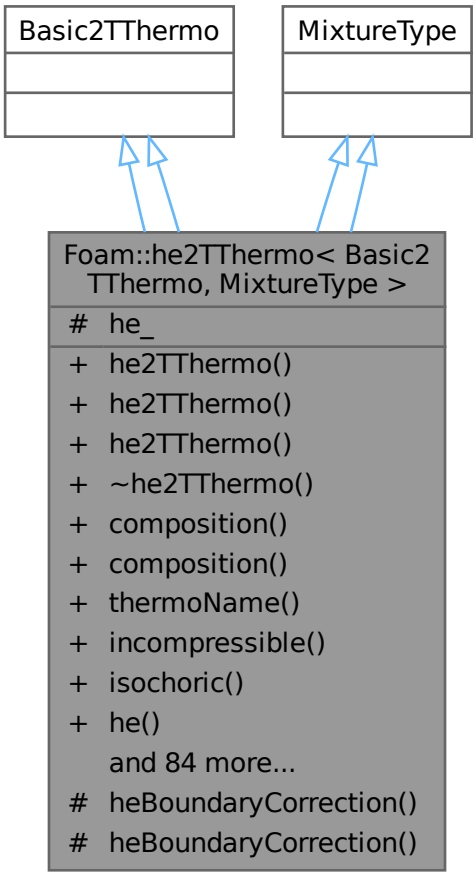
6.4 Foam::he2TThermo< Basic2TThermo, MixtureType > Class Template Reference

```
#include <he2TThermo.H>
```

Inheritance diagram for Foam::he2TThermo< Basic2TThermo, MixtureType >:



Collaboration diagram for Foam::he2TThermo< Basic2TThermo, MixtureType >:



Public Member Functions

- he2TThermo (const fvMesh &, const word &phaseName)
- he2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- he2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~he2TThermo ()
- virtual MixtureType::basicMixtureType & composition ()
- virtual const MixtureType::basicMixtureType & composition () const
- virtual word thermoName () const
- virtual bool incompressible () const
- virtual bool isochoric () const
- virtual volScalarField & he ()
- virtual const volScalarField & he () const
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > hc () const
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const

- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > CpTR () const
- virtual tmp< volScalarField > CvT () const
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvR () const
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvVib () const
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > gamma () const
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CpvTR () const
- virtual tmp< volScalarField > CpByCpvTR () const
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > W () const
- virtual tmp< volScalarField > kappaTR () const
- virtual tmp< scalarField > kappaTR (const label patchi) const
- virtual tmp< volScalarField > kappaVib () const
- virtual tmp< scalarField > kappaVib (const label patchi) const
- virtual tmp< volScalarField > alphaheTR () const
- virtual tmp< scalarField > alphaheTR (const label patchi) const
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const
- virtual bool read ()
- he2TThermo (const fvMesh &, const word &phaseName)
- he2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- he2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~he2TThermo ()
- virtual MixtureType::basicMixtureType & composition ()
- virtual const MixtureType::basicMixtureType & composition () const
- virtual word thermoName () const
- virtual bool incompressible () const
- virtual bool isochoric () const
- virtual volScalarField & he ()
- virtual const volScalarField & he () const
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > hc () const
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const

- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > CpTR () const
- virtual tmp< volScalarField > CvT () const
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvR () const
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvVib () const
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > gamma () const
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CpvTR () const
- virtual tmp< volScalarField > CpByCpvTR () const
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > W () const
- virtual tmp< volScalarField > kappaTR () const
- virtual tmp< scalarField > kappaTR (const label patchi) const
- virtual tmp< volScalarField > kappaVib () const
- virtual tmp< scalarField > kappaVib (const label patchi) const
- virtual tmp< volScalarField > alphaheTR () const
- virtual tmp< scalarField > alphaheTR (const label patchi) const
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const
- virtual bool read ()

Protected Member Functions

- void heBoundaryCorrection (volScalarField &he)
- void heBoundaryCorrection (volScalarField &he)

Protected Attributes

- volScalarField he_

6.4.1 Detailed Description

template<class Basic2TThermo, class MixtureType>
class Foam::he2TThermo< Basic2TThermo, MixtureType >

Definition at line 53 of file he2TThermo/he2TThermo.H.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 he2TThermo() [1/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo (
            const fvMesh & mesh,
            const word & phaseName )
```

Definition at line 104 of file he2TThermo/he2TThermo.C.

6.4.2.2 he2TThermo() [2/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo (
            const fvMesh & mesh,
            const dictionary & dict,
            const word & phaseName )
```

Definition at line 137 of file he2TThermo/he2TThermo.C.

References Foam::he2TThermo< Basic2TThermo, MixtureType >::he_.

6.4.2.3 he2TThermo() [3/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictionaryName )
```

Definition at line 171 of file he2TThermo/he2TThermo.C.

6.4.2.4 ~he2TThermo() [1/2]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::~he2TThermo ( )  [virtual]
```

Definition at line 208 of file he2TThermo/he2TThermo.C.

6.4.2.5 he2TThermo() [4/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo (
            const fvMesh & ,
            const word & phaseName )
```

6.4.2.6 he2TThermo() [5/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo (
            const fvMesh & ,
            const dictionary & ,
            const word & phaseName )
```

6.4.2.7 he2TThermo() [6/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo (
            const fvMesh & ,
            const word & phaseName,
            const word & dictionaryName )
```

6.4.2.8 ~he2TThermo() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual Foam::he2TThermo< Basic2TThermo, MixtureType >::~he2TThermo ( )  [virtual]
```

6.4.3 Member Function Documentation

6.4.3.1 alphaEff() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaEff (
            const scalarField & alphat,
            const label patchi ) const  [virtual]
```

Definition at line 1078 of file he2TThermo/he2TThermo.C.

6.4.3.2 alphaEff() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaEff (
            const scalarField & alphat,
            const label patchi ) const  [virtual]
```

6.4.3.3 alphaEff() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaEff (
            const volScalarField & alphat ) const  [virtual]
```

Definition at line 1065 of file he2TThermo/he2TThermo.C.

6.4.3.4 alphaEff() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaEff (
            const volScalarField & alphat ) const  [virtual]
```

6.4.3.5 alphaheTR() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaheTR (
) const  [virtual]
```

Definition at line 1005 of file he2TThermo/he2TThermo.C.

6.4.3.6 alphaheTR() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaheTR ( )
const  [virtual]
```

6.4.3.7 alphaheTR() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaheTR (
            const label patchi ) const  [virtual]
```

Definition at line 1015 of file he2TThermo/he2TThermo.C.

6.4.3.8 alphaheTR() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::alphaheTR (
            const label patchi ) const  [virtual]
```

6.4.3.9 composition() [1/4]

```
template<class Basic2TThermo , class MixtureType >
virtual MixtureType::basicMixtureType & Foam::he2TThermo< Basic2TThermo, MixtureType >::composition
( )  [inline], [virtual]
```

Definition at line 128 of file he2TThermo/he2TThermo.H.

6.4.3.10 composition() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual MixtureType::basicMixtureType & Foam::he2TThermo< Basic2TThermo, MixtureType >::composition
( )  [inline], [virtual]
```

Definition at line 128 of file lnInclude/he2TThermo.H.

6.4.3.11 composition() [3/4]

```
template<class Basic2TThermo , class MixtureType >
virtual const MixtureType::basicMixtureType & Foam::he2TThermo< Basic2TThermo, MixtureType >↩
::composition ( ) const  [inline], [virtual]
```

Definition at line 135 of file he2TThermo/he2TThermo.H.

6.4.3.12 composition() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual const MixtureType::basicMixtureType & Foam::he2TThermo< Basic2TThermo, MixtureType >↩
::composition ( ) const  [inline], [virtual]
```

Definition at line 135 of file InInclude/he2TThermo.H.

6.4.3.13 CpByCpvTR() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpByCpvTR (
) const  [virtual]
```

Definition at line 771 of file he2TThermo/he2TThermo.C.

6.4.3.14 CpByCpvTR() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpByCpvTR ( )
const  [virtual]
```

6.4.3.15 CpByCpvTR() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpByCpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 749 of file he2TThermo/he2TThermo.C.

6.4.3.16 CpByCpvTR() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpByCpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.17 CpTR() [1/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpTR ( )
const  [virtual]
```

Definition at line 385 of file he2TThermo/he2TThermo.C.

6.4.3.18 CpTR() [2/6]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpTR ( ) const
[virtual]
```

6.4.3.19 CpTR() [3/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 341 of file he2TThermo/he2TThermo.C.

6.4.3.20 CpTR() [4/6]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.21 CpTR() [5/6]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [virtual]
```

Definition at line 363 of file he2TThermo/he2TThermo.C.

6.4.3.22 CpTR() [6/6]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpTR (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [virtual]
```

6.4.3.23 CpvTR() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpvTR ( )
const  [virtual]
```

Definition at line 710 of file he2TThermo/he2TThermo.C.

6.4.3.24 CpvTR() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpvTR ( ) const
[virtual]
```

6.4.3.25 CpvTR() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 688 of file he2TThermo/he2TThermo.C.

6.4.3.26 CpvTR() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CpvTR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.27 CvR() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvR ( )
const  [virtual]
```

Definition at line 549 of file he2TThermo/he2TThermo.C.

References CvR().

Here is the call graph for this function:

6.4.3.28 CvR() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvR ( ) const
[virtual]
```

6.4.3.29 CvR() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 446 of file he2TThermo/he2TThermo.C.

6.4.3.30 CvR() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvR (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.31 CvT() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvT ( )
const  [virtual]
```

Definition at line 513 of file he2TThermo/he2TThermo.C.

References CvT().

Here is the call graph for this function:

6.4.3.32 CvT() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvT ( ) const
[virtual]
```

6.4.3.33 CvT() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvT (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 425 of file he2TThermo/he2TThermo.C.

6.4.3.34 CvT() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvT (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.35 CvVib() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvVib ( )
const  [virtual]
```

Definition at line 585 of file he2TThermo/he2TThermo.C.

References CvVib().

Here is the call graph for this function:

6.4.3.36 CvVib() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvVib ( ) const
[virtual]
```

6.4.3.37 CvVib() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvVib (
            const scalarField & p,
            const scalarField & TTR,
            const scalarField & Tvib,
            const scalar ThetaVib,
            const label patchi ) const  [virtual]
```

Definition at line 467 of file he2TThermo/he2TThermo.C.

6.4.3.38 CvVib() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::CvVib (
            const scalarField & p,
            const scalarField & TTR,
            const scalarField & Tvib,
            const scalar ThetaVib,
            const label patchi ) const  [virtual]
```

6.4.3.39 gamma() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::gamma ( )
const  [virtual]
```

Definition at line 646 of file he2TThermo/he2TThermo.C.

6.4.3.40 gamma() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::gamma ( ) const
[virtual]
```

6.4.3.41 gamma() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::gamma (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 624 of file he2TThermo/he2TThermo.C.

6.4.3.42 gamma() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::gamma (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.43 hc() [1/2]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::hc ( )
const  [virtual]
```

Definition at line 304 of file he2TThermo/he2TThermo.C.

6.4.3.44 hc() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::hc ( ) const
[virtual]
```

6.4.3.45 he() [1/10]

```
template<class Basic2TThermo , class MixtureType >
virtual volScalarField & Foam::he2TThermo< Basic2TThermo, MixtureType >::he ( )  [inline],
[virtual]
```

Definition at line 165 of file he2TThermo/he2TThermo.H.

References Foam::he2TThermo< Basic2TThermo, MixtureType >::he_.

6.4.3.46 he() [2/10]

```
template<class Basic2TThermo , class MixtureType >
virtual volScalarField & Foam::he2TThermo< Basic2TThermo, MixtureType >::he ( )  [inline],
[virtual]
```

Definition at line 165 of file InInclude/he2TThermo.H.

References Foam::he2TThermo< Basic2TThermo, MixtureType >::he_.

6.4.3.47 he() [3/10]

```
template<class Basic2TThermo , class MixtureType >
virtual const volScalarField & Foam::he2TThermo< Basic2TThermo, MixtureType >::he ( ) const
[inline], [virtual]
```

Definition at line 171 of file he2TThermo/he2TThermo.H.

References Foam::he2TThermo< Basic2TThermo, MixtureType >::he_.

6.4.3.48 he() [4/10]

```
template<class Basic2TThermo , class MixtureType >
virtual const volScalarField & Foam::he2TThermo< Basic2TThermo, MixtureType >::he ( ) const
[inline], [virtual]
```

Definition at line 171 of file InInclude/he2TThermo.H.

References Foam::he2TThermo< Basic2TThermo, MixtureType >::he_.

6.4.3.49 he() [5/10]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::he (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

Definition at line 282 of file he2TThermo/he2TThermo.C.

6.4.3.50 he() [6/10]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::he (
            const scalarField & p,
            const scalarField & TTR,
            const label patchi ) const  [virtual]
```

6.4.3.51 he() [7/10]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::he (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [virtual]
```

Definition at line 262 of file he2TThermo/he2TThermo.C.

6.4.3.52 he() [8/10]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::he (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [virtual]
```

6.4.3.53 he() [9/10]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::he (
            const volScalarField & p,
            const volScalarField & TTR ) const  [virtual]
```

Definition at line 215 of file he2TThermo/he2TThermo.C.

6.4.3.54 he() [10/10]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::he (
            const volScalarField & p,
            const volScalarField & TTR ) const  [virtual]
```

6.4.3.55 heBoundaryCorrection() [1/2]

```
template<class Basic2TThermo , class MixtureType >
void Foam::he2TThermo< Basic2TThermo, MixtureType >::heBoundaryCorrection (
            volScalarField & he )  [protected]
```

Definition at line 36 of file he2TThermo/he2TThermo.C.

6.4.3.56 heBoundaryCorrection() [2/2]

```
template<class Basic2TThermo , class MixtureType >
void Foam::he2TThermo< Basic2TThermo, MixtureType >::heBoundaryCorrection (
            volScalarField & he )  [protected]
```

6.4.3.57 incompressible() [1/2]

```
template<class Basic2TThermo , class MixtureType >
virtual bool Foam::he2TThermo< Basic2TThermo, MixtureType >::incompressible ( ) const  [inline],
[virtual]
```

Definition at line 148 of file he2TThermo/he2TThermo.H.

6.4.3.58 incompressible() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual bool Foam::he2TThermo< Basic2TThermo, MixtureType >::incompressible ( ) const  [inline],
[virtual]
```

Definition at line 148 of file InInclude/he2TThermo.H.

6.4.3.59 isochoric() [1/2]

```
template<class Basic2TThermo , class MixtureType >
virtual bool Foam::he2TThermo< Basic2TThermo, MixtureType >::isochoric ( ) const  [inline],
[virtual]
```

Definition at line 155 of file he2TThermo/he2TThermo.H.

6.4.3.60 isochoric() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual bool Foam::he2TThermo< Basic2TThermo, MixtureType >::isochoric ( ) const  [inline],
[virtual]
```

Definition at line 155 of file lnInclude/he2TThermo.H.

6.4.3.61 kappaEff() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaEff (
            const scalarField & alphat,
            const label patchi ) const  [virtual]
```

Definition at line 1043 of file he2TThermo/he2TThermo.C.

References CpTR().

Here is the call graph for this function:

6.4.3.62 kappaEff() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaEff (
            const scalarField & alphat,
            const label patchi ) const  [virtual]
```

6.4.3.63 kappaEff() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaEff (
            const volScalarField & alphat ) const  [virtual]
```

Definition at line 1030 of file he2TThermo/he2TThermo.C.

References CpTR().

Here is the call graph for this function:

6.4.3.64 kappaEff() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaEff (
            const volScalarField &  ) const  [virtual]
```

6.4.3.65 kappaTR() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaTR ( )
const  [virtual]
```

Definition at line 954 of file he2TThermo/he2TThermo.C.

References CpTR().

Here is the call graph for this function:

6.4.3.66 kappaTR() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaTR ( )
const  [virtual]
```

6.4.3.67 kappaTR() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaTR (
            const label patchi ) const  [virtual]
```

Definition at line 963 of file he2TThermo/he2TThermo.C.

References CpTR().

Here is the call graph for this function:

6.4.3.68 kappaTR() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaTR (
            const label patchi ) const  [virtual]
```

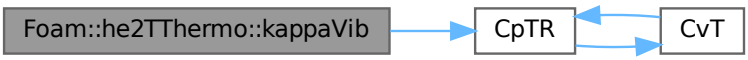
6.4.3.69 kappaVib() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaVib (
) const  [virtual]
```

Definition at line 979 of file he2TThermo/he2TThermo.C.

References CpTR().

Here is the call graph for this function:



6.4.3.70 kappaVib() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaVib ( )
const  [virtual]
```

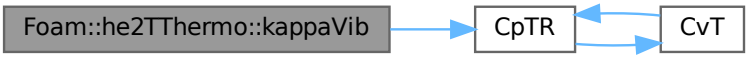
6.4.3.71 kappaVib() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaVib (
            const label patchi ) const  [virtual]
```

Definition at line 988 of file he2TThermo/he2TThermo.C.

References CpTR().

Here is the call graph for this function:



6.4.3.72 kappaVib() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaVib (
            const label patchi ) const  [virtual]
```

6.4.3.73 read() [1/2]

```
template<class Basic2TThermo , class MixtureType >
bool Foam::he2TThermo< Basic2TThermo, MixtureType >::read ( )  [virtual]
```

Definition at line 1099 of file he2TThermo/he2TThermo.C.

6.4.3.74 read() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual bool Foam::he2TThermo< Basic2TThermo, MixtureType >::read ( )  [virtual]
```

6.4.3.75 rhoEoS() [1/2]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::rhoEoS (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [virtual]
```

Definition at line 491 of file he2TThermo/he2TThermo.C.

6.4.3.76 rhoEoS() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::rhoEoS (
            const scalarField & p,
            const scalarField & TTR,
            const labelList & cells ) const  [virtual]
```

6.4.3.77 TEH_Vib() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::TEH_Vib (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const label patchi ) const  [virtual]
```

Definition at line 888 of file he2TThermo/he2TThermo.C.

6.4.3.78 TEH_Vib() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::TEH_Vib (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const label patchi ) const  [virtual]
```

6.4.3.79 TEH_Vib() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::TEH_Vib (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const labelList & cells ) const  [virtual]
```

Definition at line 864 of file he2TThermo/he2TThermo.C.

6.4.3.80 TEH_Vib() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::TEH_Vib (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const scalarField & TVib0,
            const labelList & cells ) const  [virtual]
```

6.4.3.81 THE_TR() [1/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const label patchi ) const  [virtual]
```

Definition at line 839 of file he2TThermo/he2TThermo.C.

6.4.3.82 THE_TR() [2/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const label patchi ) const  [virtual]
```

6.4.3.83 THE_TR() [3/4]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const labelList & cells ) const  [virtual]
```

Definition at line 817 of file he2TThermo/he2TThermo.C.

6.4.3.84 THE_TR() [4/4]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< scalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::THE_TR (
            const scalarField & h,
            const scalarField & p,
            const scalarField & TTR0,
            const labelList & cells ) const  [virtual]
```

6.4.3.85 thermoName() [1/2]

```
template<class Basic2TThermo , class MixtureType >
virtual word Foam::he2TThermo< Basic2TThermo, MixtureType >::thermoName ( ) const  [inline],
[virtual]
```

Definition at line 141 of file he2TThermo/he2TThermo.H.

6.4.3.86 thermoName() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual word Foam::he2TThermo< Basic2TThermo, MixtureType >::thermoName ( ) const  [inline],
[virtual]
```

Definition at line 141 of file lnInclude/he2TThermo.H.

6.4.3.87 W() [1/2]

```
template<class Basic2TThermo , class MixtureType >
Foam::tmp< Foam::volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::W ( ) const
[virtual]
```

Definition at line 915 of file he2TThermo/he2TThermo.C.

6.4.3.88 W() [2/2]

```
template<class Basic2TThermo , class MixtureType >
virtual tmp< volScalarField > Foam::he2TThermo< Basic2TThermo, MixtureType >::W ( ) const
[virtual]
```

6.4.4 Member Data Documentation

6.4.4.1 he_

```
template<class Basic2TThermo , class MixtureType >
volScalarField Foam::he2TThermo< Basic2TThermo, MixtureType >::he_  [protected]
```

Definition at line 63 of file he2TThermo/he2TThermo.H.

Referenced by Foam::he2TThermo< Basic2TThermo, MixtureType >::he(), Foam::he2TThermo< Basic2TThermo, MixtureType >::h
and Foam::he2TThermo< Basic2TThermo, MixtureType >::he2TThermo().

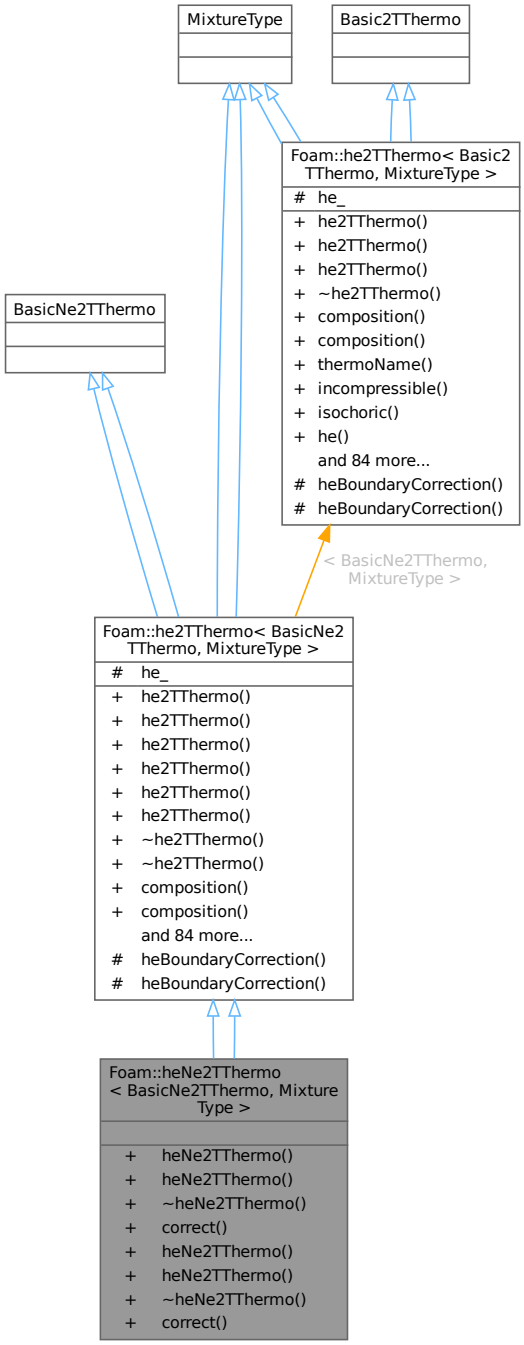
The documentation for this class was generated from the following files:

- he2TThermo/he2TThermo.H
- lnInclude/he2TThermo.H
- he2TThermo/he2TThermo.C
- lnInclude/he2TThermo.C

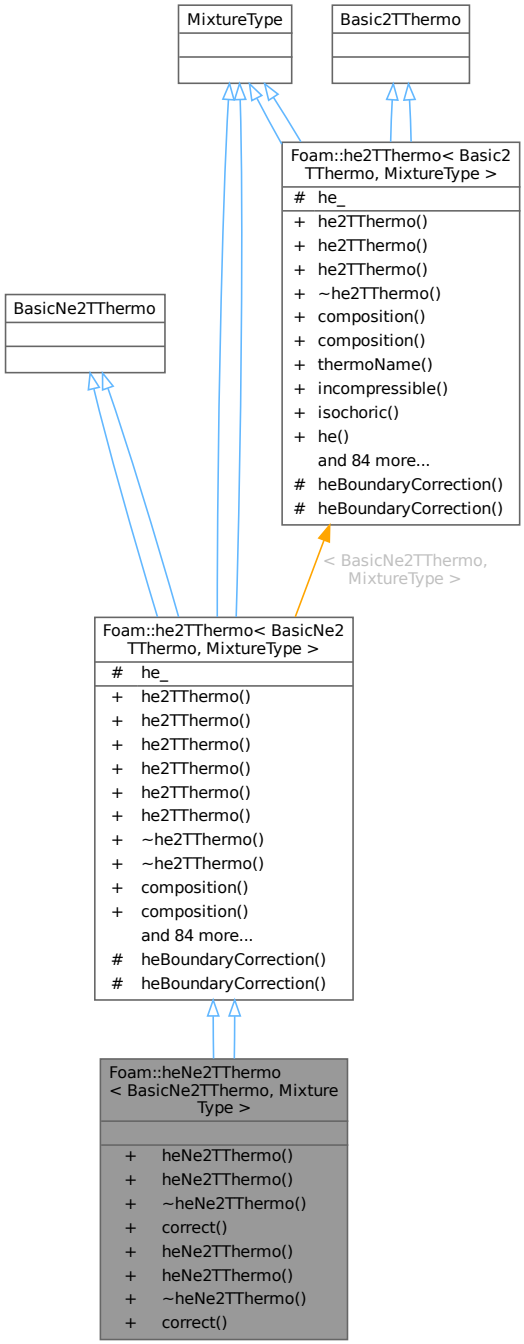
6.5 Foam::heNe2TThermo< BasicNe2TThermo, MixtureType > Class Template Reference

```
#include <heNe2TThermo.H>
```

Inheritance diagram for Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >:



Collaboration diagram for Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >:



Public Member Functions

- heNe2TThermo (const fvMesh &, const word &phaseName)
- heNe2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~heNe2TThermo ()
- virtual void correct ()
- heNe2TThermo (const fvMesh &, const word &phaseName)

- heNe2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~heNe2TThermo ()
- virtual void correct ()

Public Member Functions inherited from Foam::he2TThermo< BasicNe2TThermo, MixtureType >

- he2TThermo (const fvMesh &, const word &phaseName)
- he2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- he2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- he2TThermo (const fvMesh &, const word &phaseName)
- he2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- he2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~he2TThermo ()
- virtual ~he2TThermo ()
- virtual MixtureType::basicMixtureType & composition ()
- virtual const MixtureType::basicMixtureType & composition () const
- virtual MixtureType::basicMixtureType & composition ()
- virtual const MixtureType::basicMixtureType & composition () const
- virtual word thermoName () const
- virtual word thermoName () const
- virtual bool incompressible () const
- virtual bool incompressible () const
- virtual bool isochoric () const
- virtual bool isochoric () const
- virtual volScalarField & he ()
- virtual const volScalarField & he () const
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual volScalarField & he ()
- virtual const volScalarField & he () const
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > hc () const
- virtual tmp< volScalarField > hc () const
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const
- virtual tmp< scalarField > TEH_Vib (const scalarField &h, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const

- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > CpTR () const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > CpTR () const
- virtual tmp< volScalarField > CvT () const
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvT () const
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvR () const
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvR () const
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CvVib () const
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const
- virtual tmp< volScalarField > CvVib () const
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const
- virtual tmp< volScalarField > gamma () const
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > gamma () const
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CpvTR () const
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CpvTR () const
- virtual tmp< volScalarField > CpByCpvTR () const
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > CpByCpvTR () const
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const
- virtual tmp< volScalarField > W () const
- virtual tmp< volScalarField > W () const
- virtual tmp< volScalarField > kappaTR () const
- virtual tmp< scalarField > kappaTR (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const
- virtual tmp< scalarField > kappaTR (const label patchi) const
- virtual tmp< volScalarField > kappaVib () const
- virtual tmp< scalarField > kappaVib (const label patchi) const
- virtual tmp< volScalarField > kappaVib () const
- virtual tmp< scalarField > kappaVib (const label patchi) const
- virtual tmp< volScalarField > alphaheTR () const
- virtual tmp< scalarField > alphaheTR (const label patchi) const
- virtual tmp< volScalarField > alphaheTR () const
- virtual tmp< scalarField > alphaheTR (const label patchi) const
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const

- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const
- virtual bool read ()
- virtual bool read ()

Additional Inherited Members

Protected Member Functions inherited from Foam::he2TThermo< BasicNe2TThermo, MixtureType >

- void heBoundaryCorrection (volScalarField &he)
- void heBoundaryCorrection (volScalarField &he)

Protected Attributes inherited from Foam::he2TThermo< BasicNe2TThermo, MixtureType >

- volScalarField he_

6.5.1 Detailed Description

template<class BasicNe2TThermo, class MixtureType>
class Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >

Definition at line 54 of file lnInclude/heNe2TThermo.H.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 heNe2TThermo() [1/4]

```
template<class BasicNe2TThermo , class MixtureType >
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::heNe2TThermo (
            const fvMesh & mesh,
            const word & phaseName )
```

Definition at line 163 of file lnInclude/heNe2TThermo.C.

References Foam::he2TThermo< BasicNe2TThermo, MixtureType >::he_.

6.5.2.2 heNe2TThermo() [2/4]

```
template<class BasicNe2TThermo , class MixtureType >
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::heNe2TThermo (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictionaryName )
```

Definition at line 186 of file lnInclude/heNe2TThermo.C.

References Foam::he2TThermo< BasicNe2TThermo, MixtureType >::he_.

6.5.2.3 ~heNe2TThermo() [1/2]

```
template<class BasicNe2TThermo , class MixtureType >
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::~heNe2TThermo ( )  [virtual]
```

Definition at line 213 of file lnInclude/heNe2TThermo.C.

6.5.2.4 heNe2TThermo() [3/4]

```
template<class BasicNe2TThermo , class MixtureType >
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::heNe2TThermo (
            const fvMesh & ,
            const word & phaseName )
```

6.5.2.5 heNe2TThermo() [4/4]

```
template<class BasicNe2TThermo , class MixtureType >
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::heNe2TThermo (
            const fvMesh & ,
            const word & phaseName,
            const word & dictionaryName )
```

6.5.2.6 ~heNe2TThermo() [2/2]

```
template<class BasicNe2TThermo , class MixtureType >
virtual Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::~heNe2TThermo ( )  [virtual]
```

6.5.3 Member Function Documentation

6.5.3.1 correct() [1/2]

```
template<class BasicNe2TThermo , class MixtureType >
void Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::correct ( )  [virtual]
```

Definition at line 220 of file lnInclude/heNe2TThermo.C.

6.5.3.2 correct() [2/2]

```
template<class BasicNe2TThermo , class MixtureType >
virtual void Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >::correct ( )  [virtual]
```

The documentation for this class was generated from the following files:

- lnInclude/heNe2TThermo.H
- ne2TThermo/heNe2TThermo.H
- lnInclude/heNe2TThermo.C
- ne2TThermo/heNe2TThermo.C

6.6 Foam::janaf2TThermo< EquationOfState > Class Template Reference

```
#include <janaf2TThermo.H>
```

Inheritance diagram for Foam::janaf2TThermo< EquationOfState >:

Collaboration diagram for Foam::janaf2TThermo< EquationOfState >:

Public Types

- typedef FixedList< scalar, nCoeffs_ > coeffArray
- typedef FixedList< scalar, nCoeffs_ > coeffArray

Public Member Functions

- janaf2TThermo (const EquationOfState &st, const scalar Tlow, const scalar Thigh, const scalar Tcommon, const scalar ThetaVib, const coeffArray &highCpCoeffs, const coeffArray &lowCpCoeffs, const bool convert↩ Coeffs=false)
- janaf2TThermo (const dictionary &dict)
- janaf2TThermo (const word &, const janaf2TThermo &)
- scalar limit (const scalar TTR) const
- scalar Tlow () const
- scalar Thigh () const
- scalar Tcommon () const
- scalar ThetaVib () const

- const coeffArray & highCpCoeffs () const
- const coeffArray & lowCpCoeffs () const
- scalar CpTR (const scalar p, const scalar TTR) const
- scalar Ha (const scalar p, const scalar TTR) const
- scalar Hs (const scalar p, const scalar TTR) const
- scalar Hc () const
- scalar S (const scalar p, const scalar TTR) const
- scalar Gstd (const scalar TTR) const
- scalar dCpdT (const scalar p, const scalar TTR) const
- void write (Ostream &os) const
- void operator+= (const janaf2TThermo &)
- janaf2TThermo (const EquationOfState &st, const scalar Tlow, const scalar Thigh, const scalar Tcommon, const scalar ThetaVib, const coeffArray &highCpCoeffs, const coeffArray &lowCpCoeffs, const bool convert↩ Coeffs=false)
- janaf2TThermo (const dictionary &dict)
- janaf2TThermo (const word &, const janaf2TThermo &)
- scalar limit (const scalar TTR) const
- scalar Tlow () const
- scalar Thigh () const
- scalar Tcommon () const
- scalar ThetaVib () const
- const coeffArray & highCpCoeffs () const
- const coeffArray & lowCpCoeffs () const
- scalar CpTR (const scalar p, const scalar TTR) const
- scalar Ha (const scalar p, const scalar TTR) const
- scalar Hs (const scalar p, const scalar TTR) const
- scalar Hc () const
- scalar S (const scalar p, const scalar TTR) const
- scalar Gstd (const scalar TTR) const
- scalar dCpdT (const scalar p, const scalar TTR) const
- void write (Ostream &os) const
- void operator+= (const janaf2TThermo &)

Static Public Member Functions

- static word typeName ()
- static word typeName ()

Static Public Attributes

- static constexpr int nCoeffs_ = 7

Friends

- janaf2TThermo operator+ (const janaf2TThermo &, const janaf2TThermo &)
- janaf2TThermo operator∗ (const scalar, const janaf2TThermo &)
- janaf2TThermo operator== (const janaf2TThermo &, const janaf2TThermo &)
- Ostream & operator<< (Ostream &, const janaf2TThermo &)
- janaf2TThermo operator+ (const janaf2TThermo &, const janaf2TThermo &)
- janaf2TThermo operator∗ (const scalar, const janaf2TThermo &)
- janaf2TThermo operator== (const janaf2TThermo &, const janaf2TThermo &)
- Ostream & operator<< (Ostream &, const janaf2TThermo &)

6.6.1 Detailed Description

template<class EquationOfState>
class Foam::janaf2TThermo< EquationOfState >

Definition at line 92 of file thermo/janaf2T/janaf2TThermo.H.

6.6.2 Member Typedef Documentation

6.6.2.1 coeffArray [1/2]

```
template<class EquationOfState >
typedef FixedList<scalar, nCoeffs_> Foam::janaf2TThermo< EquationOfState >::coeffArray
```

Definition at line 101 of file lnInclude/janaf2TThermo.H.

6.6.2.2 coeffArray [2/2]

```
template<class EquationOfState >
typedef FixedList<scalar, nCoeffs_> Foam::janaf2TThermo< EquationOfState >::coeffArray
```

Definition at line 101 of file thermo/janaf2T/janaf2TThermo.H.

6.6.3 Constructor & Destructor Documentation

6.6.3.1 janaf2TThermo() [1/6]

```
template<class EquationOfState >
Foam::janaf2TThermo< EquationOfState >::janaf2TThermo (
            const EquationOfState & st,
            const scalar Tlow,
            const scalar Thigh,
            const scalar Tcommon,
            const scalar ThetaVib,
            const coeffArray & highCpCoeffs,
            const coeffArray & lowCpCoeffs,
            const bool convertCoeffs = false )  [inline]
```

6.6.3.2 janaf2TThermo() [2/6]

```
template<class EquationOfState >
Foam::janaf2TThermo< EquationOfState >::janaf2TThermo (
            const dictionary & dict )
```

Definition at line 62 of file lnInclude/janaf2TThermo.C.

References Foam::janaf2TThermo< EquationOfState >::nCoeffs_.

6.6.3.3 janaf2TThermo() [3/6]

```
template<class EquationOfState >
Foam::janaf2TThermo< EquationOfState >::janaf2TThermo (
            const word & name,
            const janaf2TThermo< EquationOfState > & jt )  [inline]
```

Definition at line 93 of file InInclude/janaf2TThermoI.H.

References Foam::janaf2TThermo< EquationOfState >::nCoeffs_.

6.6.3.4 janaf2TThermo() [4/6]

```
template<class EquationOfState >
Foam::janaf2TThermo< EquationOfState >::janaf2TThermo (
            const EquationOfState & st,
            const scalar Tlow,
            const scalar Thigh,
            const scalar Tcommon,
            const scalar ThetaVib,
            const coeffArray & highCpCoeffs,
            const coeffArray & lowCpCoeffs,
            const bool convertCoeffs = false )  [inline]
```

6.6.3.5 janaf2TThermo() [5/6]

```
template<class EquationOfState >
Foam::janaf2TThermo< EquationOfState >::janaf2TThermo (
            const dictionary & dict )
```

6.6.3.6 janaf2TThermo() [6/6]

```
template<class EquationOfState >
Foam::janaf2TThermo< EquationOfState >::janaf2TThermo (
            const word & ,
            const janaf2TThermo< EquationOfState > &  )  [inline]
```

6.6.4 Member Function Documentation

6.6.4.1 CpTR() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::CpTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 179 of file InInclude/janaf2TThermoI.H.

6.6.4.2 CpTR() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::CpTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.6.4.3 dCpdT() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::dCpdT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 269 of file InInclude/janaf2TThermoI.H.

6.6.4.4 dCpdT() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::dCpdT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.6.4.5 Gstd() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Gstd (
            const scalar TTR ) const  [inline]
```

Definition at line 250 of file InInclude/janaf2TThermoI.H.

6.6.4.6 Gstd() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Gstd (
            const scalar TTR ) const  [inline]
```

6.6.4.7 Ha() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Ha (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 193 of file InInclude/janaf2TThermoI.H.

6.6.4.8 Ha() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Ha (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.6.4.9 Hc() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Hc ( ) const  [inline]
```

Definition at line 220 of file InInclude/janaf2TThermoI.H.

6.6.4.10 Hc() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Hc ( ) const  [inline]
```

6.6.4.11 highCpCoeffs() [1/2]

```
template<class EquationOfState >
const Foam::janaf2TThermo< EquationOfState >::coeffArray & Foam::janaf2TThermo< EquationOf↩
State >::highCpCoeffs ( ) const  [inline]
```

Definition at line 164 of file InInclude/janaf2TThermoI.H.

6.6.4.12 highCpCoeffs() [2/2]

```
template<class EquationOfState >
const coeffArray & Foam::janaf2TThermo< EquationOfState >::highCpCoeffs ( ) const  [inline]
```

6.6.4.13 Hs() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Hs (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 209 of file InInclude/janaf2TThermoI.H.

6.6.4.14 Hs() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Hs (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.6.4.15 limit() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::limit (
            const scalar TTR ) const  [inline]
```

Definition at line 116 of file InInclude/janaf2TThermoI.H.

6.6.4.16 limit() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::limit (
            const scalar TTR ) const  [inline]
```

6.6.4.17 lowCpCoeffs() [1/2]

```
template<class EquationOfState >
const Foam::janaf2TThermo< EquationOfState >::coeffArray & Foam::janaf2TThermo< EquationOf↩
State >::lowCpCoeffs ( ) const  [inline]
```

Definition at line 172 of file InInclude/janaf2TThermoI.H.

6.6.4.18 lowCpCoeffs() [2/2]

```
template<class EquationOfState >
const coeffArray & Foam::janaf2TThermo< EquationOfState >::lowCpCoeffs ( ) const  [inline]
```

6.6.4.19 operator+=() [1/2]

```
template<class EquationOfState >
void Foam::janaf2TThermo< EquationOfState >::operator+= (
            const janaf2TThermo< EquationOfState > & jt )  [inline]
```

Definition at line 283 of file InInclude/janaf2TThermoI.H.

6.6.4.20 operator+=() [2/2]

```
template<class EquationOfState >
void Foam::janaf2TThermo< EquationOfState >::operator+= (
            const janaf2TThermo< EquationOfState > &  )  [inline]
```

6.6.4.21 S() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::S (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 234 of file InInclude/janaf2TThermoI.H.

6.6.4.22 S() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::S (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.6.4.23 Tcommon() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Tcommon ( ) const  [inline]
```

Definition at line 151 of file InInclude/janaf2TThermoI.H.

6.6.4.24 Tcommon() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Tcommon ( ) const  [inline]
```

6.6.4.25 ThetaVib() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::ThetaVib ( ) const  [inline]
```

Definition at line 157 of file InInclude/janaf2TThermoI.H.

6.6.4.26 ThetaVib() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::ThetaVib ( ) const  [inline]
```

6.6.4.27 Thigh() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Thigh ( ) const  [inline]
```

Definition at line 144 of file InInclude/janaf2TThermoI.H.

6.6.4.28 Thigh() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Thigh ( ) const  [inline]
```

6.6.4.29 Tlow() [1/2]

```
template<class EquationOfState >
Foam::scalar Foam::janaf2TThermo< EquationOfState >::Tlow ( ) const  [inline]
```

Definition at line 137 of file InInclude/janaf2TThermoI.H.

6.6.4.30 Tlow() [2/2]

```
template<class EquationOfState >
scalar Foam::janaf2TThermo< EquationOfState >::Tlow ( ) const  [inline]
```

6.6.4.31 typeName() [1/2]

```
template<class EquationOfState >
static word Foam::janaf2TThermo< EquationOfState >::typeName ( )  [inline], [static]
```

Definition at line 151 of file InInclude/janaf2TThermo.H.

6.6.4.32 typeName() [2/2]

```
template<class EquationOfState >
static word Foam::janaf2TThermo< EquationOfState >::typeName ( )  [inline], [static]
```

Definition at line 151 of file thermo/janaf2T/janaf2TThermo.H.

6.6.4.33 write() [1/2]

```
template<class EquationOfState >
void Foam::janaf2TThermo< EquationOfState >::write (
            Ostream & os ) const
```

Definition at line 86 of file InInclude/janaf2TThermo.C.

6.6.4.34 write() [2/2]

```
template<class EquationOfState >
void Foam::janaf2TThermo< EquationOfState >::write (
            Ostream & os ) const
```

6.6.5 Friends And Related Symbol Documentation

6.6.5.1 operator* [1/2]

```
template<class EquationOfState >
janaf2TThermo operator* (
            const scalar ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.2 operator* [2/2]

```
template<class EquationOfState >
janaf2TThermo operator* (
            const scalar ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.3 operator+ [1/2]

```
template<class EquationOfState >
janaf2TThermo operator+ (
            const janaf2TThermo< EquationOfState > & ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.4 operator+ [2/2]

```
template<class EquationOfState >
janaf2TThermo operator+ (
            const janaf2TThermo< EquationOfState > & ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.5 operator<< [1/2]

```
template<class EquationOfState >
Ostream & operator<< (
            Ostream & ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.6 operator<< [2/2]

```
template<class EquationOfState >
Ostream & operator<< (
            Ostream & ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.7 operator== [1/2]

```
template<class EquationOfState >
janaf2TThermo operator== (
            const janaf2TThermo< EquationOfState > & ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.5.8 operator== [2/2]

```
template<class EquationOfState >
janaf2TThermo operator== (
            const janaf2TThermo< EquationOfState > & ,
            const janaf2TThermo< EquationOfState > &  )  [friend]
```

6.6.6 Member Data Documentation

6.6.6.1 nCoeffs_

```
template<class EquationOfState >
static constexpr int Foam::janaf2TThermo< EquationOfState >::nCoeffs_ = 7  [static], [constexpr]
```

Definition at line 100 of file lnInclude/janaf2TThermo.H.

Referenced by Foam::janaf2TThermo< EquationOfState >::janaf2TThermo(), and Foam::janaf2TThermo< EquationOfState >::jana

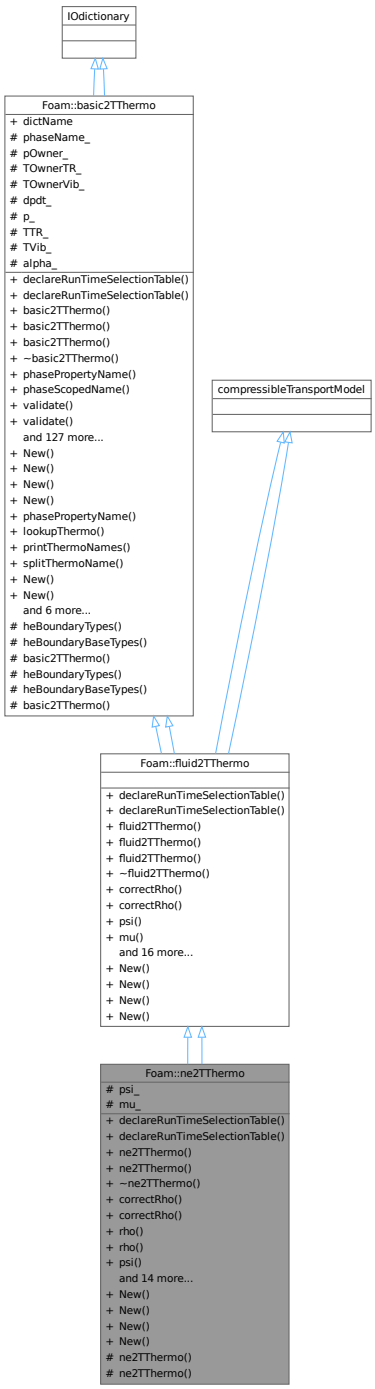
The documentation for this class was generated from the following files:

- lnInclude/janaf2TThermo.H
- thermo/janaf2T/janaf2TThermo.H
- lnInclude/janaf2TThermo.C
- lnInclude/janaf2TThermoI.H
- thermo/janaf2T/janaf2TThermo.C
- thermo/janaf2T/janaf2TThermoI.H

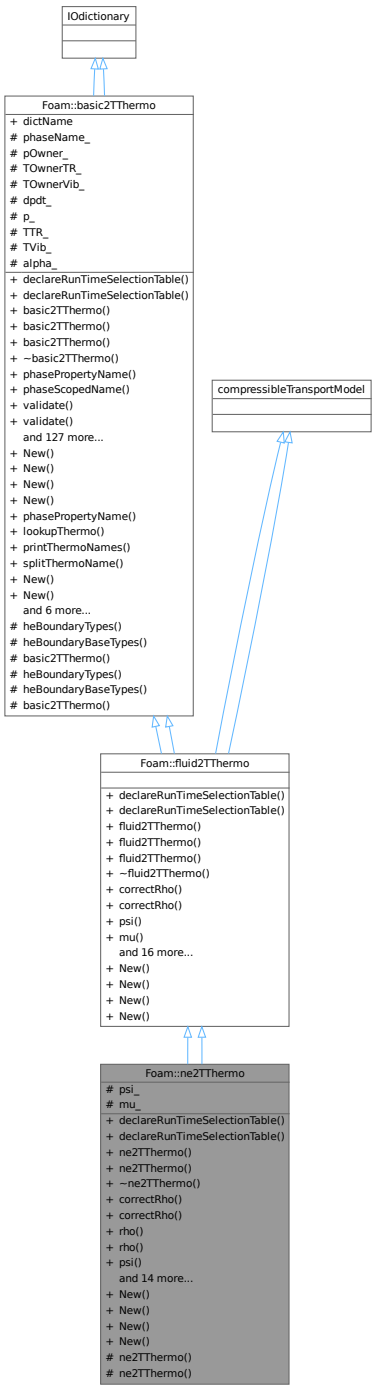
6.7 Foam::ne2TThermo Class Reference

```
#include <ne2TThermo.H>
```

Inheritance diagram for Foam::ne2TThermo:



Collaboration diagram for Foam::ne2TThermo:



Public Member Functions

- declareRunTimeSelectionTable (autoPtr, ne2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, ne2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- ne2TThermo (const fvMesh &, const word &phaseName)

- ne2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~ne2TThermo ()
- virtual void correctRho (const volScalarField &deltaRho, const dimensionedScalar &rhoMin, const dimensionedScalar &rhoMax)
- virtual void correctRho (const volScalarField &deltaRho)
- virtual tmp< volScalarField > rho () const
- virtual tmp< scalarField > rho (const label patchi) const
- virtual const volScalarField & psi () const
- virtual tmp< volScalarField > mu () const
- virtual tmp< scalarField > mu (const label patchi) const
- declareRunTimeSelectionTable (autoPtr, ne2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, ne2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- ne2TThermo (const fvMesh &, const word &phaseName)
- ne2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~ne2TThermo ()
- virtual void correctRho (const volScalarField &deltaRho, const dimensionedScalar &rhoMin, const dimensionedScalar &rhoMax)
- virtual void correctRho (const volScalarField &deltaRho)
- virtual tmp< volScalarField > rho () const
- virtual tmp< scalarField > rho (const label patchi) const
- virtual const volScalarField & psi () const
- virtual tmp< volScalarField > mu () const
- virtual tmp< scalarField > mu (const label patchi) const

Public Member Functions inherited from Foam::fluid2TThermo

- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- fluid2TThermo (const fvMesh &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~fluid2TThermo ()
- virtual tmp< volScalarField > nu () const
- virtual tmp< scalarField > nu (const label patchi) const
- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, fluid2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- fluid2TThermo (const fvMesh &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- fluid2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~fluid2TThermo ()
- virtual tmp< volScalarField > nu () const
- virtual tmp< scalarField > nu (const label patchi) const

Public Member Functions inherited from Foam::basic2TThermo

- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- basic2TThermo (const fvMesh &, const word &phaseName)
- basic2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- basic2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~basic2TThermo ()
- word phasePropertyName (const word &name) const
- word phaseScopedName (const std::string &scope, const word &name) const
- void validate (const string &app, const word &) const
- void validate (const string &app, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &, const word &) const
- virtual void correct ()=0
- virtual word thermoName () const =0
- virtual bool incompressible () const =0
- virtual bool isochoric () const =0
- bool dpdt () const noexcept
- bool updateTTR () const noexcept
- bool updateTVib () const noexcept
- virtual volScalarField & p ()
- virtual const volScalarField & p () const
- virtual volScalarField & he ()=0
- virtual const volScalarField & he () const =0
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > hc () const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const =0
- virtual const volScalarField & TTR () const
- virtual const volScalarField & TVib () const
- virtual volScalarField & TTR ()
- virtual volScalarField & TVib ()
- virtual tmp< volScalarField > CpTR () const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > CvT () const =0
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvR () const =0
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvVib () const =0
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const =0

- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > gamma () const =0
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpvTR () const =0
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpByCpvTR () const =0
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > W () const =0
- virtual const volScalarField & alpha () const
- virtual const scalarField & alpha (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const =0
- virtual tmp< scalarField > kappaTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaVib () const =0
- virtual tmp< scalarField > kappaVib (const label patchi) const =0
- virtual tmp< volScalarField > alphaheTR () const =0
- virtual tmp< scalarField > alphaheTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const =0
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const =0
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const =0
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const =0
- virtual bool read ()
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMesh,(const fvMesh &mesh, const word &phaseName),(mesh, phaseName))
- declareRunTimeSelectionTable (autoPtr, basic2TThermo, fvMeshDictPhase,(const fvMesh &mesh, const word &phaseName, const word &dictName),(mesh, phaseName, dictName))
- basic2TThermo (const fvMesh &, const word &phaseName)
- basic2TThermo (const fvMesh &, const dictionary &, const word &phaseName)
- basic2TThermo (const fvMesh &, const word &phaseName, const word &dictionaryName)
- virtual ~basic2TThermo ()
- word phasePropertyName (const word &name) const
- word phaseScopedName (const std::string &scope, const word &name) const
- void validate (const string &app, const word &) const
- void validate (const string &app, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &) const
- void validate (const string &app, const word &, const word &, const word &, const word &) const
- virtual void correct ()=0
- virtual word thermoName () const =0
- virtual bool incompressible () const =0
- virtual bool isochoric () const =0
- bool dpdt () const noexcept
- bool updateTTR () const noexcept
- bool updateTVib () const noexcept
- virtual volScalarField & p ()
- virtual const volScalarField & p () const
- virtual volScalarField & he ()=0
- virtual const volScalarField & he () const =0
- virtual tmp< volScalarField > he (const volScalarField &p, const volScalarField &TTR) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< scalarField > he (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > hc () const =0

- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const labelList &cells) const =0
- virtual tmp< scalarField > THE_TR (const scalarField &h, const scalarField &p, const scalarField &TTR0, const label patchi) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const labelList &cells) const =0
- virtual tmp< scalarField > TEH_Vib (const scalarField &esVib, const scalarField &p, const scalarField &TTR0, const scalarField &TVib0, const label patchi) const =0
- virtual const volScalarField & TTR () const
- virtual const volScalarField & TVib () const
- virtual volScalarField & TTR ()
- virtual volScalarField & TVib ()
- virtual tmp< volScalarField > CpTR () const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< scalarField > CpTR (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > CvT () const =0
- virtual tmp< scalarField > CvT (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvR () const =0
- virtual tmp< scalarField > CvR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CvVib () const =0
- virtual tmp< scalarField > CvVib (const scalarField &p, const scalarField &TTR, const scalarField &Tvib, const scalar ThetaVib, const label patchi) const =0
- virtual tmp< scalarField > rhoEoS (const scalarField &p, const scalarField &TTR, const labelList &cells) const =0
- virtual tmp< volScalarField > gamma () const =0
- virtual tmp< scalarField > gamma (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpvTR () const =0
- virtual tmp< scalarField > CpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > CpByCpvTR () const =0
- virtual tmp< scalarField > CpByCpvTR (const scalarField &p, const scalarField &TTR, const label patchi) const =0
- virtual tmp< volScalarField > W () const =0
- virtual const volScalarField & alpha () const
- virtual const scalarField & alpha (const label patchi) const
- virtual tmp< volScalarField > kappaTR () const =0
- virtual tmp< scalarField > kappaTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaVib () const =0
- virtual tmp< scalarField > kappaVib (const label patchi) const =0
- virtual tmp< volScalarField > alphaheTR () const =0
- virtual tmp< scalarField > alphaheTR (const label patchi) const =0
- virtual tmp< volScalarField > kappaEff (const volScalarField &) const =0
- virtual tmp< scalarField > kappaEff (const scalarField &alphat, const label patchi) const =0
- virtual tmp< volScalarField > alphaEff (const volScalarField &alphat) const =0
- virtual tmp< scalarField > alphaEff (const scalarField &alphat, const label patchi) const =0
- virtual bool read ()
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const word &phaseName)
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const dictionary &dict, const word &phaseName)
- template<class TThermo >
  Foam::autoPtr< TThermo > New (const fvMesh &mesh, const word &phaseName, const word &dictName)

Static Public Member Functions

- static autoPtr< ne2TThermo > New (const fvMesh &mesh, const word &phaseName=word::null)
- static autoPtr< ne2TThermo > New (const fvMesh &, const word &dictName, const word &phaseName)
- static autoPtr< ne2TThermo > New (const fvMesh &mesh, const word &phaseName=word::null)
- static autoPtr< ne2TThermo > New (const fvMesh &, const word &dictName, const word &phaseName)

Static Public Member Functions inherited from Foam::fluid2TThermo

- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static autoPtr< fluid2TThermo > New (const fvMesh &, const word &phaseName, const word &dictName)

Static Public Member Functions inherited from Foam::basic2TThermo

- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const dictionary &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< basic2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static word phasePropertyName (const word &name, const word &phaseName)
- static const basic2TThermo & lookupThermo (const fvPatchScalarField &pf)
- static Ostream & printThermoNames (Ostream &os, const wordList &cmptNames, const wordList &thermoNames)
- static wordList splitThermoName (const std::string &thermoName, const int nExpectedCmpts)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const dictionary &, const word &phaseName=word::null)
- template<class Thermo >
  static autoPtr< Thermo > New (const fvMesh &, const word &phaseName, const word &dictName)
- static autoPtr< basic2TThermo > New (const fvMesh &, const word &phaseName=word::null)
- static word phasePropertyName (const word &name, const word &phaseName)
- static const basic2TThermo & lookupThermo (const fvPatchScalarField &pf)
- static Ostream & printThermoNames (Ostream &os, const wordList &cmptNames, const wordList &thermoNames)
- static wordList splitThermoName (const std::string &thermoName, const int nExpectedCmpts)

Protected Member Functions

- ne2TThermo (const ne2TThermo &)
- ne2TThermo (const ne2TThermo &)

Protected Member Functions inherited from Foam::basic2TThermo

- wordList heBoundaryTypes ()
- wordList heBoundaryBaseTypes ()
- basic2TThermo (const basic2TThermo &)=delete
- wordList heBoundaryTypes ()
- wordList heBoundaryBaseTypes ()
- basic2TThermo (const basic2TThermo &)=delete

Protected Attributes

- volScalarField psi_
- volScalarField mu_

Protected Attributes inherited from Foam::basic2TThermo

- const word & phaseName_
- bool pOwner_
- bool TOwnerTR_
- bool TOwnerVib_
- bool dpdt_
- volScalarField & p_
- volScalarField & TTR_
- volScalarField & TVib_
- volScalarField alpha_

Additional Inherited Members

Static Public Attributes inherited from Foam::basic2TThermo

- static const word dictName

6.7.1 Detailed Description

Definition at line 56 of file lnInclude/ne2TThermo.H.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 ne2TThermo() [1/6]

```
Foam::ne2TThermo::ne2TThermo (
            const ne2TThermo &  )  [protected]
```

6.7.2.2 ne2TThermo() [2/6]

```
Foam::ne2TThermo::ne2TThermo (
            const fvMesh & mesh,
            const word & phaseName )
```

Definition at line 43 of file lnInclude/ne2TThermo.C.

6.7.2.3 ne2TThermo() [3/6]

```
Foam::ne2TThermo::ne2TThermo (
            const fvMesh & mesh,
            const word & phaseName,
            const word & dictionaryName )
```

Definition at line 79 of file lnInclude/ne2TThermo.C.

6.7.2.4 ~ne2TThermo() [1/2]

```
Foam::ne2TThermo::~ne2TThermo ( )  [virtual]
```

Definition at line 144 of file InInclude/ne2TThermo.C.

6.7.2.5 ne2TThermo() [4/6]

```
Foam::ne2TThermo::ne2TThermo (
            const ne2TThermo &  )  [protected]
```

6.7.2.6 ne2TThermo() [5/6]

```
Foam::ne2TThermo::ne2TThermo (
            const fvMesh & ,
            const word & phaseName )
```

6.7.2.7 ne2TThermo() [6/6]

```
Foam::ne2TThermo::ne2TThermo (
            const fvMesh & ,
            const word & phaseName,
            const word & dictionaryName )
```

6.7.2.8 ~ne2TThermo() [2/2]

```
virtual Foam::ne2TThermo::~ne2TThermo ( )  [virtual]
```

6.7.3 Member Function Documentation

6.7.3.1 correctRho() [1/4]

```
virtual void Foam::ne2TThermo::correctRho (
            const volScalarField & deltaRho )  [virtual]
```

Implements Foam::fluid2TThermo.

6.7.3.2 correctRho() [2/4]

```
virtual void Foam::ne2TThermo::correctRho (
            const volScalarField & deltaRho )  [virtual]
```

Implements Foam::fluid2TThermo.

6.7.3.3 correctRho() [3/4]

```
virtual void Foam::ne2TThermo::correctRho (
            const volScalarField & deltaRho,
            const dimensionedScalar & rhoMin,
            const dimensionedScalar & rhoMax )  [virtual]
```

Implements Foam::fluid2TThermo.

6.7.3.4 correctRho() [4/4]

```
virtual void Foam::ne2TThermo::correctRho (
            const volScalarField & deltaRho,
            const dimensionedScalar & rhoMin,
            const dimensionedScalar & rhoMax )  [virtual]
```

Implements Foam::fluid2TThermo.

6.7.3.5 declareRunTimeSelectionTable() [1/4]

```
Foam::ne2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            ne2TThermo ,
            fvMesh ,
            (const fvMesh &mesh, const word &phaseName) ,
            (mesh, phaseName)  )
```

6.7.3.6 declareRunTimeSelectionTable() [2/4]

```
Foam::ne2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            ne2TThermo ,
            fvMesh ,
            (const fvMesh &mesh, const word &phaseName) ,
            (mesh, phaseName)  )
```

6.7.3.7 declareRunTimeSelectionTable() [3/4]

```
Foam::ne2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            ne2TThermo ,
            fvMeshDictPhase ,
            (const fvMesh &mesh, const word &phaseName, const word &dictName) ,
            (mesh, phaseName, dictName)  )
```

6.7.3.8 declareRunTimeSelectionTable() [4/4]

```
Foam::ne2TThermo::declareRunTimeSelectionTable (
            autoPtr ,
            ne2TThermo ,
            fvMeshDictPhase ,
            (const fvMesh &mesh, const word &phaseName, const word &dictName) ,
            (mesh, phaseName, dictName)  )
```

6.7.3.9 mu() [1/4]

```
Foam::tmp< Foam::volScalarField > Foam::ne2TThermo::mu ( ) const  [virtual]
```

Implements Foam::fluid2TThermo.

Definition at line 182 of file lnInclude/ne2TThermo.C.

6.7.3.10 mu() [2/4]

```
virtual tmp< volScalarField > Foam::ne2TThermo::mu ( ) const  [virtual]
```

Implements Foam::fluid2TThermo.

6.7.3.11 mu() [3/4]

```
Foam::tmp< Foam::scalarField > Foam::ne2TThermo::mu (
            const label patchi ) const  [virtual]
```

Implements Foam::fluid2TThermo.

Definition at line 189 of file lnInclude/ne2TThermo.C.

6.7.3.12 mu() [4/4]

```
virtual tmp< scalarField > Foam::ne2TThermo::mu (
            const label patchi ) const  [virtual]
```

Implements Foam::fluid2TThermo.

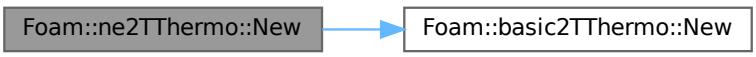
6.7.3.13 New() [1/4]

```
Foam::autoPtr< Foam::ne2TThermo > Foam::ne2TThermo::New (
            const fvMesh & mesh,
            const word & dictName,
            const word & phaseName )  [static]
```

Definition at line 131 of file lnInclude/ne2TThermo.C.

References Foam::basic2TThermo::New().

Here is the call graph for this function:



6.7.3.14 New() [2/4]

```
static autoPtr< ne2TThermo > Foam::ne2TThermo::New (
            const fvMesh & ,
            const word & dictName,
            const word & phaseName )  [static]
```

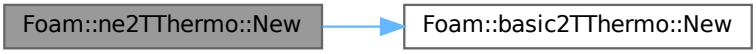
6.7.3.15 New() [3/4]

```
Foam::autoPtr< Foam::ne2TThermo > Foam::ne2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName = word::null )  [static]
```

Definition at line 121 of file InInclude/ne2TThermo.C.

References Foam::basic2TThermo::New().

Here is the call graph for this function:



6.7.3.16 New() [4/4]

```
static autoPtr< ne2TThermo > Foam::ne2TThermo::New (
            const fvMesh & mesh,
            const word & phaseName = word::null )  [static]
```

6.7.3.17 psi() [1/2]

```
const Foam::volScalarField & Foam::ne2TThermo::psi ( ) const  [virtual]
```

Implements Foam::fluid2TThermo.

Definition at line 176 of file InInclude/ne2TThermo.C.

6.7.3.18 psi() [2/2]

```
virtual const volScalarField & Foam::ne2TThermo::psi ( ) const  [virtual]
```

Implements Foam::fluid2TThermo.

6.7.3.19 rho() [1/4]

```
Foam::tmp< Foam::volScalarField > Foam::ne2TThermo::rho ( ) const  [virtual]
```

Implements Foam::basic2TThermo.

Definition at line 150 of file lnInclude/ne2TThermo.C.

6.7.3.20 rho() [2/4]

```
virtual tmp< volScalarField > Foam::ne2TThermo::rho ( ) const  [virtual]
```

Implements Foam::basic2TThermo.

6.7.3.21 rho() [3/4]

```
Foam::tmp< Foam::scalarField > Foam::ne2TThermo::rho (
            const label patchi ) const  [virtual]
```

Implements Foam::basic2TThermo.

Definition at line 156 of file lnInclude/ne2TThermo.C.

6.7.3.22 rho() [4/4]

```
virtual tmp< scalarField > Foam::ne2TThermo::rho (
            const label patchi ) const  [virtual]
```

Implements Foam::basic2TThermo.

6.7.4 Member Data Documentation

6.7.4.1 mu_

```
volScalarField Foam::ne2TThermo::mu_  [protected]
```

Definition at line 69 of file lnInclude/ne2TThermo.H.

6.7.4.2 psi_

```
volScalarField Foam::ne2TThermo::psi_  [protected]
```

Definition at line 66 of file lnInclude/ne2TThermo.H.

The documentation for this class was generated from the following files:

- lnInclude/ne2TThermo.H
- ne2TThermo/ne2TThermo.H
- lnInclude/ne2TThermo.C
- ne2TThermo/ne2TThermo.C

6.8 Foam::sensibleEnthalpy2T< Thermo2T > Class Template Reference

```
#include <sensibleEnthalpy2T.H>
```

Collaboration diagram for Foam::sensibleEnthalpy2T< Thermo2T >:

Public Member Functions

- sensibleEnthalpy2T ()
- scalar CpvTR (const Thermo2T &thermo2T, const scalar p, const scalar TTR) const
- scalar CpByCpvTR (const Thermo2T &thermo2T, const scalar p, const scalar TTR) const
- scalar HE (const Thermo2T &thermo2T, const scalar p, const scalar TTR) const
- scalar THE_TR (const Thermo2T &thermo2T, const scalar h, const scalar p, const scalar TTR0) const
- scalar TEH_Vib (const Thermo2T &thermo2T, const scalar eVib, const scalar p, const scalar TTR0, const scalar TVib0) const
- sensibleEnthalpy2T ()
- scalar CpvTR (const Thermo2T &thermo2T, const scalar p, const scalar TTR) const
- scalar CpByCpvTR (const Thermo2T &thermo2T, const scalar p, const scalar TTR) const
- scalar HE (const Thermo2T &thermo2T, const scalar p, const scalar TTR) const
- scalar THE_TR (const Thermo2T &thermo2T, const scalar h, const scalar p, const scalar TTR0) const
- scalar TEH_Vib (const Thermo2T &thermo2T, const scalar eVib, const scalar p, const scalar TTR0, const scalar TVib0) const

Static Public Member Functions

- static word typeName ()
- static word energyName ()
- static word typeName ()
- static word energyName ()

6.8.1 Detailed Description

template<class Thermo2T>
class Foam::sensibleEnthalpy2T< Thermo2T >

Definition at line 50 of file lnInclude/sensibleEnthalpy2T.H.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 sensibleEnthalpy2T() [1/2]

```
template<class Thermo2T >
Foam::sensibleEnthalpy2T< Thermo2T >::sensibleEnthalpy2T ( )  [inline]
```

Definition at line 58 of file lnInclude/sensibleEnthalpy2T.H.

6.8.2.2 sensibleEnthalpy2T() [2/2]

```
template<class Thermo2T >
Foam::sensibleEnthalpy2T< Thermo2T >::sensibleEnthalpy2T ( )  [inline]
```

Definition at line 58 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3 Member Function Documentation

6.8.3.1 CpByCpvTR() [1/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::CpByCpvTR (
            const Thermo2T & thermo2T,
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 96 of file lnInclude/sensibleEnthalpy2T.H.

6.8.3.2 CpByCpvTR() [2/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::CpByCpvTR (
            const Thermo2T & thermo2T,
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 96 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3.3 CpvTR() [1/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::CpvTR (
            const Thermo2T & thermo2T,
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 78 of file InInclude/sensibleEnthalpy2T.H.

6.8.3.4 CpvTR() [2/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::CpvTR (
            const Thermo2T & thermo2T,
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 78 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3.5 energyName() [1/2]

```
template<class Thermo2T >
static word Foam::sensibleEnthalpy2T< Thermo2T >::energyName ( )  [inline], [static]
```

Definition at line 72 of file InInclude/sensibleEnthalpy2T.H.

6.8.3.6 energyName() [2/2]

```
template<class Thermo2T >
static word Foam::sensibleEnthalpy2T< Thermo2T >::energyName ( )  [inline], [static]
```

Definition at line 72 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3.7 HE() [1/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::HE (
            const Thermo2T & thermo2T,
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 107 of file InInclude/sensibleEnthalpy2T.H.

6.8.3.8 HE() [2/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::HE (
            const Thermo2T & thermo2T,
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 107 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3.9 TEH_Vib() [1/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::TEH_Vib (
            const Thermo2T & thermo2T,
            const scalar eVib,
            const scalar p,
            const scalar TTR0,
            const scalar TVib0 ) const  [inline]
```

Definition at line 146 of file InInclude/sensibleEnthalpy2T.H.

6.8.3.10 TEH_Vib() [2/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::TEH_Vib (
            const Thermo2T & thermo2T,
            const scalar eVib,
            const scalar p,
            const scalar TTR0,
            const scalar TVib0 ) const  [inline]
```

Definition at line 146 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3.11 THE_TR() [1/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::THE_TR (
            const Thermo2T & thermo2T,
            const scalar h,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

Definition at line 126 of file InInclude/sensibleEnthalpy2T.H.

6.8.3.12 THE_TR() [2/2]

```
template<class Thermo2T >
scalar Foam::sensibleEnthalpy2T< Thermo2T >::THE_TR (
            const Thermo2T & thermo2T,
            const scalar h,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

Definition at line 126 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

6.8.3.13 typeName() [1/2]

```
template<class Thermo2T >
static word Foam::sensibleEnthalpy2T< Thermo2T >::typeName ( )  [inline], [static]
```

Definition at line 65 of file InInclude/sensibleEnthalpy2T.H.

6.8.3.14 typeName() [2/2]

```
template<class Thermo2T >
static word Foam::sensibleEnthalpy2T< Thermo2T >::typeName ( ) [inline], [static]
```

Definition at line 65 of file thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H.

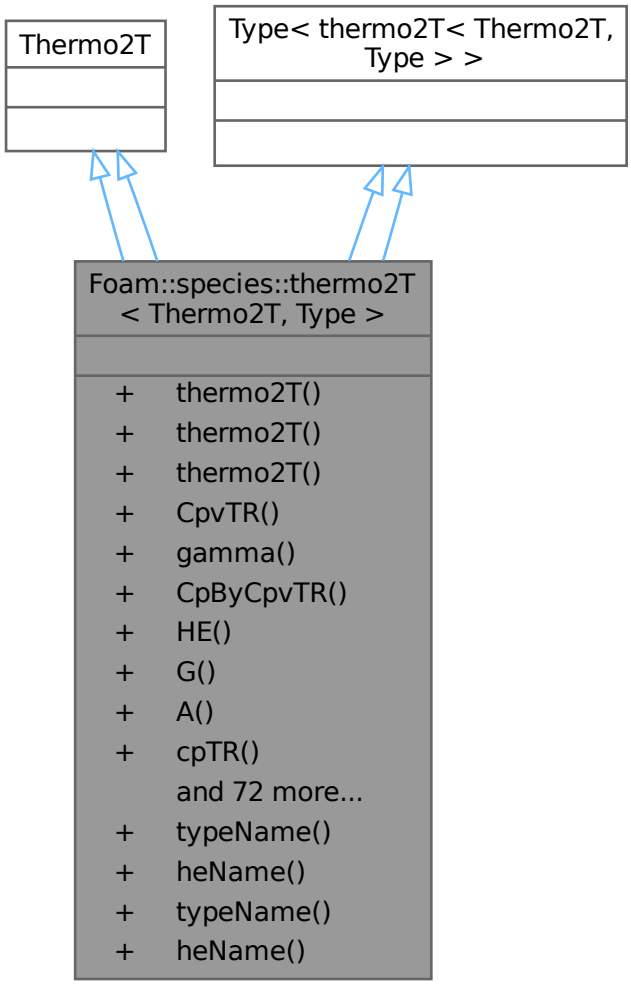
The documentation for this class was generated from the following files:

- InInclude/sensibleEnthalpy2T.H
- thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H

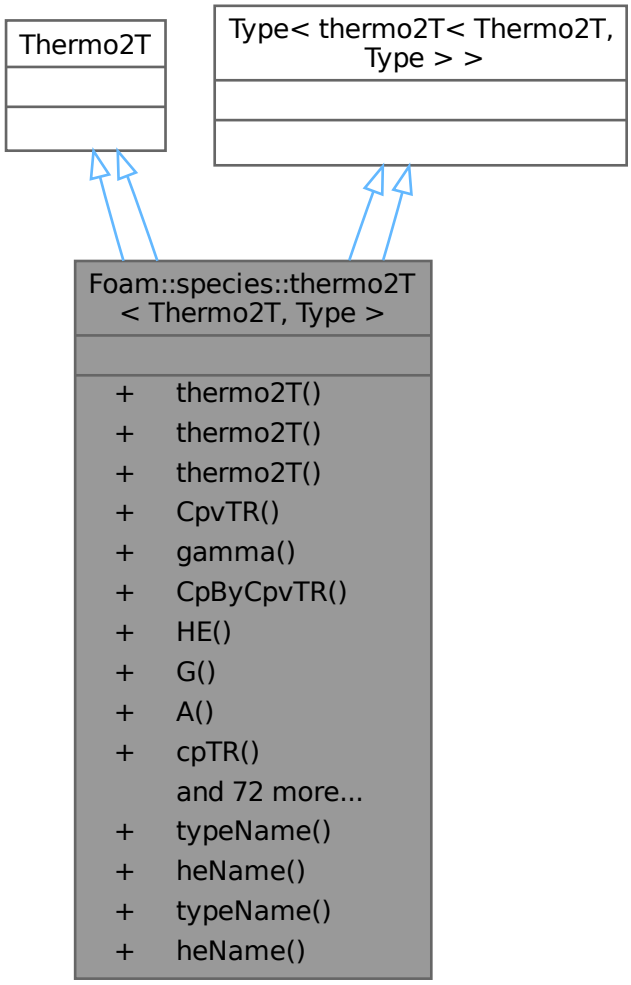
6.9 Foam::species::thermo2T< Thermo2T, Type > Class Template Reference

```
#include <thermo2T.H>
```

Inheritance diagram for Foam::species::thermo2T< Thermo2T, Type >:



Collaboration diagram for Foam::species::thermo2T< Thermo2T, Type >:



Public Types

- typedef thermo2T< Thermo2T, Type > thermoType
- typedef thermo2T< Thermo2T, Type > thermoType

Public Member Functions

- thermo2T (const Thermo2T &sp)
- thermo2T (const dictionary &dict)
- thermo2T (const word &name, const thermo2T &)
- scalar CpvTR (const scalar p, const scalar TTR) const
- scalar gamma (const scalar p, const scalar TTR) const
- scalar CpByCpvTR (const scalar p, const scalar TTR) const
- scalar HE (const scalar p, const scalar TTR) const

- scalar G (const scalar p, const scalar TTR) const
- scalar A (const scalar p, const scalar TTR) const
- scalar cpTR (const scalar p, const scalar TTR) const
- scalar ha (const scalar p, const scalar TTR) const
- scalar hs (const scalar p, const scalar TTR) const
- scalar hc () const
- scalar s (const scalar p, const scalar TTR) const
- scalar he (const scalar p, const scalar TTR) const
- scalar cvT (const scalar p, const scalar TTR) const
- scalar cvR (const scalar p, const scalar TTR) const
- scalar cvVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar esT (const scalar p, const scalar TTR) const
- scalar eaT (const scalar p, const scalar TTR) const
- scalar esR (const scalar p, const scalar TTR) const
- scalar esVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar g (const scalar p, const scalar TTR) const
- scalar a (const scalar p, const scalar TTR) const
- scalar K (const scalar p, const scalar TTR) const
- scalar Kp (const scalar p, const scalar TTR) const
- scalar Kc (const scalar p, const scalar TTR) const
- scalar Kx (const scalar p, const scalar TTR) const
- scalar Kn (const scalar p, const scalar TTR, const scalar n) const
- scalar THE_TR (const scalar H, const scalar p, const scalar TTR0) const
- scalar TEH_Vib (const scalar esVib, const scalar p, const scalar TTR0, const scalar TVib0) const
- scalar THs_TR (const scalar Hs, const scalar p, const scalar TTR0) const
- scalar THa_TR (const scalar H, const scalar p, const scalar TTR0) const
- scalar TEs_TR (const scalar E, const scalar p, const scalar TTR0) const
- scalar TEs_Vib (const scalar E, const scalar p, const scalar TTR0, const scalar TVib0) const
- scalar TEa_TR (const scalar E, const scalar p, const scalar TTR0) const
- scalar dKcdTbyKc (const scalar p, const scalar TTR) const
- scalar dcpdT (const scalar p, const scalar TTR) const
- void write (Ostream &os) const
- void operator+= (const thermo2T &)
- void operator∗= (const scalar)
- thermo2T (const Thermo2T &sp)
- thermo2T (const dictionary &dict)
- thermo2T (const word &name, const thermo2T &)
- scalar CpvTR (const scalar p, const scalar TTR) const
- scalar gamma (const scalar p, const scalar TTR) const
- scalar CpByCpvTR (const scalar p, const scalar TTR) const
- scalar HE (const scalar p, const scalar TTR) const
- scalar G (const scalar p, const scalar TTR) const
- scalar A (const scalar p, const scalar TTR) const
- scalar cpTR (const scalar p, const scalar TTR) const
- scalar ha (const scalar p, const scalar TTR) const
- scalar hs (const scalar p, const scalar TTR) const
- scalar hc () const
- scalar s (const scalar p, const scalar TTR) const
- scalar he (const scalar p, const scalar TTR) const
- scalar cvT (const scalar p, const scalar TTR) const
- scalar cvR (const scalar p, const scalar TTR) const
- scalar cvVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar esT (const scalar p, const scalar TTR) const
- scalar eaT (const scalar p, const scalar TTR) const
- scalar esR (const scalar p, const scalar TTR) const

- scalar esVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar g (const scalar p, const scalar TTR) const
- scalar a (const scalar p, const scalar TTR) const
- scalar K (const scalar p, const scalar TTR) const
- scalar Kp (const scalar p, const scalar TTR) const
- scalar Kc (const scalar p, const scalar TTR) const
- scalar Kx (const scalar p, const scalar TTR) const
- scalar Kn (const scalar p, const scalar TTR, const scalar n) const
- scalar THE_TR (const scalar H, const scalar p, const scalar TTR0) const
- scalar TEH_Vib (const scalar esVib, const scalar p, const scalar TTR0, const scalar TVib0) const
- scalar THs_TR (const scalar Hs, const scalar p, const scalar TTR0) const
- scalar THa_TR (const scalar H, const scalar p, const scalar TTR0) const
- scalar TEs_TR (const scalar E, const scalar p, const scalar TTR0) const
- scalar TEs_Vib (const scalar E, const scalar p, const scalar TTR0, const scalar TVib0) const
- scalar TEa_TR (const scalar E, const scalar p, const scalar TTR0) const
- scalar dKcdTbyKc (const scalar p, const scalar TTR) const
- scalar dcpdT (const scalar p, const scalar TTR) const
- void write (Ostream &os) const
- void operator+= (const thermo2T &)
- void operator∗= (const scalar)

Static Public Member Functions

- static word typeName ()
- static word heName ()
- static word typeName ()
- static word heName ()

Friends

- thermo2T operator+ (const thermo2T &, const thermo2T &)
- thermo2T operator∗ (const scalar s, const thermo2T &)
- thermo2T operator== (const thermo2T &, const thermo2T &)
- Ostream & operator (Ostream &, const thermo2T &)
- thermo2T operator+ (const thermo2T &, const thermo2T &)
- thermo2T operator∗ (const scalar s, const thermo2T &)
- thermo2T operator== (const thermo2T &, const thermo2T &)
- Ostream & operator (Ostream &, const thermo2T &)

6.9.1 Detailed Description

template<class Thermo2T, template< class > class Type>
class Foam::species::thermo2T< Thermo2T, Type >

Definition at line 92 of file thermo/thermo2T/thermo2T.H.

6.9.2 Member Typedef Documentation

6.9.2.1 thermoType [1/2]

```
template<class Thermo2T , template< class > class Type>
typedef thermo2T<Thermo2T, Type> Foam::species::thermo2T< Thermo2T, Type >::thermoType
```

Definition at line 136 of file lnInclude/thermo2T.H.

6.9.2.2 thermoType [2/2]

```
template<class Thermo2T , template< class > class Type>
typedef thermo2T<Thermo2T, Type> Foam::species::thermo2T< Thermo2T, Type >::thermoType
```

Definition at line 136 of file thermo/thermo2T/thermo2T.H.

6.9.3 Constructor & Destructor Documentation

6.9.3.1 thermo2T() [1/6]

```
template<class Thermo2T , template< class > class Type>
Foam::species::thermo2T< Thermo2T, Type >::thermo2T (
            const Thermo2T & sp )  [inline]
```

Definition at line 34 of file lnInclude/thermo2TI.H.

6.9.3.2 thermo2T() [2/6]

```
template<class Thermo2T , template< class > class Type>
Foam::species::thermo2T< Thermo2T, Type >::thermo2T (
            const dictionary & dict )
```

Definition at line 42 of file lnInclude/thermo2T.C.

6.9.3.3 thermo2T() [3/6]

```
template<class Thermo2T , template< class > class Type>
Foam::species::thermo2T< Thermo2T, Type >::thermo2T (
            const word & name,
            const thermo2T< Thermo2T, Type > & st )  [inline]
```

Definition at line 144 of file lnInclude/thermo2TI.H.

6.9.3.4 thermo2T() [4/6]

```
template<class Thermo2T , template< class > class Type>
Foam::species::thermo2T< Thermo2T, Type >::thermo2T (
            const Thermo2T & sp )  [inline]
```

6.9.3.5 thermo2T() [5/6]

```
template<class Thermo2T , template< class > class Type>
Foam::species::thermo2T< Thermo2T, Type >::thermo2T (
            const dictionary & dict )
```

6.9.3.6 thermo2T() [6/6]

```
template<class Thermo2T , template< class > class Type>
Foam::species::thermo2T< Thermo2T, Type >::thermo2T (
            const word & name,
            const thermo2T< Thermo2T, Type > &  )  [inline]
```

6.9.4 Member Function Documentation

6.9.4.1 A() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::A (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 216 of file InInclude/thermo2TI.H.

6.9.4.2 a() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::a (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 330 of file InInclude/thermo2TI.H.

6.9.4.3 A() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::A (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.4 a() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::a (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.5 CpByCpvTR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::CpByCpvTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 188 of file InInclude/thermo2TI.H.

6.9.4.6 CpByCpvTR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::CpByCpvTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.7 cpTR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::cpTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 224 of file InInclude/thermo2TI.H.

References CpTR().

Here is the call graph for this function:

6.9.4.8 cpTR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::cpTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.9 CpvTR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::CpvTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 166 of file InInclude/thermo2TI.H.

6.9.4.10 CpvTR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::CpvTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.11 cvR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::cvR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 279 of file InInclude/thermo2TI.H.

References CvR().

Here is the call graph for this function:

6.9.4.12 cvR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::cvR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.13 cvT() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::cvT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 272 of file InInclude/thermo2TI.H.

References CvT().

Here is the call graph for this function:

6.9.4.14 cvT() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::cvT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

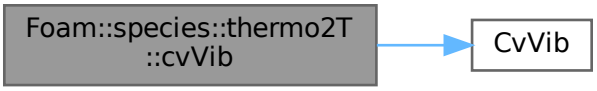
6.9.4.15 cvVib() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::cvVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

Definition at line 286 of file InInclude/thermo2TI.H.

References CvVib().

Here is the call graph for this function:



6.9.4.16 cvVib() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::cvVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

6.9.4.17 dcpdT() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::dcpdT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 569 of file InInclude/thermo2TI.H.

6.9.4.18 dcpdT() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::dcpdT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.19 dKcdTbyKc() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::dKcdTbyKc (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 546 of file InInclude/thermo2TI.H.

6.9.4.20 dKcdTbyKc() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::dKcdTbyKc (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.21 eaT() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::eaT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 314 of file InInclude/thermo2TI.H.

References EaT().

Here is the call graph for this function:

6.9.4.22 eaT() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::eaT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

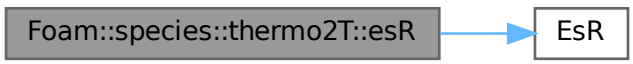
6.9.4.23 esR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::esR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 300 of file InInclude/thermo2TI.H.

References EsR().

Here is the call graph for this function:



6.9.4.24 esR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::esR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

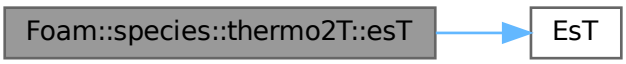
6.9.4.25 esT() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::esT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 293 of file InInclude/thermo2TI.H.

References EsT().

Here is the call graph for this function:



6.9.4.26 esT() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::esT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

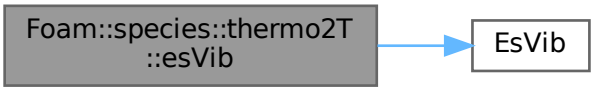
6.9.4.27 esVib() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::esVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

Definition at line 307 of file InInclude/thermo2TI.H.

References EsVib().

Here is the call graph for this function:



6.9.4.28 esVib() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::esVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

6.9.4.29 G() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::G (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 208 of file InInclude/thermo2TI.H.

6.9.4.30 g() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::g (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 322 of file InInclude/thermo2TI.H.

6.9.4.31 G() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::G (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.32 g() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::g (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.33 gamma() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::gamma (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 174 of file InInclude/thermo2TI.H.

References CpTR().

Here is the call graph for this function:

6.9.4.34 gamma() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::gamma (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.35 ha() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::ha (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 232 of file InInclude/thermo2TI.H.

6.9.4.36 ha() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::ha (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.37 hc() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::hc ( ) const  [inline]
```

Definition at line 248 of file InInclude/thermo2TI.H.

6.9.4.38 hc() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::hc ( ) const  [inline]
```

6.9.4.39 HE() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::HE (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 200 of file InInclude/thermo2TI.H.

6.9.4.40 he() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::he (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 264 of file InInclude/thermo2TI.H.

6.9.4.41 HE() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::HE (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.42 he() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::he (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.43 heName() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::word Foam::species::thermo2T< Thermo2T, Type >::heName ( )  [inline], [static]
```

Definition at line 158 of file InInclude/thermo2TI.H.

6.9.4.44 heName() [2/2]

```
template<class Thermo2T , template< class > class Type>
static word Foam::species::thermo2T< Thermo2T, Type >::heName ( )  [inline], [static]
```

6.9.4.45 hs() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::hs (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 240 of file InInclude/thermo2TI.H.

6.9.4.46 hs() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::hs (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.47 K() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::K (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 338 of file InInclude/thermo2TI.H.

6.9.4.48 K() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::K (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.49 Kc() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::Kc (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 363 of file InInclude/thermo2TI.H.

6.9.4.50 Kc() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::Kc (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.51 Kn() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::Kn (
            const scalar p,
            const scalar TTR,
            const scalar n ) const  [inline]
```

Definition at line 399 of file InInclude/thermo2TI.H.

6.9.4.52 Kn() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::Kn (
            const scalar p,
            const scalar TTR,
            const scalar n ) const  [inline]
```

6.9.4.53 Kp() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::Kp (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 355 of file InInclude/thermo2TI.H.

6.9.4.54 Kp() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::Kp (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.55 Kx() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::Kx (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 379 of file InInclude/thermo2TI.H.

6.9.4.56 Kx() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::Kx (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.57 operator*=() [1/2]

```
template<class Thermo2T , template< class > class Type>
void Foam::species::thermo2T< Thermo2T, Type >::operator*= (
            const scalar s )  [inline]
```

Definition at line 587 of file InInclude/thermo2TI.H.

6.9.4.58 operator*=() [2/2]

```
template<class Thermo2T , template< class > class Type>
void Foam::species::thermo2T< Thermo2T, Type >::operator*= (
            const scalar  )  [inline]
```

6.9.4.59 operator+=() [1/2]

```
template<class Thermo2T , template< class > class Type>
void Foam::species::thermo2T< Thermo2T, Type >::operator+= (
            const thermo2T< Thermo2T, Type > & st )  [inline]
```

Definition at line 577 of file InInclude/thermo2TI.H.

6.9.4.60 operator+=() [2/2]

```
template<class Thermo2T , template< class > class Type>
void Foam::species::thermo2T< Thermo2T, Type >::operator+= (
            const thermo2T< Thermo2T, Type > &  )  [inline]
```

6.9.4.61 s() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::s (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 256 of file InInclude/thermo2TI.H.

6.9.4.62 s() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::s (
            const scalar p,
            const scalar TTR ) const  [inline]
```

6.9.4.63 TEa_TR() [1/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::TEa_TR (
            const scalar E,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

6.9.4.64 TEa_TR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::TEa_TR (
            const scalar E,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

6.9.4.65 TEH_Vib() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::TEH_Vib (
            const scalar esVib,
            const scalar p,
            const scalar TTR0,
            const scalar TVib0 ) const  [inline]
```

Definition at line 431 of file InInclude/thermo2TI.H.

6.9.4.66 TEH_Vib() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::TEH_Vib (
            const scalar esVib,
            const scalar p,
            const scalar TTR0,
            const scalar TVib0 ) const  [inline]
```

6.9.4.67 TEs_TR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::TEs_TR (
            const scalar E,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

Definition at line 484 of file InInclude/thermo2TI.H.

6.9.4.68 TEs_TR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::TEs_TR (
            const scalar E,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

6.9.4.69 TEs_Vib() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::TEs_Vib (
            const scalar E,
            const scalar p,
            const scalar TTR0,
            const scalar TVib0 ) const  [inline]
```

Definition at line 503 of file InInclude/thermo2TI.H.

6.9.4.70 TEs_Vib() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::TEs_Vib (
            const scalar E,
            const scalar p,
            const scalar TTR0,
            const scalar TVib0 ) const  [inline]
```

6.9.4.71 THa_TR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::THa_TR (
            const scalar H,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

Definition at line 464 of file InInclude/thermo2TI.H.

6.9.4.72 THa_TR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::THa_TR (
            const scalar H,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

6.9.4.73 THE_TR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::THE_TR (
            const scalar H,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

Definition at line 420 of file InInclude/thermo2TI.H.

6.9.4.74 THE_TR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::THE_TR (
            const scalar H,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

6.9.4.75 THs_TR() [1/2]

```
template<class Thermo2T , template< class > class Type>
Foam::scalar Foam::species::thermo2T< Thermo2T, Type >::THs_TR (
            const scalar Hs,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

Definition at line 444 of file InInclude/thermo2TI.H.

6.9.4.76 THs_TR() [2/2]

```
template<class Thermo2T , template< class > class Type>
scalar Foam::species::thermo2T< Thermo2T, Type >::THs_TR (
            const scalar Hs,
            const scalar p,
            const scalar TTR0 ) const  [inline]
```

6.9.4.77 typeName() [1/2]

```
template<class Thermo2T , template< class > class Type>
static word Foam::species::thermo2T< Thermo2T, Type >::typeName ( )  [inline], [static]
```

Definition at line 154 of file InInclude/thermo2T.H.

References Foam::species::thermo2T< Thermo2T, Type >::typeName().

Referenced by Foam::species::thermo2T< Thermo2T, Type >::typeName().

Here is the call graph for this function:

Here is the caller graph for this function:

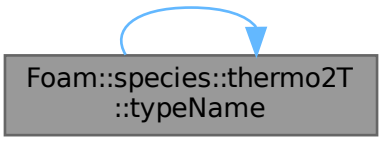
6.9.4.78 typeName() [2/2]

```
template<class Thermo2T , template< class > class Type>
static word Foam::species::thermo2T< Thermo2T, Type >::typeName ( )  [inline], [static]
```

Definition at line 154 of file thermo/thermo2T/thermo2T.H.

References Foam::species::thermo2T< Thermo2T, Type >::typeName().

Here is the call graph for this function:



6.9.4.79 write() [1/2]

```
template<class Thermo2T , template< class > class Type>
void Foam::species::thermo2T< Thermo2T, Type >::write (
            Ostream & os ) const
```

Definition at line 51 of file InInclude/thermo2T.C.

6.9.4.80 write() [2/2]

```
template<class Thermo2T , template< class > class Type>
void Foam::species::thermo2T< Thermo2T, Type >::write (
            Ostream & os ) const
```

6.9.5 Friends And Related Symbol Documentation

6.9.5.1 operator [1/2]

```
template<class Thermo2T , template< class > class Type>
Ostream & operator (
            Ostream & ,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.2 operator [2/2]

```
template<class Thermo2T , template< class > class Type>
Ostream & operator (
            Ostream & ,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.3 operator* [1/2]

```
template<class Thermo2T , template< class > class Type>
thermo2T operator* (
            const scalar s,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.4 operator* [2/2]

```
template<class Thermo2T , template< class > class Type>
thermo2T operator* (
            const scalar s,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.5 operator+ [1/2]

```
template<class Thermo2T , template< class > class Type>
thermo2T operator+ (
            const thermo2T< Thermo2T, Type > & ,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.6 operator+ [2/2]

```
template<class Thermo2T , template< class > class Type>
thermo2T operator+ (
            const thermo2T< Thermo2T, Type > & ,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.7 operator== [1/2]

```
template<class Thermo2T , template< class > class Type>
thermo2T operator== (
            const thermo2T< Thermo2T, Type > & ,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

6.9.5.8 operator== [2/2]

```
template<class Thermo2T , template< class > class Type>
thermo2T operator== (
            const thermo2T< Thermo2T, Type > & ,
            const thermo2T< Thermo2T, Type > &  )  [friend]
```

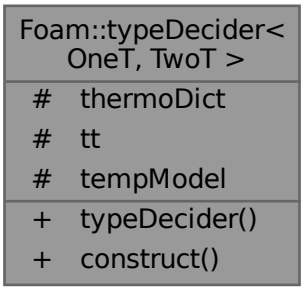
The documentation for this class was generated from the following files:

- lnInclude/thermo2T.H
- thermo/thermo2T/thermo2T.H
- lnInclude/thermo2T.C
- lnInclude/thermo2TI.H
- thermo/thermo2T/thermo2T.C
- thermo/thermo2T/thermo2TI.H

6.10 Foam::typeDecider< OneT, TwoT > Class Template Reference

```
#include <typeDecider.H>
```

Collaboration diagram for Foam::typeDecider< OneT, TwoT >:



Public Member Functions

- typeDecider (const fvMesh &)
- bool construct (autoPtr< OneT > &p1T, autoPtr< TwoT > &p2T, const fvMesh &mesh)

Protected Attributes

- IOdictionary thermoDict
- const dictionary tt
- const word tempModel

6.10.1 Detailed Description

template<class OneT, class TwoT>
class Foam::typeDecider< OneT, TwoT >

Definition at line 54 of file typeDecider.H.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 typeDecider()

```
template<class OneT , class TwoT >
Foam::typeDecider< OneT, TwoT >::typeDecider (
            const fvMesh & mesh )
```

Definition at line 30 of file typeDecider.C.

6.10.3 Member Function Documentation

6.10.3.1 construct()

```
template<class OneT , class TwoT >
bool Foam::typeDecider< OneT, TwoT >::construct (
            autoPtr< OneT > & p1T,
            autoPtr< TwoT > & p2T,
            const fvMesh & mesh )  [inline]
```

Definition at line 5 of file typeDeciderI.H.

6.10.4 Member Data Documentation

6.10.4.1 tempModel

```
template<class OneT , class TwoT >
const word Foam::typeDecider< OneT, TwoT >::tempModel  [protected]
```

Definition at line 60 of file typeDecider.H.

6.10.4.2 thermoDict

```
template<class OneT , class TwoT >
IOdictionary Foam::typeDecider< OneT, TwoT >::thermoDict  [protected]
```

Definition at line 58 of file typeDecider.H.

6.10.4.3 tt

```
template<class OneT , class TwoT >
const dictionary Foam::typeDecider< OneT, TwoT >::tt  [protected]
```

Definition at line 59 of file typeDecider.H.

The documentation for this class was generated from the following files:

- typeDecider.H
- typeDecider.C
- typeDeciderI.H

Chapter 7

File Documentation

7.1 basic2TThermo/basic2TThermo.C File Reference

```
#include "basic2TThermo.H"
#include "stringOps.H"
#include "wordIOList.H"
#include "zeroGradientFvPatchFields.H"
#include "fixedEnergyFvPatchScalarField.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"
#include "fixedJumpFvPatchFields.H"
#include "fixedJumpAMIFvPatchFields.H"
#include "energyJumpFvPatchScalarField.H"
#include "energyJumpAMIFvPatchScalarField.H"
```

Include dependency graph for basic2TThermo/basic2TThermo.C:

Namespaces

- namespace Foam

Functions

- Foam::defineRunTimeSelectionTable (basic2TThermo, fvMesh)
- Foam::defineRunTimeSelectionTable (basic2TThermo, fvMeshDictPhase)

7.2 basic2TThermo/basic2TThermo.C

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2016 OpenFOAM Foundation
    Copyright (C) 2017-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "basic2TThermo.H"
#include "stringOps.H"
#include "wordIOList.H"
#include "zeroGradientFvPatchFields.H"
#include "fixedEnergyFvPatchScalarField.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"
#include "fixedJumpFvPatchFields.H"
#include "fixedJumpAMIFvPatchFields.H"
#include "energyJumpFvPatchScalarField.H"
#include "energyJumpAMIFvPatchScalarField.H"

// * * * * * * * * * * * * * * Static Data Members * * * * * * * * * * * * * //

namespace Foam
{
    defineTypeNameAndDebug(basic2TThermo, 0);
    defineRunTimeSelectionTable(basic2TThermo, fvMesh);
    defineRunTimeSelectionTable(basic2TThermo, fvMeshDictPhase);
}

const Foam::word Foam::basic2TThermo::dictName("thermophysicalProperties");

const Foam::wordList Foam::basic2TThermo::componentHeader4
({
    "type",
    "mixture",
    "properties",
    "energy"
});

const Foam::wordList Foam::basic2TThermo::componentHeader7
({
    "type",
    "mixture",
    "transport",
    "thermo",
    "equationOfState",
    "specie",
    "energy"
});


// * * * * * * * * * * * * * * Static Member Functions * * * * * * * * * * * * //

Foam::Ostream& Foam::basic2TThermo::printThermoNames
(
    Ostream& os,
    const wordList& cmptNames,
    const wordList& thermoNames
)
{
    const int nCmpt = cmptNames.size();

```

```
    // Build a table of constituent parts by split name into constituent parts
    // - remove incompatible entries from the list
    // - note: row-0 contains the names of constituent parts (ie, the header)

    DynamicList<wordList> outputTbl;
    outputTbl.resize(thermoNames.size()+1);

    label rowi = 0;

    // Header
    outputTbl[rowi] = cmptNames;
    if (!outputTbl[rowi].empty())
    {
        ++rowi;
    }

    for (const word& thermoName : thermoNames)
    {
        outputTbl[rowi] = basic2TThermo::splitThermoName(thermoName, nCmpt);
        if (!outputTbl[rowi].empty())
        {
            ++rowi;
        }
    }

    if (rowi > 1)
    {
        outputTbl.resize(rowi);
        Foam::printTable(outputTbl, os);
    }

    return os;
}


// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

Foam::word Foam::basic2TThermo::makeThermoName
(
    const dictionary& thermoTypeDict,
    const wordList*& cmptHeaderPtr
)
{
    if (thermoTypeDict.found("properties"))
    {
        if (cmptHeaderPtr)
        {
            cmptHeaderPtr = &(componentHeader4);
        }

        return word
        (
            thermoTypeDict.get<word>("type") + '<'
          + thermoTypeDict.get<word>("mixture") + '<'
          + thermoTypeDict.get<word>("properties") + ','
          + thermoTypeDict.get<word>("energy") + "»"
        );
    }
    else
    {
        if (cmptHeaderPtr)
        {
            cmptHeaderPtr = &(componentHeader7);
        }

        return word
        (
            thermoTypeDict.get<word>("type") + '<'
          + thermoTypeDict.get<word>("mixture") + '<'
          + thermoTypeDict.get<word>("transport") + '<'
          + thermoTypeDict.get<word>("thermo") + '<'
          + thermoTypeDict.get<word>("equationOfState") + '<'
          + thermoTypeDict.get<word>("specie") + "»,"
          + thermoTypeDict.get<word>("energy") + "»>"
        );
    }
}


// * * * * * * * * * * * * Protected Member Functions  * * * * * * * * * * * //

Foam::wordList Foam::basic2TThermo::heBoundaryBaseTypes()
{
    const volScalarField::Boundary& tbf = this->TTR_.boundaryField();

    wordList hbt(tbf.size());

```

```
    forAll(tbf, patchi)
    {
        if (isA<fixedJumpFvPatchScalarField>(tbf[patchi]))
        {
            const auto& pf =
                dynamic_cast<const fixedJumpFvPatchScalarField&>
                (
                    tbf[patchi]
                );

            hbt[patchi] = pf.interfaceFieldType();
        }
        else if (isA<fixedJumpAMIFvPatchScalarField>(tbf[patchi]))
        {
            const auto& pf =
                dynamic_cast<const fixedJumpAMIFvPatchScalarField&>
                (
                    tbf[patchi]
                );

            hbt[patchi] = pf.interfaceFieldType();
        }
    }

    return hbt;
}


Foam::wordList Foam::basic2TThermo::heBoundaryTypes()
{
    const volScalarField::Boundary& tbf = this->TTR_.boundaryField();

    wordList hbt(tbf.types());

    forAll(tbf, patchi)
    {
        if (isA<fixedValueFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = fixedEnergyFvPatchScalarField::typeName;
        }
        else if
        (
            isA<zeroGradientFvPatchScalarField>(tbf[patchi])
         || isA<fixedGradientFvPatchScalarField>(tbf[patchi])
        )
        {
            hbt[patchi] = gradientEnergyFvPatchScalarField::typeName;
        }
        else if (isA<mixedFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = mixedEnergyFvPatchScalarField::typeName;
        }
        else if (isA<fixedJumpFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = energyJumpFvPatchScalarField::typeName;
        }
        else if (isA<fixedJumpAMIFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = energyJumpAMIFvPatchScalarField::typeName;
        }
    }

    return hbt;
}


// * * * * * * * * * * * * * Static Member Functions * * * * * * * * * * * * //

Foam::volScalarField& Foam::basic2TThermo::lookupOrConstruct
(
    const fvMesh& mesh,
    const word& fieldName,
    bool& isOwner
)
{
    auto* ptr = mesh.objectRegistry::getObjectPtr<volScalarField>(fieldName);

    isOwner = !ptr;

    if (!ptr)
    {
        ptr = new volScalarField
        (
            IOobject
            (
                fieldName,
                mesh.time().timeName(),
```

```
                mesh,
                IOobject::MUST_READ,
                IOobject::AUTO_WRITE,
                IOobject::REGISTER
            ),
            mesh
        );

        // Transfer ownership of this object to the objectRegistry
        ptr->store();
    }

    return *ptr;
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

Foam::basic2TThermo::basic2TThermo
(
    const fvMesh& mesh,
    const word& phaseName
)
:
    IOdictionary
    (
        IOobject
        (
            phasePropertyName(dictName, phaseName),
            mesh.time().constant(),
            mesh,
            IOobject::READ_MODIFIED,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        )
    ),

    phaseName_(phaseName),

    pOwner_(false),
    TOwnerTR_(false),
    TOwnerVib_(false),
    dpdt_(getOrDefault<bool>("dpdt", true)),

    p_(lookupOrConstruct(mesh, "p", pOwner_)),
    TTR_(lookupOrConstruct(mesh, phasePropertyName("TTR"), TOwnerTR_)),
    TVib_(lookupOrConstruct(mesh, phasePropertyName("TVib"), TOwnerVib_)),

    alpha_
    (
        IOobject
        (
            phaseScopedName("thermo", "alpha"),
            mesh.time().timeName(),
            mesh,
            IOobject::READ_IF_PRESENT,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionedScalar(dimensionSet(1, -1, -1, 0, 0), Zero)
    )
{
    this->readIfPresent("updateTTR", TOwnerTR_);  // Manual override
    this->readIfPresent("updateTVib", TOwnerVib_);  // Manual override
}


Foam::basic2TThermo::basic2TThermo
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
:
    IOdictionary
    (
        IOobject
        (
            phasePropertyName(dictName, phaseName),
            mesh.time().constant(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        dict
```

```
    ),

    phaseName_(phaseName),

    pOwner_(false),
    TOwnerTR_(false),
    TOwnerVib_(false),
    dpdt_(getOrDefault<bool>("dpdt", true)),

    p_(lookupOrConstruct(mesh, "p", pOwner_)),
    TTR_(lookupOrConstruct(mesh, phasePropertyName("TTR"), TOwnerTR_)),
    TVib_(lookupOrConstruct(mesh, phasePropertyName("TVib"), TOwnerVib_)),

    alpha_
    (
        IOobject
        (
            phaseScopedName("thermo", "alpha"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionedScalar(dimensionSet(1, -1, -1, 0, 0), Zero)
    )
{
    this->readIfPresent("updateTTR", TOwnerTR_);  // Manual override
    this->readIfPresent("updateTVib", TOwnerVib_);  // Manual override
}


Foam::basic2TThermo::basic2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    IOdictionary
    (
        IOobject
        (
            dictionaryName,
            mesh.time().constant(),
            mesh,
            IOobject::READ_MODIFIED,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        )
    ),

    phaseName_(phaseName),

    pOwner_(false),
    TOwnerTR_(false),
    TOwnerVib_(false),
    dpdt_(getOrDefault<bool>("dpdt", true)),

    p_(lookupOrConstruct(mesh, "p", pOwner_)),
    TTR_(lookupOrConstruct(mesh, "TTR", TOwnerTR_)),
    TVib_(lookupOrConstruct(mesh, "TVib", TOwnerVib_)),

    alpha_
    (
        IOobject
        (
            phaseScopedName("thermo", "alpha"),
            mesh.time().timeName(),
            mesh,
            IOobject::READ_IF_PRESENT,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionedScalar(dimensionSet(1, -1, -1, 0, 0), Zero)
    )
{
    this->readIfPresent("updateTTR", TOwnerTR_);  // Manual override

    if (debug)
    {
        Pout« "Constructed shared thermo : mesh:" « mesh.name()
            « " phase:" « phaseName
            « " dictionary:" « dictionaryName
            « " TTR:" « TTR_.name()
```

```
            « " updateTTR:" « TOwnerTR_
            « " alphaName:" « alpha_.name()
            « endl;
    }
}


// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

Foam::autoPtr<Foam::basic2TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    return New<basic2TThermo>(mesh, phaseName);
}


// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * * //

Foam::basic2TThermo::~basic2TThermo()
{
    if (pOwner_)
    {
        db().checkOut(p_.name());
    }

    if (TOwnerTR_)
    {
        db().checkOut(TTR_.name());
    }

    if (TOwnerVib_)
    {
        db().checkOut(TVib_.name());
    }
}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * * * * //

const Foam::basic2TThermo& Foam::basic2TThermo::lookupThermo
(
    const fvPatchScalarField& pf
)
{
    const basic2TThermo* thermoPtr = pf.db().cfindObject<basic2TThermo>(dictName);

    if (thermoPtr)
    {
        return *thermoPtr;
    }

    for (const basic2TThermo& thermo : pf.db().cobjects<basic2TThermo>())
    {
        if
        (
            &(thermo.he().internalField())
         == &(pf.internalField())
        )
        {
            return thermo;
        }
    }

    // Failure
    return pf.db().lookupObject<basic2TThermo>(dictName);
}


void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a
) const
{
    if (!(he().name() == phasePropertyName(a)))
    {
        FatalErrorInFunction
            « "Supported energy type is " « phasePropertyName(a)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}

void Foam::basic2TThermo::validate
```

```
(
    const string& app,
    const word& a,
    const word& b
) const
{
    if
    (
       !(
            he().name() == phasePropertyName(a)
         || he().name() == phasePropertyName(b)
        )
    )
    {
        FatalErrorInFunction
            « "Supported energy types: " « phasePropertyName(a)
            « " and " « phasePropertyName(b)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}

void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a,
    const word& b,
    const word& c
) const
{
    if
    (
       !(
            he().name() == phasePropertyName(a)
         || he().name() == phasePropertyName(b)
         || he().name() == phasePropertyName(c)
        )
    )
    {
        FatalErrorInFunction
            « "Supported energy types: " « phasePropertyName(a)
            « ", " « phasePropertyName(b)
            « " and " « phasePropertyName(c)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}

void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a,
    const word& b,
    const word& c,
    const word& d
) const
{
    if
    (
       !(
            he().name() == phasePropertyName(a)
         || he().name() == phasePropertyName(b)
         || he().name() == phasePropertyName(c)
         || he().name() == phasePropertyName(d)
        )
    )
    {
        FatalErrorInFunction
            « "Supported energy types: " « phasePropertyName(a)
            « ", " « phasePropertyName(b)
            « ", " « phasePropertyName(c)
            « " and " « phasePropertyName(d)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}


Foam::wordList Foam::basic2TThermo::splitThermoName
(
    const std::string& thermoName,
    const int nExpectedCmpts
)
{
    // Split on ",<>" but include space for good measure.
    // Splits things like
    // "hePsiThermo<pureMixture<const<hConst<perfectGas<specie»,enthalpy»>"
```

```
    const auto parsed = stringOps::splitAny<std::string>(thermoName, " ,<>");
    const int nParsed(parsed.size());

    wordList cmpts;

    if (!nExpectedCmpts || nParsed == nExpectedCmpts)
    {
        cmpts.resize(nParsed);

        auto iter = cmpts.begin();
        for (const auto& sub : parsed)
        {
            *iter = word(sub.str());
            ++iter;
        }
    }

    return cmpts;
}


Foam::volScalarField& Foam::basic2TThermo::p()
{
    return p_;
}


const Foam::volScalarField& Foam::basic2TThermo::p() const
{
    return p_;
}


const Foam::volScalarField& Foam::basic2TThermo::TTR() const
{
    return TTR_;
}


Foam::volScalarField& Foam::basic2TThermo::TTR()
{
    return TTR_;
}

const Foam::volScalarField& Foam::basic2TThermo::TVib() const
{
    return TVib_;
}


Foam::volScalarField& Foam::basic2TThermo::TVib()
{
    return TVib_;
}


const Foam::volScalarField& Foam::basic2TThermo::alpha() const
{
    return alpha_;
}


const Foam::scalarField& Foam::basic2TThermo::alpha(const label patchi) const
{
    return alpha_.boundaryField()[patchi];
}


bool Foam::basic2TThermo::read()
{
    return regIOobject::read();
}


// ************************************************************************* //
```

7.3 InInclude/basic2TThermo.C File Reference

```
#include "basic2TThermo.H"
#include "stringOps.H"
```

```
#include "wordIOList.H"
#include "zeroGradientFvPatchFields.H"
#include "fixedEnergyFvPatchScalarField.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"
#include "fixedJumpFvPatchFields.H"
#include "fixedJumpAMIFvPatchFields.H"
#include "energyJumpFvPatchScalarField.H"
#include "energyJumpAMIFvPatchScalarField.H"
```

Include dependency graph for lnInclude/basic2TThermo.C:

Namespaces

- namespace Foam

Functions

- Foam::defineRunTimeSelectionTable (basic2TThermo, fvMesh)
- Foam::defineRunTimeSelectionTable (basic2TThermo, fvMeshDictPhase)

7.4 lnInclude/basic2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2016 OpenFOAM Foundation
    Copyright (C) 2017-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "basic2TThermo.H"
#include "stringOps.H"
#include "wordIOList.H"
#include "zeroGradientFvPatchFields.H"
#include "fixedEnergyFvPatchScalarField.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"
#include "fixedJumpFvPatchFields.H"
#include "fixedJumpAMIFvPatchFields.H"
```

```
#include "energyJumpFvPatchScalarField.H"
#include "energyJumpAMIFvPatchScalarField.H"

// * * * * * * * * * * * * * * Static Data Members * * * * * * * * * * * * * //

namespace Foam
{
    defineTypeNameAndDebug(basic2TThermo, 0);
    defineRunTimeSelectionTable(basic2TThermo, fvMesh);
    defineRunTimeSelectionTable(basic2TThermo, fvMeshDictPhase);
}

const Foam::word Foam::basic2TThermo::dictName("thermophysicalProperties");

const Foam::wordList Foam::basic2TThermo::componentHeader4
({
    "type",
    "mixture",
    "properties",
    "energy"
});

const Foam::wordList Foam::basic2TThermo::componentHeader7
({
    "type",
    "mixture",
    "transport",
    "thermo",
    "equationOfState",
    "specie",
    "energy"
});


// * * * * * * * * * * * * * * Static Member Functions * * * * * * * * * * * * //

Foam::Ostream& Foam::basic2TThermo::printThermoNames
(
    Ostream& os,
    const wordList& cmptNames,
    const wordList& thermoNames
)
{
    const int nCmpt = cmptNames.size();

    // Build a table of constituent parts by split name into constituent parts
    // - remove incompatible entries from the list
    // - note: row-0 contains the names of constituent parts (ie, the header)

    DynamicList<wordList> outputTbl;
    outputTbl.resize(thermoNames.size()+1);

    label rowi = 0;

    // Header
    outputTbl[rowi] = cmptNames;
    if (!outputTbl[rowi].empty())
    {
        ++rowi;
    }

    for (const word& thermoName : thermoNames)
    {
        outputTbl[rowi] = basic2TThermo::splitThermoName(thermoName, nCmpt);
        if (!outputTbl[rowi].empty())
        {
            ++rowi;
        }
    }

    if (rowi > 1)
    {
        outputTbl.resize(rowi);
        Foam::printTable(outputTbl, os);
    }

    return os;
}


// * * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

Foam::word Foam::basic2TThermo::makeThermoName
(
    const dictionary& thermoTypeDict,
    const wordList*& cmptHeaderPtr
)
```

```
{
    if (thermoTypeDict.found("properties"))
    {
        if (cmptHeaderPtr)
        {
            cmptHeaderPtr = &(componentHeader4);
        }

        return word
        (
            thermoTypeDict.get<word>("type") + '<'
          + thermoTypeDict.get<word>("mixture") + '<'
          + thermoTypeDict.get<word>("properties") + ','
          + thermoTypeDict.get<word>("energy") + "»"
        );
    }
    else
    {
        if (cmptHeaderPtr)
        {
            cmptHeaderPtr = &(componentHeader7);
        }

        return word
        (
            thermoTypeDict.get<word>("type") + '<'
          + thermoTypeDict.get<word>("mixture") + '<'
          + thermoTypeDict.get<word>("transport") + '<'
          + thermoTypeDict.get<word>("thermo") + '<'
          + thermoTypeDict.get<word>("equationOfState") + '<'
          + thermoTypeDict.get<word>("specie") + "»,"
          + thermoTypeDict.get<word>("energy") + "»>"
        );
    }
}


// * * * * * * * * * * * * Protected Member Functions  * * * * * * * * * * * //

Foam::wordList Foam::basic2TThermo::heBoundaryBaseTypes()
{
    const volScalarField::Boundary& tbf = this->TTR_.boundaryField();

    wordList hbt(tbf.size());

    forAll(tbf, patchi)
    {
        if (isA<fixedJumpFvPatchScalarField>(tbf[patchi]))
        {
            const auto& pf =
                dynamic_cast<const fixedJumpFvPatchScalarField&>
                (
                    tbf[patchi]
                );

            hbt[patchi] = pf.interfaceFieldType();
        }
        else if (isA<fixedJumpAMIFvPatchScalarField>(tbf[patchi]))
        {
            const auto& pf =
                dynamic_cast<const fixedJumpAMIFvPatchScalarField&>
                (
                    tbf[patchi]
                );

            hbt[patchi] = pf.interfaceFieldType();
        }
    }

    return hbt;
}


Foam::wordList Foam::basic2TThermo::heBoundaryTypes()
{
    const volScalarField::Boundary& tbf = this->TTR_.boundaryField();

    wordList hbt(tbf.types());

    forAll(tbf, patchi)
    {
        if (isA<fixedValueFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = fixedEnergyFvPatchScalarField::typeName;
        }
        else if
        (
```

```
            isA<zeroGradientFvPatchScalarField>(tbf[patchi])
         || isA<fixedGradientFvPatchScalarField>(tbf[patchi])
        )
        {
            hbt[patchi] = gradientEnergyFvPatchScalarField::typeName;
        }
        else if (isA<mixedFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = mixedEnergyFvPatchScalarField::typeName;
        }
        else if (isA<fixedJumpFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = energyJumpFvPatchScalarField::typeName;
        }
        else if (isA<fixedJumpAMIFvPatchScalarField>(tbf[patchi]))
        {
            hbt[patchi] = energyJumpAMIFvPatchScalarField::typeName;
        }
    }

    return hbt;
}


// * * * * * * * * * * * * * Static Member Functions * * * * * * * * * * * * //

Foam::volScalarField& Foam::basic2TThermo::lookupOrConstruct
(
    const fvMesh& mesh,
    const word& fieldName,
    bool& isOwner
)
{
    auto* ptr = mesh.objectRegistry::getObjectPtr<volScalarField>(fieldName);

    isOwner = !ptr;

    if (!ptr)
    {
        ptr = new volScalarField
        (
            IOobject
            (
                fieldName,
                mesh.time().timeName(),
                mesh,
                IOobject::MUST_READ,
                IOobject::AUTO_WRITE,
                IOobject::REGISTER
            ),
            mesh
        );

        // Transfer ownership of this object to the objectRegistry
        ptr->store();
    }

    return *ptr;
}


// * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

Foam::basic2TThermo::basic2TThermo
(
    const fvMesh& mesh,
    const word& phaseName
)
:
    IOdictionary
    (
        IOobject
        (
            phasePropertyName(dictName, phaseName),
            mesh.time().constant(),
            mesh,
            IOobject::READ_MODIFIED,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        )
    ),

    phaseName_(phaseName),

    pOwner_(false),
    TOwnerTR_(false),
    TOwnerVib_(false),
```

```
    dpdt_(getOrDefault<bool>("dpdt", true)),

    p_(lookupOrConstruct(mesh, "p", pOwner_)),
    TTR_(lookupOrConstruct(mesh, phasePropertyName("TTR"), TOwnerTR_)),
    TVib_(lookupOrConstruct(mesh, phasePropertyName("TVib"), TOwnerVib_)),

    alpha_
    (
        IOobject
        (
            phaseScopedName("thermo", "alpha"),
            mesh.time().timeName(),
            mesh,
            IOobject::READ_IF_PRESENT,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionedScalar(dimensionSet(1, -1, -1, 0, 0), Zero)
    )
{
    this->readIfPresent("updateTTR", TOwnerTR_);  // Manual override
    this->readIfPresent("updateTVib", TOwnerVib_);  // Manual override
}


Foam::basic2TThermo::basic2TThermo
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
:
    IOdictionary
    (
        IOobject
        (
            phasePropertyName(dictName, phaseName),
            mesh.time().constant(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        dict
    ),

    phaseName_(phaseName),

    pOwner_(false),
    TOwnerTR_(false),
    TOwnerVib_(false),
    dpdt_(getOrDefault<bool>("dpdt", true)),

    p_(lookupOrConstruct(mesh, "p", pOwner_)),
    TTR_(lookupOrConstruct(mesh, phasePropertyName("TTR"), TOwnerTR_)),
    TVib_(lookupOrConstruct(mesh, phasePropertyName("TVib"), TOwnerVib_)),

    alpha_
    (
        IOobject
        (
            phaseScopedName("thermo", "alpha"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionedScalar(dimensionSet(1, -1, -1, 0, 0), Zero)
    )
{
    this->readIfPresent("updateTTR", TOwnerTR_);  // Manual override
    this->readIfPresent("updateTVib", TOwnerVib_);  // Manual override
}


Foam::basic2TThermo::basic2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    IOdictionary
    (
```

```
        IOobject
        (
            dictionaryName,
            mesh.time().constant(),
            mesh,
            IOobject::READ_MODIFIED,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        )
    ),

    phaseName_(phaseName),

    pOwner_(false),
    TOwnerTR_(false),
    TOwnerVib_(false),
    dpdt_(getOrDefault<bool>("dpdt", true)),

    p_(lookupOrConstruct(mesh, "p", pOwner_)),
    TTR_(lookupOrConstruct(mesh, "TTR", TOwnerTR_)),
    TVib_(lookupOrConstruct(mesh, "TVib", TOwnerVib_)),

    alpha_
    (
        IOobject
        (
            phaseScopedName("thermo", "alpha"),
            mesh.time().timeName(),
            mesh,
            IOobject::READ_IF_PRESENT,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionedScalar(dimensionSet(1, -1, -1, 0, 0), Zero)
    )
{
    this->readIfPresent("updateTTR", TOwnerTR_);  // Manual override

    if (debug)
    {
        Pout« "Constructed shared thermo : mesh:" « mesh.name()
            « " phase:" « phaseName
            « " dictionary:" « dictionaryName
            « " TTR:" « TTR_.name()
            « " updateTTR:" « TOwnerTR_
            « " alphaName:" « alpha_.name()
            « endl;
    }
}


// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

Foam::autoPtr<Foam::basic2TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    return New<basic2TThermo>(mesh, phaseName);
}


// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

Foam::basic2TThermo::~basic2TThermo()
{
    if (pOwner_)
    {
        db().checkOut(p_.name());
    }

    if (TOwnerTR_)
    {
        db().checkOut(TTR_.name());
    }

    if (TOwnerVib_)
    {
        db().checkOut(TVib_.name());
    }
}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * * * //

```

```
const Foam::basic2TThermo& Foam::basic2TThermo::lookupThermo
(
    const fvPatchScalarField& pf
)
{
    const basic2TThermo* thermoPtr = pf.db().cfindObject<basic2TThermo>(dictName);

    if (thermoPtr)
    {
        return *thermoPtr;
    }

    for (const basic2TThermo& thermo : pf.db().cobjects<basic2TThermo>())
    {
        if
        (
            &(thermo.he().internalField())
         == &(pf.internalField())
        )
        {
            return thermo;
        }
    }

    // Failure
    return pf.db().lookupObject<basic2TThermo>(dictName);
}


void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a
) const
{
    if (!(he().name() == phasePropertyName(a)))
    {
        FatalErrorInFunction
            « "Supported energy type is " « phasePropertyName(a)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}

void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a,
    const word& b
) const
{
    if
    (
       !(
            he().name() == phasePropertyName(a)
         || he().name() == phasePropertyName(b)
        )
    )
    {
        FatalErrorInFunction
            « "Supported energy types: " « phasePropertyName(a)
            « " and " « phasePropertyName(b)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}

void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a,
    const word& b,
    const word& c
) const
{
    if
    (
       !(
            he().name() == phasePropertyName(a)
         || he().name() == phasePropertyName(b)
         || he().name() == phasePropertyName(c)
        )
    )
    {
        FatalErrorInFunction
            « "Supported energy types: " « phasePropertyName(a)
            « ", " « phasePropertyName(b)
```

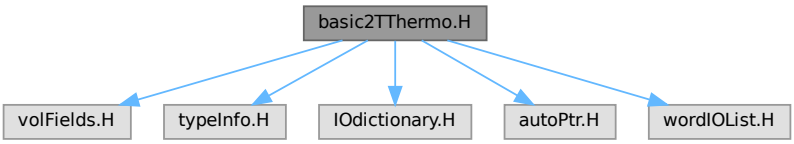
```
            « " and " « phasePropertyName(c)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}

void Foam::basic2TThermo::validate
(
    const string& app,
    const word& a,
    const word& b,
    const word& c,
    const word& d
) const
{
    if
    (
       !(
            he().name() == phasePropertyName(a)
          || he().name() == phasePropertyName(b)
          || he().name() == phasePropertyName(c)
          || he().name() == phasePropertyName(d)
        )
    )
    {
        FatalErrorInFunction
            « "Supported energy types: " « phasePropertyName(a)
            « ", " « phasePropertyName(b)
            « ", " « phasePropertyName(c)
            « " and " « phasePropertyName(d)
            « ", thermodynamics package provides " « he().name()
            « exit(FatalError);
    }
}


Foam::wordList Foam::basic2TThermo::splitThermoName
(
    const std::string& thermoName,
    const int nExpectedCmpts
)
{
    // Split on ",<>" but include space for good measure.
    // Splits things like
    // "hePsiThermo<pureMixture<const<hConst<perfectGas<specie»,enthalpy»>"

    const auto parsed = stringOps::splitAny<std::string>(thermoName, " ,<>");
    const int nParsed(parsed.size());

    wordList cmpts;

    if (!nExpectedCmpts || nParsed == nExpectedCmpts)
    {
        cmpts.resize(nParsed);

        auto iter = cmpts.begin();
        for (const auto& sub : parsed)
        {
            *iter = word(sub.str());
            ++iter;
        }
    }

    return cmpts;
}


Foam::volScalarField& Foam::basic2TThermo::p()
{
    return p_;
}


const Foam::volScalarField& Foam::basic2TThermo::p() const
{
    return p_;
}


const Foam::volScalarField& Foam::basic2TThermo::TTR() const
{
    return TTR_;
}


Foam::volScalarField& Foam::basic2TThermo::TTR()
{
```

```
    return TTR_;
}

const Foam::volScalarField& Foam::basic2TThermo::TVib() const
{
    return TVib_;
}


Foam::volScalarField& Foam::basic2TThermo::TVib()
{
    return TVib_;
}


const Foam::volScalarField& Foam::basic2TThermo::alpha() const
{
    return alpha_;
}


const Foam::scalarField& Foam::basic2TThermo::alpha(const label patchi) const
{
    return alpha_.boundaryField()[patchi];
}


bool Foam::basic2TThermo::read()
{
    return regIOobject::read();
}


// ************************************************************************* //
```

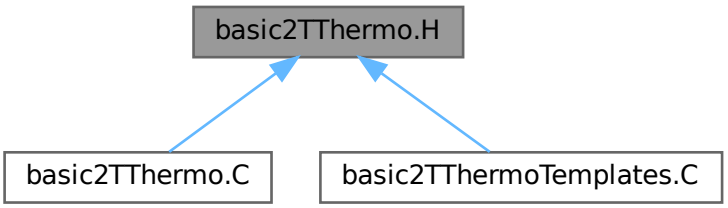
7.5 basic2TThermo/basic2TThermo.H File Reference

```
#include "volFields.H"
#include "typeInfo.H"
#include "IOdictionary.H"
#include "autoPtr.H"
#include "wordIOList.H"
```

Include dependency graph for basic2TThermo/basic2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::basic2TThermo

Namespaces

- namespace Foam

7.6 basic2TThermo/basic2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2017-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::basic2TThermo

Description
    Abstract base-class for fluid and solid thermodynamic properties

    An important concept is that the thermo can share an existing T
    (similar to p) in which case it will not try to update it. This gets
    triggered purely on construction order - the first one to register
    T is responsible for updating it. Note that the mechanism also means
    that if multiple thermos are running on the same mesh, only the first one
    will update the temperature. The behaviour can be overridden using the
```

```
    'updateT' dictionary entry.

SourceFiles
    basic2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef Foam_basic2TThermo_H
#define Foam_basic2TThermo_H

#include "volFields.H"
#include "typeInfo.H"
#include "IOdictionary.H"
#include "autoPtr.H"
#include "wordIOList.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class basic2TThermo Declaration
\*---------------------------------------------------------------------------*/

class basic2TThermo
:
    public IOdictionary
{
    // Private Data

        //- Components names/order
        static const wordList componentHeader4;

        //- Components names/order
        static const wordList componentHeader7;


    // Private Member Functions

        //- Construct name of thermo package from dictionary components
        static word makeThermoName
        (
            const dictionary& dict,
            const wordList*& cmptHeaderPtr
        );

        //- Look up field from registry or construct and store
        static volScalarField& lookupOrConstruct
        (
            const fvMesh& mesh,
            const word& fieldName,
            bool& isOwner
        );

        //- Generic lookup for thermodynamics package thermoTypeName
        //  \return constructor pointer, or FatalError
        template<class Thermo, class ThermoConstructTable>
        static typename ThermoConstructTable::mapped_type
        getThermoOrDie
        (
            const dictionary& thermoTypeDict,
            ThermoConstructTable& thermoTable,
            const word& thermoTypeName,
            const wordList& cmptNames
        );

        //- Generic lookup for each of the related thermodynamics packages
        //  \return constructor pointer, or FatalError
        template<class Thermo, class ThermoConstructTable>
        static typename ThermoConstructTable::mapped_type
        getThermoOrDie
        (
            const dictionary& thermoDict,
            ThermoConstructTable& thermoTable
        );


protected:

    // Protected Data

        //- Phase-name
        const word& phaseName_;

        //- Pressure created and stored by this instance
        bool pOwner_;
```

```

        //- Temperature created and stored by this instance
        bool TOwnerTR_;

        //- Temperature created and stored by this instance
        bool TOwnerVib_;

        //- Include dpdt term in the enthalpy equation?
        bool dpdt_;


    // Fields

        //- Pressure [Pa]
        volScalarField& p_;

        //- Translational-Rotational Temperature [K]
        volScalarField& TTR_;

        //- Vibrational Temperature [K]
        volScalarField& TVib_;

        //- Laminar thermal diffusivity [kg/m/s]
        volScalarField alpha_;


    // Protected Member Functions

        //- Return the enthalpy/internal energy field boundary types
        //- by interrogating the temperature field boundary types
        wordList heBoundaryTypes();

        //- Return the enthalpy/internal energy field boundary base types
        //- by interrogating the temperature field boundary types
        wordList heBoundaryBaseTypes();

        //- No copy construct
        basic2TThermo(const basic2TThermo&) = delete;


public:

    //- Runtime type information
    TypeName("basic2TThermo");


    //- Declare run-time constructor selection table
    declareRunTimeSelectionTable
    (
        autoPtr,
        basic2TThermo,
        fvMesh,
        (const fvMesh& mesh, const word& phaseName),
        (mesh, phaseName)
    );


    //- Declare run-time constructor selection table for dictionary based
    declareRunTimeSelectionTable
    (
        autoPtr,
        basic2TThermo,
        fvMeshDictPhase,
        (const fvMesh& mesh, const word& phaseName, const word& dictName),
        (mesh, phaseName, dictName)
    );


    // Constructors

        //- Construct from mesh and phase name
        basic2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //- Construct from mesh, dictionary and phase name
        basic2TThermo
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName
        );

        //- Construct from mesh, phase name and explicit naming of the
        //- dictionary (so it can be shared amongst phases).
```

```
        basic2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    // Selectors

        //- Generic New for each of the related thermodynamics packages
        template<class Thermo>
        static autoPtr<Thermo> New
        (
            const fvMesh&,
            const word& phaseName = word::null
        );

        //- Generic New for each of the related thermodynamics packages
        template<class Thermo>
        static autoPtr<Thermo> New
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName = word::null
        );

        //- Generic New for each of the related thermodynamics packages
        template<class Thermo>
        static autoPtr<Thermo> New
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictName
        );

        //- Specialisation of the Generic New for basic2TThermo
        static autoPtr<basic2TThermo> New
        (
            const fvMesh&,
            const word& phaseName = word::null
        );


    //- Destructor
    virtual ~basic2TThermo();


    // Member Functions

        //- The dictionary name ("thermophysicalProperties")
        static const word dictName;

        //- The phase property name as property.phase
        //- \sa IOobject::groupName
        static word phasePropertyName(const word& name, const word& phaseName)
        {
            return IOobject::groupName(name, phaseName);
        }

        //- The phase property name as property.phase (using phaseName_)
        //- \sa IOobject::groupName
        word phasePropertyName(const word& name) const
        {
            return IOobject::groupName(name, phaseName_);
        }

        //- The phase property name scoped as scope:property.phase
        //- \sa IOobject::groupName and \sa IOobject::scopedName
        word phaseScopedName(const std::string& scope, const word& name) const
        {
            return IOobject::groupName
            (
                IOobject::scopedName(scope, name),
                phaseName_
            );
        }

        static const basic2TThermo& lookupThermo(const fvPatchScalarField& pf);

        //- Print (filtered) table of thermo names, splits on \c " ,<>"
        static Ostream& printThermoNames
        (
            Ostream& os,
            const wordList& cmptNames,
            const wordList& thermoNames
        );
```

```

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&
        ) const;

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&,
            const word&
        ) const;

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&,
            const word&,
            const word&
        ) const;

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&,
            const word&,
            const word&,
            const word&
        ) const;

        //- Split thermo package name into a list of components names
        //  Splits on \c " ,<>"
        //  \return empty list if the split name does not have the
        //  expected number of components (non-zero).
        static wordList splitThermoName
        (
            const std::string& thermoName,
            const int nExpectedCmpts
        );

        //- Update properties
        virtual void correct() = 0;

        //- Return the name of the thermo physics
        virtual word thermoName() const = 0;

        //- Return true if the equation of state is incompressible
        //  i.e. rho != f(p)
        virtual bool incompressible() const = 0;

        //- Return true if the equation of state is isochoric
        //  i.e. rho = const
        virtual bool isochoric() const = 0;

        //- True if dpdt term should be included in enthalpy equation
        bool dpdt() const noexcept
        {
            return dpdt_;
        }

        //- True if T_TR should be updated
        bool updateTTR() const noexcept
        {
            return TOwnerTR_;
        }

        //- True if T_vib should be updated
        bool updateTVib() const noexcept
        {
            return TOwnerVib_;
        }


        // Access to thermodynamic state variables

            //- Pressure [Pa]
            //  Non-const access allowed for transport equations
            virtual volScalarField& p();
```

```
            //- Pressure [Pa]
            virtual const volScalarField& p() const;

            //- Density [kg/m^3]
            virtual tmp<volScalarField> rho() const = 0;

            //- Density for patch [kg/m^3]
            virtual tmp<scalarField> rho(const label patchi) const = 0;

            //- Enthalpy/Internal energy [J/kg]
            //  Non-const access allowed for transport equations
            //  Encapsulates all energy modes
            virtual volScalarField& he() = 0;

            //- Enthalpy/Internal energy [J/kg]
            virtual const volScalarField& he() const = 0;

            //- Enthalpy/Internal energy
            //  for given pressure and temperature [J/kg]
            virtual tmp<volScalarField> he
            (
                const volScalarField& p,
                const volScalarField& TTR
            ) const = 0;

            //- Enthalpy/Internal energy for cell-set [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const = 0;

            //- Enthalpy/Internal energy for patch [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Chemical enthalpy [J/kg]
            virtual tmp<volScalarField> hc() const = 0;

            //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const labelList& cells
            ) const = 0;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const label patchi
            ) const = 0;

                        //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& esVib,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const labelList& cells
            ) const = 0;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& esVib,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const label patchi
            ) const = 0;

        // Fields derived from thermodynamic state variables

            //- Translational-Rotational Temperature [K]
            virtual const volScalarField& TTR() const;
```

```
            //- Vibrational Temperature [K]
            virtual const volScalarField& TVib() const;

            //- Translational-Rotational Temperature [K]
            //  Non-const access allowed for transport equations
            virtual volScalarField& TTR();

            //- Vibrational Temperature [K]
            //  Non-const access allowed for transport equations
            virtual volScalarField& TVib();

            //- Heat capacity at constant pressure [J/kg/K]
            virtual tmp<volScalarField> CpTR() const = 0;

            //- Heat capacity at constant pressure for patch [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Heat capacity using pressure and temperature [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const = 0;

            //- Translational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvT() const = 0;

            //- Translational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvT
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Rotational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvR() const = 0;

            //- Rotational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Vibrational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvVib() const = 0;

            //- Vibrational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvVib
            (
                const scalarField& p,
                const scalarField& TTR,
                const scalarField& Tvib,
                const scalar ThetaVib,
                const label patchi
            ) const = 0;

            //- Density from pressure and temperature from EoS
            virtual tmp<scalarField> rhoEoS
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const = 0;

            //- Gamma = CpTR/CvT []
            virtual tmp<volScalarField> gamma() const = 0;

            //- Gamma = CpTR/CvT for patch []
            virtual tmp<scalarField> gamma
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Translational-Rotational heat capacity at constant pressure/volume [J/kg/K]
            virtual tmp<volScalarField> CpvTR() const = 0;
```

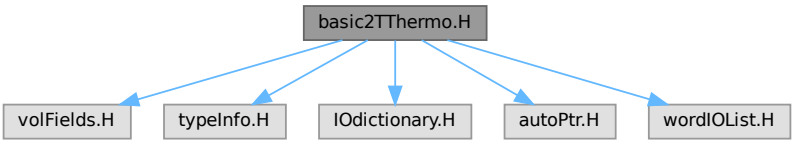
```
            //- Translational-Rotational heat capacity at constant pressure/volume for patch [J/kg/K]
            virtual tmp<scalarField> CpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Translational-Rotational heat capacity ratio []
            virtual tmp<volScalarField> CpByCpvTR() const = 0;

            //- Translational-Rotational heat capacity ratio for patch []
            virtual tmp<scalarField> CpByCpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Molecular weight [kg/kmol]
            virtual tmp<volScalarField> W() const = 0;


        // Access to transport state variables

            //- Thermal diffusivity for enthalpy of mixture [kg/m/s]
            virtual const volScalarField& alpha() const;

            //- Thermal diffusivity for enthalpy of mixture for patch [kg/m/s]
            virtual const scalarField& alpha
            (
                const label patchi
            ) const;


        // Fields derived from transport state variables

            //- Translational-rotational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaTR() const = 0;

            //- Translational-rotational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaTR
            (
                const label patchi
            ) const = 0;

            //- Vibrational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaVib() const = 0;

            //- Vibrational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaVib
            (
                const label patchi
            ) const = 0;

             //- Translational-rotational thermal diffusivity for energy of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaheTR() const = 0;

            //- Translational-rotational thermal diffusivity for energy of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaheTR(const label patchi) const = 0;

            //- Effective thermal diffusivity for temperature
            //  of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaEff
            (
                const volScalarField&
            ) const = 0;

            //- Effective thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const = 0;

            //- Effective thermal diffusivity of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaEff
            (
                const volScalarField& alphat
            ) const = 0;

            //- Effective thermal diffusivity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaEff
```

```
            (
                const scalarField& alphat,
                const label patchi
            ) const = 0;


        //- Read thermophysical properties dictionary
        virtual bool read();
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#ifdef NoRepository
    #include "basic2TThermoTemplates.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

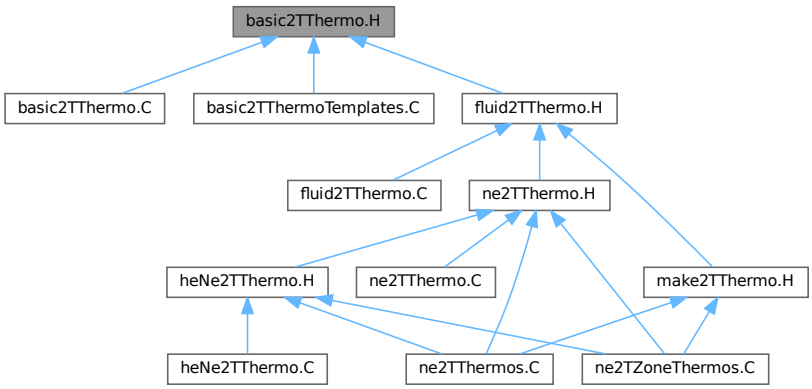
7.7 InInclude/basic2TThermo.H File Reference

```
#include "volFields.H"
#include "typeInfo.H"
#include "IOdictionary.H"
#include "autoPtr.H"
#include "wordIOList.H"
```

Include dependency graph for InInclude/basic2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::basic2TThermo

Namespaces

- namespace Foam

7.8 InInclude/basic2TThermo.H

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2017-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::basic2TThermo

Description
    Abstract base-class for fluid and solid thermodynamic properties

    An important concept is that the thermo can share an existing T
    (similar to p) in which case it will not try to update it. This gets
    triggered purely on construction order - the first one to register
    T is responsible for updating it. Note that the mechanism also means
    that if multiple thermos are running on the same mesh, only the first one
    will update the temperature. The behaviour can be overridden using the
    'updateT' dictionary entry.

SourceFiles
    basic2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef Foam_basic2TThermo_H
#define Foam_basic2TThermo_H

#include "volFields.H"
#include "typeInfo.H"
#include "IOdictionary.H"
#include "autoPtr.H"
#include "wordIOList.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class basic2TThermo Declaration
\*---------------------------------------------------------------------------*/

class basic2TThermo
:
```

```
    public IOdictionary
{
    // Private Data

        //- Components names/order
        static const wordList componentHeader4;

        //- Components names/order
        static const wordList componentHeader7;


    // Private Member Functions

        //- Construct name of thermo package from dictionary components
        static word makeThermoName
        (
            const dictionary& dict,
            const wordList*& cmptHeaderPtr
        );

        //- Look up field from registry or construct and store
        static volScalarField& lookupOrConstruct
        (
            const fvMesh& mesh,
            const word& fieldName,
            bool& isOwner
        );

        //- Generic lookup for thermodynamics package thermoTypeName
        //  \return constructor pointer, or FatalError
        template<class Thermo, class ThermoConstructTable>
        static typename ThermoConstructTable::mapped_type
        getThermoOrDie
        (
            const dictionary& thermoTypeDict,
            ThermoConstructTable& thermoTable,
            const word& thermoTypeName,
            const wordList& cmptNames
        );

        //- Generic lookup for each of the related thermodynamics packages
        //  \return constructor pointer, or FatalError
        template<class Thermo, class ThermoConstructTable>
        static typename ThermoConstructTable::mapped_type
        getThermoOrDie
        (
            const dictionary& thermoDict,
            ThermoConstructTable& thermoTable
        );


protected:

    // Protected Data

        //- Phase-name
        const word& phaseName_;

        //- Pressure created and stored by this instance
        bool pOwner_;

        //- Temperature created and stored by this instance
        bool TOwnerTR_;

        //- Temperature created and stored by this instance
        bool TOwnerVib_;

        //- Include dpdt term in the enthalpy equation?
        bool dpdt_;


    // Fields

        //- Pressure [Pa]
        volScalarField& p_;

        //- Translational-Rotational Temperature [K]
        volScalarField& TTR_;

        //- Vibrational Temperature [K]
        volScalarField& TVib_;

        //- Laminar thermal diffusivity [kg/m/s]
        volScalarField alpha_;


    // Protected Member Functions
```

```

        //- Return the enthalpy/internal energy field boundary types
        //- by interrogating the temperature field boundary types
        wordList heBoundaryTypes();

        //- Return the enthalpy/internal energy field boundary base types
        //- by interrogating the temperature field boundary types
        wordList heBoundaryBaseTypes();

        //- No copy construct
        basic2TThermo(const basic2TThermo&) = delete;


public:

    //- Runtime type information
    TypeName("basic2TThermo");


    //- Declare run-time constructor selection table
    declareRunTimeSelectionTable
    (
        autoPtr,
        basic2TThermo,
        fvMesh,
        (const fvMesh& mesh, const word& phaseName),
        (mesh, phaseName)
    );


    //- Declare run-time constructor selection table for dictionary based
    declareRunTimeSelectionTable
    (
        autoPtr,
        basic2TThermo,
        fvMeshDictPhase,
        (const fvMesh& mesh, const word& phaseName, const word& dictName),
        (mesh, phaseName, dictName)
    );


    // Constructors

        //- Construct from mesh and phase name
        basic2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //- Construct from mesh, dictionary and phase name
        basic2TThermo
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName
        );

        //- Construct from mesh, phase name and explicit naming of the
        //- dictionary (so it can be shared amongst phases).
        basic2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    // Selectors

        //- Generic New for each of the related thermodynamics packages
        template<class Thermo>
        static autoPtr<Thermo> New
        (
            const fvMesh&,
            const word& phaseName = word::null
        );

        //- Generic New for each of the related thermodynamics packages
        template<class Thermo>
        static autoPtr<Thermo> New
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName = word::null
        );

```

```
        //- Generic New for each of the related thermodynamics packages
        template<class Thermo>
        static autoPtr<Thermo> New
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictName
        );

        //- Specialisation of the Generic New for basic2TThermo
        static autoPtr<basic2TThermo> New
        (
            const fvMesh&,
            const word& phaseName = word::null
        );


    //- Destructor
    virtual ~basic2TThermo();


    // Member Functions

        //- The dictionary name ("thermophysicalProperties")
        static const word dictName;

        //- The phase property name as property.phase
        //- \sa IOobject::groupName
        static word phasePropertyName(const word& name, const word& phaseName)
        {
            return IOobject::groupName(name, phaseName);
        }

        //- The phase property name as property.phase (using phaseName_)
        //- \sa IOobject::groupName
        word phasePropertyName(const word& name) const
        {
            return IOobject::groupName(name, phaseName_);
        }

        //- The phase property name scoped as scope:property.phase
        //- \sa IOobject::groupName and \sa IOobject::scopedName
        word phaseScopedName(const std::string& scope, const word& name) const
        {
            return IOobject::groupName
            (
                IOobject::scopedName(scope, name),
                phaseName_
            );
        }

        static const basic2TThermo& lookupThermo(const fvPatchScalarField& pf);

        //- Print (filtered) table of thermo names, splits on \c " ,<>"
        static Ostream& printThermoNames
        (
            Ostream& os,
            const wordList& cmptNames,
            const wordList& thermoNames
        );

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&
        ) const;

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&,
            const word&
        ) const;

        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&,
            const word&,
            const word&
        ) const;
```

```
        //- Check that the thermodynamics package is consistent
        //  with energy forms supported by the application
        void validate
        (
            const string& app,
            const word&,
            const word&,
            const word&,
            const word&
        ) const;

        //- Split thermo package name into a list of components names
        //  Splits on \c " ,<>"
        //  \return empty list if the split name does not have the
        //  expected number of components (non-zero).
        static wordList splitThermoName
        (
            const std::string& thermoName,
            const int nExpectedCmpts
        );

        //- Update properties
        virtual void correct() = 0;

        //- Return the name of the thermo physics
        virtual word thermoName() const = 0;

        //- Return true if the equation of state is incompressible
        //  i.e. rho != f(p)
        virtual bool incompressible() const = 0;

        //- Return true if the equation of state is isochoric
        //  i.e. rho = const
        virtual bool isochoric() const = 0;

        //- True if dpdt term should be included in enthalpy equation
        bool dpdt() const noexcept
        {
            return dpdt_;
        }

        //- True if T_TR should be updated
        bool updateTTR() const noexcept
        {
            return TOwnerTR_;
        }

        //- True if T_vib should be updated
        bool updateTVib() const noexcept
        {
            return TOwnerVib_;
        }


        // Access to thermodynamic state variables

            //- Pressure [Pa]
            //  Non-const access allowed for transport equations
            virtual volScalarField& p();

            //- Pressure [Pa]
            virtual const volScalarField& p() const;

            //- Density [kg/m^3]
            virtual tmp<volScalarField> rho() const = 0;

            //- Density for patch [kg/m^3]
            virtual tmp<scalarField> rho(const label patchi) const = 0;

            //- Enthalpy/Internal energy [J/kg]
            //  Non-const access allowed for transport equations
            //  Encapsulates all energy modes
            virtual volScalarField& he() = 0;

            //- Enthalpy/Internal energy [J/kg]
            virtual const volScalarField& he() const = 0;

            //- Enthalpy/Internal energy
            //  for given pressure and temperature [J/kg]
            virtual tmp<volScalarField> he
            (
                const volScalarField& p,
                const volScalarField& TTR
            ) const = 0;

            //- Enthalpy/Internal energy for cell-set [J/kg]
```

```
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const = 0;

            //- Enthalpy/Internal energy for patch [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Chemical enthalpy [J/kg]
            virtual tmp<volScalarField> hc() const = 0;

            //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const labelList& cells
            ) const = 0;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const label patchi
            ) const = 0;

                        //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& esVib,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const labelList& cells
            ) const = 0;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& esVib,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const label patchi
            ) const = 0;

        // Fields derived from thermodynamic state variables

            //- Translational-Rotational Temperature [K]
            virtual const volScalarField& TTR() const;

            //- Vibrational Temperature [K]
            virtual const volScalarField& TVib() const;

            //- Translational-Rotational Temperature [K]
            //  Non-const access allowed for transport equations
            virtual volScalarField& TTR();

            //- Vibrational Temperature [K]
            //  Non-const access allowed for transport equations
            virtual volScalarField& TVib();

            //- Heat capacity at constant pressure [J/kg/K]
            virtual tmp<volScalarField> CpTR() const = 0;

            //- Heat capacity at constant pressure for patch [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Heat capacity using pressure and temperature [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
```

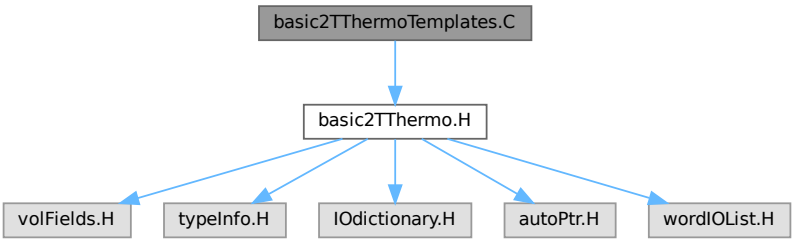
```
                const scalarField& TTR,
                const labelList& cells
            ) const = 0;

            //- Translational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvT() const = 0;

            //- Translational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvT
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Rotational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvR() const = 0;

            //- Rotational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Vibrational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvVib() const = 0;

            //- Vibrational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvVib
            (
                const scalarField& p,
                const scalarField& TTR,
                const scalarField& Tvib,
                const scalar ThetaVib,
                const label patchi
            ) const = 0;

            //- Density from pressure and temperature from EoS
            virtual tmp<scalarField> rhoEoS
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const = 0;

            //- Gamma = CpTR/CvT []
            virtual tmp<volScalarField> gamma() const = 0;

            //- Gamma = CpTR/CvT for patch []
            virtual tmp<scalarField> gamma
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Translational-Rotational heat capacity at constant pressure/volume [J/kg/K]
            virtual tmp<volScalarField> CpvTR() const = 0;

            //- Translational-Rotational heat capacity at constant pressure/volume for patch [J/kg/K]
            virtual tmp<scalarField> CpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Translational-Rotational heat capacity ratio []
            virtual tmp<volScalarField> CpByCpvTR() const = 0;

            //- Translational-Rotational heat capacity ratio for patch []
            virtual tmp<scalarField> CpByCpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const = 0;

            //- Molecular weight [kg/kmol]
            virtual tmp<volScalarField> W() const = 0;


        // Access to transport state variables

            //- Thermal diffusivity for enthalpy of mixture [kg/m/s]
```

```
            virtual const volScalarField& alpha() const;

            //- Thermal diffusivity for enthalpy of mixture for patch [kg/m/s]
            virtual const scalarField& alpha
            (
                const label patchi
            ) const;


        // Fields derived from transport state variables

            //- Translational-rotational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaTR() const = 0;

            //- Translational-rotational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaTR
            (
                const label patchi
            ) const = 0;

            //- Vibrational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaVib() const = 0;

            //- Vibrational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaVib
            (
                const label patchi
            ) const = 0;

             //- Translational-rotational thermal diffusivity for energy of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaheTR() const = 0;

            //- Translational-rotational thermal diffusivity for energy of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaheTR(const label patchi) const = 0;

            //- Effective thermal diffusivity for temperature
            //  of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaEff
            (
                const volScalarField&
            ) const = 0;

            //- Effective thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const = 0;

            //- Effective thermal diffusivity of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaEff
            (
                const volScalarField& alphat
            ) const = 0;

            //- Effective thermal diffusivity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const = 0;


        //- Read thermophysical properties dictionary
        virtual bool read();
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#ifdef NoRepository
    #include "basic2TThermoTemplates.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.9 basic2TThermo/basic2TThermoTemplates.C File Reference

`#include "basic2TThermo.H"`

Include dependency graph for basic2TThermo/basic2TThermoTemplates.C:



7.10 basic2TThermo/basic2TThermoTemplates.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
    Copyright (C) 2017-2021 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "basic2TThermo.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class TThermo, class ThermoConstructTable>
typename ThermoConstructTable::mapped_type
Foam::basic2TThermo::getThermoOrDie
(
    const dictionary& thermoTypeDict,
    ThermoConstructTable& thermoTable,
    const word& thermoTypeName,
    const wordList& cmptNames
)
{
    // Lookup the thermo package

    auto ctorIter = thermoTable.cfind(thermoTypeName);

    // Print error message if package not found in the table
    if (!ctorIter.good())
    {
        FatalIOErrorInLookup
```

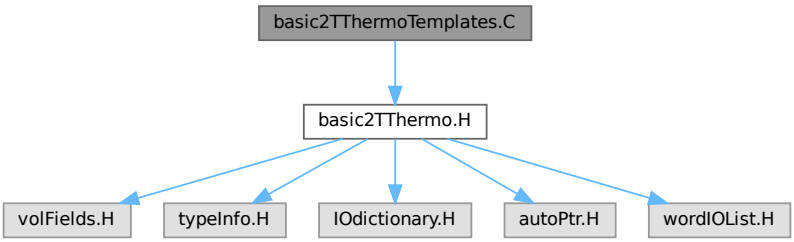
```
        (
            thermoTypeDict,
            TThermo::typeName,
            word::null, // Suppress long name? Just output dictionary (above)
            thermoTable
        );

        basic2TThermo::printThermoNames
        (
            FatalIOError,
            cmptNames,
            thermoTable.sortedToc()
        ) « exit(FatalIOError);

        // return nullptr;
    }

    return ctorIter.val();
}


template<class TThermo, class ThermoConstructTable>
typename ThermoConstructTable::mapped_type
Foam::basic2TThermo::getThermoOrDie
(
    const dictionary& thermoDict,
    ThermoConstructTable& thermoTable
)
{
    const dictionary* dictptr = thermoDict.findDict("thermoType");

    if (dictptr)
    {
        const auto& thermoTypeDict = *dictptr;

        const wordList* cmptHeaderPtr = &(wordList::null());

        // TThermo package name, constructed from components
        const word thermoTypeName
        (
            basic2TThermo::makeThermoName(thermoTypeDict, cmptHeaderPtr)
        );

        Info« "Selecting thermodynamics package " « thermoTypeDict « endl;

        return getThermoOrDie<TThermo, ThermoConstructTable>
        (
            thermoTypeDict,
            thermoTable,
            thermoTypeName,
            *cmptHeaderPtr
        );
    }
    else
    {
        const word thermoTypeName(thermoDict.get<word>("thermoType"));

        Info« "Selecting thermodynamics package " « thermoTypeName « endl;

        auto ctorIter = thermoTable.cfind(thermoTypeName);

        if (!ctorIter.good())
        {
            FatalIOErrorInLookup
            (
                thermoDict,
                TThermo::typeName,
                thermoTypeName,
                thermoTable
            ) « exit(FatalIOError);
        }

        return ctorIter.val();
    }
}


// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

template<class TThermo>
Foam::autoPtr<TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    IOdictionary thermoDict
```

```
    (
        IOobject
        (
            phasePropertyName(dictName, phaseName),
            mesh.time().constant(),
            mesh,
            IOobject::MUST_READ,
            IOobject::NO_WRITE,
            IOobject::NO_REGISTER
        )
    );

    auto* ctorPtr = getThermoOrDie<TThermo>
    (
        thermoDict,
        *(TThermo::fvMeshConstructorTablePtr_)
    );

    return autoPtr<TThermo>(ctorPtr(mesh, phaseName));
}


template<class TThermo>
Foam::autoPtr<TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
{
    auto* ctorPtr = getThermoOrDie<TThermo>
    (
        dict,
        *(TThermo::dictionaryConstructorTablePtr_)
    );

    return autoPtr<TThermo>(ctorPtr(mesh, dict, phaseName));
}


template<class TThermo>
Foam::autoPtr<TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictName
)
{
    IOdictionary thermoDict
    (
        IOobject
        (
            dictName,
            mesh.time().constant(),
            mesh,
            IOobject::MUST_READ,
            IOobject::NO_WRITE,
            IOobject::NO_REGISTER
        )
    );

    auto* ctorPtr = getThermoOrDie<TThermo>
    (
        thermoDict,
        *(TThermo::fvMeshDictPhaseConstructorTablePtr_)
    );

    return autoPtr<TThermo>(ctorPtr(mesh, phaseName, dictName));
}


// ************************************************************************* //
```

7.11 lnInclude/basic2TThermoTemplates.C File Reference

`#include "basic2TThermo.H"`

Include dependency graph for lnInclude/basic2TThermoTemplates.C:



7.12 lnInclude/basic2TThermoTemplates.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
    Copyright (C) 2017-2021 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "basic2TThermo.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class TThermo, class ThermoConstructTable>
typename ThermoConstructTable::mapped_type
Foam::basic2TThermo::getThermoOrDie
(
    const dictionary& thermoTypeDict,
    ThermoConstructTable& thermoTable,
    const word& thermoTypeName,
    const wordList& cmptNames
)
{
    // Lookup the thermo package

    auto ctorIter = thermoTable.cfind(thermoTypeName);

    // Print error message if package not found in the table
    if (!ctorIter.good())
    {
        FatalIOErrorInLookup
```

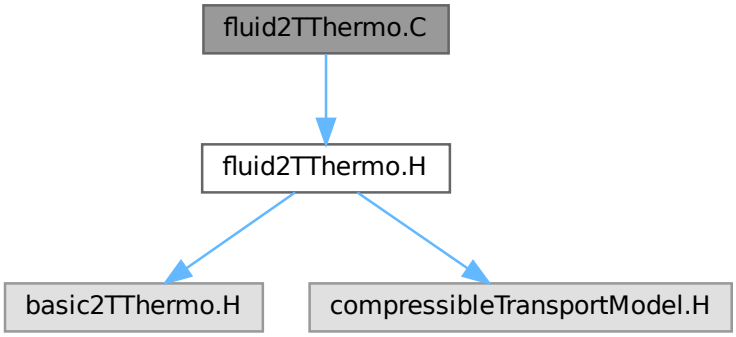
```
        (
            thermoTypeDict,
            TThermo::typeName,
            word::null, // Suppress long name? Just output dictionary (above)
            thermoTable
        );

        basic2TThermo::printThermoNames
        (
            FatalIOError,
            cmptNames,
            thermoTable.sortedToc()
        ) « exit(FatalIOError);

        // return nullptr;
    }

    return ctorIter.val();
}


template<class TThermo, class ThermoConstructTable>
typename ThermoConstructTable::mapped_type
Foam::basic2TThermo::getThermoOrDie
(
    const dictionary& thermoDict,
    ThermoConstructTable& thermoTable
)
{
    const dictionary* dictptr = thermoDict.findDict("thermoType");

    if (dictptr)
    {
        const auto& thermoTypeDict = *dictptr;

        const wordList* cmptHeaderPtr = &(wordList::null());

        // TThermo package name, constructed from components
        const word thermoTypeName
        (
            basic2TThermo::makeThermoName(thermoTypeDict, cmptHeaderPtr)
        );

        Info« "Selecting thermodynamics package " « thermoTypeDict « endl;

        return getThermoOrDie<TThermo, ThermoConstructTable>
        (
            thermoTypeDict,
            thermoTable,
            thermoTypeName,
            *cmptHeaderPtr
        );
    }
    else
    {
        const word thermoTypeName(thermoDict.get<word>("thermoType"));

        Info« "Selecting thermodynamics package " « thermoTypeName « endl;

        auto ctorIter = thermoTable.cfind(thermoTypeName);

        if (!ctorIter.good())
        {
            FatalIOErrorInLookup
            (
                thermoDict,
                TThermo::typeName,
                thermoTypeName,
                thermoTable
            ) « exit(FatalIOError);
        }

        return ctorIter.val();
    }
}


// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

template<class TThermo>
Foam::autoPtr<TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    IOdictionary thermoDict
```

```
    (
        IOobject
        (
            phasePropertyName(dictName, phaseName),
            mesh.time().constant(),
            mesh,
            IOobject::MUST_READ,
            IOobject::NO_WRITE,
            IOobject::NO_REGISTER
        )
    );

    auto* ctorPtr = getThermoOrDie<TThermo>
    (
        thermoDict,
        *(TThermo::fvMeshConstructorTablePtr_)
    );

    return autoPtr<TThermo>(ctorPtr(mesh, phaseName));
}


template<class TThermo>
Foam::autoPtr<TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
{
    auto* ctorPtr = getThermoOrDie<TThermo>
    (
        dict,
        *(TThermo::dictionaryConstructorTablePtr_)
    );

    return autoPtr<TThermo>(ctorPtr(mesh, dict, phaseName));
}


template<class TThermo>
Foam::autoPtr<TThermo> Foam::basic2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictName
)
{
    IOdictionary thermoDict
    (
        IOobject
        (
            dictName,
            mesh.time().constant(),
            mesh,
            IOobject::MUST_READ,
            IOobject::NO_WRITE,
            IOobject::NO_REGISTER
        )
    );

    auto* ctorPtr = getThermoOrDie<TThermo>
    (
        thermoDict,
        *(TThermo::fvMeshDictPhaseConstructorTablePtr_)
    );

    return autoPtr<TThermo>(ctorPtr(mesh, phaseName, dictName));
}


// ************************************************************************* //
```

7.13 fluid2TThermo/fluid2TThermo.C File Reference

`#include "fluid2TThermo.H"`

Include dependency graph for fluid2TThermo/fluid2TThermo.C:



Namespaces

- namespace Foam

Functions

- Foam::defineRunTimeSelectionTable (fluid2TThermo, fvMesh)
- Foam::defineRunTimeSelectionTable (fluid2TThermo, fvMeshDictPhase)

7.14 fluid2TThermo/fluid2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012 OpenFOAM Foundation
    Copyright (C) 2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.
```

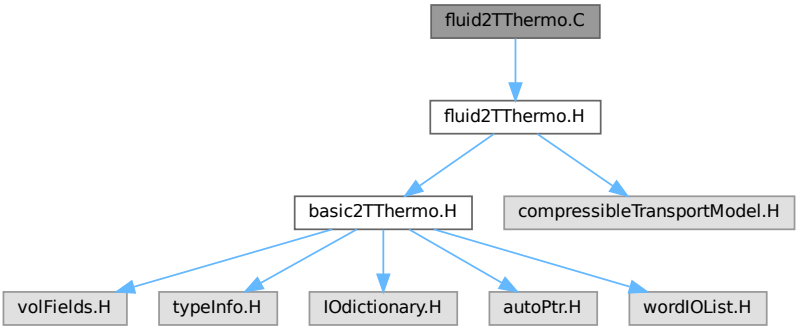
```

\*---------------------------------------------------------------------------*/

#include "fluid2TThermo.H"

/* * * * * * * * * * * * * * private static data * * * * * * * * * * * * * * */

namespace Foam
{
    defineTypeNameAndDebug(fluid2TThermo, 0);
    defineRunTimeSelectionTable(fluid2TThermo, fvMesh);
    defineRunTimeSelectionTable(fluid2TThermo, fvMeshDictPhase);
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

Foam::fluid2TThermo::fluid2TThermo(const fvMesh& mesh, const word& phaseName)
:
    basic2TThermo(mesh, phaseName)
{}



Foam::fluid2TThermo::fluid2TThermo
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
:
    basic2TThermo(mesh, dict, phaseName)
{}


Foam::fluid2TThermo::fluid2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    basic2TThermo(mesh, phaseName, dictionaryName)
{}


// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

Foam::autoPtr<Foam::fluid2TThermo> Foam::fluid2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    return basic2TThermo::New<fluid2TThermo>(mesh, phaseName);
}


Foam::autoPtr<Foam::fluid2TThermo> Foam::fluid2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictName
)
{
    return basic2TThermo::New<fluid2TThermo>(mesh, phaseName, dictName);
}


// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

Foam::fluid2TThermo::~fluid2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

Foam::tmp<Foam::volScalarField> Foam::fluid2TThermo::nu() const
{
    return mu()/rho();
}


Foam::tmp<Foam::scalarField> Foam::fluid2TThermo::nu(const label patchi) const
{
    return mu(patchi)/rho(patchi);
}
```

```
00113
00114
00115 // ************************************************************************* //
```

7.15 InInclude/fluid2TThermo.C File Reference

`#include "fluid2TThermo.H"`

Include dependency graph for InInclude/fluid2TThermo.C:



Namespaces

- namespace Foam

Functions

- Foam::defineRunTimeSelectionTable (fluid2TThermo, fvMesh)
- Foam::defineRunTimeSelectionTable (fluid2TThermo, fvMeshDictPhase)

7.16 InInclude/fluid2TThermo.C

Go to the documentation of this file.

```
00001 /*-------------------------------------------------------------------------*\
00002   =========                 |
00003   \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
00004    \\    /   O peration     |
00005     \\  /    A nd           | www.openfoam.com
00006      \\/     M anipulation  |
00007 -------------------------------------------------------------------------------
00008     Copyright (C) 2012 OpenFOAM Foundation
00009     Copyright (C) 2017 OpenCFD Ltd.
00010 -------------------------------------------------------------------------------
00011 License
00012     This file is part of OpenFOAM.
00013
00014     OpenFOAM is free software: you can redistribute it and/or modify it
00015     under the terms of the GNU General Public License as published by
00016     the Free Software Foundation, either version 3 of the License, or
00017     (at your option) any later version.
00018
00019     OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
00020     ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
00021     FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
```

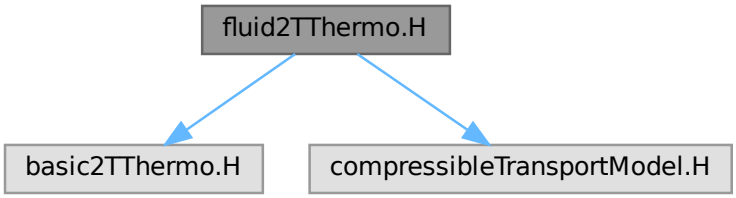
```
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "fluid2TThermo.H"

/* * * * * * * * * * * * * * private static data * * * * * * * * * * * * * * */

namespace Foam
{
    defineTypeNameAndDebug(fluid2TThermo, 0);
    defineRunTimeSelectionTable(fluid2TThermo, fvMesh);
    defineRunTimeSelectionTable(fluid2TThermo, fvMeshDictPhase);
}


// * * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

Foam::fluid2TThermo::fluid2TThermo(const fvMesh& mesh, const word& phaseName)
:
    basic2TThermo(mesh, phaseName)
{}



Foam::fluid2TThermo::fluid2TThermo
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
:
    basic2TThermo(mesh, dict, phaseName)
{}


Foam::fluid2TThermo::fluid2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    basic2TThermo(mesh, phaseName, dictionaryName)
{}


// * * * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

Foam::autoPtr<Foam::fluid2TThermo> Foam::fluid2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    return basic2TThermo::New<fluid2TThermo>(mesh, phaseName);
}


Foam::autoPtr<Foam::fluid2TThermo> Foam::fluid2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictName
)
{
    return basic2TThermo::New<fluid2TThermo>(mesh, phaseName, dictName);
}


// * * * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

Foam::fluid2TThermo::~fluid2TThermo()
{}


// * * * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

Foam::tmp<Foam::volScalarField> Foam::fluid2TThermo::nu() const
{
    return mu()/rho();
}


```

```
Foam::tmp<Foam::scalarField> Foam::fluid2TThermo::nu(const label patchi) const
{
    return mu(patchi)/rho(patchi);
}


// ************************************************************************* //
```

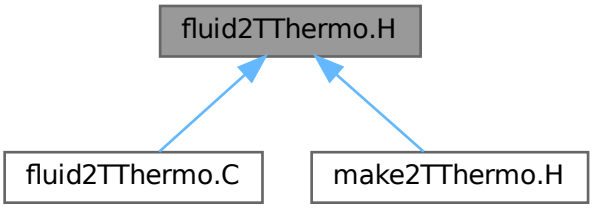
7.17 fluid2TThermo/fluid2TThermo.H File Reference

```
#include "basic2TThermo.H"
#include "compressibleTransportModel.H"
```

Include dependency graph for fluid2TThermo/fluid2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::fluid2TThermo

Namespaces

- namespace Foam

7.18 fluid2TThermo/fluid2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
    Copyright (C) 2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::fluid2TThermo

Description
    Fundamental fluid thermodynamic properties

SourceFiles
    fluid2TThermo.C

\*-------------------------------------------------------------------------*/

#ifndef fluid2TThermo_H
#define fluid2TThermo_H

#include "basic2TThermo.H"
#include "compressibleTransportModel.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*-------------------------------------------------------------------------*\
                        Class fluid2TThermo Declaration
\*-------------------------------------------------------------------------*/

class fluid2TThermo
:
    public basic2TThermo,
    public compressibleTransportModel
{

public:

    //- Runtime type information
    TypeName("fluid2TThermo");


    //- Declare run-time constructor selection table
    declareRunTimeSelectionTable
    (
        autoPtr,
        fluid2TThermo,
        fvMesh,
        (const fvMesh& mesh, const word& phaseName),
        (mesh, phaseName)
    );


    //- Declare run-time constructor selection table for dictionary based
    declareRunTimeSelectionTable
    (
        autoPtr,
        fluid2TThermo,
        fvMeshDictPhase,
        (const fvMesh& mesh, const word& phaseName, const word& dictName),
```

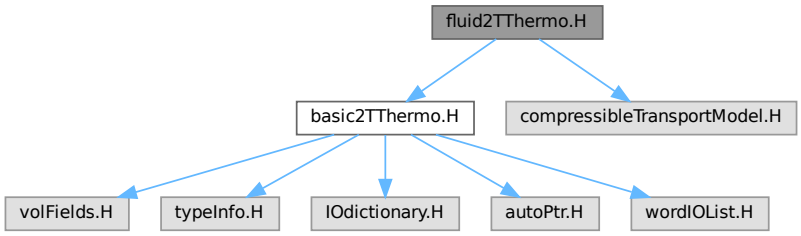
```
        (mesh, phaseName, dictName)
    );

    // Constructors

        //- Construct from mesh and phase name
        fluid2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //- Construct from mesh and phase name
        fluid2TThermo
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName
        );


        //- Construct from mesh,dictionary,phase name with a single temperature
        fluid2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Selector
    static autoPtr<fluid2TThermo> New
    (
        const fvMesh&,
        const word& phaseName=word::null
    );


    //- Selector
    static autoPtr<fluid2TThermo> New
    (
        const fvMesh&,
        const word& phaseName,
        const word& dictName
    );


    //- Destructor
    virtual ~fluid2TThermo();


    // Member functions

        // Access to thermodynamic state variables

            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution
            //  Limit thermo rho between rhoMin and rhoMax
            virtual void correctRho
            (
                const volScalarField& deltaRho,
                const dimensionedScalar& rhoMin,
                const dimensionedScalar& rhoMax
            ) = 0;


            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution
            virtual void correctRho
            (
                const volScalarField& deltaRho
            ) = 0;

            //- Compressibility [s^2/m^2]
            virtual const volScalarField& psi() const = 0;


        // Access to transport state variables

            //- Dynamic viscosity of mixture [kg/m/s]
            virtual tmp<volScalarField> mu() const = 0;

            //- Dynamic viscosity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> mu(const label patchi) const = 0;

            //- Kinematic viscosity of mixture [m^2/s]
            virtual tmp<volScalarField> nu() const;
```

```
00170
00171             //- Kinematic viscosity of mixture for patch [m^2/s]
00172             virtual tmp<scalarField> nu(const label patchi) const;
00173 };
00174
00175
00176 // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
00177
00178 } // End namespace Foam
00179
00180 // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
00181
00182 #endif
00183
00184 // ************************************************************************* //
```

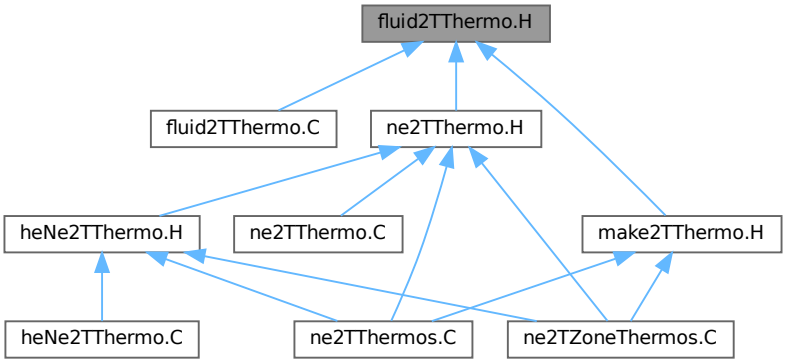
7.19 lnInclude/fluid2TThermo.H File Reference

```
#include "basic2TThermo.H"
#include "compressibleTransportModel.H"
```

Include dependency graph for lnInclude/fluid2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::fluid2TThermo

Namespaces

- namespace Foam

7.20 InInclude/fluid2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
    Copyright (C) 2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::fluid2TThermo

Description
    Fundamental fluid thermodynamic properties

SourceFiles
    fluid2TThermo.C

\*-------------------------------------------------------------------------*/

#ifndef fluid2TThermo_H
#define fluid2TThermo_H

#include "basic2TThermo.H"
#include "compressibleTransportModel.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*-------------------------------------------------------------------------*\
                        Class fluid2TThermo Declaration
\*-------------------------------------------------------------------------*/

class fluid2TThermo
:
    public basic2TThermo,
    public compressibleTransportModel
{

public:

    //- Runtime type information
    TypeName("fluid2TThermo");


    //- Declare run-time constructor selection table
    declareRunTimeSelectionTable
    (
        autoPtr,
        fluid2TThermo,
        fvMesh,
        (const fvMesh& mesh, const word& phaseName),
        (mesh, phaseName)
    );
```

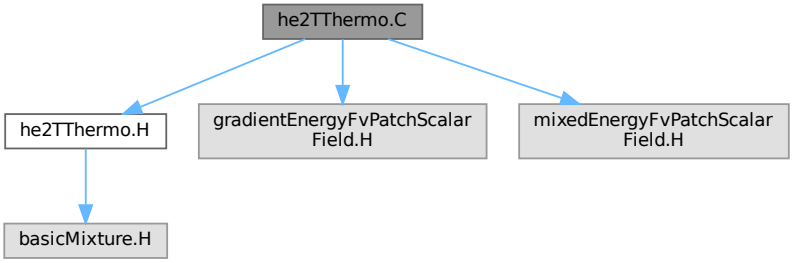
```
    //- Declare run-time constructor selection table for dictionary based
    declareRunTimeSelectionTable
    (
        autoPtr,
        fluid2TThermo,
        fvMeshDictPhase,
        (const fvMesh& mesh, const word& phaseName, const word& dictName),
        (mesh, phaseName, dictName)
    );

    // Constructors

        //- Construct from mesh and phase name
        fluid2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //- Construct from mesh and phase name
        fluid2TThermo
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName
        );


        //- Construct from mesh,dictionary,phase name with a single temperature
        fluid2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Selector
    static autoPtr<fluid2TThermo> New
    (
        const fvMesh&,
        const word& phaseName=word::null
    );


    //- Selector
    static autoPtr<fluid2TThermo> New
    (
        const fvMesh&,
        const word& phaseName,
        const word& dictName
    );


    //- Destructor
    virtual ~fluid2TThermo();


    // Member functions

        // Access to thermodynamic state variables

            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution
            //  Limit thermo rho between rhoMin and rhoMax
            virtual void correctRho
            (
                const volScalarField& deltaRho,
                const dimensionedScalar& rhoMin,
                const dimensionedScalar& rhoMax
            ) = 0;


            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution
            virtual void correctRho
            (
                const volScalarField& deltaRho
            ) = 0;

            //- Compressibility [s^2/m^2]
            virtual const volScalarField& psi() const = 0;


        // Access to transport state variables
```

```

            //- Dynamic viscosity of mixture [kg/m/s]
            virtual tmp<volScalarField> mu() const = 0;

            //- Dynamic viscosity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> mu(const label patchi) const = 0;

            //- Kinematic viscosity of mixture [m^2/s]
            virtual tmp<volScalarField> nu() const;

            //- Kinematic viscosity of mixture for patch [m^2/s]
            virtual tmp<scalarField> nu(const label patchi) const;
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.21 he2TThermo/he2TThermo.C File Reference

```
#include "he2TThermo.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"
```

Include dependency graph for he2TThermo/he2TThermo.C:



7.22 he2TThermo/he2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2015-2021 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.
```

```

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "he2TThermo.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"

// * * * * * * * * * * * * Protected Member Functions  * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
void Foam::he2TThermo<Basic2TThermo, MixtureType>::
heBoundaryCorrection(volScalarField& h)
{
    volScalarField::Boundary& hBf = h.boundaryFieldRef();

    forAll(hBf, patchi)
    {
        if (isA<gradientEnergyFvPatchScalarField>(hBf[patchi]))
        {
            refCast<gradientEnergyFvPatchScalarField>(hBf[patchi]).gradient()
                = hBf[patchi].fvPatchField::snGrad();
        }
        else if (isA<mixedEnergyFvPatchScalarField>(hBf[patchi]))
        {
            refCast<mixedEnergyFvPatchScalarField>(hBf[patchi]).refGrad()
                = hBf[patchi].fvPatchField::snGrad();
        }
    }
}


template<class Basic2TThermo, class MixtureType>
void Foam::he2TThermo<Basic2TThermo, MixtureType>::init
(
    const volScalarField& p,
    const volScalarField& TTR,
    const volScalarField& TVib,
    volScalarField& he
)
{
    scalarField& heCells = he.primitiveFieldRef();
    const scalarField& pCells = p.primitiveField();
    const scalarField& TTRCells = TTR.primitiveField();
    //const scalarField& TVibCells = TVib.primitiveField();

    forAll(heCells, celli)
    {
        heCells[celli] =
            this->cellMixture(celli).HE(pCells[celli], TTRCells[celli]);
    }

    volScalarField::Boundary& heBf = he.boundaryFieldRef();

    forAll(heBf, patchi)
    {
        heBf[patchi] == this->he
        (
            p.boundaryField()[patchi],
            TTR.boundaryField()[patchi],
            patchi
        );

        heBf[patchi].useImplicit(TTR.boundaryField()[patchi].useImplicit());
    }

    this->heBoundaryCorrection(he);

    // Note: TTR does not have oldTime
    if (p.nOldTimes())
    {
        init(p.oldTime(), TTR.oldTime(), TVib.oldTime(), he.oldTime());
    }
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::he2TThermo
```

```
(
    const fvMesh& mesh,
    const word& phaseName
)
:
    Basic2TThermo(mesh, phaseName),
    MixtureType(*this, mesh, phaseName),

    he_
    (
        IOobject
        (
            Basic2TThermo::phasePropertyName
            (
                MixtureType::thermoType::heName()
            ),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE
        ),
        mesh,
        dimEnergy/dimMass,
        this->heBoundaryTypes(),
        this->heBoundaryBaseTypes()
    )
{
    init(this->p_, this->TTR_, this->TVib_, he_);
}


template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::he2TThermo
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
:
    Basic2TThermo(mesh, dict, phaseName),
    MixtureType(*this, mesh, phaseName),

    he_
    (
        IOobject
        (
            Basic2TThermo::phasePropertyName
            (
                MixtureType::thermoType::heName()
            ),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE
        ),
        mesh,
        dimEnergy/dimMass,
        this->heBoundaryTypes(),
        this->heBoundaryBaseTypes()
    )
{
    init(this->p_, this->TTR_, this->TVib_, he_);
}


template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::he2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    Basic2TThermo(mesh, phaseName, dictionaryName),
    MixtureType(*this, mesh, phaseName),

    he_
    (
        IOobject
        (
            Basic2TThermo::phasePropertyName
            (
                MixtureType::thermoType::heName()
            ),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
```

```
            IOobject::NO_WRITE
        ),
        mesh,
        dimEnergy/dimMass,
        this->heBoundaryTypes(),
        this->heBoundaryBaseTypes()
    )
{
    init(this->p_, this->TTR_, this->TVib_, he_);
}


// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::~he2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::he
(
    const volScalarField& p,
    const volScalarField& TTR
) const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto the = volScalarField::New
    (
        "he",
        IOobject::NO_REGISTER,
        mesh,
        he_.dimensions()
    );
    auto& he = the.ref();

    scalarField& heCells = he.primitiveFieldRef();
    const scalarField& pCells = p;
    const scalarField& TTRCells = TTR;

    forAll(heCells, celli)
    {
        heCells[celli] =
            this->cellMixture(celli).HE(pCells[celli], TTRCells[celli]);
    }

    volScalarField::Boundary& heBf = he.boundaryFieldRef();

    forAll(heBf, patchi)
    {
        scalarField& hep = heBf[patchi];
        const scalarField& pp = p.boundaryField()[patchi];
        const scalarField& TTRp = TTR.boundaryField()[patchi];

        forAll(hep, facei)
        {
            hep[facei] =
                this->patchFaceMixture(patchi, facei).HE(pp[facei], TTRp[facei]);
        }
    }

    return the;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::he
(
    const scalarField& p,
    const scalarField& TTR,
    const labelList& cells
) const
{
    auto the = tmp<scalarField>::New(TTR.size());
    auto& he = the.ref();

    forAll(TTR, celli)
    {
        he[celli] = this->cellMixture(cells[celli]).HE(p[celli], TTR[celli]);
    }

    return the;
}
```

```


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::he
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto the = tmp<scalarField>::New(TTR.size());
    auto& he = the.ref();

    forAll(TTR, facei)
    {
        he[facei] =
            this->patchFaceMixture(patchi, facei).HE(p[facei], TTR[facei]);
    }

    return the;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::hc() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto thc = volScalarField::New
    (
        "hc",
        IOobject::NO_REGISTER,
        mesh,
        he_.dimensions()
    );
    auto& hcf = thc.ref();

    scalarField& hcCells = hcf.primitiveFieldRef();

    forAll(hcCells, celli)
    {
        hcCells[celli] = this->cellMixture(celli).Hc();
    }

    volScalarField::Boundary& hcfBf = hcf.boundaryFieldRef();

    forAll(hcfBf, patchi)
    {
        scalarField& hcp = hcfBf[patchi];

        forAll(hcp, facei)
        {
            hcp[facei] = this->patchFaceMixture(patchi, facei).Hc();
        }
    }

    return thc;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::CpTR
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCpTR = tmp<scalarField>::New(TTR.size());
    auto& cpTR = tCpTR.ref();

    forAll(TTR, facei)
    {
        cpTR[facei] =
            this->patchFaceMixture(patchi, facei).CpTR(p[facei], TTR[facei]);
    }

    return tCpTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpTR
(
    const scalarField& p,
```

```
    const scalarField& TTR,
    const labelList& cells
) const
{
    auto tCpTR = tmp<scalarField>::New(TTR.size());
    auto& CpTR = tCpTR.ref();

    forAll(cells, i)
    {
        const label celli = cells[i];
        CpTR[i] = this->cellMixture(celli).CpTR(p[i], TTR[i]);
    }

    return tCpTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpTR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCpTR = volScalarField::New
    (
        "CpTR",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cpTR = tCpTR.ref();

    forAll(this->TTR_, celli)
    {
        cpTR[celli] =
            this->cellMixture(celli).CpTR(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& cpTRBf = cpTR.boundaryFieldRef();

    forAll(cpTRBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pCpTR = cpTRBf[patchi];

        forAll(pTTR, facei)
        {
            pCpTR[facei] =
                this->patchFaceMixture(patchi, facei).CpTR(pp[facei], pTTR[facei]);
        }
    }

    return tCpTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvT
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCvT = tmp<scalarField>::New(TTR.size());
    auto& cvT = tCvT.ref();

    forAll(TTR, facei)
    {
        cvT[facei] =
            this->patchFaceMixture(patchi, facei).CvT(p[facei], TTR[facei]);
    }

    return tCvT;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvR
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
```

```
    auto tCvR = tmp<scalarField>::New(TTR.size());
    auto& cvR = tCvR.ref();

    forAll(TTR, facei)
    {
        cvR[facei] =
            this->patchFaceMixture(patchi, facei).CvR(p[facei], TTR[facei]);
    }

    return tCvR;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvVib
(
    const scalarField& p,
    const scalarField& TTR,
    const scalarField& TVib,
    const scalar ThetaVib,
    const label patchi
) const
{
    auto tCvVib = tmp<scalarField>::New(TVib.size());
    auto& cvVib = tCvVib.ref();

    forAll(TVib, facei)
    {
        cvVib[facei] =
            this->patchFaceMixture(patchi, facei).CvVib(p[facei], TTR[facei], TVib[facei], ThetaVib);
    }

    return tCvVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::rhoEoS
(
    const scalarField& p,
    const scalarField& TTR,
    const labelList& cells
) const
{
    auto tRho = tmp<scalarField>::New(TTR.size());
    auto& rho = tRho.ref();

    forAll(cells, i)
    {
        const label celli = cells[i];
        rho[i] = this->cellMixture(celli).rho(p[i], TTR[i]);
    }

    return tRho;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvT() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCvT = volScalarField::New
    (
        "CvT",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cvT = tCvT.ref();

    forAll(this->TTR_, celli)
    {
        cvT[celli] =
            this->cellMixture(celli).CvT(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& cvTBf = cvT.boundaryFieldRef();

    forAll(cvTBf, patchi)
    {
        cvTBf[patchi] = CvT
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
```

```
            patchi
        );
    }

    return tCvT;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCvR = volScalarField::New
    (
        "CvR",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cvR = tCvR.ref();

    forAll(this->TTR_, celli)
    {
        cvR[celli] =
            this->cellMixture(celli).CvR(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& cvRBf = cvR.boundaryFieldRef();

    forAll(cvRBf, patchi)
    {
        cvRBf[patchi] = CvR
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
        );
    }

    return tCvR;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvVib() const
{
    const fvMesh& mesh = this->TVib_.mesh();
    const scalar theta = this->cellMixture(0).ThetaVib();  // getter on Thermo2T

    auto tCvVib = volScalarField::New
    (
        "CvVib",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cvVib = tCvVib.ref();

    forAll(this->TVib_, celli)
    {
        cvVib[celli] =
            this->cellMixture(celli).CvVib(this->p_[celli], this->TTR_[celli] ,this->TVib_[celli],
theta);
    }

    volScalarField::Boundary& cvVibBf = cvVib.boundaryFieldRef();

    forAll(cvVibBf, patchi)
    {
        cvVibBf[patchi] = CvVib
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            this->TVib_.boundaryField()[patchi],
            theta,
            patchi
        );
    }

    return tCvVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::gamma
(
```

```
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tgamma = tmp<scalarField>::New(TTR.size());
    auto& gamma = tgamma.ref();

    forAll(TTR, facei)
    {
        gamma[facei] =
            this->patchFaceMixture(patchi, facei).gamma(p[facei], TTR[facei]);
    }

    return tgamma;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::gamma() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tgamma = volScalarField::New
    (
        "gamma",
        IOobject::NO_REGISTER,
        mesh,
        dimless
    );
    auto& gamma = tgamma.ref();

    forAll(this->TTR_, celli)
    {
        gamma[celli] =
            this->cellMixture(celli).gamma(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& gammaBf = gamma.boundaryFieldRef();

    forAll(gammaBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pgamma = gammaBf[patchi];

        forAll(pTTR, facei)
        {
            pgamma[facei] = this->patchFaceMixture(patchi, facei).gamma
            (
                pp[facei],
                pTTR[facei]
            );
        }
    }

    return tgamma;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::CpvTR
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCpvTR = tmp<scalarField>::New(TTR.size());
    auto& CpvTR = tCpvTR.ref();

    forAll(TTR, facei)
    {
        CpvTR[facei] =
            this->patchFaceMixture(patchi, facei).CpvTR(p[facei], TTR[facei]);
    }

    return tCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpvTR() const
{
    const fvMesh& mesh = this->TTR_.mesh();
```

```

    auto tCpvTR = volScalarField::New
    (
        "CpvTR",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& CpvTR = tCpvTR.ref();

    forAll(this->TTR_, celli)
    {
        CpvTR[celli] =
            this->cellMixture(celli).CpvTR(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& CpvTRBf = CpvTR.boundaryFieldRef();

    forAll(CpvTRBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pCpvTR = CpvTRBf[patchi];

        forAll(pTTR, facei)
        {
            pCpvTR[facei] =
                this->patchFaceMixture(patchi, facei).CpvTR(pp[facei], pTTR[facei]);
        }
    }

    return tCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::CpByCpvTR
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCpByCpvTR = tmp<scalarField>::New(TTR.size());
    auto& CpByCpvTR = tCpByCpvTR.ref();

    forAll(TTR, facei)
    {
        CpByCpvTR[facei] =
            this->patchFaceMixture(patchi, facei).CpByCpvTR(p[facei], TTR[facei]);
    }

    return tCpByCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpByCpvTR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCpByCpvTR = volScalarField::New
    (
        "CpByCpvTR",
        IOobject::NO_REGISTER,
        mesh,
        dimless
    );
    auto& CpByCpvTR = tCpByCpvTR.ref();

    forAll(this->TTR_, celli)
    {
        CpByCpvTR[celli] = this->cellMixture(celli).CpByCpvTR
        (
            this->p_[celli],
            this->TTR_[celli]
        );
    }

    volScalarField::Boundary& CpByCpvTRBf =
        CpByCpvTR.boundaryFieldRef();

    forAll(CpByCpvTRBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
```

```
        fvPatchScalarField& pCpByCpvTR = CpByCpvTRBf[patchi];

        forAll(pTTR, facei)
        {
            pCpByCpvTR[facei] = this->patchFaceMixture(patchi, facei).CpByCpvTR
            (
                pp[facei],
                pTTR[facei]
            );
        }
    }

    return tCpByCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::THE_TR
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const labelList& cells
) const
{
    auto tTTR = tmp<scalarField>::New(h.size());
    auto& TTR = tTTR.ref();

    forAll(h, celli)
    {
        TTR[celli] =
            this->cellMixture(cells[celli]).THE_TR(h[celli], p[celli], TTR0[celli]);
    }

    return tTTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::THE_TR
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const label patchi
) const
{

    auto tTTR = tmp<scalarField>::New(h.size());
    auto& TTR = tTTR.ref();

    forAll(h, facei)
    {
        TTR[facei] = this->patchFaceMixture
        (
            patchi,
            facei
        ).THE_TR(h[facei], p[facei], TTR0[facei]);
    }

    return tTTR;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::TEH_Vib
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const scalarField& TVib0,
    const labelList& cells
) const
{

    auto tTVib = tmp<scalarField>::New(h.size());
    auto& TVib = tTVib.ref();

    forAll(h, celli)
    {
        TVib[celli] =
            this->cellMixture(cells[celli]).TEH_Vib(h[celli], p[celli], TTR0[celli], TVib0[celli]);
    }

    return tTVib;
}


```

```
template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::TEH_Vib
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const scalarField& TVib0,
    const label patchi
) const
{

    auto tTVib = tmp<scalarField>::New(h.size());
    auto& TVib = tTVib.ref();

    forAll(h, facei)
    {
        TVib[facei] = this->patchFaceMixture
        (
            patchi,
            facei
        ).TEH_Vib(h[facei], p[facei], TTR0[facei], TVib0[facei]);
    }

    return tTVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::W
(
) const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tW = volScalarField::New
    (
        "W",
        IOobject::NO_REGISTER,
        mesh,
        dimMass/dimMoles
    );
    auto& W = tW.ref();

    scalarField& WCells = W.primitiveFieldRef();

    forAll(WCells, celli)
    {
        WCells[celli] = this->cellMixture(celli).W();
    }

    auto& WBf = W.boundaryFieldRef();

    forAll(WBf, patchi)
    {
        scalarField& Wp = WBf[patchi];
        forAll(Wp, facei)
        {
            Wp[facei] = this->patchFaceMixture(patchi, facei).W();
        }
    }

    return tW;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaTR() const
{
    tmp<Foam::volScalarField> kappaTR(CpTR()*this->alpha_);
    kappaTR.ref().rename("kappaTR");
    return kappaTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaTR
(
    const label patchi
) const
{
    return
        CpTR
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
```

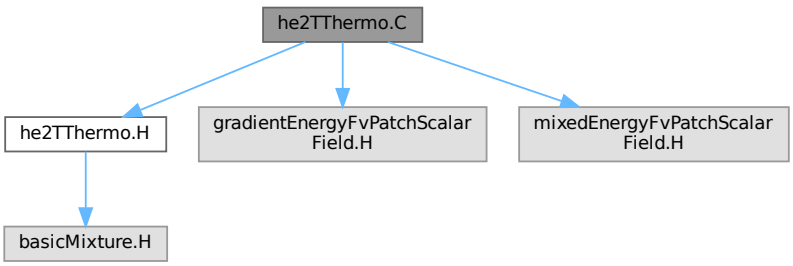
```
        )*this->alpha_.boundaryField()[patchi];
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaVib() const
{
    tmp<Foam::volScalarField> kappaVib(CpTR()*this->alpha_);
    kappaVib.ref().rename("kappaVib");
    return kappaVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaVib
(
    const label patchi
) const
{
    return
        CpTR
        (
            this->p_.boundaryField()[patchi],
            this->TVib_.boundaryField()[patchi],
            patchi
        )*this->alpha_.boundaryField()[patchi];
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaheTR() const
{
    tmp<Foam::volScalarField> alphaEff(this->CpByCpvTR()*this->alpha_);
    alphaEff.ref().rename("alphaheTR");
    return alphaEff;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaheTR(const label patchi) const
{
    return
    this->CpByCpvTR
    (
        this->p_.boundaryField()[patchi],
        this->TTR_.boundaryField()[patchi],
        patchi
    )
   *this->alpha_.boundaryField()[patchi];
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaEff
(
    const volScalarField& alphat
) const
{
    tmp<Foam::volScalarField> kappaEff(CpTR()*(this->alpha_ + alphat));
    kappaEff.ref().rename("kappaEff");
    return kappaEff;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaEff
(
    const scalarField& alphat,
    const label patchi
) const
{
    return
        CpTR
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
        )
       *(
           this->alpha_.boundaryField()[patchi]
         + alphat
        );
}
```

```
template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaEff
(
    const volScalarField& alphat
) const
{
    tmp<Foam::volScalarField> alphaEff(this->CpByCpvTR()*(this->alpha_ + alphat));
    alphaEff.ref().rename("alphaEff");
    return alphaEff;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaEff
(
    const scalarField& alphat,
    const label patchi
) const
{
    return
    this->CpByCpvTR
    (
        this->p_.boundaryField()[patchi],
        this->TTR_.boundaryField()[patchi],
        patchi
    )
   *(
        this->alpha_.boundaryField()[patchi]
      + alphat
    );
}


template<class Basic2TThermo, class MixtureType>
bool Foam::he2TThermo<Basic2TThermo, MixtureType>::read()
{
    if (Basic2TThermo::read())
    {
        MixtureType::read(*this);
        return true;
    }

    return false;
}


// ************************************************************************* //
```

7.23 InInclude/he2TThermo.C File Reference

```
#include "he2TThermo.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"
```

Include dependency graph for InInclude/he2TThermo.C:



7.24 InInclude/he2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2015-2021 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "he2TThermo.H"
#include "gradientEnergyFvPatchScalarField.H"
#include "mixedEnergyFvPatchScalarField.H"

// * * * * * * * * * * * * Protected Member Functions  * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
void Foam::he2TThermo<Basic2TThermo, MixtureType>::
heBoundaryCorrection(volScalarField& h)
{
    volScalarField::Boundary& hBf = h.boundaryFieldRef();

    forAll(hBf, patchi)
    {
        if (isA<gradientEnergyFvPatchScalarField>(hBf[patchi]))
        {
            refCast<gradientEnergyFvPatchScalarField>(hBf[patchi]).gradient()
                = hBf[patchi].fvPatchField::snGrad();
        }
        else if (isA<mixedEnergyFvPatchScalarField>(hBf[patchi]))
        {
            refCast<mixedEnergyFvPatchScalarField>(hBf[patchi]).refGrad()
                = hBf[patchi].fvPatchField::snGrad();
        }
    }
}


template<class Basic2TThermo, class MixtureType>
void Foam::he2TThermo<Basic2TThermo, MixtureType>::init
(
    const volScalarField& p,
    const volScalarField& TTR,
    const volScalarField& TVib,
    volScalarField& he
)
{
    scalarField& heCells = he.primitiveFieldRef();
    const scalarField& pCells = p.primitiveField();
    const scalarField& TTRCells = TTR.primitiveField();
    //const scalarField& TVibCells = TVib.primitiveField();

    forAll(heCells, celli)
    {
        heCells[celli] =
            this->cellMixture(celli).HE(pCells[celli], TTRCells[celli]);
    }

    volScalarField::Boundary& heBf = he.boundaryFieldRef();

    forAll(heBf, patchi)
    {
        heBf[patchi] == this->he
        (
```

```
            p.boundaryField()[patchi],
            TTR.boundaryField()[patchi],
            patchi
        );

        heBf[patchi].useImplicit(TTR.boundaryField()[patchi].useImplicit());
    }

    this->heBoundaryCorrection(he);

    // Note: TTR does not have oldTime
    if (p.nOldTimes())
    {
        init(p.oldTime(), TTR.oldTime(), TVib.oldTime(), he.oldTime());
    }
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::he2TThermo
(
    const fvMesh& mesh,
    const word& phaseName
)
:
    Basic2TThermo(mesh, phaseName),
    MixtureType(*this, mesh, phaseName),

    he_
    (
        IOobject
        (
            Basic2TThermo::phasePropertyName
            (
                MixtureType::thermoType::heName()
            ),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE
        ),
        mesh,
        dimEnergy/dimMass,
        this->heBoundaryTypes(),
        this->heBoundaryBaseTypes()
    )
{
    init(this->p_, this->TTR_, this->TVib_, he_);
}


template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::he2TThermo
(
    const fvMesh& mesh,
    const dictionary& dict,
    const word& phaseName
)
:
    Basic2TThermo(mesh, dict, phaseName),
    MixtureType(*this, mesh, phaseName),

    he_
    (
        IOobject
        (
            Basic2TThermo::phasePropertyName
            (
                MixtureType::thermoType::heName()
            ),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE
        ),
        mesh,
        dimEnergy/dimMass,
        this->heBoundaryTypes(),
        this->heBoundaryBaseTypes()
    )
{
    init(this->p_, this->TTR_, this->TVib_, he_);
}


```

```
template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::he2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    Basic2TThermo(mesh, phaseName, dictionaryName),
    MixtureType(*this, mesh, phaseName),

    he_
    (
        IOobject
        (
            Basic2TThermo::phasePropertyName
            (
                MixtureType::thermoType::heName()
            ),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE
        ),
        mesh,
        dimEnergy/dimMass,
        this->heBoundaryTypes(),
        this->heBoundaryBaseTypes()
    )
{
    init(this->p_, this->TTR_, this->TVib_, he_);
}



// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
Foam::he2TThermo<Basic2TThermo, MixtureType>::~he2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::he
(
    const volScalarField& p,
    const volScalarField& TTR
) const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto the = volScalarField::New
    (
        "he",
        IOobject::NO_REGISTER,
        mesh,
        he_.dimensions()
    );
    auto& he = the.ref();

    scalarField& heCells = he.primitiveFieldRef();
    const scalarField& pCells = p;
    const scalarField& TTRCells = TTR;

    forAll(heCells, celli)
    {
        heCells[celli] =
            this->cellMixture(celli).HE(pCells[celli], TTRCells[celli]);
    }

    volScalarField::Boundary& heBf = he.boundaryFieldRef();

    forAll(heBf, patchi)
    {
        scalarField& hep = heBf[patchi];
        const scalarField& pp = p.boundaryField()[patchi];
        const scalarField& TTRp = TTR.boundaryField()[patchi];

        forAll(hep, facei)
        {
            hep[facei] =
                this->patchFaceMixture(patchi, facei).HE(pp[facei], TTRp[facei]);
        }
    }

```

```
    return the;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::he
(
    const scalarField& p,
    const scalarField& TTR,
    const labelList& cells
) const
{
    auto the = tmp<scalarField>::New(TTR.size());
    auto& he = the.ref();

    forAll(TTR, celli)
    {
        he[celli] = this->cellMixture(cells[celli]).HE(p[celli], TTR[celli]);
    }

    return the;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::he
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto the = tmp<scalarField>::New(TTR.size());
    auto& he = the.ref();

    forAll(TTR, facei)
    {
        he[facei] =
            this->patchFaceMixture(patchi, facei).HE(p[facei], TTR[facei]);
    }

    return the;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::hc() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto thc = volScalarField::New
    (
        "hc",
        IOobject::NO_REGISTER,
        mesh,
        he_.dimensions()
    );
    auto& hcf = thc.ref();

    scalarField& hcCells = hcf.primitiveFieldRef();

    forAll(hcCells, celli)
    {
        hcCells[celli] = this->cellMixture(celli).Hc();
    }

    volScalarField::Boundary& hcfBf = hcf.boundaryFieldRef();

    forAll(hcfBf, patchi)
    {
        scalarField& hcp = hcfBf[patchi];

        forAll(hcp, facei)
        {
            hcp[facei] = this->patchFaceMixture(patchi, facei).Hc();
        }
    }

    return thc;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::CpTR
(
    const scalarField& p,
```

```
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCpTR = tmp<scalarField>::New(TTR.size());
    auto& cpTR = tCpTR.ref();

    forAll(TTR, facei)
    {
        cpTR[facei] =
            this->patchFaceMixture(patchi, facei).CpTR(p[facei], TTR[facei]);
    }

    return tCpTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpTR
(
    const scalarField& p,
    const scalarField& TTR,
    const labelList& cells
) const
{
    auto tCpTR = tmp<scalarField>::New(TTR.size());
    auto& CpTR = tCpTR.ref();

    forAll(cells, i)
    {
        const label celli = cells[i];
        CpTR[i] = this->cellMixture(celli).CpTR(p[i], TTR[i]);
    }

    return tCpTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpTR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCpTR = volScalarField::New
    (
        "CpTR",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cpTR = tCpTR.ref();

    forAll(this->TTR_, celli)
    {
        cpTR[celli] =
            this->cellMixture(celli).CpTR(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& cpTRBf = cpTR.boundaryFieldRef();

    forAll(cpTRBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pCpTR = cpTRBf[patchi];

        forAll(pTTR, facei)
        {
            pCpTR[facei] =
                this->patchFaceMixture(patchi, facei).CpTR(pp[facei], pTTR[facei]);
        }
    }

    return tCpTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvT
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
```

```
{
    auto tCvT = tmp<scalarField>::New(TTR.size());
    auto& cvT = tCvT.ref();

    forAll(TTR, facei)
    {
        cvT[facei] =
            this->patchFaceMixture(patchi, facei).CvT(p[facei], TTR[facei]);
    }

    return tCvT;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvR
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCvR = tmp<scalarField>::New(TTR.size());
    auto& cvR = tCvR.ref();

    forAll(TTR, facei)
    {
        cvR[facei] =
            this->patchFaceMixture(patchi, facei).CvR(p[facei], TTR[facei]);
    }

    return tCvR;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvVib
(
    const scalarField& p,
    const scalarField& TTR,
    const scalarField& TVib,
    const scalar ThetaVib,
    const label patchi
) const
{
    auto tCvVib = tmp<scalarField>::New(TVib.size());
    auto& cvVib = tCvVib.ref();

    forAll(TVib, facei)
    {
        cvVib[facei] =
            this->patchFaceMixture(patchi, facei).CvVib(p[facei], TTR[facei], TVib[facei], ThetaVib);
    }

    return tCvVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::rhoEoS
(
    const scalarField& p,
    const scalarField& TTR,
    const labelList& cells
) const
{
    auto tRho = tmp<scalarField>::New(TTR.size());
    auto& rho = tRho.ref();

    forAll(cells, i)
    {
        const label celli = cells[i];
        rho[i] = this->cellMixture(celli).rho(p[i], TTR[i]);
    }

    return tRho;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvT() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCvT = volScalarField::New
```

```
    (
        "CvT",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cvT = tCvT.ref();

    forAll(this->TTR_, celli)
    {
        cvT[celli] =
            this->cellMixture(celli).CvT(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& cvTBf = cvT.boundaryFieldRef();

    forAll(cvTBf, patchi)
    {
        cvTBf[patchi] = CvT
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
        );
    }

    return tCvT;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCvR = volScalarField::New
    (
        "CvR",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cvR = tCvR.ref();

    forAll(this->TTR_, celli)
    {
        cvR[celli] =
            this->cellMixture(celli).CvR(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& cvRBf = cvR.boundaryFieldRef();

    forAll(cvRBf, patchi)
    {
        cvRBf[patchi] = CvR
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
        );
    }

    return tCvR;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CvVib() const
{
    const fvMesh& mesh = this->TVib_.mesh();
    const scalar theta = this->cellMixture(0).ThetaVib();  // getter on Thermo2T

    auto tCvVib = volScalarField::New
    (
        "CvVib",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& cvVib = tCvVib.ref();

    forAll(this->TVib_, celli)
    {
        cvVib[celli] =
            this->cellMixture(celli).CvVib(this->p_[celli], this->TTR_[celli] ,this->TVib_[celli],
theta);
    }
```

```

    volScalarField::Boundary& cvVibBf = cvVib.boundaryFieldRef();

    forAll(cvVibBf, patchi)
    {
        cvVibBf[patchi] = CvVib
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            this->TVib_.boundaryField()[patchi],
            theta,
            patchi
        );
    }

    return tCvVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::gamma
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tgamma = tmp<scalarField>::New(TTR.size());
    auto& gamma = tgamma.ref();

    forAll(TTR, facei)
    {
        gamma[facei] =
            this->patchFaceMixture(patchi, facei).gamma(p[facei], TTR[facei]);
    }

    return tgamma;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::gamma() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tgamma = volScalarField::New
    (
        "gamma",
        IOobject::NO_REGISTER,
        mesh,
        dimless
    );
    auto& gamma = tgamma.ref();

    forAll(this->TTR_, celli)
    {
        gamma[celli] =
            this->cellMixture(celli).gamma(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& gammaBf = gamma.boundaryFieldRef();

    forAll(gammaBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pgamma = gammaBf[patchi];

        forAll(pTTR, facei)
        {
            pgamma[facei] = this->patchFaceMixture(patchi, facei).gamma
            (
                pp[facei],
                pTTR[facei]
            );
        }
    }

    return tgamma;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::CpvTR
(
    const scalarField& p,
```

```
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCpvTR = tmp<scalarField>::New(TTR.size());
    auto& CpvTR = tCpvTR.ref();

    forAll(TTR, facei)
    {
        CpvTR[facei] =
            this->patchFaceMixture(patchi, facei).CpvTR(p[facei], TTR[facei]);
    }

    return tCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpvTR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCpvTR = volScalarField::New
    (
        "CpvTR",
        IOobject::NO_REGISTER,
        mesh,
        dimEnergy/dimMass/dimTemperature
    );
    auto& CpvTR = tCpvTR.ref();

    forAll(this->TTR_, celli)
    {
        CpvTR[celli] =
            this->cellMixture(celli).CpvTR(this->p_[celli], this->TTR_[celli]);
    }

    volScalarField::Boundary& CpvTRBf = CpvTR.boundaryFieldRef();

    forAll(CpvTRBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pCpvTR = CpvTRBf[patchi];

        forAll(pTTR, facei)
        {
            pCpvTR[facei] =
                this->patchFaceMixture(patchi, facei).CpvTR(pp[facei], pTTR[facei]);
        }
    }

    return tCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::CpByCpvTR
(
    const scalarField& p,
    const scalarField& TTR,
    const label patchi
) const
{
    auto tCpByCpvTR = tmp<scalarField>::New(TTR.size());
    auto& CpByCpvTR = tCpByCpvTR.ref();

    forAll(TTR, facei)
    {
        CpByCpvTR[facei] =
            this->patchFaceMixture(patchi, facei).CpByCpvTR(p[facei], TTR[facei]);
    }

    return tCpByCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::CpByCpvTR() const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tCpByCpvTR = volScalarField::New
    (
        "CpByCpvTR",
```

```
        IOobject::NO_REGISTER,
        mesh,
        dimless
    );
    auto& CpByCpvTR = tCpByCpvTR.ref();

    forAll(this->TTR_, celli)
    {
        CpByCpvTR[celli] = this->cellMixture(celli).CpByCpvTR
        (
            this->p_[celli],
            this->TTR_[celli]
        );
    }

    volScalarField::Boundary& CpByCpvTRBf =
        CpByCpvTR.boundaryFieldRef();

    forAll(CpByCpvTRBf, patchi)
    {
        const fvPatchScalarField& pp = this->p_.boundaryField()[patchi];
        const fvPatchScalarField& pTTR = this->TTR_.boundaryField()[patchi];
        fvPatchScalarField& pCpByCpvTR = CpByCpvTRBf[patchi];

        forAll(pTTR, facei)
        {
            pCpByCpvTR[facei] = this->patchFaceMixture(patchi, facei).CpByCpvTR
            (
                pp[facei],
                pTTR[facei]
            );
        }
    }

    return tCpByCpvTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::THE_TR
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const labelList& cells
) const
{
    auto tTTR = tmp<scalarField>::New(h.size());
    auto& TTR = tTTR.ref();

    forAll(h, celli)
    {
        TTR[celli] =
            this->cellMixture(cells[celli]).THE_TR(h[celli], p[celli], TTR0[celli]);
    }

    return tTTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::THE_TR
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const label patchi
) const
{

    auto tTTR = tmp<scalarField>::New(h.size());
    auto& TTR = tTTR.ref();

    forAll(h, facei)
    {
        TTR[facei] = this->patchFaceMixture
        (
            patchi,
            facei
        ).THE_TR(h[facei], p[facei], TTR0[facei]);
    }

    return tTTR;
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::TEH_Vib
```

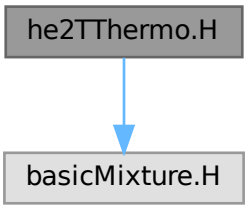
```
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const scalarField& TVib0,
    const labelList& cells
) const
{

    auto tTVib = tmp<scalarField>::New(h.size());
    auto& TVib = tTVib.ref();

    forAll(h, celli)
    {
        TVib[celli] =
            this->cellMixture(cells[celli]).TEH_Vib(h[celli], p[celli], TTR0[celli], TVib0[celli]);
    }

    return tTVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::TEH_Vib
(
    const scalarField& h,
    const scalarField& p,
    const scalarField& TTR0,
    const scalarField& TVib0,
    const label patchi
) const
{

    auto tTVib = tmp<scalarField>::New(h.size());
    auto& TVib = tTVib.ref();

    forAll(h, facei)
    {
        TVib[facei] = this->patchFaceMixture
        (
            patchi,
            facei
        ).TEH_Vib(h[facei], p[facei], TTR0[facei], TVib0[facei]);
    }

    return tTVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::W
(
) const
{
    const fvMesh& mesh = this->TTR_.mesh();

    auto tW = volScalarField::New
    (
        "W",
        IOobject::NO_REGISTER,
        mesh,
        dimMass/dimMoles
    );
    auto& W = tW.ref();

    scalarField& WCells = W.primitiveFieldRef();

    forAll(WCells, celli)
    {
        WCells[celli] = this->cellMixture(celli).W();
    }

    auto& WBf = W.boundaryFieldRef();

    forAll(WBf, patchi)
    {
        scalarField& Wp = WBf[patchi];
        forAll(Wp, facei)
        {
            Wp[facei] = this->patchFaceMixture(patchi, facei).W();
        }
    }

    return tW;
}


```

```
template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaTR() const
{
    tmp<Foam::volScalarField> kappaTR(CpTR()*this->alpha_);
    kappaTR.ref().rename("kappaTR");
    return kappaTR;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaTR
(
    const label patchi
) const
{
    return
        CpTR
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
        )*this->alpha_.boundaryField()[patchi];
}

template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaVib() const
{
    tmp<Foam::volScalarField> kappaVib(CpTR()*this->alpha_);
    kappaVib.ref().rename("kappaVib");
    return kappaVib;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField> Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaVib
(
    const label patchi
) const
{
    return
        CpTR
        (
            this->p_.boundaryField()[patchi],
            this->TVib_.boundaryField()[patchi],
            patchi
        )*this->alpha_.boundaryField()[patchi];
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaheTR() const
{
    tmp<Foam::volScalarField> alphaEff(this->CpByCpvTR()*this->alpha_);
    alphaEff.ref().rename("alphaheTR");
    return alphaEff;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaheTR(const label patchi) const
{
    return
    this->CpByCpvTR
    (
        this->p_.boundaryField()[patchi],
        this->TTR_.boundaryField()[patchi],
        patchi
    )
   *this->alpha_.boundaryField()[patchi];
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaEff
(
    const volScalarField& alphat
) const
{
    tmp<Foam::volScalarField> kappaEff(CpTR()*(this->alpha_ + alphat));
    kappaEff.ref().rename("kappaEff");
    return kappaEff;
}
```

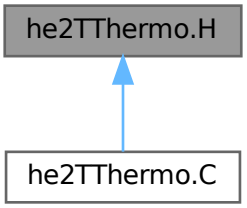
```


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::kappaEff
(
    const scalarField& alphat,
    const label patchi
) const
{
    return
        CpTR
        (
            this->p_.boundaryField()[patchi],
            this->TTR_.boundaryField()[patchi],
            patchi
        )
       *(
           this->alpha_.boundaryField()[patchi]
         + alphat
        );
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::volScalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaEff
(
    const volScalarField& alphat
) const
{
    tmp<Foam::volScalarField> alphaEff(this->CpByCpvTR()*(this->alpha_ + alphat));
    alphaEff.ref().rename("alphaEff");
    return alphaEff;
}


template<class Basic2TThermo, class MixtureType>
Foam::tmp<Foam::scalarField>
Foam::he2TThermo<Basic2TThermo, MixtureType>::alphaEff
(
    const scalarField& alphat,
    const label patchi
) const
{
    return
    this->CpByCpvTR
    (
        this->p_.boundaryField()[patchi],
        this->TTR_.boundaryField()[patchi],
        patchi
    )
   *(
        this->alpha_.boundaryField()[patchi]
      + alphat
    );
}


template<class Basic2TThermo, class MixtureType>
bool Foam::he2TThermo<Basic2TThermo, MixtureType>::read()
{
    if (Basic2TThermo::read())
    {
        MixtureType::read(*this);
        return true;
    }

    return false;
}


// ************************************************************************* //
```

7.25 he2TThermo/he2TThermo.H File Reference

`#include "basicMixture.H"`
Include dependency graph for he2TThermo/he2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::he2TThermo< Basic2TThermo, MixtureType >

Namespaces

- namespace Foam

7.26 he2TThermo/he2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
```

```
---------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2015-2021 OpenCFD Ltd.
---------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::he2TThermo

Description
    Enthalpy/Internal energy for a mixture

SourceFiles
    he2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef he2TThermo_H
#define he2TThermo_H

#include "basicMixture.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class he2TThermo Declaration
\*---------------------------------------------------------------------------*/

template<class Basic2TThermo, class MixtureType>
class he2TThermo
:
    public Basic2TThermo,
    public MixtureType
{
protected:

    // Protected data

        //- Energy field
        volScalarField he_;


    // Protected Member Functions

        // Enthalpy/Internal energy

            //- Correct the enthalpy/internal energy field boundaries
            void heBoundaryCorrection(volScalarField& he);


private:

    // Private Member Functions

        //- Construct as copy (not implemented)
        he2TThermo(const he2TThermo<Basic2TThermo, MixtureType>&);


        //- Initialize he2TThermo
        void init
        (
            const volScalarField& p,
            const volScalarField& TTR,
            const volScalarField& TVib,
            volScalarField& he
        );


public:

```

```
    // Constructors

        //- Construct from mesh
        he2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //- Construct from mesh and dictionary
        he2TThermo
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName
        );

        //- Construct from mesh,dictionary,phase name with a single temperature
        he2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Destructor
    virtual ~he2TThermo();


    // Member functions

        //- Return the composition of the mixture
        virtual typename MixtureType::basicMixtureType&
        composition()
        {
            return *this;
        }

        //- Return the composition of the mixture
        virtual const typename MixtureType::basicMixtureType&
        composition() const
        {
            return *this;
        }

        //- Return the name of the thermo physics
        virtual word thermoName() const
        {
            return MixtureType::thermoType::typeName();
        }

        //- Return true if the equation of state is incompressible
        //  i.e. rho != f(p)
        virtual bool incompressible() const
        {
            return MixtureType::thermoType::incompressible;
        }

        //- Return true if the equation of state is isochoric
        //  i.e. rho = const
        virtual bool isochoric() const
        {
            return MixtureType::thermoType::isochoric;
        }


        // Access to thermodynamic state variables

            //- Enthalpy/Internal energy [J/kg]
            //  Non-const access allowed for transport equations
            virtual volScalarField& he()
            {
                return he_;
            }

            //- Enthalpy/Internal energy [J/kg]
            virtual const volScalarField& he() const
            {
                return he_;
            }


        // Fields derived from thermodynamic state variables

            //- Enthalpy/Internal energy
            //  for given pressure and temperature [J/kg]
```

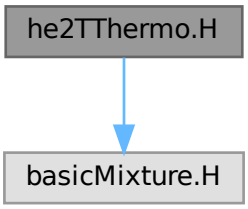
```
            virtual tmp<volScalarField> he
            (
                const volScalarField& p,
                const volScalarField& TTR
            ) const;

            //- Enthalpy/Internal energy for cell-set [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const;

            //- Enthalpy/Internal energy for patch [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Chemical enthalpy [J/kg]
            virtual tmp<volScalarField> hc() const;

            //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const labelList& cells
            ) const;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const label patchi
            ) const;

            //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const labelList& cells
            ) const;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const label patchi
            ) const;

            //- Heat capacity at constant pressure for patch [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Heat capacity using pressure and temperature [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const;

            //- Heat capacity at constant pressure [J/kg/K]
            virtual tmp<volScalarField> CpTR() const;

            //- Translational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvT() const;

            //- Translational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvT
```

```
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Rotational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvR() const;

            //- Rotational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Vibrational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvVib() const;

            //- Vibrational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvVib
            (
                const scalarField& p,
                const scalarField& TTR,
                const scalarField& Tvib,
                const scalar ThetaVib,
                const label patchi
            ) const;

            //- Density from pressure and temperature
            virtual tmp<scalarField> rhoEoS
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const;

            //- Gamma = CpTR/CvT []
            virtual tmp<volScalarField> gamma() const;

            //- Gamma = CpTR/CvT for patch []
            virtual tmp<scalarField> gamma
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Translational-Rotational heat capacity at constant pressure/volume for patch [J/kg/K]
            virtual tmp<scalarField> CpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Translational-Rotational heat capacity at constant pressure/volume [J/kg/K]
            virtual tmp<volScalarField> CpvTR() const;

            //- Translational-Rotational heat capacity ratio []
            virtual tmp<volScalarField> CpByCpvTR() const;

            //- Translational-Rotational heat capacity ratio for patch []
            virtual tmp<scalarField> CpByCpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Molecular weight [kg/kmol]
            virtual tmp<volScalarField> W() const;


        // Fields derived from transport state variables

            //- Translational-rotational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaTR() const;

            //- Translational-rotational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaTR
            (
                const label patchi
            ) const;

```

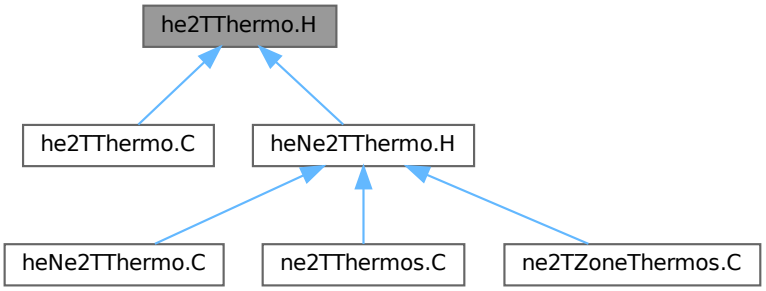
```
            //- Vibrational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaVib() const;

            //- Vibrational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaVib
            (
                const label patchi
            ) const;

             //- Translational-rotational thermal diffusivity for energy of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaheTR() const;

            //- Translational-rotational thermal diffusivity for energy of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaheTR(const label patchi) const;

            //- Effective thermal diffusivity for temperature
            //  of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaEff(const volScalarField&) const;

            //- Effective thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const;

            //- Effective thermal diffusivity of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaEff
            (
                const volScalarField& alphat
            ) const;

            //- Effective thermal diffusivity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const;


        //- Read thermophysical properties dictionary
        virtual bool read();
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

#ifdef NoRepository
    #include "he2TThermo.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.27 lnInclude/he2TThermo.H File Reference

`#include "basicMixture.H"`
Include dependency graph for lnInclude/he2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::he2TThermo< Basic2TThermo, MixtureType >

Namespaces

- namespace Foam

7.28 lnInclude/he2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
```

```
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2015-2021 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::he2TThermo

Description
    Enthalpy/Internal energy for a mixture

SourceFiles
    he2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef he2TThermo_H
#define he2TThermo_H

#include "basicMixture.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class he2TThermo Declaration
\*---------------------------------------------------------------------------*/

template<class Basic2TThermo, class MixtureType>
class he2TThermo
:
    public Basic2TThermo,
    public MixtureType
{
protected:

    // Protected data

        //- Energy field
        volScalarField he_;


    // Protected Member Functions

        // Enthalpy/Internal energy

            //- Correct the enthalpy/internal energy field boundaries
            void heBoundaryCorrection(volScalarField& he);


private:

    // Private Member Functions

        //- Construct as copy (not implemented)
        he2TThermo(const he2TThermo<Basic2TThermo, MixtureType>&);


        //- Initialize he2TThermo
        void init
        (
            const volScalarField& p,
            const volScalarField& TTR,
            const volScalarField& TVib,
            volScalarField& he
        );


```

```
public:

    // Constructors

        //- Construct from mesh
        he2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //- Construct from mesh and dictionary
        he2TThermo
        (
            const fvMesh&,
            const dictionary&,
            const word& phaseName
        );

        //- Construct from mesh,dictionary,phase name with a single temperature
        he2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Destructor
    virtual ~he2TThermo();


    // Member functions

        //- Return the composition of the mixture
        virtual typename MixtureType::basicMixtureType&
        composition()
        {
            return *this;
        }

        //- Return the composition of the mixture
        virtual const typename MixtureType::basicMixtureType&
        composition() const
        {
            return *this;
        }

        //- Return the name of the thermo physics
        virtual word thermoName() const
        {
            return MixtureType::thermoType::typeName();
        }

        //- Return true if the equation of state is incompressible
        //  i.e. rho != f(p)
        virtual bool incompressible() const
        {
            return MixtureType::thermoType::incompressible;
        }

        //- Return true if the equation of state is isochoric
        //  i.e. rho = const
        virtual bool isochoric() const
        {
            return MixtureType::thermoType::isochoric;
        }


        // Access to thermodynamic state variables

            //- Enthalpy/Internal energy [J/kg]
            //  Non-const access allowed for transport equations
            virtual volScalarField& he()
            {
                return he_;
            }

            //- Enthalpy/Internal energy [J/kg]
            virtual const volScalarField& he() const
            {
                return he_;
            }


        // Fields derived from thermodynamic state variables

```

```
            //- Enthalpy/Internal energy
            //  for given pressure and temperature [J/kg]
            virtual tmp<volScalarField> he
            (
                const volScalarField& p,
                const volScalarField& TTR
            ) const;

            //- Enthalpy/Internal energy for cell-set [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const;

            //- Enthalpy/Internal energy for patch [J/kg]
            virtual tmp<scalarField> he
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Chemical enthalpy [J/kg]
            virtual tmp<volScalarField> hc() const;

            //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const labelList& cells
            ) const;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> THE_TR
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const label patchi
            ) const;

            //- Temperature from enthalpy/internal energy for cell-set
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const labelList& cells
            ) const;

            //- Temperature from enthalpy/internal energy for patch
            virtual tmp<scalarField> TEH_Vib
            (
                const scalarField& h,
                const scalarField& p,
                const scalarField& TTR0,      // starting translational-rotational temperature
                const scalarField& TVib0,      // starting vibrational temperature
                const label patchi
            ) const;

            //- Heat capacity at constant pressure for patch [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Heat capacity using pressure and temperature [J/kg/K]
            virtual tmp<scalarField> CpTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const;

            //- Heat capacity at constant pressure [J/kg/K]
            virtual tmp<volScalarField> CpTR() const;

            //- Translational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvT() const;

```

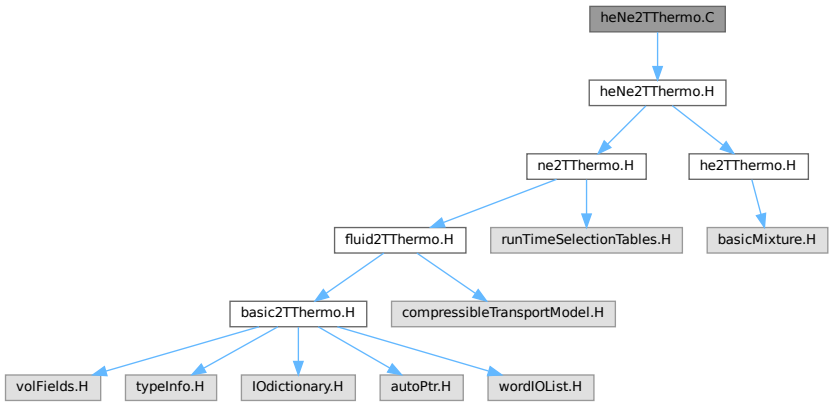
```
            //- Translational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvT
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Rotational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvR() const;

            //- Rotational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Vibrational heat capacity at constant volume [J/kg/K]
            virtual tmp<volScalarField> CvVib() const;

            //- Vibrational heat capacity at constant volume for patch [J/kg/K]
            virtual tmp<scalarField> CvVib
            (
                const scalarField& p,
                const scalarField& TTR,
                const scalarField& Tvib,
                const scalar ThetaVib,
                const label patchi
            ) const;

            //- Density from pressure and temperature
            virtual tmp<scalarField> rhoEoS
            (
                const scalarField& p,
                const scalarField& TTR,
                const labelList& cells
            ) const;

            //- Gamma = CpTR/CvT []
            virtual tmp<volScalarField> gamma() const;

            //- Gamma = CpTR/CvT for patch []
            virtual tmp<scalarField> gamma
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Translational-Rotational heat capacity at constant pressure/volume for patch [J/kg/K]
            virtual tmp<scalarField> CpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Translational-Rotational heat capacity at constant pressure/volume [J/kg/K]
            virtual tmp<volScalarField> CpvTR() const;

            //- Translational-Rotational heat capacity ratio []
            virtual tmp<volScalarField> CpByCpvTR() const;

            //- Translational-Rotational heat capacity ratio for patch []
            virtual tmp<scalarField> CpByCpvTR
            (
                const scalarField& p,
                const scalarField& TTR,
                const label patchi
            ) const;

            //- Molecular weight [kg/kmol]
            virtual tmp<volScalarField> W() const;


        // Fields derived from transport state variables

            //- Translational-rotational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaTR() const;

            //- Translational-rotational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaTR
            (
                const label patchi
```

```
            ) const;

            //- Vibrational thermal diffusivity for temperature of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaVib() const;

            //- Vibrational thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaVib
            (
                const label patchi
            ) const;

             //- Translational-rotational thermal diffusivity for energy of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaheTR() const;

            //- Translational-rotational thermal diffusivity for energy of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaheTR(const label patchi) const;

            //- Effective thermal diffusivity for temperature
            //  of mixture [J/m/s/K]
            virtual tmp<volScalarField> kappaEff(const volScalarField&) const;

            //- Effective thermal diffusivity for temperature
            //  of mixture for patch [J/m/s/K]
            virtual tmp<scalarField> kappaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const;

            //- Effective thermal diffusivity of mixture [kg/m/s]
            virtual tmp<volScalarField> alphaEff
            (
                const volScalarField& alphat
            ) const;

            //- Effective thermal diffusivity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> alphaEff
            (
                const scalarField& alphat,
                const label patchi
            ) const;


        //- Read thermophysical properties dictionary
        virtual bool read();
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * *

#ifdef NoRepository
    #include "he2TThermo.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.29 InInclude/heNe2TThermo.C File Reference

`#include "heNe2TThermo.H"`

Include dependency graph for InInclude/heNe2TThermo.C:



7.30 InInclude/heNe2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2016 OpenFOAM Foundation
    Copyright (C) 2015-2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "heNe2TThermo.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
void Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::calculate
(
    const volScalarField& p,
    volScalarField& TTR,
    volScalarField& TVib,
    volScalarField& he,
    volScalarField& psi,
    volScalarField& mu,
    volScalarField& alpha,
    const bool doOldTimes
)
```

```
{
    // Note: update oldTimes before current time so that if TTR.oldTime() is
    // created from TTR, it starts from the unconverted TTR
    if (doOldTimes && (p.nOldTimes() || TTR.nOldTimes() || TVib.nOldTimes()))
    {
        calculate
        (
            p.oldTime(),
            TTR.oldTime(),
            TVib.oldTime(),
            he.oldTime(),
            psi.oldTime(),
            mu.oldTime(),
            alpha.oldTime(),
            true
        );
    }

    const scalarField& hCells = he.primitiveField();
    const scalarField& pCells = p.primitiveField();

    scalarField& TTRCells = TTR.primitiveFieldRef();
    scalarField& TVibCells = TVib.primitiveFieldRef();
    scalarField& psiCells = psi.primitiveFieldRef();
    scalarField& muCells = mu.primitiveFieldRef();
    scalarField& alphaCells = alpha.primitiveFieldRef();

    forAll(TTRCells, celli)
    {
        const typename MixtureType::thermoType& mixture_ =
            this->cellMixture(celli);

        if (this->updateTTR())
        {
            TTRCells[celli] = mixture_.THE_TR
            (
                hCells[celli],
                pCells[celli],
                TTRCells[celli]
            );
        }

        if (this->updateTVib())
        {
            TVibCells[celli] = mixture_.TEH_Vib
            (
                hCells[celli],
                pCells[celli],
                TTRCells[celli],
                TVibCells[celli]
            );
        }
        psiCells[celli] = mixture_.psi(pCells[celli], TTRCells[celli]);

        muCells[celli] = mixture_.mu(TTRCells[celli]);
        alphaCells[celli] = mixture_.alphah(pCells[celli], TTRCells[celli]);
    }

    const volScalarField::Boundary& pBf = p.boundaryField();
    volScalarField::Boundary& TTRBf = TTR.boundaryFieldRef();
    volScalarField::Boundary& TVibBf = TVib.boundaryFieldRef();
    volScalarField::Boundary& psiBf = psi.boundaryFieldRef();
    volScalarField::Boundary& heBf = he.boundaryFieldRef();
    volScalarField::Boundary& muBf = mu.boundaryFieldRef();
    volScalarField::Boundary& alphaBf = alpha.boundaryFieldRef();

    forAll(pBf, patchi)
    {
        const fvPatchScalarField& pp = pBf[patchi];
        fvPatchScalarField& pTTR = TTRBf[patchi];
        fvPatchScalarField& pTVib = TVibBf[patchi];
        fvPatchScalarField& ppsi = psiBf[patchi];
        fvPatchScalarField& phe = heBf[patchi];
        fvPatchScalarField& pmu = muBf[patchi];
        fvPatchScalarField& palpha = alphaBf[patchi];

        if (pTTR.fixesValue())
        {
            forAll(pTTR, facei)
            {
                const typename MixtureType::thermoType& mixture_ =
                    this->patchFaceMixture(patchi, facei);

                phe[facei] = mixture_.HE(pp[facei], pTTR[facei]);

                ppsi[facei] = mixture_.psi(pp[facei], pTTR[facei]);
                pmu[facei] = mixture_.mu(pTTR[facei]);
```

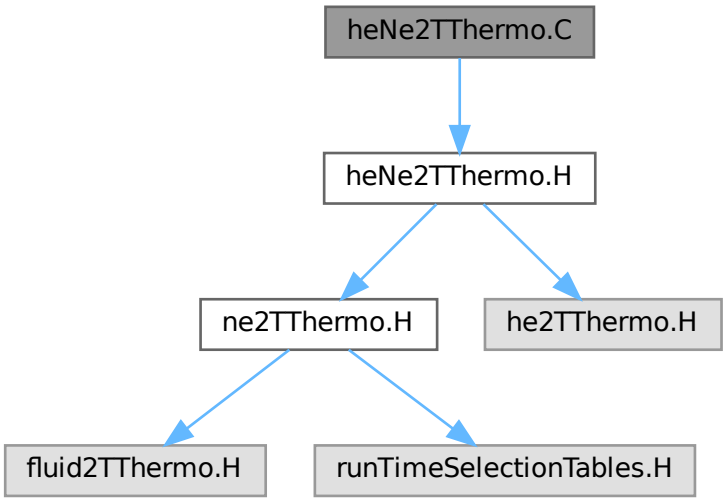
```
                palpha[facei] = mixture_.alphah(pp[facei], pTTR[facei]);
            }
        }
        else
        {
            forAll(pTTR, facei)
            {
                const typename MixtureType::thermoType& mixture_ =
                    this->patchFaceMixture(patchi, facei);

                if (this->updateTTR())
                {
                    pTTR[facei] = mixture_.THE_TR(phe[facei], pp[facei], pTTR[facei]);
                }

                if (this->updateTVib())
                {
                    pTVib[facei] = mixture_.TEH_Vib(phe[facei], pp[facei], pTTR[facei], pTVib[facei]);
                }

                ppsi[facei] = mixture_.psi(pp[facei], pTTR[facei]);
                pmu[facei] = mixture_.mu(pTTR[facei]);
                palpha[facei] = mixture_.alphah(pp[facei], pTTR[facei]);
            }
        }
    }
}

// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::heNe2TThermo
(
    const fvMesh& mesh,
    const word& phaseName
)
:
    he2TThermo<BasicNe2TThermo, MixtureType>(mesh, phaseName)
{
    calculate
    (
        this->p_,
        this->TTR_,
        this->TVib_,
        this->he_,
        this->psi_,
        this->mu_,
        this->alpha_,
        true                    // Create old time fields
    );
}


template<class BasicNe2TThermo, class MixtureType>
Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::heNe2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    he2TThermo<BasicNe2TThermo, MixtureType>(mesh, phaseName, dictionaryName)
{
    calculate
    (
        this->p_,
        this->TTR_,
        this->TVib_,
        this->he_,
        this->psi_,
        this->mu_,
        this->alpha_,
        true                    // Create old time fields
    );
}



// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::~heNe2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

```

```
template<class BasicNe2TThermo, class MixtureType>
void Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::correct()
{
    DebugInFunction « endl;

    calculate
    (
        this->p_,
        this->TTR_,
        this->TVib_,
        this->he_,
        this->psi_,
        this->mu_,
        this->alpha_,
        false           // No need to update old times
    );

    DebugInFunction « "Finished" « endl;
}

// ************************************************************************* //
```

7.31 ne2TThermo/heNe2TThermo.C File Reference

`#include "heNe2TThermo.H"`

Include dependency graph for ne2TThermo/heNe2TThermo.C:



7.32 ne2TThermo/heNe2TThermo.C

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2016 OpenFOAM Foundation
    Copyright (C) 2015-2017 OpenCFD Ltd.
```

```
---------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "heNe2TThermo.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
void Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::calculate
(
    const volScalarField& p,
    volScalarField& TTR,
    volScalarField& TVib,
    volScalarField& he,
    volScalarField& psi,
    volScalarField& mu,
    volScalarField& alpha,
    const bool doOldTimes
)
{
    // Note: update oldTimes before current time so that if TTR.oldTime() is
    // created from TTR, it starts from the unconverted TTR
    if (doOldTimes && (p.nOldTimes() || TTR.nOldTimes() || TVib.nOldTimes()))
    {
        calculate
        (
            p.oldTime(),
            TTR.oldTime(),
            TVib.oldTime(),
            he.oldTime(),
            psi.oldTime(),
            mu.oldTime(),
            alpha.oldTime(),
            true
        );
    }

    const scalarField& hCells = he.primitiveField();
    const scalarField& pCells = p.primitiveField();

    scalarField& TTRCells = TTR.primitiveFieldRef();
    scalarField& TVibCells = TVib.primitiveFieldRef();
    scalarField& psiCells = psi.primitiveFieldRef();
    scalarField& muCells = mu.primitiveFieldRef();
    scalarField& alphaCells = alpha.primitiveFieldRef();

    forAll(TTRCells, celli)
    {
        const typename MixtureType::thermoType& mixture_ =
            this->cellMixture(celli);

        if (this->updateTTR())
        {
            TTRCells[celli] = mixture_.THE_TR
            (
                hCells[celli],
                pCells[celli],
                TTRCells[celli]
            );
        }

        if (this->updateTVib())
        {
            TVibCells[celli] = mixture_.TEH_Vib
            (
                hCells[celli],
                pCells[celli],
                TTRCells[celli],
                TVibCells[celli]
            );
        }
```

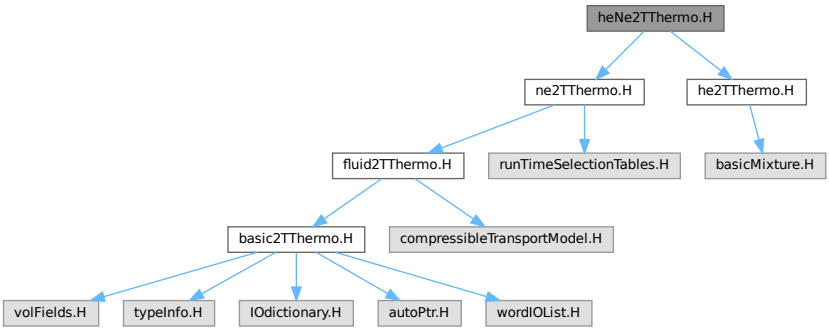
```
        psiCells[celli] = mixture_.psi(pCells[celli], TTRCells[celli]);

        muCells[celli] = mixture_.mu(TTRCells[celli]);
        alphaCells[celli] = mixture_.alphah(pCells[celli], TTRCells[celli]);
    }

    const volScalarField::Boundary& pBf = p.boundaryField();
    volScalarField::Boundary& TTRBf = TTR.boundaryFieldRef();
    volScalarField::Boundary& TVibBf = TVib.boundaryFieldRef();
    volScalarField::Boundary& psiBf = psi.boundaryFieldRef();
    volScalarField::Boundary& heBf = he.boundaryFieldRef();
    volScalarField::Boundary& muBf = mu.boundaryFieldRef();
    volScalarField::Boundary& alphaBf = alpha.boundaryFieldRef();

    forAll(pBf, patchi)
    {
        const fvPatchScalarField& pp = pBf[patchi];
        fvPatchScalarField& pTTR = TTRBf[patchi];
        fvPatchScalarField& pTVib = TVibBf[patchi];
        fvPatchScalarField& ppsi = psiBf[patchi];
        fvPatchScalarField& phe = heBf[patchi];
        fvPatchScalarField& pmu = muBf[patchi];
        fvPatchScalarField& palpha = alphaBf[patchi];

        if (pTTR.fixesValue())
        {
            forAll(pTTR, facei)
            {
                const typename MixtureType::thermoType& mixture_ =
                    this->patchFaceMixture(patchi, facei);

                phe[facei] = mixture_.HE(pp[facei], pTTR[facei]);

                ppsi[facei] = mixture_.psi(pp[facei], pTTR[facei]);
                pmu[facei] = mixture_.mu(pTTR[facei]);
                palpha[facei] = mixture_.alphah(pp[facei], pTTR[facei]);
            }
        }
        else
        {
            forAll(pTTR, facei)
            {
                const typename MixtureType::thermoType& mixture_ =
                    this->patchFaceMixture(patchi, facei);

                if (this->updateTTR())
                {
                    pTTR[facei] = mixture_.THE_TR(phe[facei], pp[facei], pTTR[facei]);
                }

                if (this->updateTVib())
                {
                    pTVib[facei] = mixture_.TEH_Vib(phe[facei], pp[facei], pTTR[facei], pTVib[facei]);
                }

                ppsi[facei] = mixture_.psi(pp[facei], pTTR[facei]);
                pmu[facei] = mixture_.mu(pTTR[facei]);
                palpha[facei] = mixture_.alphah(pp[facei], pTTR[facei]);
            }
        }
    }
}

// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::heNe2TThermo
(
    const fvMesh& mesh,
    const word& phaseName
)
:
    he2TThermo<BasicNe2TThermo, MixtureType>(mesh, phaseName)
{
    calculate
    (
        this->p_,
        this->TTR_,
        this->TVib_,
        this->he_,
        this->psi_,
        this->mu_,
        this->alpha_,
        true                    // Create old time fields
    );
}
```

```

template<class BasicNe2TThermo, class MixtureType>
Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::heNe2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    he2TThermo<BasicNe2TThermo, MixtureType>(mesh, phaseName, dictionaryName)
{
    calculate
    (
        this->p_,
        this->TTR_,
        this->TVib_,
        this->he_,
        this->psi_,
        this->mu_,
        this->alpha_,
        true                    // Create old time fields
    );
}



// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::~heNe2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * * * //

template<class BasicNe2TThermo, class MixtureType>
void Foam::heNe2TThermo<BasicNe2TThermo, MixtureType>::correct()
{
    DebugInFunction « endl;

    calculate
    (
        this->p_,
        this->TTR_,
        this->TVib_,
        this->he_,
        this->psi_,
        this->mu_,
        this->alpha_,
        false           // No need to update old times
    );

    DebugInFunction « "Finished" « endl;
}

// ************************************************************************* //
```

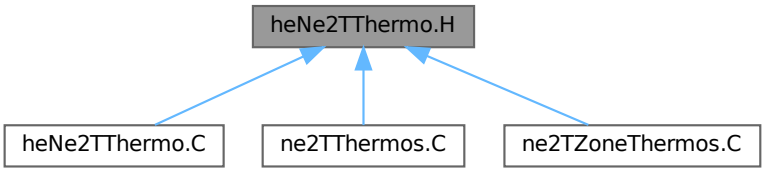
7.33 InInclude/heNe2TThermo.H File Reference

```
#include "ne2TThermo.H"
#include "he2TThermo.H"
```

Include dependency graph for lnInclude/heNe2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >

Namespaces

- namespace Foam

7.34 lnInclude/heNe2TThermo.H

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2016 OpenFOAM Foundation
    Copyright (C) 2015-2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
```

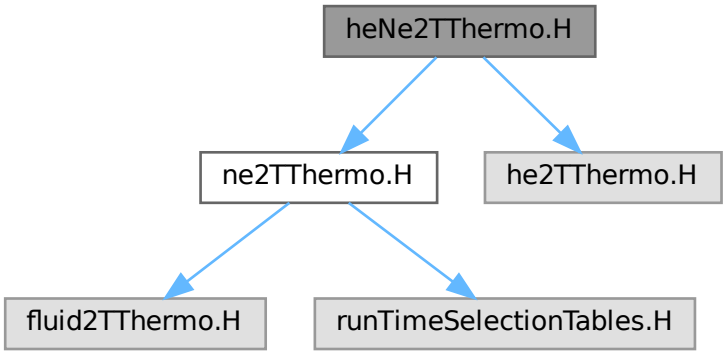
```
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::heNe2TThermo

Description
    Energy for a mixture based on compressibility

SourceFiles
    heNe2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef heNe2TThermo_H
#define heNe2TThermo_H

#include "ne2TThermo.H"
#include "he2TThermo.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class heNe2TThermo Declaration
\*---------------------------------------------------------------------------*/

template<class BasicNe2TThermo, class MixtureType>
class heNe2TThermo
:
    public he2TThermo<BasicNe2TThermo, MixtureType>
{
    // Private Member Functions

        //- Calculate the thermo variables
        void calculate
        (
            const volScalarField& p,
            volScalarField& TTR,
            volScalarField& TVib,
            volScalarField& he,
            volScalarField& psi,
            volScalarField& mu,
            volScalarField& alpha,
            const bool doOldTimes
        );

        //- Construct as copy (not implemented)
        heNe2TThermo(const heNe2TThermo<BasicNe2TThermo, MixtureType>&);


public:

    //- Runtime type information
    TypeName("heNe2TThermo");


    // Constructors

        //- Construct from mesh and phase name
        heNe2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //-  Construct from mesh and phase name and dict name
        heNe2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Destructor
```

```
    virtual ~heNe2TThermo();


    // Member functions

        //- Update properties
        virtual void correct();

};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#ifdef NoRepository
#   include "heNe2TThermo.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

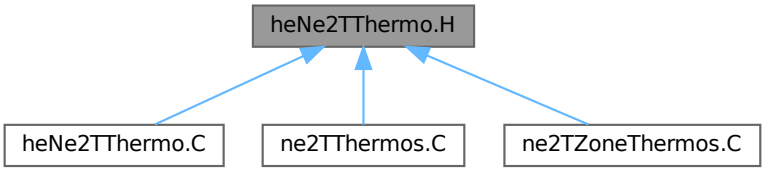
7.35 ne2TThermo/heNe2TThermo.H File Reference

```
#include "ne2TThermo.H"
#include "he2TThermo.H"
```

Include dependency graph for ne2TThermo/heNe2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >

Namespaces

- namespace Foam

7.36 ne2TThermo/heNe2TThermo.H

Go to the documentation of this file.

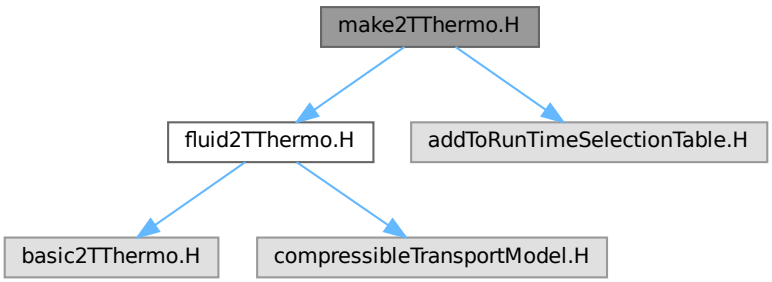
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2016 OpenFOAM Foundation
    Copyright (C) 2015-2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::heNe2TThermo

Description
    Energy for a mixture based on compressibility

SourceFiles
    heNe2TThermo.C

\*-------------------------------------------------------------------------*/

#ifndef heNe2TThermo_H
#define heNe2TThermo_H

#include "ne2TThermo.H"
#include "he2TThermo.H"
```

```

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class heNe2TThermo Declaration
\*---------------------------------------------------------------------------*/

template<class BasicNe2TThermo, class MixtureType>
class heNe2TThermo
:
    public he2TThermo<BasicNe2TThermo, MixtureType>
{
    // Private Member Functions

        //- Calculate the thermo variables
        void calculate
        (
            const volScalarField& p,
            volScalarField& TTR,
            volScalarField& TVib,
            volScalarField& he,
            volScalarField& psi,
            volScalarField& mu,
            volScalarField& alpha,
            const bool doOldTimes
        );

        //- Construct as copy (not implemented)
        heNe2TThermo(const heNe2TThermo<BasicNe2TThermo, MixtureType>&);


public:

    //- Runtime type information
    TypeName("heNe2TThermo");


    // Constructors

        //- Construct from mesh and phase name
        heNe2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );

        //-  Construct from mesh and phase name and dict name
        heNe2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Destructor
    virtual ~heNe2TThermo();


    // Member functions

        //- Update properties
        virtual void correct();

};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#ifdef NoRepository
#   include "heNe2TThermo.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.37 fluid2TThermo/make2TThermo.H File Reference

```
#include "fluid2TThermo.H"
#include "addToRunTimeSelectionTable.H"
```

Include dependency graph for fluid2TThermo/make2TThermo.H:



Macros

- #define typedefThermoPhysics(Transport, Type, Thermo, EqnOfState, Specie)
- #define defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys)
- #define addThermoPhysicsThermo(BaseThermo, CThermoMixtureThermoPhys)
- #define makeThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys)
- #define makeThermoPhysicsThermos(BaseThermo, CThermo, Mixture, ThermoPhys)
- #define makeThermo(BaseThermo, CThermo, Mixture, Transport, Type, Thermo, EqnOfState, Specie)
- #define makeThermos(BaseThermo, CThermo, Mixture, Transport, Type, Thermo, EqnOfState, Specie)

7.37.1 Macro Definition Documentation

7.37.1.1 addThermoPhysicsThermo

```
#define addThermoPhysicsThermo(
            BaseThermo,
            CThermoMixtureThermoPhys )
```

Value:

```
    addToRunTimeSelectionTable                                          \
    (                                                                   \
        BaseThermo,                                                     \
        CThermoMixtureThermoPhys,                                       \
        fvMesh                                                          \
    );                                                                  \
                                                                        \
    addToRunTimeSelectionTable                                          \
    (                                                                   \
        BaseThermo,                                                     \
        CThermoMixtureThermoPhys,                                       \
        fvMeshDictPhase                                                 \
    );
```

Definition at line 81 of file fluid2TThermo/make2TThermo.H.

7.37.1.2 defineThermoPhysicsThermo

```
#define defineThermoPhysicsThermo(
                BaseThermo,
                CThermo,
                Mixture,
                ThermoPhys )
```

Value:

```
                                                                        \
    typedef                                                             \
        CThermo                                                         \
        <                                                               \
            BaseThermo,                                                 \
            Mixture<ThermoPhys>                                         \
        >                                                               \
        CThermo##Mixture##ThermoPhys;                                   \
                                                                        \
    defineTemplateTypeNameAndDebugWithName                              \
    (                                                                   \
        CThermo##Mixture##ThermoPhys,                                   \
        (#CThermo"<" + Mixture<ThermoPhys>::typeName() + ">").c_str(),  \
        0                                                               \
    )
```

Definition at line 63 of file fluid2TThermo/make2TThermo.H.

7.37.1.3 makeThermo

```
#define makeThermo(
                BaseThermo,
                CThermo,
                Mixture,
                Transport,
                Type,
                Thermo,
                EqnOfState,
                Specie )
```

Value:

```
                                                                        \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);      \
                                                                        \
    makeThermoPhysicsThermo                                             \
    (                                                                   \
        BaseThermo,                                                     \
        CThermo,                                                        \
        Mixture,                                                        \
        Transport##Type##Thermo##EqnOfState##Specie                     \
    )
```

Definition at line 115 of file fluid2TThermo/make2TThermo.H.

7.37.1.4 makeThermoPhysicsThermo

```
#define makeThermoPhysicsThermo(
                BaseThermo,
                CThermo,
                Mixture,
                ThermoPhys )
```

Value:

```
                                                                        \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);\
                                                                        \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)
```

Definition at line 98 of file fluid2TThermo/make2TThermo.H.

7.37.1.5 makeThermoPhysicsThermos

```
#define makeThermoPhysicsThermos(
              BaseThermo,
              CThermo,
              Mixture,
              ThermoPhys )
```

Value:

```
                                                                    \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);   \
                                                                    \
                                                                    \
    addThermoPhysicsThermo(basic2TThermo, CThermo##Mixture##ThermoPhys);     \
    addThermoPhysicsThermo(fluid2TThermo, CThermo##Mixture##ThermoPhys);     \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)
```

Definition at line 105 of file fluid2TThermo/make2TThermo.H.

7.37.1.6 makeThermos

```
#define makeThermos(
              BaseThermo,
              CThermo,
              Mixture,
              Transport,
              Type,
              Thermo,
              EqnOfState,
              Specie )
```

Value:

```
                                                                    \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);  \
                                                                    \
    makeThermoPhysicsThermos                                        \
    (                                                               \
        BaseThermo,                                                 \
        CThermo,                                                    \
        Mixture,                                                    \
        Transport##Type##Thermo##EqnOfState##Specie                 \
    )
```

Definition at line 128 of file fluid2TThermo/make2TThermo.H.

7.37.1.7 typedefThermoPhysics

```
#define typedefThermoPhysics(
              Transport,
              Type,
              Thermo,
              EqnOfState,
              Specie )
```

Value:

```
                                                                    \
    typedef                                                         \
        Transport                                                   \
        <                                                           \
            species::thermo2T                                         \
            <                                                       \
                Thermo                                              \
                <                                                   \
```

```
                EqnOfState                                                  \
                <                                                           \
                    Specie                                                  \
                >                                                           \
            >,                                                              \
            Type                                                            \
        >                                                                   \
    >                                                                       \
    Transport##Type##Thermo##EqnOfState##Specie
```

Definition at line 43 of file fluid2TThermo/make2TThermo.H.

7.38 fluid2TThermo/make2TThermo.H

Go to the documentation of this file.

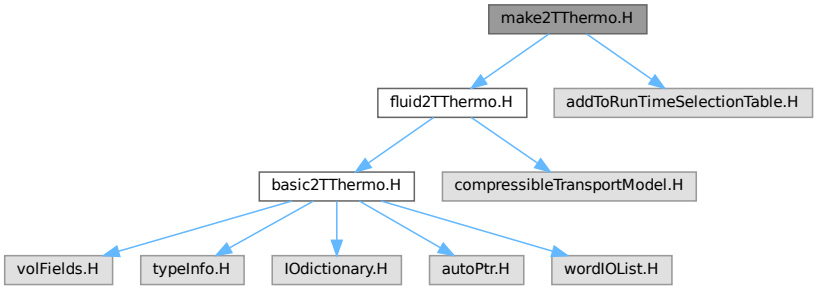
```
00001 /*-------------------------------------------------------------------------*\
00002   =========                 |
00003   \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
00004    \\    /   O peration     |
00005     \\  /    A nd           | www.openfoam.com
00006      \\/     M anipulation  |
00007 -------------------------------------------------------------------------------
00008     Copyright (C) 2012-2017 OpenFOAM Foundation
00009     Copyright (C) 2018 OpenCFD Ltd
00010 -------------------------------------------------------------------------------
00011 License
00012     This file is part of OpenFOAM.
00013
00014     OpenFOAM is free software: you can redistribute it and/or modify it
00015     under the terms of the GNU General Public License as published by
00016     the Free Software Foundation, either version 3 of the License, or
00017     (at your option) any later version.
00018
00019     OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
00020     ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
00021     FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
00022     for more details.
00023
00024     You should have received a copy of the GNU General Public License
00025     along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.
00026
00027 InClass
00028     Foam::fluid2TThermo
00029
00030 Description
00031     Macros for creating basic fluid thermo packages
00032
00033 \*-------------------------------------------------------------------------*/
00034
00035 #ifndef make2TThermo_H
00036 #define make2TThermo_H
00037
00038 #include "fluid2TThermo.H"
00039 #include "addToRunTimeSelectionTable.H"
00040
00041 // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
00042
00043 #define typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie)          \
00044                                                                                \
00045     typedef                                                                    \
00046         Transport                                                              \
00047         <                                                                      \
00048             species::thermo2T                                                  \
00049             <                                                                  \
00050                 Thermo                                                         \
00051                 <                                                              \
00052                     EqnOfState                                                 \
00053                     <                                                          \
00054                         Specie                                                 \
00055                     >                                                          \
00056                 >,                                                             \
00057                 Type                                                           \
00058             >                                                                  \
00059         >                                                                      \
00060         Transport##Type##Thermo##EqnOfState##Specie
00061
00062
00063 #define defineThermoPhysicsThermo(BaseThermo,CThermo,Mixture,ThermoPhys)       \
00064                                                                                \
00065     typedef                                                                    \
```

```
        CThermo                                                         \
        <                                                               \
            BaseThermo,                                                 \
            Mixture<ThermoPhys>                                         \
        >                                                               \
        CThermo##Mixture##ThermoPhys;                                   \
                                                                        \
    defineTemplateTypeNameAndDebugWithName                              \
    (                                                                   \
        CThermo##Mixture##ThermoPhys,                                   \
        (#CThermo"<" + Mixture<ThermoPhys>::typeName() + ">").c_str(),  \
        0                                                               \
    )


#define addThermoPhysicsThermo(BaseThermo,CThermoMixtureThermoPhys)     \
                                                                        \
    addToRunTimeSelectionTable                                          \
    (                                                                   \
        BaseThermo,                                                     \
        CThermoMixtureThermoPhys,                                       \
        fvMesh                                                          \
    );                                                                  \
                                                                        \
    addToRunTimeSelectionTable                                          \
    (                                                                   \
        BaseThermo,                                                     \
        CThermoMixtureThermoPhys,                                       \
        fvMeshDictPhase                                                 \
    );


#define makeThermoPhysicsThermo(BaseThermo,CThermo,Mixture,ThermoPhys)  \
                                                                        \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);\
                                                                        \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)


#define makeThermoPhysicsThermos(BaseThermo,CThermo,Mixture,ThermoPhys) \
                                                                        \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);\
                                                                        \
                                                                        \
    addThermoPhysicsThermo(basic2TThermo, CThermo##Mixture##ThermoPhys);  \
    addThermoPhysicsThermo(fluid2TThermo, CThermo##Mixture##ThermoPhys);  \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)


#define makeThermo(BaseThermo,CThermo,Mixture,Transport,Type,Thermo,EqnOfState,Specie) \
                                                                        \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);      \
                                                                        \
    makeThermoPhysicsThermo                                             \
    (                                                                   \
        BaseThermo,                                                     \
        CThermo,                                                        \
        Mixture,                                                        \
        Transport##Type##Thermo##EqnOfState##Specie                     \
    )


#define makeThermos(BaseThermo,CThermo,Mixture,Transport,Type,Thermo,EqnOfState,Specie) \
                                                                        \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);      \
                                                                        \
    makeThermoPhysicsThermos                                            \
    (                                                                   \
        BaseThermo,                                                     \
        CThermo,                                                        \
        Mixture,                                                        \
        Transport##Type##Thermo##EqnOfState##Specie                     \
    )

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

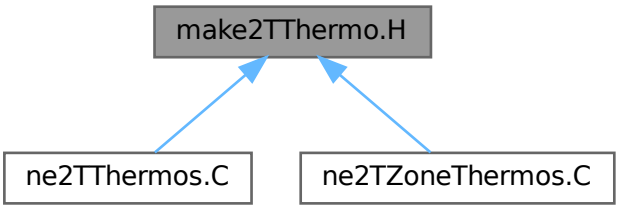
7.39 lnInclude/make2TThermo.H File Reference

```
#include "fluid2TThermo.H"
#include "addToRunTimeSelectionTable.H"
```

Include dependency graph for lnInclude/make2TThermo.H:



This graph shows which files directly or indirectly include this file:



Macros

- #define typedefThermoPhysics(Transport, Type, Thermo, EqnOfState, Specie)
- #define defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys)
- #define addThermoPhysicsThermo(BaseThermo, CThermoMixtureThermoPhys)
- #define makeThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys)
- #define makeThermoPhysicsThermos(BaseThermo, CThermo, Mixture, ThermoPhys)
- #define makeThermo(BaseThermo, CThermo, Mixture, Transport, Type, Thermo, EqnOfState, Specie)
- #define makeThermos(BaseThermo, CThermo, Mixture, Transport, Type, Thermo, EqnOfState, Specie)

7.39.1 Macro Definition Documentation

7.39.1.1 addThermoPhysicsThermo

```
#define addThermoPhysicsThermo(
            BaseThermo,
            CThermoMixtureThermoPhys )
```

Value:

```
                                                                        \
    addToRunTimeSelectionTable                                          \
    (                                                                   \
        BaseThermo,                                                     \
        CThermoMixtureThermoPhys,                                       \
        fvMesh                                                          \
    );                                                                  \
                                                                        \
    addToRunTimeSelectionTable                                          \
    (                                                                   \
        BaseThermo,                                                     \
        CThermoMixtureThermoPhys,                                       \
        fvMeshDictPhase                                                 \
    );
```

Definition at line 81 of file lnInclude/make2TThermo.H.

7.39.1.2 defineThermoPhysicsThermo

```
#define defineThermoPhysicsThermo(
              BaseThermo,
              CThermo,
              Mixture,
              ThermoPhys )
```

Value:

```
                                                                        \
    typedef                                                             \
        CThermo                                                         \
        <                                                               \
            BaseThermo,                                                 \
            Mixture<ThermoPhys>                                         \
        >                                                               \
        CThermo##Mixture##ThermoPhys;                                   \
                                                                        \
    defineTemplateTypeNameAndDebugWithName                              \
    (                                                                   \
        CThermo##Mixture##ThermoPhys,                                   \
        (#CThermo"<" + Mixture<ThermoPhys>::typeName() + ">").c_str(),  \
        0                                                               \
    )
```

Definition at line 63 of file lnInclude/make2TThermo.H.

7.39.1.3 makeThermo

```
#define makeThermo(
              BaseThermo,
              CThermo,
              Mixture,
              Transport,
              Type,
              Thermo,
              EqnOfState,
              Specie )
```

Value:

```
                                                                        \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);      \
                                                                        \
    makeThermoPhysicsThermo                                             \
    (                                                                   \
        BaseThermo,                                                     \
        CThermo,                                                        \
        Mixture,                                                        \
        Transport##Type##Thermo##EqnOfState##Specie                     \
    )
```

Definition at line 115 of file lnInclude/make2TThermo.H.

7.39.1.4 makeThermoPhysicsThermo

```
#define makeThermoPhysicsThermo(
              BaseThermo,
              CThermo,
              Mixture,
              ThermoPhys )
```

Value:

```
                                                                    \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);    \
                                                                    \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)
```

Definition at line 98 of file InInclude/make2TThermo.H.

7.39.1.5 makeThermoPhysicsThermos

```
#define makeThermoPhysicsThermos(
              BaseThermo,
              CThermo,
              Mixture,
              ThermoPhys )
```

Value:

```
                                                                    \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);    \
                                                                    \
                                                                    \
    addThermoPhysicsThermo(basic2TThermo, CThermo##Mixture##ThermoPhys);      \
    addThermoPhysicsThermo(fluid2TThermo, CThermo##Mixture##ThermoPhys);      \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)
```

Definition at line 105 of file InInclude/make2TThermo.H.

7.39.1.6 makeThermos

```
#define makeThermos(
              BaseThermo,
              CThermo,
              Mixture,
              Transport,
              Type,
              Thermo,
              EqnOfState,
              Specie )
```

Value:

```
                                                                    \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);          \
                                                                    \
    makeThermoPhysicsThermos                                        \
    (                                                               \
        BaseThermo,                                                 \
        CThermo,                                                    \
        Mixture,                                                    \
        Transport##Type##Thermo##EqnOfState##Specie                 \
    )
```

Definition at line 128 of file InInclude/make2TThermo.H.

7.39.1.7 typedefThermoPhysics

```
#define typedefThermoPhysics(
              Transport,
              Type,
              Thermo,
              EqnOfState,
              Specie )
```

Value:

```
    typedef                                                                 \
        Transport                                                           \
        <                                                                   \
            species::thermo2T                                                 \
            <                                                               \
                Thermo                                                      \
                <                                                           \
                    EqnOfState                                              \
                    <                                                       \
                        Specie                                              \
                    >                                                       \
                >,                                                          \
                Type                                                        \
            >                                                               \
        >                                                                   \
        Transport##Type##Thermo##EqnOfState##Specie
```

Definition at line 43 of file InInclude/make2TThermo.H.

7.40 InInclude/make2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
    Copyright (C) 2018 OpenCFD Ltd
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

InClass
    Foam::fluid2TThermo

Description
    Macros for creating basic fluid thermo packages

\*-------------------------------------------------------------------------*/

#ifndef make2TThermo_H
#define make2TThermo_H

#include "fluid2TThermo.H"
#include "addToRunTimeSelectionTable.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

```

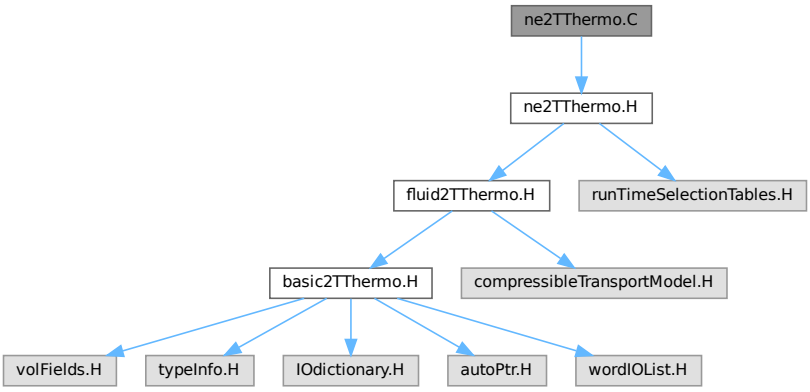
```
#define typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie)          \
                                                                               \
    typedef                                                                    \
        Transport                                                              \
        <                                                                      \
            species::thermo2T                                                   \
            <                                                                  \
                Thermo                                                         \
                <                                                              \
                    EqnOfState                                                 \
                    <                                                          \
                        Specie                                                 \
                    >                                                          \
                >,                                                             \
                Type                                                           \
            >                                                                  \
        >                                                                      \
        Transport##Type##Thermo##EqnOfState##Specie


#define defineThermoPhysicsThermo(BaseThermo,CThermo,Mixture,ThermoPhys)       \
                                                                               \
    typedef                                                                    \
        CThermo                                                                \
        <                                                                      \
            BaseThermo,                                                        \
            Mixture<ThermoPhys>                                                \
        >                                                                      \
        CThermo##Mixture##ThermoPhys;                                          \
                                                                               \
    defineTemplateTypeNameAndDebugWithName                                     \
    (                                                                          \
        CThermo##Mixture##ThermoPhys,                                          \
        (#CThermo"<" + Mixture<ThermoPhys>::typeName() + ">").c_str(),         \
        0                                                                      \
    )


#define addThermoPhysicsThermo(BaseThermo,CThermoMixtureThermoPhys)            \
                                                                               \
    addToRunTimeSelectionTable                                                 \
    (                                                                          \
        BaseThermo,                                                            \
        CThermoMixtureThermoPhys,                                              \
        fvMesh                                                                 \
    );                                                                         \
                                                                               \
    addToRunTimeSelectionTable                                                 \
    (                                                                          \
        BaseThermo,                                                            \
        CThermoMixtureThermoPhys,                                              \
        fvMeshDictPhase                                                        \
    );


#define makeThermoPhysicsThermo(BaseThermo,CThermo,Mixture,ThermoPhys)         \
                                                                               \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);       \
                                                                               \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)


#define makeThermoPhysicsThermos(BaseThermo,CThermo,Mixture,ThermoPhys)        \
                                                                               \
    defineThermoPhysicsThermo(BaseThermo, CThermo, Mixture, ThermoPhys);       \
                                                                               \
                                                                               \
    addThermoPhysicsThermo(basic2TThermo, CThermo##Mixture##ThermoPhys);         \
    addThermoPhysicsThermo(fluid2TThermo, CThermo##Mixture##ThermoPhys);         \
    addThermoPhysicsThermo(BaseThermo, CThermo##Mixture##ThermoPhys)


#define makeThermo(BaseThermo,CThermo,Mixture,Transport,Type,Thermo,EqnOfState,Specie) \
                                                                               \
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);             \
                                                                               \
    makeThermoPhysicsThermo                                                    \
    (                                                                          \
        BaseThermo,                                                            \
        CThermo,                                                               \
        Mixture,                                                               \
        Transport##Type##Thermo##EqnOfState##Specie                            \
    )


#define makeThermos(BaseThermo,CThermo,Mixture,Transport,Type,Thermo,EqnOfState,Specie) \
                                                                               \
```

```
    typedefThermoPhysics(Transport,Type,Thermo,EqnOfState,Specie);            \
                                                                              \
    makeThermoPhysicsThermos                                                  \
    (                                                                         \
        BaseThermo,                                                           \
        CThermo,                                                              \
        Mixture,                                                              \
        Transport##Type##Thermo##EqnOfState##Specie                           \
    )

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.41 InInclude/ne2TThermo.C File Reference

`#include "ne2TThermo.H"`

Include dependency graph for InInclude/ne2TThermo.C:



Namespaces

- namespace Foam

Functions

- Foam::defineRunTimeSelectionTable (ne2TThermo, fvMesh)
- Foam::defineRunTimeSelectionTable (ne2TThermo, fvMeshDictPhase)

7.42 InInclude/ne2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------
```

```
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2017-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "ne2TThermo.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    defineTypeNameAndDebug(ne2TThermo, 0);
    defineRunTimeSelectionTable(ne2TThermo, fvMesh);
    defineRunTimeSelectionTable(ne2TThermo, fvMeshDictPhase);
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

Foam::ne2TThermo::ne2TThermo(const fvMesh& mesh, const word& phaseName)
:
    fluid2TThermo(mesh, phaseName),

    psi_
    (
        IOobject
        (
            phaseScopedName("thermo", "psi"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(0, -2, 2, 0, 0)
    ),

    mu_
    (
        IOobject
        (
            phaseScopedName("thermo", "mu"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(1, -1, -1, 0, 0)
    )
{}


Foam::ne2TThermo::ne2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    fluid2TThermo(mesh, phaseName, dictionaryName),

    psi_
    (
        IOobject
        (
            phaseScopedName("thermo", "psi"),
            mesh.time().timeName(),
            mesh,
```

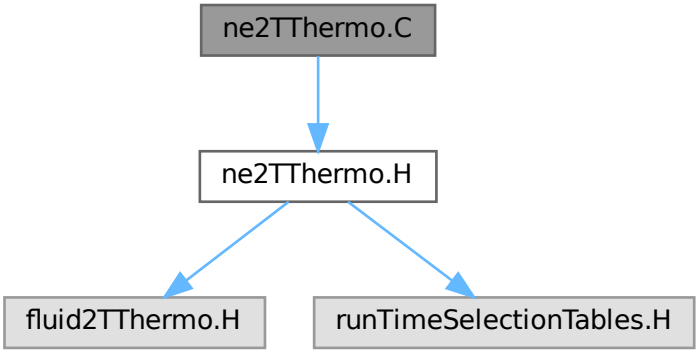
```
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(0, -2, 2, 0, 0)
    ),

    mu_
    (
        IOobject
        (
            phaseScopedName("thermo", "mu"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(1, -1, -1, 0, 0)
    )
{}

// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

Foam::autoPtr<Foam::ne2TThermo> Foam::ne2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    return basic2TThermo::New<ne2TThermo>(mesh, phaseName);
}


Foam::autoPtr<Foam::ne2TThermo> Foam::ne2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
{
     return basic2TThermo::New<ne2TThermo>(mesh, phaseName, dictionaryName);
}


// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

Foam::ne2TThermo::~ne2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

Foam::tmp<Foam::volScalarField> Foam::ne2TThermo::rho() const
{
    return p_*psi_;
}


Foam::tmp<Foam::scalarField> Foam::ne2TThermo::rho(const label patchi) const
{
    return p_.boundaryField()[patchi]*psi_.boundaryField()[patchi];
}


void Foam::ne2TThermo::correctRho
(
    const Foam::volScalarField& deltaRho,
    const dimensionedScalar& rhoMin,
    const dimensionedScalar& rhoMax
)
{}

void Foam::ne2TThermo::correctRho
(
    const Foam::volScalarField& deltaRho
)
{}

const Foam::volScalarField& Foam::ne2TThermo::psi() const
{
    return psi_;
}


```

```
Foam::tmp<Foam::volScalarField> Foam::ne2TThermo::mu() const
{

    return mu_;
}


Foam::tmp<Foam::scalarField> Foam::ne2TThermo::mu(const label patchi) const
{
    return mu_.boundaryField()[patchi];
}

// ************************************************************************* //
```

7.43 ne2TThermo/ne2TThermo.C File Reference

`#include "ne2TThermo.H"`

Include dependency graph for ne2TThermo/ne2TThermo.C:



Namespaces

- namespace Foam

Functions

- Foam::defineRunTimeSelectionTable (ne2TThermo, fvMesh)
- Foam::defineRunTimeSelectionTable (ne2TThermo, fvMeshDictPhase)

7.44 ne2TThermo/ne2TThermo.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
```

```
---------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2017-2023 OpenCFD Ltd.
---------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "ne2TThermo.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    defineTypeNameAndDebug(ne2TThermo, 0);
    defineRunTimeSelectionTable(ne2TThermo, fvMesh);
    defineRunTimeSelectionTable(ne2TThermo, fvMeshDictPhase);
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

Foam::ne2TThermo::ne2TThermo(const fvMesh& mesh, const word& phaseName)
:
    fluid2TThermo(mesh, phaseName),

    psi_
    (
        IOobject
        (
            phaseScopedName("thermo", "psi"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(0, -2, 2, 0, 0)
    ),

    mu_
    (
        IOobject
        (
            phaseScopedName("thermo", "mu"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(1, -1, -1, 0, 0)
    )
{}


Foam::ne2TThermo::ne2TThermo
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
:
    fluid2TThermo(mesh, phaseName, dictionaryName),

    psi_
    (
        IOobject
        (
            phaseScopedName("thermo", "psi"),
            mesh.time().timeName(),
```

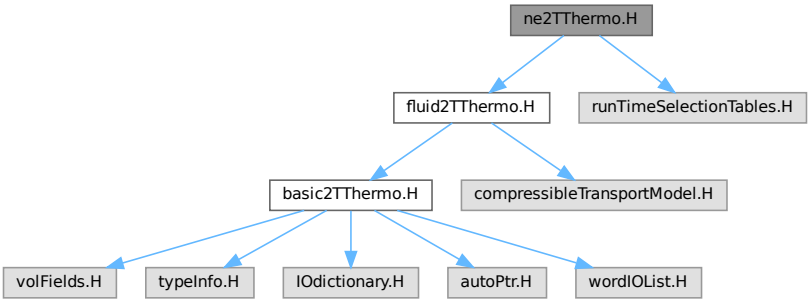
```
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(0, -2, 2, 0, 0)
    ),

    mu_
    (
        IOobject
        (
            phaseScopedName("thermo", "mu"),
            mesh.time().timeName(),
            mesh,
            IOobject::NO_READ,
            IOobject::NO_WRITE,
            IOobject::REGISTER
        ),
        mesh,
        dimensionSet(1, -1, -1, 0, 0)
    )
{}

// * * * * * * * * * * * * * * * * Selectors * * * * * * * * * * * * * * * * //

Foam::autoPtr<Foam::ne2TThermo> Foam::ne2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName
)
{
    return basic2TThermo::New<ne2TThermo>(mesh, phaseName);
}


Foam::autoPtr<Foam::ne2TThermo> Foam::ne2TThermo::New
(
    const fvMesh& mesh,
    const word& phaseName,
    const word& dictionaryName
)
{
     return basic2TThermo::New<ne2TThermo>(mesh, phaseName, dictionaryName);
}


// * * * * * * * * * * * * * * * * Destructor  * * * * * * * * * * * * * * * //

Foam::ne2TThermo::~ne2TThermo()
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

Foam::tmp<Foam::volScalarField> Foam::ne2TThermo::rho() const
{
    return p_*psi_;
}


Foam::tmp<Foam::scalarField> Foam::ne2TThermo::rho(const label patchi) const
{
    return p_.boundaryField()[patchi]*psi_.boundaryField()[patchi];
}


void Foam::ne2TThermo::correctRho
(
    const Foam::volScalarField& deltaRho,
    const dimensionedScalar& rhoMin,
    const dimensionedScalar& rhoMax
)
{}

void Foam::ne2TThermo::correctRho
(
    const Foam::volScalarField& deltaRho
)
{}

const Foam::volScalarField& Foam::ne2TThermo::psi() const
{
    return psi_;
}

```

```
Foam::tmp<Foam::volScalarField> Foam::ne2TThermo::mu() const
{
    return mu_;
}


Foam::tmp<Foam::scalarField> Foam::ne2TThermo::mu(const label patchi) const
{
    return mu_.boundaryField()[patchi];
}

// ************************************************************************* //
```

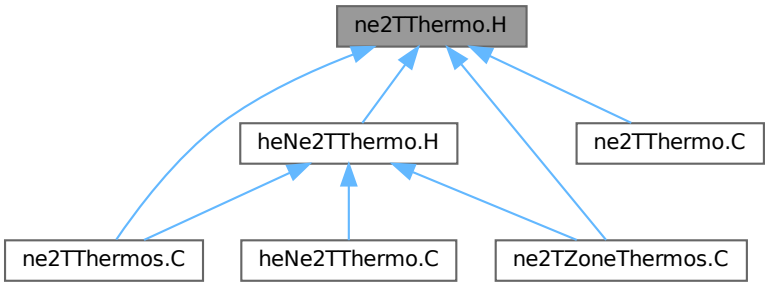
7.45 InInclude/ne2TThermo.H File Reference

```
#include "fluid2TThermo.H"
#include "runTimeSelectionTables.H"
```

Include dependency graph for InInclude/ne2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::ne2TThermo

Namespaces

- namespace Foam

7.46 InInclude/ne2TThermo.H

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::ne2TThermo

Group
    grpneThermo

Description
    Basic thermodynamic properties based on compressibility

SourceFiles
    ne2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef ne2TThermo_H
#define ne2TThermo_H

#include "fluid2TThermo.H"
#include "runTimeSelectionTables.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                           Class ne2TThermo Declaration
\*---------------------------------------------------------------------------*/

class ne2TThermo
:
    public fluid2TThermo
{

protected:

    // Protected data

        //- Compressibility [s^2/m^2]
        volScalarField psi_;

        //- Dynamic viscosity [kg/m/s]
        volScalarField mu_;


    // Protected Member Functions

```

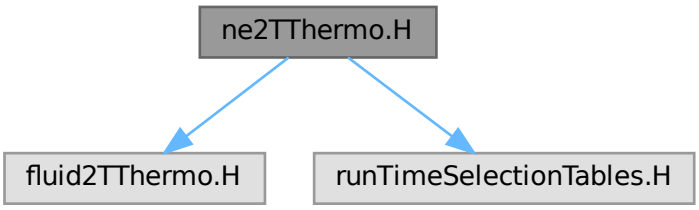
```
        //- Construct as copy (not implemented)
        ne2TThermo(const ne2TThermo&);


public:

    //- Runtime type information
    TypeName("ne2TThermo");


    //- Declare run-time constructor selection table
    declareRunTimeSelectionTable
    (
        autoPtr,
        ne2TThermo,
        fvMesh,
        (const fvMesh& mesh, const word& phaseName),
        (mesh, phaseName)
    );


    //- Declare run-time constructor selection table for dictionary based
    declareRunTimeSelectionTable
    (
        autoPtr,
        ne2TThermo,
        fvMeshDictPhase,
        (const fvMesh& mesh, const word& phaseName, const word& dictName),
        (mesh, phaseName, dictName)
    );


    // Constructors

        //- Construct from mesh and phase name
        ne2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );


        //- Construct from mesh,dictionary,phase name with a single temperature
        ne2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


    //- Selector
    static autoPtr<ne2TThermo> New
    (
        const fvMesh& mesh,
        const word& phaseName=word::null
    );

    //- Selector
    static autoPtr<ne2TThermo> New
    (
        const fvMesh&,
        const word& dictName,
        const word& phaseName
    );


    //- Destructor
    virtual ~ne2TThermo();


    // Member functions

        // Fields derived from thermodynamic state variables

            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution.
            //  For ne2TThermo does nothing.
            virtual void correctRho
            (
                const volScalarField& deltaRho,
                const dimensionedScalar& rhoMin,
                const dimensionedScalar& rhoMax
            );

            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution
```

```
            virtual void correctRho
            (
                const volScalarField& deltaRho
            );

            //- Density [kg/m^3] - uses current value of pressure
            virtual tmp<volScalarField> rho() const;

            //- Density for patch [kg/m^3]
            virtual tmp<scalarField> rho(const label patchi) const;

            //- Compressibility [s^2/m^2]
            virtual const volScalarField& psi() const;

        // Access to transport state variables

            //- Dynamic viscosity of mixture [kg/m/s]
            virtual tmp<volScalarField> mu() const;

            //- Dynamic viscosity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> mu(const label patchi) const;
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

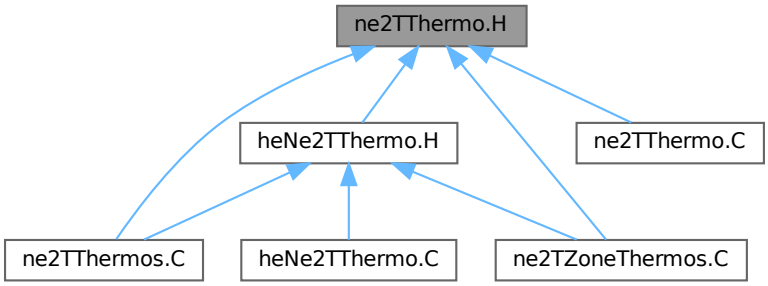
7.47 ne2TThermo/ne2TThermo.H File Reference

```
#include "fluid2TThermo.H"
#include "runTimeSelectionTables.H"
```

Include dependency graph for ne2TThermo/ne2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::ne2TThermo

Namespaces

- namespace Foam

7.48 ne2TThermo/ne2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2017 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::ne2TThermo

Group
    grpneThermo

Description
    Basic thermodynamic properties based on compressibility

SourceFiles
    ne2TThermo.C
```

```

\*---------------------------------------------------------------------------*/

#ifndef ne2TThermo_H
#define ne2TThermo_H

#include "fluid2TThermo.H"
#include "runTimeSelectionTables.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class ne2TThermo Declaration
\*---------------------------------------------------------------------------*/

class ne2TThermo
:
    public fluid2TThermo
{

protected:

    // Protected data

        //- Compressibility [s^2/m^2]
        volScalarField psi_;

        //- Dynamic viscosity [kg/m/s]
        volScalarField mu_;


    // Protected Member Functions

        //- Construct as copy (not implemented)
        ne2TThermo(const ne2TThermo&);


public:

    //- Runtime type information
    TypeName("ne2TThermo");


    //- Declare run-time constructor selection table
    declareRunTimeSelectionTable
    (
        autoPtr,
        ne2TThermo,
        fvMesh,
        (const fvMesh& mesh, const word& phaseName),
        (mesh, phaseName)
    );


    //- Declare run-time constructor selection table for dictionary based
    declareRunTimeSelectionTable
    (
        autoPtr,
        ne2TThermo,
        fvMeshDictPhase,
        (const fvMesh& mesh, const word& phaseName, const word& dictName),
        (mesh, phaseName, dictName)
    );


    // Constructors

        //- Construct from mesh and phase name
        ne2TThermo
        (
            const fvMesh&,
            const word& phaseName
        );


        //- Construct from mesh,dictionary,phase name with a single temperature
        ne2TThermo
        (
            const fvMesh&,
            const word& phaseName,
            const word& dictionaryName
        );


```

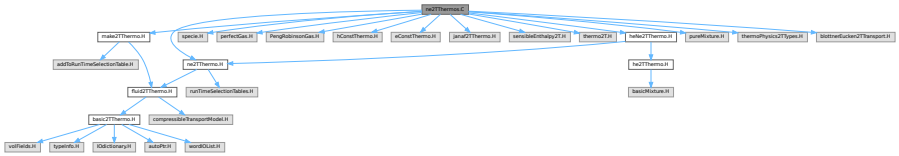
```
    //- Selector
    static autoPtr<ne2TThermo> New
    (
        const fvMesh& mesh,
        const word& phaseName=word::null
    );

    //- Selector
    static autoPtr<ne2TThermo> New
    (
        const fvMesh&,
        const word& dictName,
        const word& phaseName
    );


    //- Destructor
    virtual ~ne2TThermo();


    // Member functions

        // Fields derived from thermodynamic state variables

            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution.
            //  For ne2TThermo does nothing.
            virtual void correctRho
            (
                const volScalarField& deltaRho,
                const dimensionedScalar& rhoMin,
                const dimensionedScalar& rhoMax
            );

            //- Add the given density correction to the density field.
            //  Used to update the density field following pressure solution
            virtual void correctRho
            (
                const volScalarField& deltaRho
            );

            //- Density [kg/m^3] - uses current value of pressure
            virtual tmp<volScalarField> rho() const;

            //- Density for patch [kg/m^3]
            virtual tmp<scalarField> rho(const label patchi) const;

            //- Compressibility [s^2/m^2]
            virtual const volScalarField& psi() const;

        // Access to transport state variables

            //- Dynamic viscosity of mixture [kg/m/s]
            virtual tmp<volScalarField> mu() const;

            //- Dynamic viscosity of mixture for patch [kg/m/s]
            virtual tmp<scalarField> mu(const label patchi) const;
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.49 InInclude/ne2TThermos.C File Reference

```
#include "ne2TThermo.H"
#include "make2TThermo.H"
#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
```

```
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"
#include "heNe2TThermo.H"
#include "pureMixture.H"
#include "thermoPhysics2TTypes.H"
#include "blottnerEucken2TTransport.H"
```

Include dependency graph for lnInclude/ne2TThermos.C:



Namespaces

- namespace Foam

Typedefs

- typedef blottnerEucken2TTransport< species::thermo2T< janaf2TThermo< perfectGas< specie > >, sensibleEnthalpy2T > > Foam::blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
- typedef heNe2TThermo< ne2TThermo, pureMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGa > > Foam::heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie

Functions

- Foam::defineTemplateTypeNameAndDebugWithName (heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalp Ne2TThermo""<"+pureMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie >::typeName()+">").c_str(), 0)
- Foam::addThermoPhysicsThermo (basic2TThermo, heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2T
- Foam::addThermoPhysicsThermo (fluid2TThermo, heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tj
- Foam::addThermoPhysicsThermo (ne2TThermo, heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tja

7.50 lnInclude/ne2TThermos.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

```

```
    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "ne2TThermo.H"
#include "make2TThermo.H"

#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"

#include "heNe2TThermo.H"
#include "pureMixture.H"

#include "thermoPhysics2TTypes.H"

#include "blottnerEucken2TTransport.H"
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/* * * * * * * * * * * * * * * * Enthalpy-based * * * * * * * * * * * * * * */

makeThermos
(
    ne2TThermo,
    heNe2TThermo,
    pureMixture,
    blottnerEucken2TTransport,
    sensibleEnthalpy2T,
    janaf2TThermo,
    perfectGas,
    specie
);


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// ************************************************************************* //
```

7.51 ne2TThermo/ne2TThermos.C File Reference

```
#include "ne2TThermo.H"
#include "make2TThermo.H"
#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"
#include "heNe2TThermo.H"
#include "pureMixture.H"
#include "thermoPhysics2TTypes.H"
#include "blottnerEucken2TTransport.H"
```

Include dependency graph for ne2TThermo/ne2TThermos.C:

Namespaces

- namespace Foam

Functions

- Foam::makeThermos (ne2TThermo, heNe2TThermo, pureMixture, blottnerEucken2TTransport, sensibleEnthalpy2T, janaf2TThermo, perfectGas, specie)

7.52 ne2TThermo/ne2TThermos.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "ne2TThermo.H"
#include "make2TThermo.H"

#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"

#include "heNe2TThermo.H"
#include "pureMixture.H"

#include "thermoPhysics2TTypes.H"

#include "blottnerEucken2TTransport.H"
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
```

```
/* * * * * * * * * * * * * * * * Enthalpy-based * * * * * * * * * * * * * * */

makeThermos
(
    ne2TThermo,
    heNe2TThermo,
    pureMixture,
    blottnerEucken2TTransport,
    sensibleEnthalpy2T,
    janaf2TThermo,
    perfectGas,
    specie
);


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// ************************************************************************* //
```

7.53 InInclude/ne2TZoneThermos.C File Reference

```
#include "ne2TThermo.H"
#include "make2TThermo.H"
#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"
#include "hPolynomialThermo.H"
#include "heNe2TThermo.H"
#include "pureZoneMixture.H"
#include "thermoPhysics2TTypes.H"
#include "blottnerEucken2TTransport.H"
```

Include dependency graph for InInclude/ne2TZoneThermos.C:

Namespaces

- namespace Foam

Typedefs

- typedef heNe2TThermo< ne2TThermo, pureZoneMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperf
  > > Foam::heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie

Functions

- Foam::defineTemplateTypeNameAndDebugWithName (heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEn
Ne2TThermo""<"+pureZoneMixture< blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
>::typeName()+">").c_str(), 0)
- Foam::addThermoPhysicsThermo (basic2TThermo, heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEntha
- Foam::addThermoPhysicsThermo (fluid2TThermo, heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthal
- Foam::addThermoPhysicsThermo (ne2TThermo, heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy

7.54 InInclude/ne2TZoneThermos.C

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2021,2022 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "ne2TThermo.H"
#include "make2TThermo.H"

#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"

#include "hPolynomialThermo.H"

#include "heNe2TThermo.H"
#include "pureZoneMixture.H"

#include "thermoPhysics2TTypes.H"

#include "blottnerEucken2TTransport.H"
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/* * * * * * * * * * * * * * * * * * Enthalpy-based * * * * * * * * * * * * * */

makeThermos
(
    ne2TThermo,
    heNe2TThermo,
    pureZoneMixture,
    blottnerEucken2TTransport,
    sensibleEnthalpy2T,
    janaf2TThermo,
    perfectGas,
    specie
);
```

```
00066
00067
00068 // * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //
00069
00070 } // End namespace Foam
00071
00072 // ************************************************************************* //
```

7.55 ne2TThermo/ne2TZoneThermos.C File Reference

```
#include "ne2TThermo.H"
#include "make2TThermo.H"
#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"
#include "hPolynomialThermo.H"
#include "heNe2TThermo.H"
#include "pureZoneMixture.H"
#include "thermoPhysics2TTypes.H"
#include "blottnerEucken2TTransport.H"
```

Include dependency graph for ne2TThermo/ne2TZoneThermos.C:

Namespaces

- namespace Foam

Functions

- Foam::makeThermos (ne2TThermo, heNe2TThermo, pureZoneMixture, blottnerEucken2TTransport, sensibleEnthalpy2T, janaf2TThermo, perfectGas, specie)

7.56 ne2TThermo/ne2TZoneThermos.C

Go to the documentation of this file.

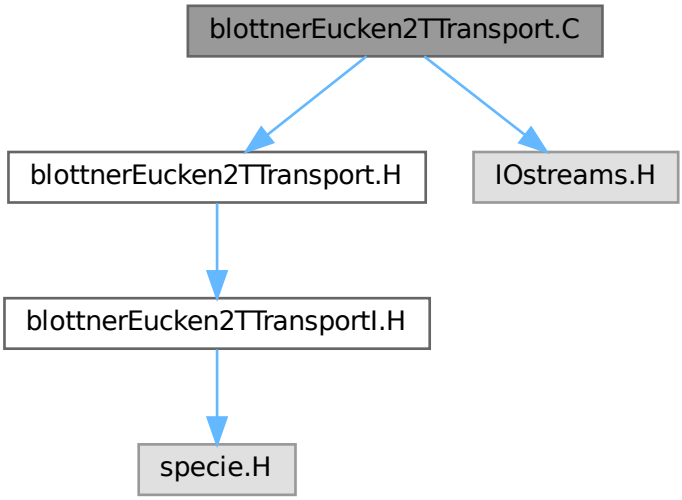
```
00001 /*-------------------------------------------------------------------------------*\
00002   =========                 |
00003   \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
00004    \\    /   O peration     |
00005     \\  /    A nd           | www.openfoam.com
00006      \\/     M anipulation  |
00007 -------------------------------------------------------------------------------
00008     Copyright (C) 2021,2022 OpenCFD Ltd.
00009 -------------------------------------------------------------------------------
00010 License
00011     This file is part of OpenFOAM.
```

```

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "ne2TThermo.H"
#include "make2TThermo.H"

#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "sensibleEnthalpy2T.H"
#include "thermo2T.H"

#include "hPolynomialThermo.H"

#include "heNe2TThermo.H"
#include "pureZoneMixture.H"

#include "thermoPhysics2TTypes.H"

#include "blottnerEucken2TTransport.H"
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/* * * * * * * * * * * * * * * * Enthalpy-based * * * * * * * * * * * * * * */

makeThermos
(
    ne2TThermo,
    heNe2TThermo,
    pureZoneMixture,
    blottnerEucken2TTransport,
    sensibleEnthalpy2T,
    janaf2TThermo,
    perfectGas,
    specie
);


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// ************************************************************************* //
```

7.57 lnInclude/blottnerEucken2TTransport.C File Reference

```
#include "blottnerEucken2TTransport.H"
#include "IOstreams.H"
```

Include dependency graph for lnInclude/blottnerEucken2TTransport.C:



7.58 lnInclude/blottnerEucken2TTransport.C

Go to the documentation of this file.

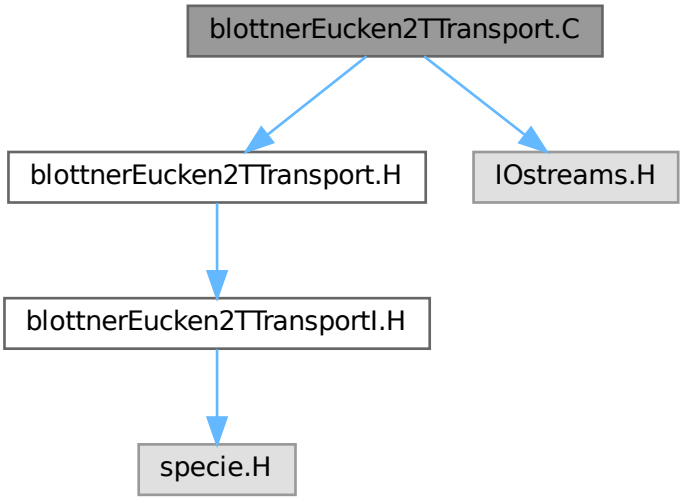
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "blottnerEucken2TTransport.H"
#include "IOstreams.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class Thermo2T>
Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::readCoeff
(
    const word& coeffName,
    const dictionary& dict
)
{
    return dict.subDict("transport").get<scalar>(coeffName);
```

```
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class Thermo2T>
Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport(const dictionary& dict)
:
    Thermo2T(dict),
    AB_(readCoeff("AB", dict)),
    BB_(readCoeff("BB", dict)),
    CB_(readCoeff("CB", dict))
{}


template<class Thermo2T>
Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport
(
    const Thermo2T& t,
    const dictionary& dict
)
:
    Thermo2T(t),
    AB_(readCoeff("AB", dict)),
    BB_(readCoeff("BB", dict)),
    CB_(readCoeff("CB", dict))
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * * //

template<class Thermo2T>
void Foam::blottnerEucken2TTransport<Thermo2T>::write(Ostream& os) const
{
    os.beginBlock(this->specie::name());

    Thermo2T::write(os);

    // Entries in dictionary format
    {
        os.beginBlock("transport");
        os.writeEntry("AB", AB_);
        os.writeEntry("BB", BB_);
        os.writeEntry("CB", CB_);
        os.endBlock();
    }

    os.endBlock();
}


// * * * * * * * * * * * * * * * * IOstream Operators  * * * * * * * * * * * * * //

template<class Thermo2T>
Foam::Ostream& Foam::operator«
(
    Ostream& os,
    const blottnerEucken2TTransport<Thermo2T>& st
)
{
    st.write(os);
    return os;
}


// ************************************************************************* //
```

7.59 transport/blottnerEucken2T/blottnerEucken2TTransport.C File Reference

```
#include "blottnerEucken2TTransport.H"
#include "IOstreams.H"
```

Include dependency graph for transport/blottnerEucken2T/blottnerEucken2TTransport.C:



7.60 transport/blottnerEucken2T/blottnerEucken2TTransport.C

Go to the documentation of this file.

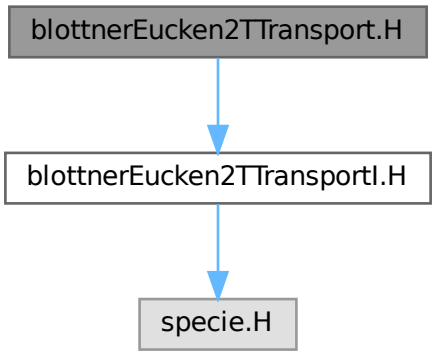
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "blottnerEucken2TTransport.H"
#include "IOstreams.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class Thermo2T>
Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::readCoeff
(
    const word& coeffName,
    const dictionary& dict
)
{
    return dict.subDict("transport").get<scalar>(coeffName);
```

```
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class Thermo2T>
Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport(const dictionary& dict)
:
    Thermo2T(dict),
    AB_(readCoeff("AB", dict)),
    BB_(readCoeff("BB", dict)),
    CB_(readCoeff("CB", dict))
{}


template<class Thermo2T>
Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport
(
    const Thermo2T& t,
    const dictionary& dict
)
:
    Thermo2T(t),
    AB_(readCoeff("AB", dict)),
    BB_(readCoeff("BB", dict)),
    CB_(readCoeff("CB", dict))
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * * //

template<class Thermo2T>
void Foam::blottnerEucken2TTransport<Thermo2T>::write(Ostream& os) const
{
    os.beginBlock(this->specie::name());

    Thermo2T::write(os);

    // Entries in dictionary format
    {
        os.beginBlock("transport");
        os.writeEntry("AB", AB_);
        os.writeEntry("BB", BB_);
        os.writeEntry("CB", CB_);
        os.endBlock();
    }

    os.endBlock();
}


// * * * * * * * * * * * * * * * * IOstream Operators  * * * * * * * * * * * * * //

template<class Thermo2T>
Foam::Ostream& Foam::operator«
(
    Ostream& os,
    const blottnerEucken2TTransport<Thermo2T>& st
)
{
    st.write(os);
    return os;
}


// ************************************************************************* //
```

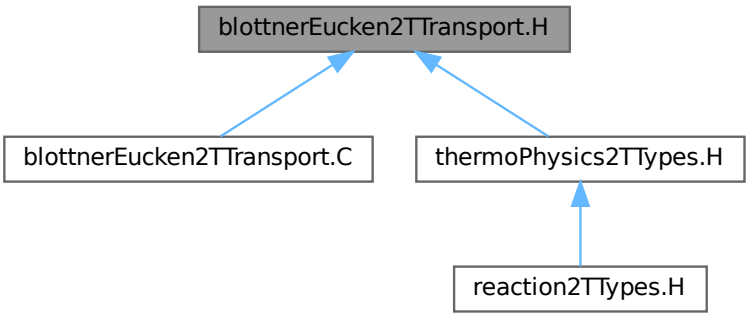
7.61 lnInclude/blottnerEucken2TTransport.H File Reference

```
#include "blottnerEucken2TTransportI.H"
```

Include dependency graph for lnInclude/blottnerEucken2TTransport.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::blottnerEucken2TTransport< Thermo2T >

Namespaces

- namespace Foam

Functions

- template<class Thermo2T >
  blottnerEucken2TTransport< Thermo2T > Foam::operator+ (const blottnerEucken2TTransport< Thermo2T > &, const blottnerEucken2TTransport< Thermo2T > &)
- template<class Thermo2T >
  blottnerEucken2TTransport< Thermo2T > Foam::operator* (const scalar, const blottnerEucken2TTransport< Thermo2T > &)
- template<class Thermo2T >
  Ostream & Foam::operator<< (Ostream &, const blottnerEucken2TTransport< Thermo2T > &)

7.62 lnInclude/blottnerEucken2TTransport.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::blottnerEucken2TTransport

Group
    grpSpecieTransport

Description
    Transport package using Blottner's viscosity formula and Eucken's
    thermal conductivity formula.

    Templated into a given thermodynamics package (needed for thermal
    conductivity).

    Dynamic viscosity [kg/m.s]
    \f[
        \mu_s = 0.1 * exp((A_B*ln(T) + B_B)*ln(T) + C_B)
    \f]
    T is translational temperature.

    Thermal conductivity [kg/m.s]
    \f[
        \kappa_tr = \mu_s (\frac{5}{2}*c_{v,t,s} + c_{v,r,s})
        \kappa_v = \nu_v * \mu_s
        \nu_v = 1.2
    \f]
    Holds until 6000 K.

SourceFiles
    blottnerEuckenTTransportI.H
    blottnerEucken2TTransport.C

\*-------------------------------------------------------------------------*/

#ifndef blottnerEuckenTTransport_H
#define blottnerEuckenTTransport_H

```

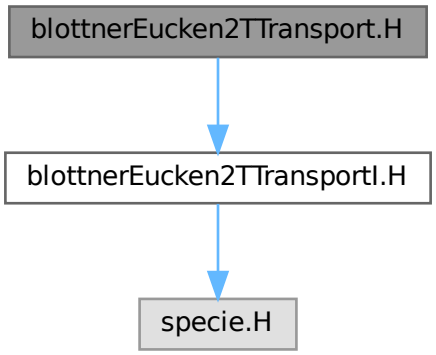
```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

// Forward Declarations

template<class Thermo2T> class blottnerEucken2TTransport;

template<class Thermo2T>
inline blottnerEucken2TTransport<Thermo2T> operator+
(
    const blottnerEucken2TTransport<Thermo2T>&,
    const blottnerEucken2TTransport<Thermo2T>&
);

template<class Thermo2T>
inline blottnerEucken2TTransport<Thermo2T> operator*
(
    const scalar,
    const blottnerEucken2TTransport<Thermo2T>&
);

template<class Thermo2T>
Ostream& operator«
(
    Ostream&,
    const blottnerEucken2TTransport<Thermo2T>&
);


/*---------------------------------------------------------------------------*\
                    Class blottnerEucken2TTransport Declaration
\*---------------------------------------------------------------------------*/

template<class Thermo2T>
class blottnerEucken2TTransport
:
    public Thermo2T
{
    // Private Data

        // Blottner's coefficients
        scalar AB_, BB_, CB_;


    // Private Member Functions

        //- Read coefficient from dictionary
        scalar readCoeff(const word& coeffName, const dictionary& dict);


public:

    // Generated Methods: copy construct, copy assignment


    // Constructors

        //- Construct from components
        inline blottnerEucken2TTransport
        (
            const Thermo2T& t,
            const scalar AB,
            const scalar BB,
            const scalar CB
        );

        //- Construct as named copy
        inline blottnerEucken2TTransport(const word&, const blottnerEucken2TTransport&);

        //- Construct from dictionary
        explicit blottnerEucken2TTransport(const dictionary& dict);

        //- Construct from base thermo and dictionary
        blottnerEucken2TTransport(const Thermo2T& t, const dictionary& dict);

        //- Construct and return a clone
        inline autoPtr<blottnerEucken2TTransport> clone() const;

        // Selector from dictionary
        inline static autoPtr<blottnerEucken2TTransport> New(const dictionary& dict);


    // Member Functions

        //- Return the instantiated type name
```

```
        static word typeName()
        {
            return "blottnerEucken2T<" + Thermo2T::typeName() + '>';
        }

        //- Dynamic viscosity [kg/ms]
        inline scalar mu(const scalar TTR) const;

        //- Thermal translational-rotational conductivity [W/mK]
        inline scalar kappaTR(const scalar p, const scalar TTR) const;

        //- Thermal vibrational conductivity [W/mK]
        inline scalar kappaVib(const scalar TTR) const;

        //- Thermal diffusivity of enthalpy [kg/ms]
        inline scalar alphah(const scalar p, const scalar TTR) const;

        // Species diffusivity
        //inline scalar D(const scalar p, const scalar TTR) const;

        //- Write to Ostream
        void write(Ostream& os) const;


    // Member Operators

        inline void operator+=(const blottnerEucken2TTransport&);

        inline void operator*=(const scalar);


    // Friend Operators

        friend blottnerEucken2TTransport operator+ <Thermo2T>
        (
            const blottnerEucken2TTransport&,
            const blottnerEucken2TTransport&
        );

        friend blottnerEucken2TTransport operator* <Thermo2T>
        (
            const scalar,
            const blottnerEucken2TTransport&
        );


    // IOstream Operators

        friend Ostream& operator« <Thermo2T>
        (
            Ostream&,
            const blottnerEucken2TTransport&
        );
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "blottnerEucken2TTransportI.H"

#ifdef NoRepository
    #include "blottnerEucken2TTransport.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

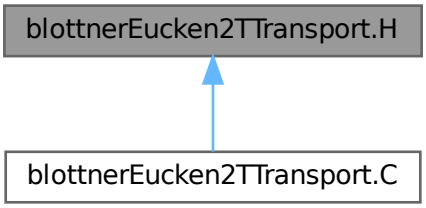
7.63 transport/blottnerEucken2T/blottnerEucken2TTransport.H File Reference

```
#include "blottnerEucken2TTransportI.H"
```

Include dependency graph for transport/blottnerEucken2T/blottnerEucken2TTransport.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::blottnerEucken2TTransport< Thermo2T >

Namespaces

- namespace Foam

Functions

- template<class Thermo2T >
  blottnerEucken2TTransport< Thermo2T > Foam::operator+ (const blottnerEucken2TTransport< Thermo2T > &, const blottnerEucken2TTransport< Thermo2T > &)
- template<class Thermo2T >
  blottnerEucken2TTransport< Thermo2T > Foam::operator* (const scalar, const blottnerEucken2TTransport< Thermo2T > &)
- template<class Thermo2T >
  Ostream & Foam::operator<< (Ostream &, const blottnerEucken2TTransport< Thermo2T > &)

7.64 transport/blottnerEucken2T/blottnerEucken2TTransport.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::blottnerEucken2TTransport

Group
    grpSpecieTransport

Description
    Transport package using Blottner's viscosity formula and Eucken's
    thermal conductivity formula.

    Templated into a given thermodynamics package (needed for thermal
    conductivity).

    Dynamic viscosity [kg/m.s]
    \f[
        \mu_s = 0.1 * exp((A_B*ln(T) + B_B)*ln(T) + C_B)
    \f]
    T is translational temperature.

    Thermal conductivity [kg/m.s]
    \f[
        \kappa_tr = \mu_s (\frac{5}{2}*c_{v,t,s} + c_{v,r,s})
        \kappa_v = \nu_v * \mu_s
        \nu_v = 1.2
    \f]
    Holds until 6000 K.

SourceFiles
    blottnerEuckenTTransportI.H
    blottnerEucken2TTransport.C

\*-------------------------------------------------------------------------*/

#ifndef blottnerEuckenTTransport_H
#define blottnerEuckenTTransport_H

```

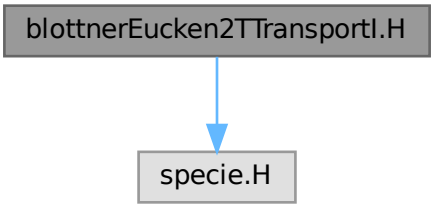
```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

// Forward Declarations

template<class Thermo2T> class blottnerEucken2TTransport;

template<class Thermo2T>
inline blottnerEucken2TTransport<Thermo2T> operator+
(
    const blottnerEucken2TTransport<Thermo2T>&,
    const blottnerEucken2TTransport<Thermo2T>&
);

template<class Thermo2T>
inline blottnerEucken2TTransport<Thermo2T> operator*
(
    const scalar,
    const blottnerEucken2TTransport<Thermo2T>&
);

template<class Thermo2T>
Ostream& operator«
(
    Ostream&,
    const blottnerEucken2TTransport<Thermo2T>&
);


/*---------------------------------------------------------------------------*\
                    Class blottnerEucken2TTransport Declaration
\*---------------------------------------------------------------------------*/

template<class Thermo2T>
class blottnerEucken2TTransport
:
    public Thermo2T
{
    // Private Data

        // Blottner's coefficients
        scalar AB_, BB_, CB_;


    // Private Member Functions

        //- Read coefficient from dictionary
        scalar readCoeff(const word& coeffName, const dictionary& dict);


public:

    // Generated Methods: copy construct, copy assignment


    // Constructors

        //- Construct from components
        inline blottnerEucken2TTransport
        (
            const Thermo2T& t,
            const scalar AB,
            const scalar BB,
            const scalar CB
        );

        //- Construct as named copy
        inline blottnerEucken2TTransport(const word&, const blottnerEucken2TTransport&);

        //- Construct from dictionary
        explicit blottnerEucken2TTransport(const dictionary& dict);

        //- Construct from base thermo and dictionary
        blottnerEucken2TTransport(const Thermo2T& t, const dictionary& dict);

        //- Construct and return a clone
        inline autoPtr<blottnerEucken2TTransport> clone() const;

        // Selector from dictionary
        inline static autoPtr<blottnerEucken2TTransport> New(const dictionary& dict);


    // Member Functions

        //- Return the instantiated type name
```

```
        static word typeName()
        {
            return "blottnerEucken2T<" + Thermo2T::typeName() + '>';
        }

        //- Dynamic viscosity [kg/ms]
        inline scalar mu(const scalar TTR) const;

        //- Thermal translational-rotational conductivity [W/mK]
        inline scalar kappaTR(const scalar p, const scalar TTR) const;

        //- Thermal vibrational conductivity [W/mK]
        inline scalar kappaVib(const scalar TTR) const;

        //- Thermal diffusivity of enthalpy [kg/ms]
        inline scalar alphah(const scalar p, const scalar TTR) const;

        // Species diffusivity
        //inline scalar D(const scalar p, const scalar TTR) const;

        //- Write to Ostream
        void write(Ostream& os) const;


    // Member Operators

        inline void operator+=(const blottnerEucken2TTransport&);

        inline void operator*=(const scalar);


    // Friend Operators

        friend blottnerEucken2TTransport operator+ <Thermo2T>
        (
            const blottnerEucken2TTransport&,
            const blottnerEucken2TTransport&
        );

        friend blottnerEucken2TTransport operator* <Thermo2T>
        (
            const scalar,
            const blottnerEucken2TTransport&
        );


    // IOstream Operators

        friend Ostream& operator« <Thermo2T>
        (
            Ostream&,
            const blottnerEucken2TTransport&
        );
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "blottnerEucken2TTransportI.H"

#ifdef NoRepository
    #include "blottnerEucken2TTransport.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

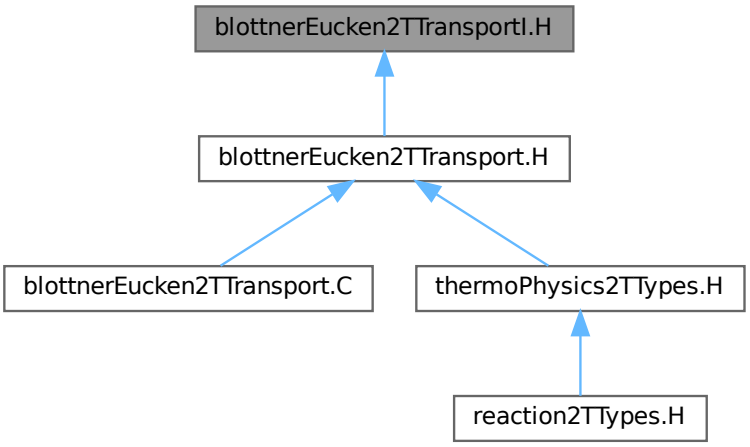
7.65 lnInclude/blottnerEucken2TTransportI.H File Reference

`#include "specie.H"`

Include dependency graph for lnInclude/blottnerEucken2TTransportI.H:



This graph shows which files directly or indirectly include this file:



7.66 lnInclude/blottnerEucken2TTransportI.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.
```

```

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "specie.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //


// * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport
(
    const Thermo2T& t,
    const scalar AB,
    const scalar BB,
    const scalar CB
)
:
    Thermo2T(t),
    AB_(AB),
    BB_(BB),
    CB_(CB)
{}


template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport
(
    const word& name,
    const blottnerEucken2TTransport& st
)
:
    Thermo2T(name, st),
    AB_(st.AB_),
    BB_(st.BB_),
    CB_(st.CB_)
{}


template<class Thermo2T>
inline Foam::autoPtr<Foam::blottnerEucken2TTransport<Thermo2T»
Foam::blottnerEucken2TTransport<Thermo2T>::clone() const
{
    return autoPtr<blottnerEucken2TTransport<Thermo2T»::New(*this);
}


template<class Thermo2T>
inline Foam::autoPtr<Foam::blottnerEucken2TTransport<Thermo2T»
Foam::blottnerEucken2TTransport<Thermo2T>::New
(
    const dictionary& dict
)
{
    return autoPtr<blottnerEucken2TTransport<Thermo2T»::New(dict);
}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * //

template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::mu
(
    const scalar TTR
) const
{
    //Info « "Function called" « endl;
    return 0.1*exp((AB_*log(TTR) + BB_)*log(TTR) + CB_);
}


template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::kappaTR
```

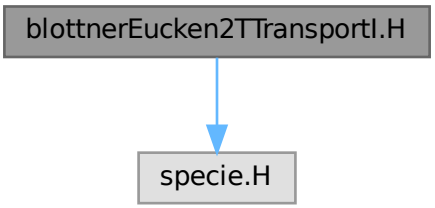
```
(
    const scalar p, const scalar TTR
) const
{
    // This function needs to be corrected
    //Info « "kappaTR function called" « endl;
    scalar Cv_T_ = this->CvT(p, TTR);
    scalar Cv_R_ = this->CvT(p, TTR);
    return mu(TTR)*((5/2)*Cv_T_ + Cv_R_);
}

template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::kappaVib
(
    const scalar TTR
) const
{
    scalar nv_ = 1.2;
    return nv_*mu(TTR);
}


template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::alphah
(
    const scalar p,
    const scalar TTR
) const
{

    return kappaTR(p, TTR)/this->CpTR(p, TTR);
}


// * * * * * * * * * * * * * * * Member Operators  * * * * * * * * * * * * * //

template<class Thermo2T>
inline void Foam::blottnerEucken2TTransport<Thermo2T>::operator+=
(
    const blottnerEucken2TTransport<Thermo2T>& st
)
{
    scalar Y1 = this->Y();

    Thermo2T::operator+=(st);

    if (mag(this->Y()) > SMALL)
    {
        Y1 /= this->Y();
        scalar Y2 = st.Y()/this->Y();

        AB_ = Y1*AB_ + Y2*st.AB_;
        BB_ = Y1*BB_ + Y2*st.BB_;
        CB_ = Y1*CB_ + Y2*st.CB_;
    }
}


template<class Thermo2T>
inline void Foam::blottnerEucken2TTransport<Thermo2T>::operator*=
(
    const scalar s
)
{
    Thermo2T::operator*=(s);
}


// * * * * * * * * * * * * * * * Friend Operators  * * * * * * * * * * * * * //

template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T> Foam::operator+
(
    const blottnerEucken2TTransport<Thermo2T>& st1,
    const blottnerEucken2TTransport<Thermo2T>& st2
)
{
    Thermo2T t
    (
        static_cast<const Thermo2T&>(st1) + static_cast<const Thermo2T&>(st2)
    );

    if (mag(t.Y()) < SMALL)
    {
        return blottnerEucken2TTransport<Thermo2T>
        (
            t,
```

```
            st1.AB_,
            st1.BB_,
            st1.CB_
        );
    }
    else
    {
        scalar Y1 = st1.Y()/t.Y();
        scalar Y2 = st2.Y()/t.Y();

        return blottnerEucken2TTransport<Thermo2T>
        (
            t,
            Y1*st1.AB_ + Y2*st2.AB_,
            Y1*st1.BB_ + Y2*st2.BB_,
            Y1*st1.CB_ + Y2*st2.CB_
        );
    }
}


template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T> Foam::operator*
(
    const scalar s,
    const blottnerEucken2TTransport<Thermo2T>& st
)
{
    return blottnerEucken2TTransport<Thermo2T>
    (
        s*static_cast<const Thermo2T&>(st),
        st.AB_,
        st.BB_,
        st.CB_
    );
}


// ************************************************************************* //
```

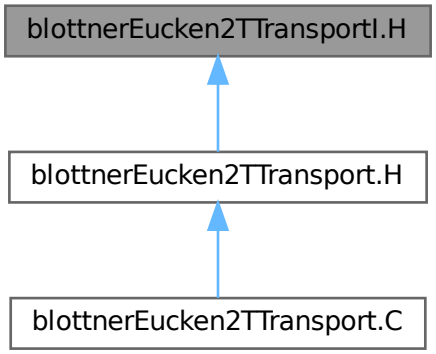
7.67 transport/blottnerEucken2T/blottnerEucken2TTransportI.H File Reference

`#include "specie.H"`

Include dependency graph for transport/blottnerEucken2T/blottnerEucken2TTransportI.H:



This graph shows which files directly or indirectly include this file:



7.68 transport/blottnerEucken2T/blottnerEucken2TTransportI.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "specie.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //


// * * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport
(
    const Thermo2T& t,
    const scalar AB,
    const scalar BB,
    const scalar CB
)
:
    Thermo2T(t),
    AB_(AB),
    BB_(BB),
    CB_(CB)
```

```
{}


template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T>::blottnerEucken2TTransport
(
    const word& name,
    const blottnerEucken2TTransport& st
)
:
    Thermo2T(name, st),
    AB_(st.AB_),
    BB_(st.BB_),
    CB_(st.CB_)
{}


template<class Thermo2T>
inline Foam::autoPtr<Foam::blottnerEucken2TTransport<Thermo2T»
Foam::blottnerEucken2TTransport<Thermo2T>::clone() const
{
    return autoPtr<blottnerEucken2TTransport<Thermo2T»::New(*this);
}


template<class Thermo2T>
inline Foam::autoPtr<Foam::blottnerEucken2TTransport<Thermo2T»
Foam::blottnerEucken2TTransport<Thermo2T>::New
(
    const dictionary& dict
)
{
    return autoPtr<blottnerEucken2TTransport<Thermo2T»::New(dict);
}


// * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::mu
(
    const scalar TTR
) const
{
    //Info « "Function called" « endl;
    return 0.1*exp((AB_*log(TTR) + BB_)*log(TTR) + CB_);
}


template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::kappaTR
(
    const scalar p, const scalar TTR
) const
{
    // This function needs to be corrected
    //Info « "kappaTR function called" « endl;
    scalar Cv_T_ = this->CvT(p, TTR);
    scalar Cv_R_ = this->CvT(p, TTR);
    return mu(TTR)*((5/2)*Cv_T_ + Cv_R_);
}

template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::kappaVib
(
    const scalar TTR
) const
{
    scalar nv_ = 1.2;
    return nv_*mu(TTR);
}


template<class Thermo2T>
inline Foam::scalar Foam::blottnerEucken2TTransport<Thermo2T>::alphah
(
    const scalar p,
    const scalar TTR
) const
{

    return kappaTR(p, TTR)/this->CpTR(p, TTR);
}


// * * * * * * * * * * * * * * * Member Operators  * * * * * * * * * * * * * //

```

```
template<class Thermo2T>
inline void Foam::blottnerEucken2TTransport<Thermo2T>::operator+=
(
    const blottnerEucken2TTransport<Thermo2T>& st
)
{
    scalar Y1 = this->Y();

    Thermo2T::operator+=(st);

    if (mag(this->Y()) > SMALL)
    {
        Y1 /= this->Y();
        scalar Y2 = st.Y()/this->Y();

        AB_ = Y1*AB_ + Y2*st.AB_;
        BB_ = Y1*BB_ + Y2*st.BB_;
        CB_ = Y1*CB_ + Y2*st.CB_;
    }
}


template<class Thermo2T>
inline void Foam::blottnerEucken2TTransport<Thermo2T>::operator*=
(
    const scalar s
)
{
    Thermo2T::operator*=(s);
}


// * * * * * * * * * * * * * * * Friend Operators  * * * * * * * * * * * * * //

template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T> Foam::operator+
(
    const blottnerEucken2TTransport<Thermo2T>& st1,
    const blottnerEucken2TTransport<Thermo2T>& st2
)
{
    Thermo2T t
    (
        static_cast<const Thermo2T&>(st1) + static_cast<const Thermo2T&>(st2)
    );

    if (mag(t.Y()) < SMALL)
    {
        return blottnerEucken2TTransport<Thermo2T>
        (
            t,
            st1.AB_,
            st1.BB_,
            st1.CB_
        );
    }
    else
    {
        scalar Y1 = st1.Y()/t.Y();
        scalar Y2 = st2.Y()/t.Y();

        return blottnerEucken2TTransport<Thermo2T>
        (
            t,
            Y1*st1.AB_ + Y2*st2.AB_,
            Y1*st1.BB_ + Y2*st2.BB_,
            Y1*st1.CB_ + Y2*st2.CB_
        );
    }
}


template<class Thermo2T>
inline Foam::blottnerEucken2TTransport<Thermo2T> Foam::operator*
(
    const scalar s,
    const blottnerEucken2TTransport<Thermo2T>& st
)
{
    return blottnerEucken2TTransport<Thermo2T>
    (
        s*static_cast<const Thermo2T&>(st),
        st.AB_,
        st.BB_,
        st.CB_
    );
}
```

```
00222
00223
00224 // ************************************************************************* //
```

7.69 InInclude/EtoHthermo2T.H File Reference

Functions

- scalar CpTR (const scalar p, const scalar TTR) const
- scalar HsTR (const scalar p, const scalar TTR) const
- scalar HaTR (const scalar p, const scalar TTR) const

7.69.1 Function Documentation

7.69.1.1 CpTR()

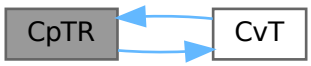
```
scalar CpTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 1 of file InInclude/EtoHthermo2T.H.

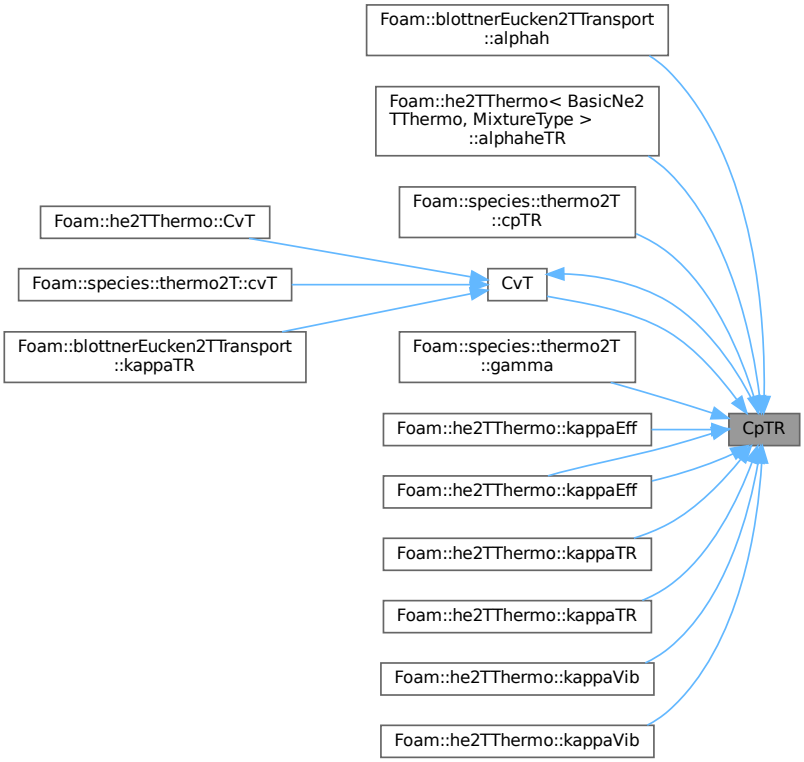
References CvT().

Referenced by Foam::blottnerEucken2TTransport< Thermo2T >::alphah(), Foam::he2TThermo< BasicNe2TThermo, MixtureType >
Foam::species::thermo2T< Thermo2T, Type >::cpTR(), CvT(), Foam::species::thermo2T< Thermo2T, Type >::gamma(),
Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaEff(), Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaEff
Foam::he2TThermo< BasicNe2TThermo, MixtureType >::kappaEff(), Foam::he2TThermo< Basic2TThermo, MixtureType >::kappa
Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaTR(), Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaVib
and Foam::he2TThermo< Basic2TThermo, MixtureType >::kappaVib().

Here is the call graph for this function:



Here is the caller graph for this function:



7.69.1.2 HaTR()

```
scalar HaTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 31 of file InInclude/EtoHthermo2T.H.

7.69.1.3 HsTR()

```
scalar HsTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 16 of file InInclude/EtoHthermo2T.H.

7.70 InInclude/EtoHthermo2T.H

Go to the documentation of this file.

```
00001 inline scalar CpTR
00002 (
00003     const scalar p,
00004     const scalar TTR
00005 ) const
00006 {
00007     #ifdef __clang__
00008     volatile const scalar cvT = CvT(p, TTR);
00009     volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
00010     return cvT + cpmcvTR;
00011     #else
00012     return CvT(p, TTR) + EquationOfState::CpMCv(p, TTR);
00013     #endif
00014 }
00015
00016 inline scalar HsTR
00017 (
00018     const scalar p,
00019     const scalar TTR
00020 ) const
00021 {
00022     #ifdef __clang__
00023     volatile const scalar esTR = EsTR(p, TTR);
00024     volatile const scalar rho = EquationOfState::rho(p, TTR);
00025     return esTR + p/rho;
00026     #else
00027     return EsTR(p, TTR) + p/EquationOfState::rho(p, TTR);
00028     #endif
00029 }
00030
00031 inline scalar HaTR
00032 (
00033     const scalar p,
00034     const scalar TTR
00035 ) const
00036 {
00037     #ifdef __clang__
00038     volatile const scalar eaTR = EaTR(p, TTR);
00039     volatile const scalar rho = EquationOfState::rho(p, TTR);
00040     return eaTR + p/rho;
00041     #else
00042     return EaTR(p, TTR) + p/EquationOfState::rho(p, TTR);
00043     #endif
00044 }
```

7.71 thermo/thermo2T/EtoHthermo2T.H File Reference

Functions

- scalar CpTR (const scalar p, const scalar TTR) const
- scalar HsTR (const scalar p, const scalar TTR) const
- scalar HaTR (const scalar p, const scalar TTR) const

7.71.1 Function Documentation

7.71.1.1 CpTR()

```
scalar CpTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 1 of file thermo/thermo2T/EtoHthermo2T.H.

References CvT().

Here is the call graph for this function:

7.71.1.2 HaTR()

```
scalar HaTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 31 of file thermo/thermo2T/EtoHthermo2T.H.

7.71.1.3 HsTR()

```
scalar HsTR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 16 of file thermo/thermo2T/EtoHthermo2T.H.

7.72 thermo/thermo2T/EtoHthermo2T.H

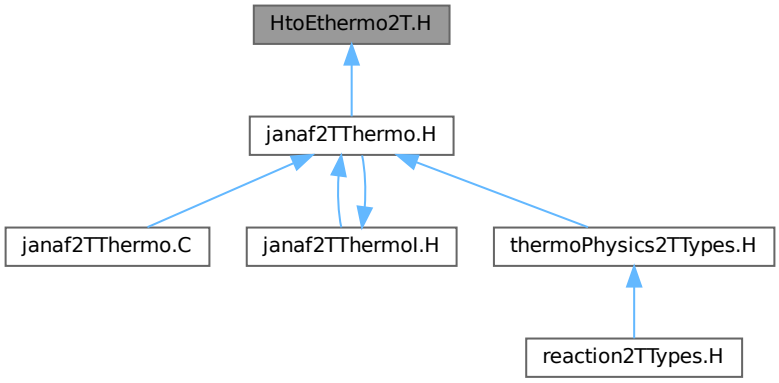
Go to the documentation of this file.

```
00001 inline scalar CpTR
00002 (
00003     const scalar p,
00004     const scalar TTR
00005 ) const
00006 {
00007     #ifdef __clang__
00008     volatile const scalar cvT = CvT(p, TTR);
00009     volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
00010     return cvT + cpmcvTR;
00011     #else
00012     return CvT(p, TTR) + EquationOfState::CpMCv(p, TTR);
00013     #endif
00014 }
00015
00016 inline scalar HsTR
00017 (
00018     const scalar p,
00019     const scalar TTR
00020 ) const
00021 {
00022     #ifdef __clang__
00023     volatile const scalar esTR = EsTR(p, TTR);
00024     volatile const scalar rho = EquationOfState::rho(p, TTR);
00025     return esTR + p/rho;
00026     #else
00027     return EsTR(p, TTR) + p/EquationOfState::rho(p, TTR);
00028     #endif
00029 }
00030
```

```
inline scalar HaTR
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar eaTR = EaTR(p, TTR);
    volatile const scalar rho = EquationOfState::rho(p, TTR);
    return eaTR + p/rho;
    #else
    return EaTR(p, TTR) + p/EquationOfState::rho(p, TTR);
    #endif
}
```

7.73 InInclude/HtoEthermo2T.H File Reference

This graph shows which files directly or indirectly include this file:



Functions

- scalar CvT (const scalar p, const scalar TTR) const
- scalar CvR (const scalar p, const scalar TTR) const
- scalar CvVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar EsT (const scalar p, const scalar TTR) const
- scalar EsR (const scalar p, const scalar TTR) const
- scalar EsVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar EaT (const scalar p, const scalar TTR) const

7.73.1 Function Documentation

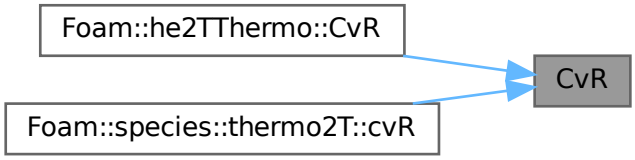
7.73.1.1 CvR()

```
scalar CvR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 16 of file InInclude/HtoEthermo2T.H.

Referenced by Foam::he2TThermo< Basic2TThermo, MixtureType >::CvR(), and Foam::species::thermo2T< Thermo2T, Type >::cv

Here is the caller graph for this function:



7.73.1.2 CvT()

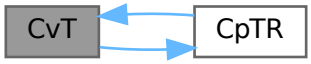
```
scalar CvT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 1 of file InInclude/HtoEthermo2T.H.

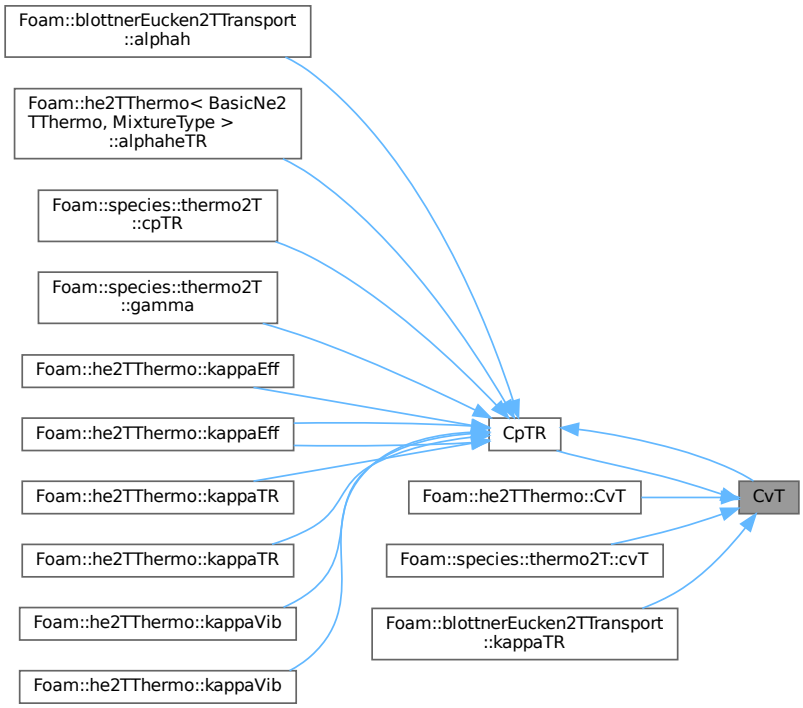
References CpTR().

Referenced by CpTR(), Foam::he2TThermo< Basic2TThermo, MixtureType >::CvT(), Foam::species::thermo2T< Thermo2T, Type >
and Foam::blottnerEucken2TTransport< Thermo2T >::kappaTR().

Here is the call graph for this function:



Here is the caller graph for this function:



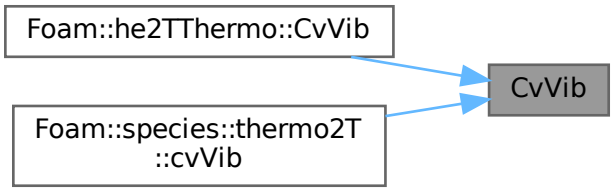
7.73.1.3 CvVib()

```
scalar CvVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

Definition at line 30 of file InInclude/HtoEthermo2T.H.

Referenced by Foam::he2TThermo< Basic2TThermo, MixtureType >::CvVib(), and Foam::species::thermo2T< Thermo2T, Type >::c

Here is the caller graph for this function:



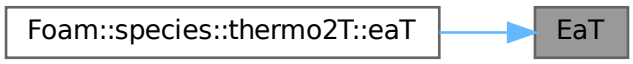
7.73.1.4 EaT()

```
scalar EaT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 94 of file InInclude/HtoEthermo2T.H.

Referenced by Foam::species::thermo2T< Thermo2T, Type >::eaT().

Here is the caller graph for this function:



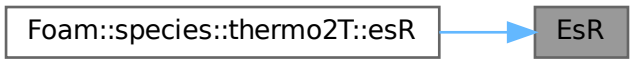
7.73.1.5 EsR()

```
scalar EsR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 63 of file InInclude/HtoEthermo2T.H.

Referenced by Foam::species::thermo2T< Thermo2T, Type >::esR().

Here is the caller graph for this function:



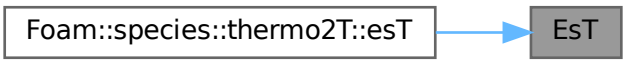
7.73.1.6 EsT()

```
scalar EsT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 48 of file InInclude/HtoEthermo2T.H.

Referenced by Foam::species::thermo2T< Thermo2T, Type >::esT().

Here is the caller graph for this function:



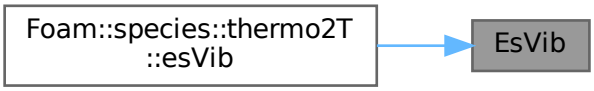
7.73.1.7 EsVib()

```
scalar EsVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

Definition at line 77 of file InInclude/HtoEthermo2T.H.

Referenced by Foam::species::thermo2T< Thermo2T, Type >::esVib().

Here is the caller graph for this function:



7.74 InInclude/HtoEthermo2T.H

Go to the documentation of this file.

```
inline scalar CvT
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar cpTR = CpTR(p, TTR);
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpTR - cpmcvTR;
    #else
    return CpTR(p, TTR) - EquationOfState::CpMCv(p, TTR);
    #endif
}

inline scalar CvR
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpmcvTR;
    #else
    return EquationOfState::CpMCv(p, TTR);
    #endif
}

inline scalar CvVib
(
    const scalar p,
    const scalar TTR,
    const scalar TVib,
    const scalar ThetaVib
) const
{
    #ifdef __clang__
    volatile const scalar dimensionlessVibTemp = 0.5*ThetaVib/TVib
    volatile const scalar div = sinh(0.5*ThetaVib/TVib);
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpmcvTR*dimensionlessVibTemp/div;
    #else
    return EquationOfState::CpMCv(p, TTR)*(0.5*ThetaVib/TVib)/sinh(0.5*ThetaVib/TVib);
    #endif
}

inline scalar EsT
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar hsT = Hs(p, TTR);
    volatile const scalar rho = EquationOfState::rho(p, TTR);
    return hsT - p/rho;
    #else
    return Hs(p, TTR) - p/EquationOfState::rho(p, TTR);
    #endif
}

inline scalar EsR
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpmcvTR*TTR;
    #else
    return EquationOfState::CpMCv(p, TTR)*TTR;
    #endif
}

inline scalar EsVib
(
    const scalar p,
    const scalar TTR,
    const scalar TVib,
    const scalar ThetaVib
```

```
) const
{
    #ifdef __clang__
    volatile const scalar div = exp(ThetaVib/TVib) - 1;
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpmcvTR*ThetaVib/div;
    #else
    return EquationOfState::CpMCv(p, TTR)*ThetaVib/(exp(ThetaVib/TVib) - 1);
    #endif
}

inline scalar EaT
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar haT = Ha(p, TTR);
    volatile const scalar rho = EquationOfState::rho(p, TTR);
    return haT - p/rho;
    #else
    return Ha(p, TTR) - p/EquationOfState::rho(p, TTR);
    #endif
}
```

7.75 thermo/thermo2T/HtoEthermo2T.H File Reference

Functions

- scalar CvT (const scalar p, const scalar TTR) const
- scalar CvR (const scalar p, const scalar TTR) const
- scalar CvVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar EsT (const scalar p, const scalar TTR) const
- scalar EsR (const scalar p, const scalar TTR) const
- scalar EsVib (const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
- scalar EaT (const scalar p, const scalar TTR) const

7.75.1 Function Documentation

7.75.1.1 CvR()

```
scalar CvR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 16 of file thermo/thermo2T/HtoEthermo2T.H.

7.75.1.2 CvT()

```
scalar CvT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 1 of file thermo/thermo2T/HtoEthermo2T.H.

References CpTR().

Here is the call graph for this function:

7.75.1.3 CvVib()

```
scalar CvVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

Definition at line 30 of file thermo/thermo2T/HtoEthermo2T.H.

7.75.1.4 EaT()

```
scalar EaT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 94 of file thermo/thermo2T/HtoEthermo2T.H.

7.75.1.5 EsR()

```
scalar EsR (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 63 of file thermo/thermo2T/HtoEthermo2T.H.

7.75.1.6 EsT()

```
scalar EsT (
            const scalar p,
            const scalar TTR ) const  [inline]
```

Definition at line 48 of file thermo/thermo2T/HtoEthermo2T.H.

7.75.1.7 EsVib()

```
scalar EsVib (
            const scalar p,
            const scalar TTR,
            const scalar TVib,
            const scalar ThetaVib ) const  [inline]
```

Definition at line 77 of file thermo/thermo2T/HtoEthermo2T.H.

7.76 thermo/thermo2T/HtoEthermo2T.H

Go to the documentation of this file.

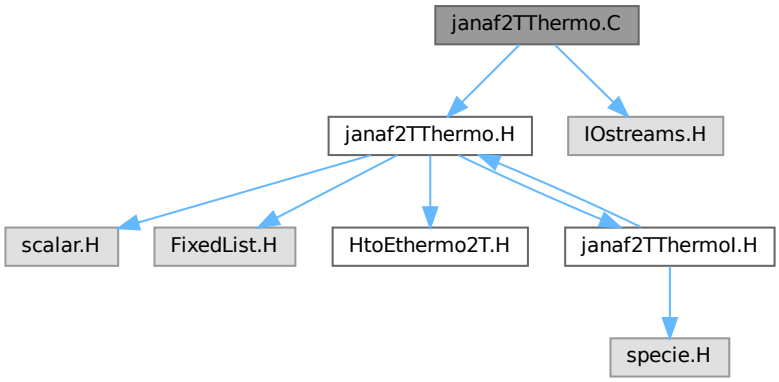
```
00001 inline scalar CvT
00002 (
00003     const scalar p,
00004     const scalar TTR
00005 ) const
00006 {
00007     #ifdef __clang__
00008     volatile const scalar cpTR = CpTR(p, TTR);
00009     volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
00010     return cpTR - cpmcvTR;
00011     #else
00012     return CpTR(p, TTR) - EquationOfState::CpMCv(p, TTR);
00013     #endif
00014 }
00015
00016 inline scalar CvR
00017 (
00018     const scalar p,
00019     const scalar TTR
00020 ) const
00021 {
00022     #ifdef __clang__
00023     volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
00024     return cpmcvTR;
00025     #else
00026     return EquationOfState::CpMCv(p, TTR);
00027     #endif
00028 }
00029
00030 inline scalar CvVib
00031 (
00032     const scalar p,
00033     const scalar TTR,
00034     const scalar TVib,
00035     const scalar ThetaVib
00036 ) const
00037 {
00038     #ifdef __clang__
00039     volatile const scalar dimensionlessVibTemp = 0.5*ThetaVib/TVib
00040     volatile const scalar div = sinh(0.5*ThetaVib/TVib);
00041     volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
00042     return cpmcvTR*dimensionlessVibTemp/div;
00043     #else
00044     return EquationOfState::CpMCv(p, TTR)*(0.5*ThetaVib/TVib)/sinh(0.5*ThetaVib/TVib);
00045     #endif
00046 }
00047
00048 inline scalar EsT
00049 (
00050     const scalar p,
00051     const scalar TTR
00052 ) const
00053 {
00054     #ifdef __clang__
00055     volatile const scalar hsT = Hs(p, TTR);
00056     volatile const scalar rho = EquationOfState::rho(p, TTR);
00057     return hsT - p/rho;
00058     #else
00059     return Hs(p, TTR) - p/EquationOfState::rho(p, TTR);
00060     #endif
00061 }
00062
00063 inline scalar EsR
```

```
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpmcvTR*TTR;
    #else
    return EquationOfState::CpMCv(p, TTR)*TTR;
    #endif
}

inline scalar EsVib
(
    const scalar p,
    const scalar TTR,
    const scalar TVib,
    const scalar ThetaVib
) const
{
    #ifdef __clang__
    volatile const scalar div = exp(ThetaVib/TVib) - 1;
    volatile const scalar cpmcvTR = EquationOfState::CpMCv(p, TTR);
    return cpmcvTR*ThetaVib/div;
    #else
    return EquationOfState::CpMCv(p, TTR)*ThetaVib/(exp(ThetaVib/TVib) - 1);
    #endif
}

inline scalar EaT
(
    const scalar p,
    const scalar TTR
) const
{
    #ifdef __clang__
    volatile const scalar haT = Ha(p, TTR);
    volatile const scalar rho = EquationOfState::rho(p, TTR);
    return haT - p/rho;
    #else
    return Ha(p, TTR) - p/EquationOfState::rho(p, TTR);
    #endif
}
```

7.77 InInclude/janaf2TThermo.C File Reference

```
#include "janaf2TThermo.H"
#include "IOstreams.H"
```

Include dependency graph for InInclude/janaf2TThermo.C:



7.78 lnInclude/janaf2TThermo.C

Go to the documentation of this file.

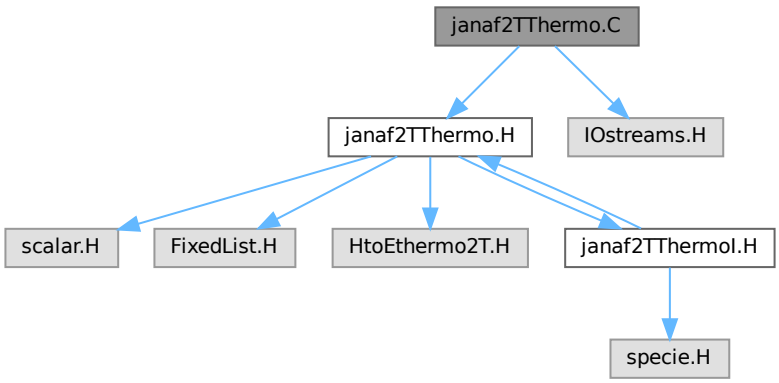
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "janaf2TThermo.H"
#include "IOstreams.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class EquationOfState>
void Foam::janaf2TThermo<EquationOfState>::checkInputData() const
{
    if (Tlow_ >= Thigh_)
    {
        FatalErrorInFunction
            « "Tlow(" « Tlow_ « ") >= Thigh(" « Thigh_ « ')'
            « exit(FatalError);
    }

    if (Tcommon_ <= Tlow_)
    {
        FatalErrorInFunction
            « "Tcommon(" « Tcommon_ « ") <= Tlow(" « Tlow_ « ')'
            « exit(FatalError);
    }

    if (Tcommon_ > Thigh_)
    {
        FatalErrorInFunction
            « "Tcommon(" « Tcommon_ « ") > Thigh(" « Thigh_ « ')'
            « exit(FatalError);
    }
}


// * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

template<class EquationOfState>
Foam::janaf2TThermo<EquationOfState>::janaf2TThermo(const dictionary& dict)
:
    EquationOfState(dict),
    Tlow_(dict.subDict("thermodynamics").get<scalar>("Tlow")),
    Thigh_(dict.subDict("thermodynamics").get<scalar>("Thigh")),
    Tcommon_(dict.subDict("thermodynamics").get<scalar>("Tcommon")),
    ThetaVib_(dict.subDict("thermodynamics").get<scalar>("ThetaVibrational")),
    highCpCoeffs_(dict.subDict("thermodynamics").lookup("highCpCoeffs")),
    lowCpCoeffs_(dict.subDict("thermodynamics").lookup("lowCpCoeffs"))
{
    // Convert coefficients to mass-basis
    for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
    {
        highCpCoeffs_[coefLabel] *= this->R();
        lowCpCoeffs_[coefLabel] *= this->R();
    }

    checkInputData();
}


```

```
// * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class EquationOfState>
void Foam::janaf2TThermo<EquationOfState>::write(Ostream& os) const
{
    EquationOfState::write(os);

    // Convert coefficients back to dimensionless form
    coeffArray highCpCoeffs;
    coeffArray lowCpCoeffs;
    for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
    {
        highCpCoeffs[coefLabel] = highCpCoeffs_[coefLabel]/this->R();
        lowCpCoeffs[coefLabel] = lowCpCoeffs_[coefLabel]/this->R();
    }

    // Entries in dictionary format
    {
        os.beginBlock("thermodynamics");
        os.writeEntry("Tlow", Tlow_);
        os.writeEntry("Thigh", Thigh_);
        os.writeEntry("Tcommon", Tcommon_);
        os.writeEntry("ThetaVib", ThetaVib_);
        os.writeEntry("highCpCoeffs", highCpCoeffs);
        os.writeEntry("lowCpCoeffs", lowCpCoeffs);
        os.endBlock();
    }
}


// * * * * * * * * * * * * * * Ostream Operator  * * * * * * * * * * * * * //

template<class EquationOfState>
Foam::Ostream& Foam::operator«
(
    Ostream& os,
    const janaf2TThermo<EquationOfState>& jt
)
{
    jt.write(os);
    return os;
}


// ************************************************************************* //
```

7.79 thermo/janaf2T/janaf2TThermo.C File Reference

```
#include "janaf2TThermo.H"
#include "IOstreams.H"
```

Include dependency graph for thermo/janaf2T/janaf2TThermo.C:



7.80 thermo/janaf2T/janaf2TThermo.C

Go to the documentation of this file.

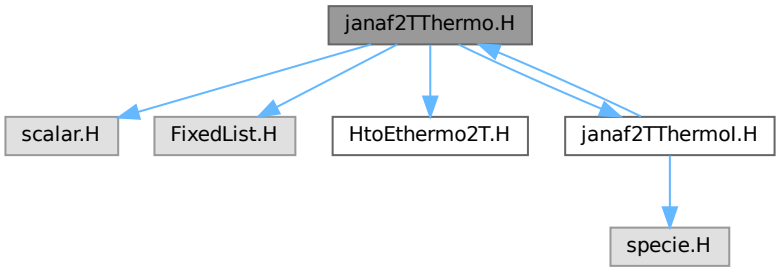
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "janaf2TThermo.H"
#include "IOstreams.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class EquationOfState>
void Foam::janaf2TThermo<EquationOfState>::checkInputData() const
{
    if (Tlow_ >= Thigh_)
    {
        FatalErrorInFunction
            « "Tlow(" « Tlow_ « ") >= Thigh(" « Thigh_ « ')'
            « exit(FatalError);
    }

    if (Tcommon_ <= Tlow_)
    {
        FatalErrorInFunction
            « "Tcommon(" « Tcommon_ « ") <= Tlow(" « Tlow_ « ')'
            « exit(FatalError);
    }

    if (Tcommon_ > Thigh_)
    {
        FatalErrorInFunction
            « "Tcommon(" « Tcommon_ « ") > Thigh(" « Thigh_ « ')'
            « exit(FatalError);
    }
}


// * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * //

template<class EquationOfState>
Foam::janaf2TThermo<EquationOfState>::janaf2TThermo(const dictionary& dict)
:
    EquationOfState(dict),
    Tlow_(dict.subDict("thermodynamics").get<scalar>("Tlow")),
    Thigh_(dict.subDict("thermodynamics").get<scalar>("Thigh")),
    Tcommon_(dict.subDict("thermodynamics").get<scalar>("Tcommon")),
    ThetaVib_(dict.subDict("thermodynamics").get<scalar>("ThetaVibrational")),
    highCpCoeffs_(dict.subDict("thermodynamics").lookup("highCpCoeffs")),
    lowCpCoeffs_(dict.subDict("thermodynamics").lookup("lowCpCoeffs"))
{
    // Convert coefficients to mass-basis
    for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
    {
        highCpCoeffs_[coefLabel] *= this->R();
        lowCpCoeffs_[coefLabel] *= this->R();
    }

    checkInputData();
}


```

```
// * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class EquationOfState>
void Foam::janaf2TThermo<EquationOfState>::write(Ostream& os) const
{
    EquationOfState::write(os);

    // Convert coefficients back to dimensionless form
    coeffArray highCpCoeffs;
    coeffArray lowCpCoeffs;
    for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
    {
        highCpCoeffs[coefLabel] = highCpCoeffs_[coefLabel]/this->R();
        lowCpCoeffs[coefLabel] = lowCpCoeffs_[coefLabel]/this->R();
    }

    // Entries in dictionary format
    {
        os.beginBlock("thermodynamics");
        os.writeEntry("Tlow", Tlow_);
        os.writeEntry("Thigh", Thigh_);
        os.writeEntry("Tcommon", Tcommon_);
        os.writeEntry("ThetaVib", ThetaVib_);
        os.writeEntry("highCpCoeffs", highCpCoeffs);
        os.writeEntry("lowCpCoeffs", lowCpCoeffs);
        os.endBlock();
    }
}


// * * * * * * * * * * * * * * Ostream Operator  * * * * * * * * * * * * * //

template<class EquationOfState>
Foam::Ostream& Foam::operator«
(
    Ostream& os,
    const janaf2TThermo<EquationOfState>& jt
)
{
    jt.write(os);
    return os;
}


// ************************************************************************* //
```

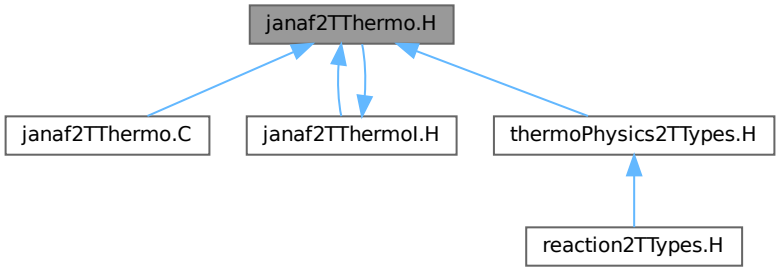
7.81 InInclude/janaf2TThermo.H File Reference

```
#include "scalar.H"
#include "FixedList.H"
#include "HtoEthermo2T.H"
#include "janaf2TThermoI.H"
```

Include dependency graph for InInclude/janaf2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::janaf2TThermo< EquationOfState >

Namespaces

- namespace Foam

Functions

- template<class EquationOfState >
  janaf2TThermo< EquationOfState > Foam::operator+ (const janaf2TThermo< EquationOfState > &, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState >
  janaf2TThermo< EquationOfState > Foam::operator* (const scalar, const janaf2TThermo< EquationOf↩State > &)
- template<class EquationOfState >
  janaf2TThermo< EquationOfState > Foam::operator== (const janaf2TThermo< EquationOfState > &, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState >
  Ostream & Foam::operator<< (Ostream &, const janaf2TThermo< EquationOfState > &)

7.82 InInclude/janaf2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
```

```
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::janaf2TThermo

Group
    grpSpecieThermo

Description
    JANAF tables based thermodynamics package templated
    into the equation of state.

SourceFiles
    janafTThermoI.H
    janaf2TThermo.C

\*---------------------------------------------------------------------------*/

#ifndef janaf2TThermo_H
#define janaf2TThermo_H

#include "scalar.H"
#include "FixedList.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

// Forward Declarations

template<class EquationOfState> class janaf2TThermo;

template<class EquationOfState>
inline janaf2TThermo<EquationOfState> operator+
(
    const janaf2TThermo<EquationOfState>&,
    const janaf2TThermo<EquationOfState>&
);

template<class EquationOfState>
inline janaf2TThermo<EquationOfState> operator*
(
    const scalar,
    const janaf2TThermo<EquationOfState>&
);

template<class EquationOfState>
inline janaf2TThermo<EquationOfState> operator==
(
    const janaf2TThermo<EquationOfState>&,
    const janaf2TThermo<EquationOfState>&
);

template<class EquationOfState>
Ostream& operator«
(
    Ostream&,
    const janaf2TThermo<EquationOfState>&
);


/*---------------------------------------------------------------------------*\
                           Class janaf2TThermo Declaration
\*---------------------------------------------------------------------------*/

template<class EquationOfState>
class janaf2TThermo
:
    public EquationOfState
{
public:

    // Public Data

        static constexpr int nCoeffs_ = 7;
        typedef FixedList<scalar, nCoeffs_> coeffArray;

```

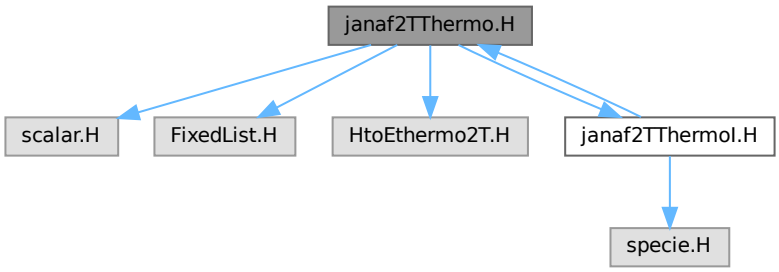
```

private:

    // Private Data

        // Temperature limits of applicability of functions
        scalar Tlow_, Thigh_, Tcommon_, ThetaVib_;

        coeffArray highCpCoeffs_;
        coeffArray lowCpCoeffs_;


    // Private Member Functions

        //- Check that input data is valid
        void checkInputData() const;

        //- Return the coefficients corresponding to the given temperature
        inline const coeffArray& coeffs(const scalar TTR) const;


public:

    // Constructors

        //- Construct from components
        inline janaf2TThermo
        (
            const EquationOfState& st,
            const scalar Tlow,
            const scalar Thigh,
            const scalar Tcommon,
            const scalar ThetaVib,
            const coeffArray& highCpCoeffs,
            const coeffArray& lowCpCoeffs,
            const bool convertCoeffs = false
        );

        //- Construct from dictionary
        janaf2TThermo(const dictionary& dict);

        //- Construct as a named copy
        inline janaf2TThermo(const word&, const janaf2TThermo&);


    // Member Functions

        //- Return the instantiated type name
        static word typeName()
        {
            return "janaf2T<" + EquationOfState::typeName() + '>';
        }

        //- Limit the temperature to be in the range Tlow_ to Thigh_
        inline scalar limit(const scalar TTR) const;


        // Access

            //- Return const access to the low temperature limit
            inline scalar Tlow() const;

            //- Return const access to the high temperature limit
            inline scalar Thigh() const;

            //- Return const access to the common temperature
            inline scalar Tcommon() const;

            //- Return const access to the vibrational temperature coefficient
            inline scalar ThetaVib() const;

            //- Return const access to the high temperature poly coefficients
            inline const coeffArray& highCpCoeffs() const;

            //- Return const access to the low temperature poly coefficients
            inline const coeffArray& lowCpCoeffs() const;


        // Fundamental properties

            //- Heat capacity at constant pressure [J/(kg K)]
            inline scalar CpTR(const scalar p, const scalar TTR) const;

            //- Absolute Enthalpy [J/kg]
            inline scalar Ha(const scalar p, const scalar TTR) const;

            //- Sensible enthalpy [J/kg]
```

```
            inline scalar Hs(const scalar p, const scalar TTR) const;

            //- Chemical enthalpy [J/kg]
            inline scalar Hc() const;

            //- Entropy [J/(kg K)]
            inline scalar S(const scalar p, const scalar TTR) const;

            //- Gibbs free energy of the mixture in the standard state [J/kg]
            inline scalar Gstd(const scalar TTR) const;

            #include "HtoEthermo2T.H"


        // Derivative term used for Jacobian


            //- Temperature derivative of heat capacity at constant pressure
            inline scalar dCpdT(const scalar p, const scalar TTR) const;


        // I-O

            //- Write to Ostream
            void write(Ostream& os) const;


    // Member operators

        inline void operator+=(const janaf2TThermo&);


    // Friend operators

        friend janaf2TThermo operator+ <EquationOfState>
        (
            const janaf2TThermo&,
            const janaf2TThermo&
        );

        friend janaf2TThermo operator* <EquationOfState>
        (
            const scalar,
            const janaf2TThermo&
        );

        friend janaf2TThermo operator== <EquationOfState>
        (
            const janaf2TThermo&,
            const janaf2TThermo&
        );


    // Ostream Operator

        friend Ostream& operator« <EquationOfState>
        (
            Ostream&,
            const janaf2TThermo&
        );
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "janaf2TThermoI.H"

#ifdef NoRepository
    #include "janaf2TThermo.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

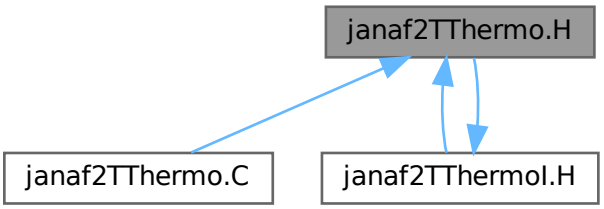
7.83 thermo/janaf2T/janaf2TThermo.H File Reference

```
#include "scalar.H"
#include "FixedList.H"
#include "HtoEthermo2T.H"
#include "janaf2TThermoI.H"
```

Include dependency graph for thermo/janaf2T/janaf2TThermo.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::janaf2TThermo< EquationOfState >

Namespaces

- namespace Foam

Functions

- template<class EquationOfState >
  janaf2TThermo< EquationOfState > Foam::operator+ (const janaf2TThermo< EquationOfState > &, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState >
  janaf2TThermo< EquationOfState > Foam::operator* (const scalar, const janaf2TThermo< EquationOf↩State > &)
- template<class EquationOfState >
  janaf2TThermo< EquationOfState > Foam::operator== (const janaf2TThermo< EquationOfState > &, const janaf2TThermo< EquationOfState > &)
- template<class EquationOfState >
  Ostream & Foam::operator<< (Ostream &, const janaf2TThermo< EquationOfState > &)

7.84 thermo/janaf2T/janaf2TThermo.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::janaf2TThermo

Group
    grpSpecieThermo

Description
    JANAF tables based thermodynamics package templated
    into the equation of state.

SourceFiles
    janafTThermoI.H
    janaf2TThermo.C

\*-------------------------------------------------------------------------*/

#ifndef janaf2TThermo_H
#define janaf2TThermo_H

#include "scalar.H"
#include "FixedList.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

// Forward Declarations

template<class EquationOfState> class janaf2TThermo;

template<class EquationOfState>
```

```
inline janaf2TThermo<EquationOfState> operator+
(
    const janaf2TThermo<EquationOfState>&,
    const janaf2TThermo<EquationOfState>&
);

template<class EquationOfState>
inline janaf2TThermo<EquationOfState> operator*
(
    const scalar,
    const janaf2TThermo<EquationOfState>&
);

template<class EquationOfState>
inline janaf2TThermo<EquationOfState> operator==
(
    const janaf2TThermo<EquationOfState>&,
    const janaf2TThermo<EquationOfState>&
);

template<class EquationOfState>
Ostream& operator«
(
    Ostream&,
    const janaf2TThermo<EquationOfState>&
);


/*---------------------------------------------------------------------------*\
                           Class janaf2TThermo Declaration
\*---------------------------------------------------------------------------*/

template<class EquationOfState>
class janaf2TThermo
:
    public EquationOfState
{
public:

    // Public Data

        static constexpr int nCoeffs_ = 7;
        typedef FixedList<scalar, nCoeffs_> coeffArray;


private:

    // Private Data

        // Temperature limits of applicability of functions
        scalar Tlow_, Thigh_, Tcommon_, ThetaVib_;

        coeffArray highCpCoeffs_;
        coeffArray lowCpCoeffs_;


    // Private Member Functions

        //- Check that input data is valid
        void checkInputData() const;

        //- Return the coefficients corresponding to the given temperature
        inline const coeffArray& coeffs(const scalar TTR) const;


public:

    // Constructors

        //- Construct from components
        inline janaf2TThermo
        (
            const EquationOfState& st,
            const scalar Tlow,
            const scalar Thigh,
            const scalar Tcommon,
            const scalar ThetaVib,
            const coeffArray& highCpCoeffs,
            const coeffArray& lowCpCoeffs,
            const bool convertCoeffs = false
        );

        //- Construct from dictionary
        janaf2TThermo(const dictionary& dict);

        //- Construct as a named copy
        inline janaf2TThermo(const word&, const janaf2TThermo&);
```

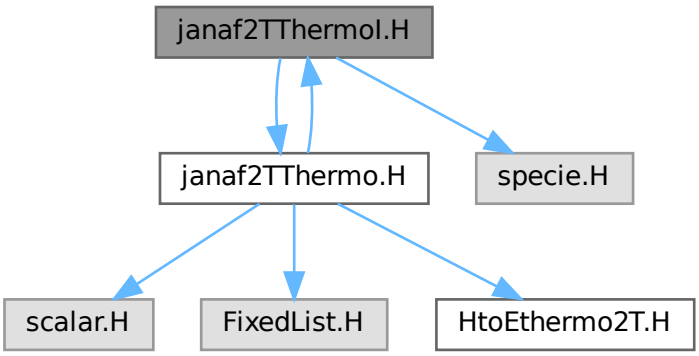
```
00146
00147
00148     // Member Functions
00149
00150         //- Return the instantiated type name
00151         static word typeName()
00152         {
00153             return "janaf2T<" + EquationOfState::typeName() + '>';
00154         }
00155
00156         //- Limit the temperature to be in the range Tlow_ to Thigh_
00157         inline scalar limit(const scalar TTR) const;
00158
00159
00160         // Access
00161
00162             //- Return const access to the low temperature limit
00163             inline scalar Tlow() const;
00164
00165             //- Return const access to the high temperature limit
00166             inline scalar Thigh() const;
00167
00168             //- Return const access to the common temperature
00169             inline scalar Tcommon() const;
00170
00171             //- Return const access to the vibrational temperature coefficient
00172             inline scalar ThetaVib() const;
00173
00174             //- Return const access to the high temperature poly coefficients
00175             inline const coeffArray& highCpCoeffs() const;
00176
00177             //- Return const access to the low temperature poly coefficients
00178             inline const coeffArray& lowCpCoeffs() const;
00179
00180
00181         // Fundamental properties
00182
00183             //- Heat capacity at constant pressure [J/(kg K)]
00184             inline scalar CpTR(const scalar p, const scalar TTR) const;
00185
00186             //- Absolute Enthalpy [J/kg]
00187             inline scalar Ha(const scalar p, const scalar TTR) const;
00188
00189             //- Sensible enthalpy [J/kg]
00190             inline scalar Hs(const scalar p, const scalar TTR) const;
00191
00192             //- Chemical enthalpy [J/kg]
00193             inline scalar Hc() const;
00194
00195             //- Entropy [J/(kg K)]
00196             inline scalar S(const scalar p, const scalar TTR) const;
00197
00198             //- Gibbs free energy of the mixture in the standard state [J/kg]
00199             inline scalar Gstd(const scalar TTR) const;
00200
00201             #include "HtoEthermo2T.H"
00202
00203
00204         // Derivative term used for Jacobian
00205
00206
00207             //- Temperature derivative of heat capacity at constant pressure
00208             inline scalar dCpdT(const scalar p, const scalar TTR) const;
00209
00210
00211         // I-O
00212
00213             //- Write to Ostream
00214             void write(Ostream& os) const;
00215
00216
00217     // Member operators
00218
00219         inline void operator+=(const janaf2TThermo&);
00220
00221
00222     // Friend operators
00223
00224         friend janaf2TThermo operator+ <EquationOfState>
00225         (
00226             const janaf2TThermo&,
00227             const janaf2TThermo&
00228         );
00229
00230         friend janaf2TThermo operator* <EquationOfState>
00231         (
00232             const scalar,
```

```
            const janaf2TThermo&
        );

        friend janaf2TThermo operator== <EquationOfState>
        (
            const janaf2TThermo&,
            const janaf2TThermo&
        );


    // Ostream Operator

        friend Ostream& operator« <EquationOfState>
        (
            Ostream&,
            const janaf2TThermo&
        );
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "janaf2TThermoI.H"

#ifdef NoRepository
    #include "janaf2TThermo.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

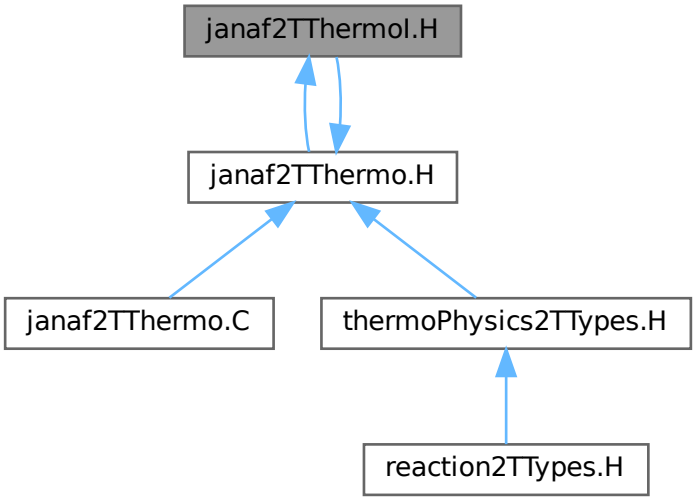
7.85 InInclude/janaf2TThermoI.H File Reference

```
#include "janaf2TThermo.H"
#include "specie.H"
```

Include dependency graph for InInclude/janaf2TThermoI.H:



This graph shows which files directly or indirectly include this file:



7.86 InInclude/janaf2TThermoI.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "janaf2TThermo.H"
#include "specie.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState>::janaf2TThermo
(
    const EquationOfState& st,
    const scalar Tlow,
    const scalar Thigh,
    const scalar Tcommon,
```

```
    const scalar ThetaVib,
    const typename janaf2TThermo<EquationOfState>::coeffArray& highCpCoeffs,
    const typename janaf2TThermo<EquationOfState>::coeffArray& lowCpCoeffs,
    const bool convertCoeffs
)
:
    EquationOfState(st),
    Tlow_(Tlow),
    Thigh_(Thigh),
    Tcommon_(Tcommon),
    ThetaVib_(ThetaVib)
{
    if (convertCoeffs)
    {
        for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
        {
            highCpCoeffs_[coefLabel] = highCpCoeffs[coefLabel]*this->R();
            lowCpCoeffs_[coefLabel] = lowCpCoeffs[coefLabel]*this->R();
        }
    }
    else
    {
        for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
        {
            highCpCoeffs_[coefLabel] = highCpCoeffs[coefLabel];
            lowCpCoeffs_[coefLabel] = lowCpCoeffs[coefLabel];
        }
    }
}


template<class EquationOfState>
inline const typename Foam::janaf2TThermo<EquationOfState>::coeffArray&
Foam::janaf2TThermo<EquationOfState>::coeffs
(
    const scalar TTR
) const
{
    if (TTR < Tcommon_)
    {
        return lowCpCoeffs_;
    }
    else
    {
        return highCpCoeffs_;
    }
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState>::janaf2TThermo
(
    const word& name,
    const janaf2TThermo& jt
)
:
    EquationOfState(name, jt),
    Tlow_(jt.Tlow_),
    Thigh_(jt.Thigh_),
    Tcommon_(jt.Tcommon_),
    ThetaVib_(jt.ThetaVib_)
{
    for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
    {
        highCpCoeffs_[coefLabel] = jt.highCpCoeffs_[coefLabel];
        lowCpCoeffs_[coefLabel] = jt.lowCpCoeffs_[coefLabel];
    }
}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::limit
(
    const scalar TTR
) const
{
    if (TTR < Tlow_ || TTR > Thigh_)
    {
        WarningInFunction
            « "attempt to use janaf2TThermo<EquationOfState>"
               " out of temperature range "
            « Tlow_ « " -> " « Thigh_ « ";  TTR = " « TTR
            « endl;
```

```

        return clamp(TTR, Tlow_, Thigh_);
    }

    return TTR;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Tlow() const
{
    return Tlow_;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Thigh() const
{
    return Thigh_;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Tcommon() const
{
    return Tcommon_;
}

template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::ThetaVib() const
{
    return ThetaVib_;
}

template<class EquationOfState>
inline const typename Foam::janaf2TThermo<EquationOfState>::coeffArray&
Foam::janaf2TThermo<EquationOfState>::highCpCoeffs() const
{
    return highCpCoeffs_;
}


template<class EquationOfState>
inline const typename Foam::janaf2TThermo<EquationOfState>::coeffArray&
Foam::janaf2TThermo<EquationOfState>::lowCpCoeffs() const
{
    return lowCpCoeffs_;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::CpTR
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
        ((((a[4]*TTR + a[3])*TTR + a[2])*TTR + a[1])*TTR + a[0])
      + EquationOfState::Cp(p, TTR);
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Ha
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
    (
        ((((a[4]/5.0*TTR + a[3]/4.0)*TTR + a[2]/3.0)*TTR + a[1]/2.0)*TTR + a[0])*TTR
      + a[5]
    ) + EquationOfState::H(p, TTR);
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Hs
(
    const scalar p,
    const scalar TTR
) const
{
```

```
    return Ha(p, TTR) - Hc();
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Hc() const
{
    const coeffArray& a = lowCpCoeffs_;
    return
    (
        (
            (((a[4]/5.0*Tstd + a[3]/4.0)*Tstd + a[2]/3.0)*Tstd + a[1]/2.0)*Tstd
          + a[0]
        )*Tstd + a[5]
    );
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::S
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
    (
        (((a[4]/4.0*TTR + a[3]/3.0)*TTR + a[2]/2.0)*TTR + a[1])*TTR + a[0]*log(TTR)
      + a[6]
    ) + EquationOfState::S(p, TTR);
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Gstd
(
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
    (
        (
            a[0]*(1 - log(TTR))
          - (((a[4]/20.0*TTR + a[3]/12.0)*TTR + a[2]/6.0)*TTR + a[1]/2.0)*TTR
          - a[6]
        )*TTR
      + a[5]
    );
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::dCpdT
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
        (((4*a[4]*TTR + 3*a[3])*TTR + 2*a[2])*TTR + a[1]);
}

// * * * * * * * * * * * * * * * Member Operators  * * * * * * * * * * * * * //

template<class EquationOfState>
inline void Foam::janaf2TThermo<EquationOfState>::operator+=
(
    const janaf2TThermo<EquationOfState>& jt
)
{
    scalar Y1 = this->Y();

    EquationOfState::operator+=(jt);

    if (mag(this->Y()) > SMALL)
    {
        Y1 /= this->Y();
        const scalar Y2 = jt.Y()/this->Y();

        Tlow_ = max(Tlow_, jt.Tlow_);
        Thigh_ = min(Thigh_, jt.Thigh_);

        if
        (
```

```
            janaf2TThermo<EquationOfState>::debug
         && !equal(Tcommon_, jt.Tcommon_)
        )
        {
            FatalErrorInFunction
                « "Tcommon " « Tcommon_ « " for "
                « (this->name().size() ? this->name() : "others")
                « " != " « jt.Tcommon_ « " for "
                « (jt.name().size() ? jt.name() : "others")
                « exit(FatalError);
        }

        for
        (
            label coefLabel=0;
            coefLabel<janaf2TThermo<EquationOfState>::nCoeffs_;
            coefLabel++
        )
        {
            highCpCoeffs_[coefLabel] =
                Y1*highCpCoeffs_[coefLabel]
              + Y2*jt.highCpCoeffs_[coefLabel];

            lowCpCoeffs_[coefLabel] =
                Y1*lowCpCoeffs_[coefLabel]
              + Y2*jt.lowCpCoeffs_[coefLabel];
        }
    }
}


// * * * * * * * * * * * * * * * Friend Operators  * * * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState> Foam::operator+
(
    const janaf2TThermo<EquationOfState>& jt1,
    const janaf2TThermo<EquationOfState>& jt2
)
{
    EquationOfState eofs = jt1;
    eofs += jt2;

    if (mag(eofs.Y()) < SMALL)
    {
        return janaf2TThermo<EquationOfState>
        (
            eofs,
            jt1.Tlow_,
            jt1.Thigh_,
            jt1.Tcommon_,
            jt1.highCpCoeffs_,
            jt1.lowCpCoeffs_
        );
    }
    else
    {
        const scalar Y1 = jt1.Y()/eofs.Y();
        const scalar Y2 = jt2.Y()/eofs.Y();

        typename janaf2TThermo<EquationOfState>::coeffArray highCpCoeffs;
        typename janaf2TThermo<EquationOfState>::coeffArray lowCpCoeffs;

        for
        (
            label coefLabel=0;
            coefLabel<janaf2TThermo<EquationOfState>::nCoeffs_;
            coefLabel++
        )
        {
            highCpCoeffs[coefLabel] =
                Y1*jt1.highCpCoeffs_[coefLabel]
              + Y2*jt2.highCpCoeffs_[coefLabel];

            lowCpCoeffs[coefLabel] =
                Y1*jt1.lowCpCoeffs_[coefLabel]
              + Y2*jt2.lowCpCoeffs_[coefLabel];
        }

        if
        (
            janaf2TThermo<EquationOfState>::debug
         && !equal(jt1.Tcommon_, jt2.Tcommon_)
        )
        {
            FatalErrorInFunction
                « "Tcommon " « jt1.Tcommon_ « " for "
```

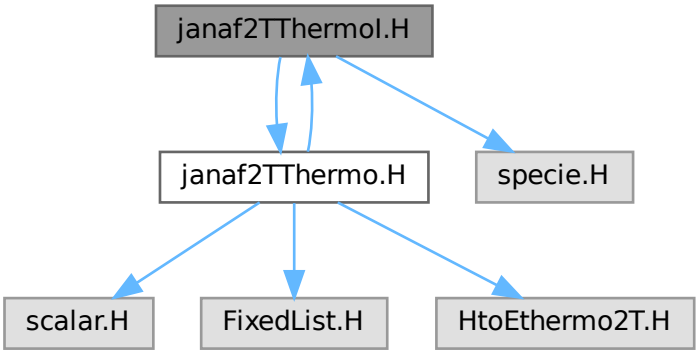
```
                « (jt1.name().size() ? jt1.name() : "others")
                « " != " « jt2.Tcommon_ « " for "
                « (jt2.name().size() ? jt2.name() : "others")
                « exit(FatalError);
        }

        return janaf2TThermo<EquationOfState>
        (
            eofs,
            max(jt1.Tlow_, jt2.Tlow_),
            min(jt1.Thigh_, jt2.Thigh_),
            jt1.Tcommon_,
            highCpCoeffs,
            lowCpCoeffs
        );
    }
}


template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState> Foam::operator*
(
    const scalar s,
    const janaf2TThermo<EquationOfState>& jt
)
{
    return janaf2TThermo<EquationOfState>
    (
        s*static_cast<const EquationOfState&>(jt),
        jt.Tlow_,
        jt.Thigh_,
        jt.Tcommon_,
        jt.highCpCoeffs_,
        jt.lowCpCoeffs_
    );
}


template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState> Foam::operator==
(
    const janaf2TThermo<EquationOfState>& jt1,
    const janaf2TThermo<EquationOfState>& jt2
)
{
    EquationOfState eofs
    (
        static_cast<const EquationOfState&>(jt1)
     == static_cast<const EquationOfState&>(jt2)
    );

    const scalar Y1 = jt2.Y()/eofs.Y();
    const scalar Y2 = jt1.Y()/eofs.Y();

    typename janaf2TThermo<EquationOfState>::coeffArray highCpCoeffs;
    typename janaf2TThermo<EquationOfState>::coeffArray lowCpCoeffs;

    for
    (
        label coefLabel=0;
        coefLabel<janaf2TThermo<EquationOfState>::nCoeffs_;
        coefLabel++
    )
    {
        highCpCoeffs[coefLabel] =
            Y1*jt2.highCpCoeffs_[coefLabel]
          - Y2*jt1.highCpCoeffs_[coefLabel];

        lowCpCoeffs[coefLabel] =
            Y1*jt2.lowCpCoeffs_[coefLabel]
          - Y2*jt1.lowCpCoeffs_[coefLabel];
    }

    if
    (
        janaf2TThermo<EquationOfState>::debug
     && !equal(jt2.Tcommon_, jt1.Tcommon_)
    )
    {
        FatalErrorInFunction
            « "Tcommon " « jt2.Tcommon_ « " for "
            « (jt2.name().size() ? jt2.name() : "others")
            « " != " « jt1.Tcommon_ « " for "
            « (jt1.name().size() ? jt1.name() : "others")
            « exit(FatalError);
    }

```

```
    return janaf2TThermo<EquationOfState>
    (
        eofs,
        max(jt2.Tlow_, jt1.Tlow_),
        min(jt2.Thigh_, jt1.Thigh_),
        jt2.Tcommon_,
        highCpCoeffs,
        lowCpCoeffs
    );
}


// ************************************************************************* //
```

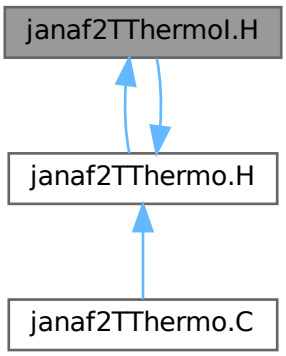
7.87 thermo/janaf2T/janaf2TThermoI.H File Reference

```
#include "janaf2TThermo.H"
#include "specie.H"
```

Include dependency graph for thermo/janaf2T/janaf2TThermoI.H:



This graph shows which files directly or indirectly include this file:



7.88 thermo/janaf2T/janaf2TThermoI.H

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "janaf2TThermo.H"
#include "specie.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState>::janaf2TThermo
(
    const EquationOfState& st,
    const scalar Tlow,
    const scalar Thigh,
    const scalar Tcommon,
    const scalar ThetaVib,
    const typename janaf2TThermo<EquationOfState>::coeffArray& highCpCoeffs,
    const typename janaf2TThermo<EquationOfState>::coeffArray& lowCpCoeffs,
    const bool convertCoeffs
)
:
    EquationOfState(st),
    Tlow_(Tlow),
    Thigh_(Thigh),
    Tcommon_(Tcommon),
    ThetaVib_(ThetaVib)
{
    if (convertCoeffs)
    {
        for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
        {
            highCpCoeffs_[coefLabel] = highCpCoeffs[coefLabel]*this->R();
            lowCpCoeffs_[coefLabel] = lowCpCoeffs[coefLabel]*this->R();
        }
    }
    else
    {
        for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
        {
            highCpCoeffs_[coefLabel] = highCpCoeffs[coefLabel];
            lowCpCoeffs_[coefLabel] = lowCpCoeffs[coefLabel];
        }
    }
}


template<class EquationOfState>
inline const typename Foam::janaf2TThermo<EquationOfState>::coeffArray&
Foam::janaf2TThermo<EquationOfState>::coeffs
(
    const scalar TTR
) const
{
    if (TTR < Tcommon_)
    {
        return lowCpCoeffs_;
    }
```

```
    else
    {
        return highCpCoeffs_;
    }
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState>::janaf2TThermo
(
    const word& name,
    const janaf2TThermo& jt
)
:
    EquationOfState(name, jt),
    Tlow_(jt.Tlow_),
    Thigh_(jt.Thigh_),
    Tcommon_(jt.Tcommon_),
    ThetaVib_(jt.ThetaVib_)
{
    for (label coefLabel=0; coefLabel<nCoeffs_; coefLabel++)
    {
        highCpCoeffs_[coefLabel] = jt.highCpCoeffs_[coefLabel];
        lowCpCoeffs_[coefLabel] = jt.lowCpCoeffs_[coefLabel];
    }
}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::limit
(
    const scalar TTR
) const
{
    if (TTR < Tlow_ || TTR > Thigh_)
    {
        WarningInFunction
            « "attempt to use janaf2TThermo<EquationOfState>"
               " out of temperature range "
            « Tlow_ « " -> " « Thigh_ « ";  TTR = " « TTR
            « endl;

        return clamp(TTR, Tlow_, Thigh_);
    }

    return TTR;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Tlow() const
{
    return Tlow_;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Thigh() const
{
    return Thigh_;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Tcommon() const
{
    return Tcommon_;
}

template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::ThetaVib() const
{
    return ThetaVib_;
}

template<class EquationOfState>
inline const typename Foam::janaf2TThermo<EquationOfState>::coeffArray&
Foam::janaf2TThermo<EquationOfState>::highCpCoeffs() const
{
    return highCpCoeffs_;
}


```

```
template<class EquationOfState>
inline const typename Foam::janaf2TThermo<EquationOfState>::coeffArray&
Foam::janaf2TThermo<EquationOfState>::lowCpCoeffs() const
{
    return lowCpCoeffs_;
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::CpTR
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
        ((((a[4]*TTR + a[3])*TTR + a[2])*TTR + a[1])*TTR + a[0])
      + EquationOfState::Cp(p, TTR);
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Ha
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
    (
        ((((a[4]/5.0*TTR + a[3]/4.0)*TTR + a[2]/3.0)*TTR + a[1]/2.0)*TTR + a[0])*TTR
      + a[5]
    ) + EquationOfState::H(p, TTR);
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Hs
(
    const scalar p,
    const scalar TTR
) const
{
    return Ha(p, TTR) - Hc();
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Hc() const
{
    const coeffArray& a = lowCpCoeffs_;
    return
    (
        (
            (((a[4]/5.0*Tstd + a[3]/4.0)*Tstd + a[2]/3.0)*Tstd + a[1]/2.0)*Tstd
          + a[0]
        )*Tstd + a[5]
    );
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::S
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
    (
        (((a[4]/4.0*TTR + a[3]/3.0)*TTR + a[2]/2.0)*TTR + a[1])*TTR + a[0]*log(TTR)
      + a[6]
    ) + EquationOfState::S(p, TTR);
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::Gstd
(
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
```

```
    (
        (
            a[0]*(1 - log(TTR))
          - (((a[4]/20.0*TTR + a[3]/12.0)*TTR + a[2]/6.0)*TTR + a[1]/2.0)*TTR
          - a[6]
        )*TTR
      + a[5]
    );
}


template<class EquationOfState>
inline Foam::scalar Foam::janaf2TThermo<EquationOfState>::dCpdT
(
    const scalar p,
    const scalar TTR
) const
{
    const coeffArray& a = coeffs(TTR);
    return
        (((4*a[4]*TTR + 3*a[3])*TTR + 2*a[2])*TTR + a[1]);
}

// * * * * * * * * * * * * * * * Member Operators  * * * * * * * * * * * * * //

template<class EquationOfState>
inline void Foam::janaf2TThermo<EquationOfState>::operator+=
(
    const janaf2TThermo<EquationOfState>& jt
)
{
    scalar Y1 = this->Y();

    EquationOfState::operator+=(jt);

    if (mag(this->Y()) > SMALL)
    {
        Y1 /= this->Y();
        const scalar Y2 = jt.Y()/this->Y();

        Tlow_ = max(Tlow_, jt.Tlow_);
        Thigh_ = min(Thigh_, jt.Thigh_);

        if
        (
            janaf2TThermo<EquationOfState>::debug
         && !equal(Tcommon_, jt.Tcommon_)
        )
        {
            FatalErrorInFunction
                << "Tcommon " << Tcommon_ << " for "
                << (this->name().size() ? this->name() : "others")
                << " != " << jt.Tcommon_ << " for "
                << (jt.name().size() ? jt.name() : "others")
                << exit(FatalError);
        }

        for
        (
            label coefLabel=0;
            coefLabel<janaf2TThermo<EquationOfState>::nCoeffs_;
            coefLabel++
        )
        {
            highCpCoeffs_[coefLabel] =
                Y1*highCpCoeffs_[coefLabel]
              + Y2*jt.highCpCoeffs_[coefLabel];

            lowCpCoeffs_[coefLabel] =
                Y1*lowCpCoeffs_[coefLabel]
              + Y2*jt.lowCpCoeffs_[coefLabel];
        }
    }
}


// * * * * * * * * * * * * * * * Friend Operators  * * * * * * * * * * * * * //

template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState> Foam::operator+
(
    const janaf2TThermo<EquationOfState>& jt1,
    const janaf2TThermo<EquationOfState>& jt2
)
{
    EquationOfState eofs = jt1;
    eofs += jt2;
```

```
    if (mag(eofs.Y()) < SMALL)
    {
        return janaf2TThermo<EquationOfState>
        (
            eofs,
            jt1.Tlow_,
            jt1.Thigh_,
            jt1.Tcommon_,
            jt1.highCpCoeffs_,
            jt1.lowCpCoeffs_
        );
    }
    else
    {
        const scalar Y1 = jt1.Y()/eofs.Y();
        const scalar Y2 = jt2.Y()/eofs.Y();

        typename janaf2TThermo<EquationOfState>::coeffArray highCpCoeffs;
        typename janaf2TThermo<EquationOfState>::coeffArray lowCpCoeffs;

        for
        (
            label coefLabel=0;
            coefLabel<janaf2TThermo<EquationOfState>::nCoeffs_;
            coefLabel++
        )
        {
            highCpCoeffs[coefLabel] =
                Y1*jt1.highCpCoeffs_[coefLabel]
              + Y2*jt2.highCpCoeffs_[coefLabel];

            lowCpCoeffs[coefLabel] =
                Y1*jt1.lowCpCoeffs_[coefLabel]
              + Y2*jt2.lowCpCoeffs_[coefLabel];
        }

        if
        (
            janaf2TThermo<EquationOfState>::debug
         && !equal(jt1.Tcommon_, jt2.Tcommon_)
        )
        {
            FatalErrorInFunction
                « "Tcommon " « jt1.Tcommon_ « " for "
                « (jt1.name().size() ? jt1.name() : "others")
                « " != " « jt2.Tcommon_ « " for "
                « (jt2.name().size() ? jt2.name() : "others")
                « exit(FatalError);
        }

        return janaf2TThermo<EquationOfState>
        (
            eofs,
            max(jt1.Tlow_, jt2.Tlow_),
            min(jt1.Thigh_, jt2.Thigh_),
            jt1.Tcommon_,
            highCpCoeffs,
            lowCpCoeffs
        );
    }
}


template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState> Foam::operator*
(
    const scalar s,
    const janaf2TThermo<EquationOfState>& jt
)
{
    return janaf2TThermo<EquationOfState>
    (
        s*static_cast<const EquationOfState&>(jt),
        jt.Tlow_,
        jt.Thigh_,
        jt.Tcommon_,
        jt.highCpCoeffs_,
        jt.lowCpCoeffs_
    );
}


template<class EquationOfState>
inline Foam::janaf2TThermo<EquationOfState> Foam::operator==
(
    const janaf2TThermo<EquationOfState>& jt1,
```

```
    const janaf2TThermo<EquationOfState>& jt2
)
{
    EquationOfState eofs
    (
        static_cast<const EquationOfState&>(jt1)
     == static_cast<const EquationOfState&>(jt2)
    );

    const scalar Y1 = jt2.Y()/eofs.Y();
    const scalar Y2 = jt1.Y()/eofs.Y();

    typename janaf2TThermo<EquationOfState>::coeffArray highCpCoeffs;
    typename janaf2TThermo<EquationOfState>::coeffArray lowCpCoeffs;

    for
    (
        label coefLabel=0;
        coefLabel<janaf2TThermo<EquationOfState>::nCoeffs_;
        coefLabel++
    )
    {
        highCpCoeffs[coefLabel] =
            Y1*jt2.highCpCoeffs_[coefLabel]
          - Y2*jt1.highCpCoeffs_[coefLabel];

        lowCpCoeffs[coefLabel] =
            Y1*jt2.lowCpCoeffs_[coefLabel]
          - Y2*jt1.lowCpCoeffs_[coefLabel];
    }

    if
    (
        janaf2TThermo<EquationOfState>::debug
     && !equal(jt2.Tcommon_, jt1.Tcommon_)
    )
    {
        FatalErrorInFunction
            « "Tcommon " « jt2.Tcommon_ « " for "
            « (jt2.name().size() ? jt2.name() : "others")
            « " != " « jt1.Tcommon_ « " for "
            « (jt1.name().size() ? jt1.name() : "others")
            « exit(FatalError);
    }

    return janaf2TThermo<EquationOfState>
    (
        eofs,
        max(jt2.Tlow_, jt1.Tlow_),
        min(jt2.Thigh_, jt1.Thigh_),
        jt2.Tcommon_,
        highCpCoeffs,
        lowCpCoeffs
    );
}


// ************************************************************************* //
```

7.89 include/reaction2TTypes.H File Reference

```
#include "thermoPhysics2TTypes.H"
#include "Reaction.H"
#include "icoPolynomial.H"
#include "hPolynomialThermo.H"
#include "polynomialTransport.H"
```

Include dependency graph for include/reaction2TTypes.H:

Namespaces

- namespace Foam

Typedefs

- typedef Reaction< blottnerEucken2TGasHThermoPhysics > Foam::blottnerEucken2TGasHThermoPhysicsHReaction

7.90 include/reaction2TTypes.H

Go to the documentation of this file.

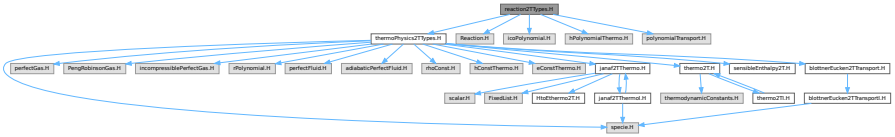
```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2013 OpenFOAM Foundation
    Copyright (C) 2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Typedefs
    Foam::reactionTypes

Description
    Type definitions for reactions

\*---------------------------------------------------------------------------*/

#ifndef reaction2TTypes_H
#define reaction2TTypes_H

#include "thermoPhysics2TTypes.H"
#include "Reaction.H"

#include "icoPolynomial.H"
#include "hPolynomialThermo.H"
#include "polynomialTransport.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    // sensible enthalpy based reactions
    typedef Reaction<blottnerEucken2TGasHThermoPhysics> blottnerEucken2TGasHThermoPhysicsHReaction;
}

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.91 InInclude/reaction2TTypes.H File Reference

```
#include "thermoPhysics2TTypes.H"
#include "Reaction.H"
#include "icoPolynomial.H"
#include "hPolynomialThermo.H"
```

`#include "polynomialTransport.H"`

Include dependency graph for lnInclude/reaction2TTypes.H:



Namespaces

- namespace Foam

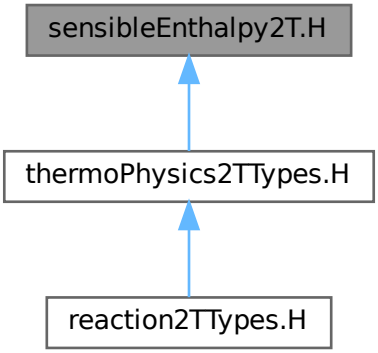
7.92 lnInclude/reaction2TTypes.H

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2013 OpenFOAM Foundation
    Copyright (C) 2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Typedefs
    Foam::reactionTypes

Description
    Type definitions for reactions

\*---------------------------------------------------------------------------*/

#ifndef reaction2TTypes_H
#define reaction2TTypes_H

#include "thermoPhysics2TTypes.H"
#include "Reaction.H"

#include "icoPolynomial.H"
#include "hPolynomialThermo.H"
#include "polynomialTransport.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    // sensible enthalpy based reactions
    typedef Reaction<blottnerEucken2TGasHThermoPhysics> blottnerEucken2TGasHThermoPhysicsHReaction;
}

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.93 InInclude/sensibleEnthalpy2T.H File Reference

This graph shows which files directly or indirectly include this file:



Classes

- class Foam::sensibleEnthalpy2T< Thermo2T >

Namespaces

- namespace Foam

7.94 InInclude/sensibleEnthalpy2T.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::sensibleEnthalpy2T
```

```

Group
    grpSpecieThermo

Description
    Thermodynamics mapping class to expose the sensible enthalpy functions.

\*---------------------------------------------------------------------------*/

#ifndef sensibleEnthalpy2T_H
#define sensibleEnthalpy2T_H

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*---------------------------------------------------------------------------*\
                        Class sensibleEnthalpy2T Declaration
\*---------------------------------------------------------------------------*/

template<class Thermo2T>
class sensibleEnthalpy2T
{

public:

    // Constructors

        //- Construct
        sensibleEnthalpy2T()
        {}


    // Member Functions

        //- Return the instantiated type name
        static word typeName()
        {
            return "sensibleEnthalpy2T";
        }

        // Fundamental properties

            static word energyName()
            {
                return "h";
            }

            //- Heat capacity at constant pressure [J/(kg K)]
            scalar CpvTR
            (
                const Thermo2T& thermo2T,
                const scalar p,
                const scalar TTR
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar cp = thermo2T.CpTR(p, TTR);
                return cp;
                #else
                return thermo2T.CpTR(p, TTR);
                #endif
            }

            //- CpTR/CpTR []
            scalar CpByCpvTR
            (
                const Thermo2T& thermo2T,
                const scalar p,
                const scalar TTR
            ) const
            {
                return 1;
            }

            //- Sensible enthalpy [J/kg]
            scalar HE
            (
                const Thermo2T& thermo2T,
                const scalar p,
                const scalar TTR
            ) const
            {
                #ifdef __clang__
```

```
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar hs = thermo2T.Hs(p, TTR);
                return hs;
                #else
                return thermo2T.Hs(p, TTR);
                #endif
            }

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            scalar THE_TR
            (
                const Thermo2T& thermo2T,
                const scalar h,
                const scalar p,
                const scalar TTR0
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar ths = thermo2T.THs_TR(h, p, TTR0);
                return ths;
                #else
                return thermo2T.THs_TR(h, p, TTR0);
                #endif
            }

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            scalar TEH_Vib
            (
                const Thermo2T& thermo2T,
                const scalar eVib,
                const scalar p,
                const scalar TTR0,
                const scalar TVib0
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar tesVib = thermo2T.TEs_Vib(eVib, p, TVib0);
                return tesVib;
                #else
                return thermo2T.TEs_Vib(eVib, p, TTR0, TVib0);
                #endif
            }
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.95 thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H File Reference

Classes

- class Foam::sensibleEnthalpy2T< Thermo2T >

Namespaces

- namespace Foam

7.96 thermo/sensibleEnthalpy2T/sensibleEnthalpy2T.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2012-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::sensibleEnthalpy2T

Group
    grpSpecieThermo

Description
    Thermodynamics mapping class to expose the sensible enthalpy functions.

\*-------------------------------------------------------------------------*/

#ifndef sensibleEnthalpy2T_H
#define sensibleEnthalpy2T_H

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{

/*-------------------------------------------------------------------------*\
                       Class sensibleEnthalpy2T Declaration
\*-------------------------------------------------------------------------*/

template<class Thermo2T>
class sensibleEnthalpy2T
{

public:

    // Constructors

        //- Construct
        sensibleEnthalpy2T()
        {}


    // Member Functions

        //- Return the instantiated type name
        static word typeName()
        {
            return "sensibleEnthalpy2T";
        }

        // Fundamental properties

            static word energyName()
            {
                return "h";
            }

            //- Heat capacity at constant pressure [J/(kg K)]
            scalar CpvTR
            (
                const Thermo2T& thermo2T,
                const scalar p,
                const scalar TTR
```

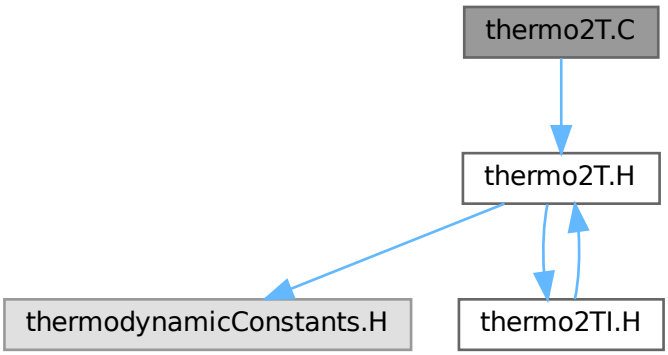
```
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar cp = thermo2T.CpTR(p, TTR);
                return cp;
                #else
                return thermo2T.CpTR(p, TTR);
                #endif
            }

            //- CpTR/CpTR []
            scalar CpByCpvTR
            (
                const Thermo2T& thermo2T,
                const scalar p,
                const scalar TTR
            ) const
            {
                return 1;
            }

            //- Sensible enthalpy [J/kg]
            scalar HE
            (
                const Thermo2T& thermo2T,
                const scalar p,
                const scalar TTR
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar hs = thermo2T.Hs(p, TTR);
                return hs;
                #else
                return thermo2T.Hs(p, TTR);
                #endif
            }

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            scalar THE_TR
            (
                const Thermo2T& thermo2T,
                const scalar h,
                const scalar p,
                const scalar TTR0
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar ths = thermo2T.THs_TR(h, p, TTR0);
                return ths;
                #else
                return thermo2T.THs_TR(h, p, TTR0);
                #endif
            }

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            scalar TEH_Vib
            (
                const Thermo2T& thermo2T,
                const scalar eVib,
                const scalar p,
                const scalar TTR0,
                const scalar TVib0
            ) const
            {
                #ifdef __clang__
                // Using volatile to prevent compiler optimisations leading to
                // a sigfpe
                volatile const scalar tesVib = thermo2T.TEs_Vib(eVib, p, TVib0);
                return tesVib;
                #else
                return thermo2T.TEs_Vib(eVib, p, TTR0, TVib0);
                #endif
            }
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace Foam
```

```
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.97 InInclude/thermo2T.C File Reference

`#include "thermo2T.H"`

Include dependency graph for InInclude/thermo2T.C:



7.98 InInclude/thermo2T.C

Go to the documentation of this file.

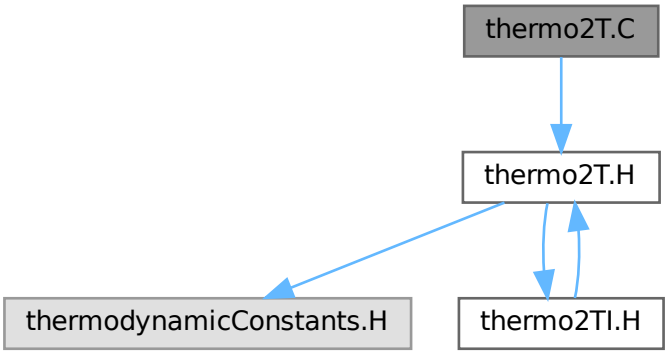
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "thermo2T.H"

/* * * * * * * * * * * * * private static data * * * * * * * * * * * * * */
```

```

template<class Thermo2T, template<class> class Type>
const Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::tol_ = 1.0e-4;

template<class Thermo2T, template<class> class Type>
const int Foam::species::thermo2T<Thermo2T, Type>::maxIter_ = 100;


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
Foam::species::thermo2T<Thermo2T, Type>::thermo2T(const dictionary& dict)
:
    Thermo2T(dict)
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
void Foam::species::thermo2T<Thermo2T, Type>::write(Ostream& os) const
{
    Thermo2T::write(os);
}


// * * * * * * * * * * * * * * * * Ostream Operator  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
Foam::Ostream& Foam::species::operator«
(
    Ostream& os, const thermo2T<Thermo2T, Type>& st
)
{
    st.write(os);
    return os;
}


// ************************************************************************* //
```

7.99 thermo/thermo2T/thermo2T.C File Reference

`#include "thermo2T.H"`

Include dependency graph for thermo/thermo2T/thermo2T.C:



7.100 thermo/thermo2T/thermo2T.C

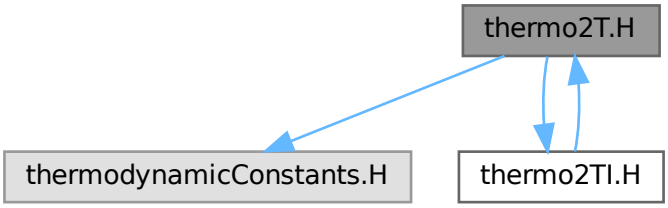
Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "thermo2T.H"

/* * * * * * * * * * * * * * * private static data * * * * * * * * * * * * * * */

template<class Thermo2T, template<class> class Type>
const Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::tol_ = 1.0e-4;

template<class Thermo2T, template<class> class Type>
const int Foam::species::thermo2T<Thermo2T, Type>::maxIter_ = 100;


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
Foam::species::thermo2T<Thermo2T, Type>::thermo2T(const dictionary& dict)
:
    Thermo2T(dict)
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
void Foam::species::thermo2T<Thermo2T, Type>::write(Ostream& os) const
{
    Thermo2T::write(os);
}


// * * * * * * * * * * * * * * * * Ostream Operator  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
Foam::Ostream& Foam::species::operator«
(
    Ostream& os, const thermo2T<Thermo2T, Type>& st
)
{
    st.write(os);
    return os;
}


// ************************************************************************* //
```

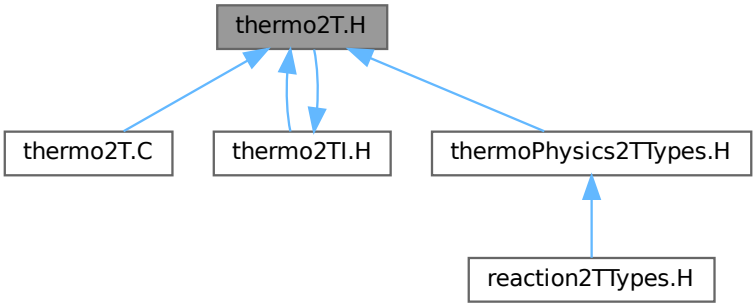
7.101 InInclude/thermo2T.H File Reference

```
#include "thermodynamicConstants.H"
#include "thermo2TI.H"
```

Include dependency graph for lnInclude/thermo2T.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::species::thermo2T< Thermo2T, Type >

Namespaces

- namespace Foam
- namespace Foam::species

Functions

- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > Foam::species::operator+ (const thermo2T< Thermo2T, Type > &, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > Foam::species::operator* (const scalar, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > Foam::species::operator== (const thermo2T< Thermo2T, Type > &, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  Ostream & Foam::species::operator<< (Ostream &, const thermo2T< Thermo2T, Type > &)

7.102 InInclude/thermo2T.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::thermo2T

Description
    Basic thermodynamics type based on the use of fitting functions for
    cp, h, s obtained from the template argument type thermo2T.  All other
    properties are derived from these primitive functions.

SourceFiles
    thermoTI.H
    thermo2T.C

\*-------------------------------------------------------------------------*/

#ifndef thermo2T_H
#define thermo2T_H

#include "thermodynamicConstants.H"
using namespace Foam::constant::thermodynamic;

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
namespace species
{

// Forward declaration of friend functions and operators

template<class Thermo2T, template<class> class Type> class thermo2T;

template<class Thermo2T, template<class> class Type>
inline thermo2T<Thermo2T, Type> operator+
(
    const thermo2T<Thermo2T, Type>&,
    const thermo2T<Thermo2T, Type>&
);

template<class Thermo2T, template<class> class Type>
inline thermo2T<Thermo2T, Type> operator*
(
    const scalar,
    const thermo2T<Thermo2T, Type>&
);

template<class Thermo2T, template<class> class Type>
inline thermo2T<Thermo2T, Type> operator==
(
    const thermo2T<Thermo2T, Type>&,
    const thermo2T<Thermo2T, Type>&
);

template<class Thermo2T, template<class> class Type>
Ostream& operator«
(
    Ostream&,
```

```
    const thermo2T<Thermo2T, Type>&
);


/*----------------------------------------------------------------------*\
                           Class thermo2T Declaration
\*----------------------------------------------------------------------*/

template<class Thermo2T, template<class> class Type>
class thermo2T
:
    public Thermo2T,
    public Type<thermo2T<Thermo2T, Type»
{
    // Private data

        //- Convergence tolerance of energy -> temperature inversion functions
        static const scalar tol_;

        //- Max number of iterations in energy->temperature inversion functions
        static const int maxIter_;


    // Private Member Functions

        //- Return the temperature corresponding to the value of the
        //  thermodynamic property f, given the function f = F(p, T)
        //  and dF(p, T)/dT
        inline scalar T
        (
            scalar f,
            scalar p,
            scalar T0,
            scalar (thermo2T::*F)(const scalar, const scalar) const,
            scalar (thermo2T::*dFdT)(const scalar, const scalar) const,
            scalar (thermo2T::*limit)(const scalar) const
        ) const;

        inline scalar TVib
        (
            scalar f,
            scalar p,
            scalar TTR,
            scalar TVib0,
            scalar (thermo2T::*F)(const scalar, const scalar, const scalar, const scalar) const,
            scalar (thermo2T::*dFdT)(const scalar, const scalar) const,
            scalar (thermo2T::*limit)(const scalar) const
        ) const;


public:

    //- The thermodynamics of the individual species'
    typedef thermo2T<Thermo2T, Type> thermoType;


    // Constructors

        //- Construct from components
        inline thermo2T(const Thermo2T& sp);

        //- Construct from dictionary
        thermo2T(const dictionary& dict);

        //- Construct as named copy
        inline thermo2T(const word& name, const thermo2T&);


    // Member Functions

        //- Return the instantiated type name
        static word typeName()
        {
            return
                  Thermo2T::typeName() + ','
                + Type<thermo2T<Thermo2T, Type»::typeName();
        }

        //- Name of Enthalpy/Internal energy
        static inline word heName();


        // Fundamental properties
        // (These functions must be provided in derived types)

            // Heat capacity at constant pressure [J/(kg K)]
            // inline scalar Cp(const scalar p, const scalar T) const;
```

```

            // Sensible enthalpy [J/kg]
            // inline scalar Hs(const scalar p, const scalar T) const;

            // Chemical enthalpy [J/kg]
            // inline scalar Hc() const;

            // Absolute Enthalpy [J/kg]
            // inline scalar Ha(const scalar p, const scalar T) const;

            // Heat capacity at constant volume [J/(kg K)]
            // inline scalar Cv(const scalar p, const scalar T) const;

            // Sensible internal energy [J/kg]
            // inline scalar Es(const scalar p, const scalar T) const;

            // Absolute internal energy [J/kg]
            // inline scalar Ea(const scalar p, const scalar T) const;

            // Entropy [J/(kg K)]
            // inline scalar S(const scalar p, const scalar T) const;


        // Mass specific derived properties


            //- Heat capacity at constant pressure/volume [J/(kg K)]
            inline scalar CpvTR(const scalar p, const scalar TTR) const;

            //- Gamma = CpTR/CvT []
            inline scalar gamma(const scalar p, const scalar TTR) const;

            //- Ratio of heat capacity at constant pressure to that at
            //  constant pressure/volume []
            inline scalar CpByCpvTR(const scalar p, const scalar TTR) const;

            //- Enthalpy/Internal energy [J/kg]
            inline scalar HE(const scalar p, const scalar TTR) const;

            //- Gibbs free energy [J/kg]
            inline scalar G(const scalar p, const scalar TTR) const;

            //- Helmholtz free energy [J/kg]
            inline scalar A(const scalar p, const scalar TTR) const;


        // Mole specific derived properties

            //- Heat capacity at constant pressure [J/(kmol K)]
            inline scalar cpTR(const scalar p, const scalar TTR) const;

            //- Absolute Enthalpy [J/kmol]
            inline scalar ha(const scalar p, const scalar TTR) const;

            //- Sensible enthalpy [J/kmol]
            inline scalar hs(const scalar p, const scalar TTR) const;

            //- Chemical enthalpy [J/kmol]
            inline scalar hc() const;

            //- Entropy [J/(kmol K)]
            inline scalar s(const scalar p, const scalar TTR) const;

            //- Enthalpy/Internal energy [J/kmol]
            inline scalar he(const scalar p, const scalar TTR) const;

            //- Translational heat capacity at constant volume [J/(kmol K)]
            inline scalar cvT(const scalar p, const scalar TTR) const;

            //- Rotational heat capacity at constant volume [J/(kmol K)]
            inline scalar cvR(const scalar p, const scalar TTR) const;

            //- Vibrational heat capacity at constant volume [J/(kmol K)]
            inline scalar cvVib(const scalar p, const scalar TTR, const scalar TVib, const scalar
    ThetaVib) const;

            //- Translational sensible internal energy [J/kmol]
            inline scalar esT(const scalar p, const scalar TTR) const;

            //- Translational absolute internal energy [J/kmol]
            inline scalar eaT(const scalar p, const scalar TTR) const;

            //- Rotational sensible internal energy [J/kmol]
            inline scalar esR(const scalar p, const scalar TTR) const;

            //- Vibrational sensible internal energy [J/kmol]
            inline scalar esVib(const scalar p, const scalar TTR, const scalar TVib, const scalar
```

```
    ThetaVib) const;

            //- Gibbs free energy [J/kmol]
            inline scalar g(const scalar p, const scalar TTR) const;

            //- Helmholtz free energy [J/kmol]
            inline scalar a(const scalar p, const scalar TTR) const;


        // Equilibrium reaction thermodynamics

            //- Equilibrium constant [] i.t.o fugacities
            //  = PIi(fi/Pstd)^nui
            inline scalar K(const scalar p, const scalar TTR) const;

            //- Equilibrium constant [] i.t.o. partial pressures
            //  = PIi(pi/Pstd)^nui
            //  For low pressures (where the gas mixture is near perfect) Kp = K
            inline scalar Kp(const scalar p, const scalar TTR) const;

            //- Equilibrium constant i.t.o. molar concentration
            //  = PIi(ci/cstd)^nui
            //  For low pressures (where the gas mixture is near perfect)
            //  Kc = Kp(pstd/(RR*TTR))^nu
            inline scalar Kc(const scalar p, const scalar TTR) const;

            //- Equilibrium constant [] i.t.o. mole-fractions
            //  For low pressures (where the gas mixture is near perfect)
            //  Kx = Kp(pstd/p)^nui
            inline scalar Kx
            (
                const scalar p,
                const scalar TTR
            ) const;

            //- Equilibrium constant [] i.t.o. number of moles
            //  For low pressures (where the gas mixture is near perfect)
            //  Kn = Kp(n*pstd/p)^nui where n = number of moles in mixture
            inline scalar Kn
            (
                const scalar p,
                const scalar TTR,
                const scalar n
            ) const;


        // Energy->temperature  inversion functions

            //- Temperature from enthalpy or internal energy
            //  given an initial temperature TTR0
            inline scalar THE_TR
            (
                const scalar H,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from enthalpy or internal energy
            //  given an initial temperature TVib0
            inline scalar TEH_Vib
            (
                const scalar esVib,
                const scalar p,
                const scalar TTR0,
                const scalar TVib0
            ) const;

            //- Temperature from sensible enthalpy given an initial T0
            inline scalar THs_TR
            (
                const scalar Hs,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from absolute enthalpy
            //  given an initial temperature T0
            inline scalar THa_TR
            (
                const scalar H,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            inline scalar TEs_TR
```

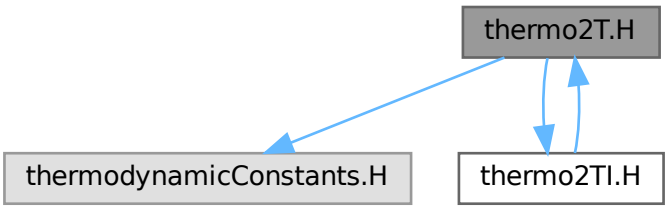
```
            (
                const scalar E,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            inline scalar TEs_Vib
            (
                const scalar E,
                const scalar p,
                const scalar TTR0,
                const scalar TVib0
            ) const;

            //- Temperature from absolute internal energy
            //  given an initial temperature T0
            inline scalar TEa_TR
            (
                const scalar E,
                const scalar p,
                const scalar TTR0
            ) const;


        // Derivative term used for Jacobian

            //- Derivative of B (acooding to Niemeyer et al.) w.r.t. temperature
            inline scalar dKcdTbyKc(const scalar p, const scalar TTR) const;

            //- Derivative of cp w.r.t. temperature
            inline scalar dcpdT(const scalar p, const scalar TTR) const;


        // I-O

            //- Write to Ostream
            void write(Ostream& os) const;


    // Member operators

        inline void operator+=(const thermo2T&);
        inline void operator*=(const scalar);


    // Friend operators

        friend thermo2T operator+ <Thermo2T, Type>
        (
            const thermo2T&,
            const thermo2T&
        );

        friend thermo2T operator* <Thermo2T, Type>
        (
            const scalar s,
            const thermo2T&
        );

        friend thermo2T operator== <Thermo2T, Type>
        (
            const thermo2T&,
            const thermo2T&
        );


    // Ostream Operator

        friend Ostream& operator« <Thermo2T, Type>
        (
            Ostream&,
            const thermo2T&
        );
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace species
} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "thermo2TI.H"

```

```
#ifdef NoRepository
    #include "thermo2T.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

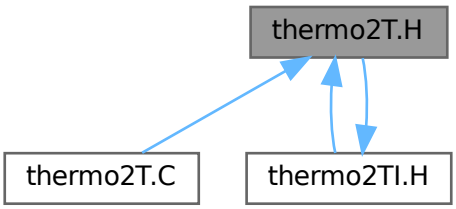
7.103 thermo/thermo2T/thermo2T.H File Reference

```
#include "thermodynamicConstants.H"
#include "thermo2TI.H"
```

Include dependency graph for thermo/thermo2T/thermo2T.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::species::thermo2T< Thermo2T, Type >

Namespaces

- namespace Foam
- namespace Foam::species

Functions

- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > Foam::species::operator+ (const thermo2T< Thermo2T, Type > &, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > Foam::species::operator* (const scalar, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  thermo2T< Thermo2T, Type > Foam::species::operator== (const thermo2T< Thermo2T, Type > &, const thermo2T< Thermo2T, Type > &)
- template<class Thermo2T , template< class > class Type>
  Ostream & Foam::species::operator<< (Ostream &, const thermo2T< Thermo2T, Type > &)

7.104 thermo/thermo2T/thermo2T.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::thermo2T

Description
    Basic thermodynamics type based on the use of fitting functions for
    cp, h, s obtained from the template argument type thermo2T.  All other
    properties are derived from these primitive functions.

SourceFiles
    thermoTI.H
    thermo2T.C

\*-------------------------------------------------------------------------*/

#ifndef thermo2T_H
#define thermo2T_H

#include "thermodynamicConstants.H"
using namespace Foam::constant::thermodynamic;

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
namespace species
{

// Forward declaration of friend functions and operators

template<class Thermo2T, template<class> class Type> class thermo2T;

template<class Thermo2T, template<class> class Type>
```

```
inline thermo2T<Thermo2T, Type> operator+
(
    const thermo2T<Thermo2T, Type>&,
    const thermo2T<Thermo2T, Type>&
);

template<class Thermo2T, template<class> class Type>
inline thermo2T<Thermo2T, Type> operator*
(
    const scalar,
    const thermo2T<Thermo2T, Type>&
);

template<class Thermo2T, template<class> class Type>
inline thermo2T<Thermo2T, Type> operator==
(
    const thermo2T<Thermo2T, Type>&,
    const thermo2T<Thermo2T, Type>&
);

template<class Thermo2T, template<class> class Type>
Ostream& operator«
(
    Ostream&,
    const thermo2T<Thermo2T, Type>&
);


/*---------------------------------------------------------------------------*\
                           Class thermo2T Declaration
\*---------------------------------------------------------------------------*/

template<class Thermo2T, template<class> class Type>
class thermo2T
:
    public Thermo2T,
    public Type<thermo2T<Thermo2T, Type»
{
    // Private data

        //- Convergence tolerance of energy -> temperature inversion functions
        static const scalar tol_;

        //- Max number of iterations in energy->temperature inversion functions
        static const int maxIter_;


    // Private Member Functions

        //- Return the temperature corresponding to the value of the
        //  thermodynamic property f, given the function f = F(p, T)
        //  and dF(p, T)/dT
        inline scalar T
        (
            scalar f,
            scalar p,
            scalar T0,
            scalar (thermo2T::*F)(const scalar, const scalar) const,
            scalar (thermo2T::*dFdT)(const scalar, const scalar) const,
            scalar (thermo2T::*limit)(const scalar) const
        ) const;

        inline scalar TVib
        (
            scalar f,
            scalar p,
            scalar TTR,
            scalar TVib0,
            scalar (thermo2T::*F)(const scalar, const scalar, const scalar, const scalar) const,
            scalar (thermo2T::*dFdT)(const scalar, const scalar) const,
            scalar (thermo2T::*limit)(const scalar) const
        ) const;


public:

    //- The thermodynamics of the individual species'
    typedef thermo2T<Thermo2T, Type> thermoType;


    // Constructors

        //- Construct from components
        inline thermo2T(const Thermo2T& sp);

        //- Construct from dictionary
        thermo2T(const dictionary& dict);
```

```

        //- Construct as named copy
        inline thermo2T(const word& name, const thermo2T&);


    // Member Functions

        //- Return the instantiated type name
        static word typeName()
        {
            return
                  Thermo2T::typeName() + ','
                + Type<thermo2T<Thermo2T, Type>>::typeName();
        }

        //- Name of Enthalpy/Internal energy
        static inline word heName();


        // Fundamental properties
        // (These functions must be provided in derived types)

            // Heat capacity at constant pressure [J/(kg K)]
            // inline scalar Cp(const scalar p, const scalar T) const;

            // Sensible enthalpy [J/kg]
            // inline scalar Hs(const scalar p, const scalar T) const;

            // Chemical enthalpy [J/kg]
            // inline scalar Hc() const;

            // Absolute Enthalpy [J/kg]
            // inline scalar Ha(const scalar p, const scalar T) const;

            // Heat capacity at constant volume [J/(kg K)]
            // inline scalar Cv(const scalar p, const scalar T) const;

            // Sensible internal energy [J/kg]
            // inline scalar Es(const scalar p, const scalar T) const;

            // Absolute internal energy [J/kg]
            // inline scalar Ea(const scalar p, const scalar T) const;

            // Entropy [J/(kg K)]
            // inline scalar S(const scalar p, const scalar T) const;


        // Mass specific derived properties


            //- Heat capacity at constant pressure/volume [J/(kg K)]
            inline scalar CpvTR(const scalar p, const scalar TTR) const;

            //- Gamma = CpTR/CvT []
            inline scalar gamma(const scalar p, const scalar TTR) const;

            //- Ratio of heat capacity at constant pressure to that at
            //  constant pressure/volume []
            inline scalar CpByCpvTR(const scalar p, const scalar TTR) const;

            //- Enthalpy/Internal energy [J/kg]
            inline scalar HE(const scalar p, const scalar TTR) const;

            //- Gibbs free energy [J/kg]
            inline scalar G(const scalar p, const scalar TTR) const;

            //- Helmholtz free energy [J/kg]
            inline scalar A(const scalar p, const scalar TTR) const;


        // Mole specific derived properties

            //- Heat capacity at constant pressure [J/(kmol K)]
            inline scalar cpTR(const scalar p, const scalar TTR) const;

            //- Absolute Enthalpy [J/kmol]
            inline scalar ha(const scalar p, const scalar TTR) const;

            //- Sensible enthalpy [J/kmol]
            inline scalar hs(const scalar p, const scalar TTR) const;

            //- Chemical enthalpy [J/kmol]
            inline scalar hc() const;

            //- Entropy [J/(kmol K)]
            inline scalar s(const scalar p, const scalar TTR) const;

```

```
            //- Enthalpy/Internal energy [J/kmol]
            inline scalar he(const scalar p, const scalar TTR) const;

            //- Translational heat capacity at constant volume [J/(kmol K)]
            inline scalar cvT(const scalar p, const scalar TTR) const;

            //- Rotational heat capacity at constant volume [J/(kmol K)]
            inline scalar cvR(const scalar p, const scalar TTR) const;

            //- Vibrational heat capacity at constant volume [J/(kmol K)]
            inline scalar cvVib(const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const;

            //- Translational sensible internal energy [J/kmol]
            inline scalar esT(const scalar p, const scalar TTR) const;

            //- Translational absolute internal energy [J/kmol]
            inline scalar eaT(const scalar p, const scalar TTR) const;

            //- Rotational sensible internal energy [J/kmol]
            inline scalar esR(const scalar p, const scalar TTR) const;

            //- Vibrational sensible internal energy [J/kmol]
            inline scalar esVib(const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const;

            //- Gibbs free energy [J/kmol]
            inline scalar g(const scalar p, const scalar TTR) const;

            //- Helmholtz free energy [J/kmol]
            inline scalar a(const scalar p, const scalar TTR) const;


        // Equilibrium reaction thermodynamics

            //- Equilibrium constant [] i.t.o fugacities
            //  = PIi(fi/Pstd)^nui
            inline scalar K(const scalar p, const scalar TTR) const;

            //- Equilibrium constant [] i.t.o. partial pressures
            //  = PIi(pi/Pstd)^nui
            //  For low pressures (where the gas mixture is near perfect) Kp = K
            inline scalar Kp(const scalar p, const scalar TTR) const;

            //- Equilibrium constant i.t.o. molar concentration
            //  = PIi(ci/cstd)^nui
            //  For low pressures (where the gas mixture is near perfect)
            //  Kc = Kp(pstd/(RR*TTR))^nu
            inline scalar Kc(const scalar p, const scalar TTR) const;

            //- Equilibrium constant [] i.t.o. mole-fractions
            //  For low pressures (where the gas mixture is near perfect)
            //  Kx = Kp(pstd/p)^nui
            inline scalar Kx
            (
                const scalar p,
                const scalar TTR
            ) const;

            //- Equilibrium constant [] i.t.o. number of moles
            //  For low pressures (where the gas mixture is near perfect)
            //  Kn = Kp(n*pstd/p)^nui where n = number of moles in mixture
            inline scalar Kn
            (
                const scalar p,
                const scalar TTR,
                const scalar n
            ) const;


        // Energy->temperature  inversion functions

            //- Temperature from enthalpy or internal energy
            //  given an initial temperature TTR0
            inline scalar THE_TR
            (
                const scalar H,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from enthalpy or internal energy
            //  given an initial temperature TVib0
            inline scalar TEH_Vib
            (
                const scalar esVib,
                const scalar p,
```

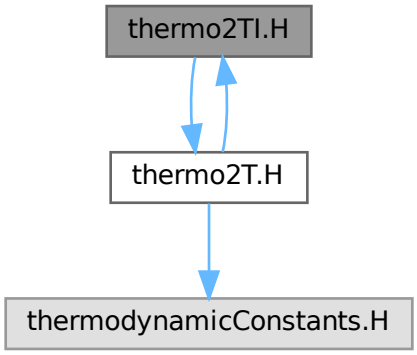
```
                const scalar TTR0,
                const scalar TVib0
            ) const;

            //- Temperature from sensible enthalpy given an initial T0
            inline scalar THs_TR
            (
                const scalar Hs,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from absolute enthalpy
            //  given an initial temperature T0
            inline scalar THa_TR
            (
                const scalar H,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            inline scalar TEs_TR
            (
                const scalar E,
                const scalar p,
                const scalar TTR0
            ) const;

            //- Temperature from sensible internal energy
            //  given an initial temperature T0
            inline scalar TEs_Vib
            (
                const scalar E,
                const scalar p,
                const scalar TTR0,
                const scalar TVib0
            ) const;

            //- Temperature from absolute internal energy
            //  given an initial temperature T0
            inline scalar TEa_TR
            (
                const scalar E,
                const scalar p,
                const scalar TTR0
            ) const;


        // Derivative term used for Jacobian

            //- Derivative of B (acooding to Niemeyer et al.) w.r.t. temperature
            inline scalar dKcdTbyKc(const scalar p, const scalar TTR) const;

            //- Derivative of cp w.r.t. temperature
            inline scalar dcpdT(const scalar p, const scalar TTR) const;


        // I-O

            //- Write to Ostream
            void write(Ostream& os) const;


    // Member operators

        inline void operator+=(const thermo2T&);
        inline void operator*=(const scalar);


    // Friend operators

        friend thermo2T operator+ <Thermo2T, Type>
        (
            const thermo2T&,
            const thermo2T&
        );

        friend thermo2T operator* <Thermo2T, Type>
        (
            const scalar s,
            const thermo2T&
        );

        friend thermo2T operator== <Thermo2T, Type>
        (
```

```
            const thermo2T&,
            const thermo2T&
        );


    // Ostream Operator

        friend Ostream& operator« <Thermo2T, Type>
        (
            Ostream&,
            const thermo2T&
        );
};


// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

} // End namespace species
} // End namespace Foam

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#include "thermo2TI.H"

#ifdef NoRepository
    #include "thermo2T.C"
#endif

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

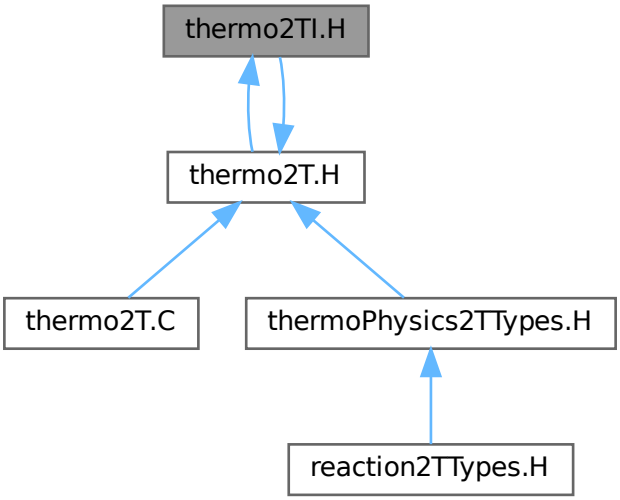
7.105 InInclude/thermo2TI.H File Reference

`#include "thermo2T.H"`

Include dependency graph for InInclude/thermo2TI.H:



This graph shows which files directly or indirectly include this file:



7.106 InInclude/thermo2TI.H

Go to the documentation of this file.

```
/*---------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "thermo2T.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type>::thermo2T
(
    const Thermo2T& sp
)
:
    Thermo2T(sp)
{}
```

```


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::T
(
    scalar f,
    scalar p,
    scalar T0,
    scalar (thermo2T<Thermo2T, Type>::*F)(const scalar, const scalar) const,
    scalar (thermo2T<Thermo2T, Type>::*dFdT)(const scalar, const scalar)
        const,
    scalar (thermo2T<Thermo2T, Type>::*limit)(const scalar) const
) const
{
    if (T0 < 0)
    {
        FatalErrorInFunction
            « "Negative initial temperature T0: " « T0
            « abort(FatalError);
    }

    scalar Test = T0;
    scalar Tnew = T0;
    scalar Ttol = T0*tol_;
    int    iter = 0;

    do
    {
        Test = Tnew;
        Tnew =
            (this->*limit)
            (Test - ((this->*F)(p, Test) - f)/(this->*dFdT)(p, Test));

        if (iter++ > maxIter_)
        {
            FatalErrorInFunction
                « "Maximum number of iterations exceeded: " « maxIter_
                « " when starting from T0:" « T0
                « " old T:" « Test « " new T:" « Tnew
                « " f:" « f
                « " p:" « p
                « " tol:" « Ttol
                « abort(FatalError);
        }

    } while (mag(Tnew - Test) > Ttol);

    return Tnew;
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TVib
(
    scalar f,
    scalar p,
    scalar TTR,
    scalar TVib0,
    scalar (thermo2T<Thermo2T, Type>::*F)(const scalar, const scalar, const scalar, const scalar)
const,
    scalar (thermo2T<Thermo2T, Type>::*dFdT)(const scalar, const scalar)
        const,
    scalar (thermo2T<Thermo2T, Type>::*limit)(const scalar) const
) const
{
    if (TVib0 < 0)
    {
        FatalErrorInFunction
            « "Negative initial temperature TVib0: " « TVib0
            « abort(FatalError);
    }

    scalar Test = TVib0;
    scalar Tnew = TVib0;
    scalar Ttol = TVib0*tol_;
    int    iter = 0;

    do
    {
        Test = Tnew;
        Tnew =
            (this->*limit)
            (Test - ((this->*F)(p, TTR, Test, this->ThetaVib()) - f)/(this->*dFdT)(p, Test));

        if (iter++ > maxIter_)
        {
            FatalErrorInFunction
                « "Maximum number of iterations exceeded: " « maxIter_
```

```
                « " when starting from TVib0:" « TVib0
                « " old T:" « Test « " new T:" « Tnew
                « " f:" « f
                « " p:" « p
                « " tol:" « Ttol
                « abort(FatalError);
        }

    } while (mag(Tnew - Test) > Ttol);

    return Tnew;
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type>::thermo2T
(
    const word& name,
    const thermo2T& st
)
:
    Thermo2T(name, st)
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::word
Foam::species::thermo2T<Thermo2T, Type>::heName()
{
    return Type<thermo2T<Thermo2T, Type»::energyName();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::CpvTR(const scalar p, const scalar TTR) const
{
    return Type<thermo2T<Thermo2T, Type»::CpvTR(*this, p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::gamma(const scalar p, const scalar TTR) const
{
    #ifdef __clang__
    volatile const scalar CpTR = this->CpTR(p, TTR);
    #else
    const scalar CpTR = this->CpTR(p, TTR);
    #endif

    return CpTR/(CpTR - this->CpMCv(p, TTR));
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::CpByCpvTR
(
    const scalar p,
    const scalar TTR
) const
{
    return Type<thermo2T<Thermo2T, Type»::CpByCpvTR(*this, p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::HE(const scalar p, const scalar TTR) const
{
    return Type<thermo2T<Thermo2T, Type»::HE(*this, p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::G(const scalar p, const scalar TTR) const
{
    return this->Ha(p, TTR) - TTR*this->S(p, TTR);
}


```

```
template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::A(const scalar p, const scalar TTR) const
{
    return this->Ea(p, TTR) - TTR*this->S(p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cpTR(const scalar p, const scalar TTR) const
{
    return this->CpTR(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::ha(const scalar p, const scalar TTR) const
{
    return this->Ha(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::hs(const scalar p, const scalar TTR) const
{
    return this->Hs(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::hc() const
{
    return this->Hc()*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::s(const scalar p, const scalar TTR) const
{
    return this->S(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::he(const scalar p, const scalar TTR) const
{
    return this->HE(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cvT(const scalar p, const scalar TTR) const
{
    return this->CvT(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cvR(const scalar p, const scalar TTR) const
{
    return this->CvR(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cvVib(const scalar p, const scalar TTR, const scalar TVib,
    const scalar ThetaVib) const
{
    return this->CvVib(p, TTR, TVib, ThetaVib)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::esT(const scalar p, const scalar TTR) const
{
    return this->EsT(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
```

```
Foam::species::thermo2T<Thermo2T, Type>::esR(const scalar p, const scalar TTR) const
{
    return this->EsR(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::esVib(const scalar p, const scalar TTR, const scalar TVib,
const scalar ThetaVib) const
{
    return this->EsVib(p, TTR, TVib, ThetaVib)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::eaT(const scalar p, const scalar TTR) const
{
    return this->EaT(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::g(const scalar p, const scalar TTR) const
{
    return this->G(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::a(const scalar p, const scalar TTR) const
{
    return this->A(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::K(const scalar p, const scalar TTR) const
{
    scalar arg = -this->Y()*this->G(Pstd, TTR)/(RR*TTR);

    if (arg < 600)
    {
        return exp(arg);
    }
    else
    {
        return VGREAT;
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::Kp(const scalar p, const scalar TTR) const
{
    return K(p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::Kc(const scalar p, const scalar TTR) const
{
    const scalar nm = this->Y()/this->W_s();

    if (equal(nm, SMALL))
    {
        return Kp(p, TTR);
    }
    else
    {
        return Kp(p, TTR)*pow(Pstd/(RR*TTR), nm);
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::Kx
(
    const scalar p,
    const scalar TTR
) const
{
    const scalar nm = this->Y()/this->W_s();
```

```

    if (equal(nm, SMALL))
    {
        return Kp(p, TTR);
    }
    else
    {
        return Kp(p, TTR)*pow(Pstd/p, nm);
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::Kn
(
    const scalar p,
    const scalar TTR,
    const scalar n
) const
{
    const scalar nm = this->Y()/this->W_s();

    if (equal(nm, SMALL))
    {
        return Kp(p, TTR);
    }
    else
    {
        return Kp(p, TTR)*pow(n*Pstd/p, nm);
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::THE_TR
(
    const scalar he,
    const scalar p,
    const scalar TTR0
) const
{
    return Type<thermo2T<Thermo2T, Type»::THE_TR(*this, he, p, TTR0);
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEH_Vib
(
    const scalar esVib,
    const scalar p,
    const scalar TTR0,
    const scalar TVib0
) const
{
    return Type<thermo2T<Thermo2T, Type»::TEH_Vib(*this, esVib, p, TTR0, TVib0);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::THs_TR
(
    const scalar hs,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        hs,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::Hs,
        &thermo2T<Thermo2T, Type>::CpTR,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::THa_TR
(
    const scalar ha,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
```

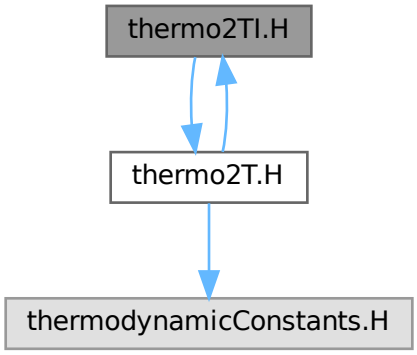
```
        ha,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::Ha,
        &thermo2T<Thermo2T, Type>::CpTR,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEs_TR
(
    const scalar es,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        es,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::EsT,
        &thermo2T<Thermo2T, Type>::CvT,
        &thermo2T<Thermo2T, Type>::limit
    );
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEs_Vib
(
    const scalar esVib,
    const scalar p,
    const scalar TVib0,
    const scalar TTR0
) const
{
    return TVib
    (
        esVib,
        p,
        TTR0,
        TVib0,
        &thermo2T<Thermo2T, Type>::EsVib,
        &thermo2T<Thermo2T, Type>::CvVib,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEa_TR
(thermo2T_H
    const scalar ea,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        ea,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::EaT,
        &thermo2T<Thermo2T, Type>::CvT,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::dKcdTbyKc
(
    const scalar p,
    const scalar TTR
) const
{
    const scalar dKcdTbyKc =
        (this->S(Pstd, TTR) + this->Gstd(TTR)/TTR)*this->Y()/(RR*TTR);

    const scalar nm = this->Y()/this->W_s();
    if (equal(nm, SMALL))
    {
        return dKcdTbyKc;
    }
```

```
    else
    {
        return dKcdTbyKc - nm/TTR;
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::dcpdT(const scalar p, const scalar TTR) const
{
    return this->dCpdT(p, TTR)*this->W_s();
}

// * * * * * * * * * * * * * * * Member Operators  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline void Foam::species::thermo2T<Thermo2T, Type>::operator+=
(
    const thermo2T<Thermo2T, Type>& st
)
{
    Thermo2T::operator+=(st);
}


template<class Thermo2T, template<class> class Type>
inline void Foam::species::thermo2T<Thermo2T, Type>::operator*=(const scalar s)
{
    Thermo2T::operator*=(s);
}


// * * * * * * * * * * * * * * * Friend Operators  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type> Foam::species::operator+
(
    const thermo2T<Thermo2T, Type>& st1,
    const thermo2T<Thermo2T, Type>& st2
)
{
    return thermo2T<Thermo2T, Type>
    (
        static_cast<const Thermo2T&>(st1) + static_cast<const Thermo2T&>(st2)
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type> Foam::species::operator*
(
    const scalar s,
    const thermo2T<Thermo2T, Type>& st
)
{
    return thermo2T<Thermo2T, Type>
    (
        s*static_cast<const Thermo2T&>(st)
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type> Foam::species::operator==
(
    const thermo2T<Thermo2T, Type>& st1,
    const thermo2T<Thermo2T, Type>& st2
)
{
    return thermo2T<Thermo2T, Type>
    (
        static_cast<const Thermo2T&>(st1) == static_cast<const Thermo2T&>(st2)
    );
}


// ************************************************************************* //
```

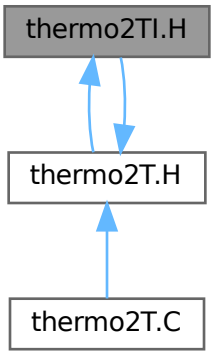
7.107 thermo/thermo2T/thermo2TI.H File Reference

`#include "thermo2T.H"`

Include dependency graph for thermo/thermo2T/thermo2TI.H:



This graph shows which files directly or indirectly include this file:



7.108 thermo/thermo2T/thermo2TI.H

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
```

```
    Copyright (C) 2020 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/

#include "thermo2T.H"

// * * * * * * * * * * * * * Private Member Functions  * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type>::thermo2T
(
    const Thermo2T& sp
)
:
    Thermo2T(sp)
{}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::T
(
    scalar f,
    scalar p,
    scalar T0,
    scalar (thermo2T<Thermo2T, Type>::*F)(const scalar, const scalar) const,
    scalar (thermo2T<Thermo2T, Type>::*dFdT)(const scalar, const scalar)
        const,
    scalar (thermo2T<Thermo2T, Type>::*limit)(const scalar) const
) const
{
    if (T0 < 0)
    {
        FatalErrorInFunction
            « "Negative initial temperature T0: " « T0
            « abort(FatalError);
    }

    scalar Test = T0;
    scalar Tnew = T0;
    scalar Ttol = T0*tol_;
    int    iter = 0;

    do
    {
        Test = Tnew;
        Tnew =
            (this->*limit)
            (Test - ((this->*F)(p, Test) - f)/(this->*dFdT)(p, Test));

        if (iter++ > maxIter_)
        {
            FatalErrorInFunction
                « "Maximum number of iterations exceeded: " « maxIter_
                « " when starting from T0:" « T0
                « " old T:" « Test « " new T:" « Tnew
                « " f:" « f
                « " p:" « p
                « " tol:" « Ttol
                « abort(FatalError);
        }

    } while (mag(Tnew - Test) > Ttol);

    return Tnew;
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TVib
(
    scalar f,
    scalar p,
```

```
    scalar TTR,
    scalar TVib0,
    scalar (thermo2T<Thermo2T, Type>::*F)(const scalar, const scalar, const scalar, const scalar) const,
    scalar (thermo2T<Thermo2T, Type>::*dFdT)(const scalar, const scalar)
        const,
    scalar (thermo2T<Thermo2T, Type>::*limit)(const scalar) const
) const
{
    if (TVib0 < 0)
    {
        FatalErrorInFunction
            << "Negative initial temperature TVib0: " << TVib0
            << abort(FatalError);
    }

    scalar Test = TVib0;
    scalar Tnew = TVib0;
    scalar Ttol = TVib0*tol_;
    int    iter = 0;

    do
    {
        Test = Tnew;
        Tnew =
            (this->*limit)
            (Test - ((this->*F)(p, TTR, Test, this->ThetaVib()) - f)/(this->*dFdT)(p, Test));

        if (iter++ > maxIter_)
        {
            FatalErrorInFunction
                << "Maximum number of iterations exceeded: " << maxIter_
                << " when starting from TVib0:" << TVib0
                << " old T:" << Test << " new T:" << Tnew
                << " f:" << f
                << " p:" << p
                << " tol:" << Ttol
                << abort(FatalError);
        }

    } while (mag(Tnew - Test) > Ttol);

    return Tnew;
}


// * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type>::thermo2T
(
    const word& name,
    const thermo2T& st
)
:
    Thermo2T(name, st)
{}


// * * * * * * * * * * * * * * * * Member Functions  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::word
Foam::species::thermo2T<Thermo2T, Type>::heName()
{
    return Type<thermo2T<Thermo2T, Type>>::energyName();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::CpvTR(const scalar p, const scalar TTR) const
{
    return Type<thermo2T<Thermo2T, Type>>::CpvTR(*this, p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::gamma(const scalar p, const scalar TTR) const
{
    #ifdef __clang__
    volatile const scalar CpTR = this->CpTR(p, TTR);
    #else
    const scalar CpTR = this->CpTR(p, TTR);
    #endif

```

```
    return CpTR/(CpTR - this->CpMCv(p, TTR));
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::CpByCpvTR
(
    const scalar p,
    const scalar TTR
) const
{
    return Type<thermo2T<Thermo2T, Type»::CpByCpvTR(*this, p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::HE(const scalar p, const scalar TTR) const
{
    return Type<thermo2T<Thermo2T, Type»::HE(*this, p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::G(const scalar p, const scalar TTR) const
{
    return this->Ha(p, TTR) - TTR*this->S(p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::A(const scalar p, const scalar TTR) const
{
    return this->Ea(p, TTR) - TTR*this->S(p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cpTR(const scalar p, const scalar TTR) const
{
    return this->CpTR(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::ha(const scalar p, const scalar TTR) const
{
    return this->Ha(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::hs(const scalar p, const scalar TTR) const
{
    return this->Hs(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::hc() const
{
    return this->Hc()*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::s(const scalar p, const scalar TTR) const
{
    return this->S(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::he(const scalar p, const scalar TTR) const
{
    return this->HE(p, TTR)*this->W_s();
}

```

```
template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cvT(const scalar p, const scalar TTR) const
{
    return this->CvT(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cvR(const scalar p, const scalar TTR) const
{
    return this->CvR(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::cvVib(const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
{
    return this->CvVib(p, TTR, TVib, ThetaVib)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::esT(const scalar p, const scalar TTR) const
{
    return this->EsT(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::esR(const scalar p, const scalar TTR) const
{
    return this->EsR(p, TTR)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::esVib(const scalar p, const scalar TTR, const scalar TVib, const scalar ThetaVib) const
{
    return this->EsVib(p, TTR, TVib, ThetaVib)*this->W_s();
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::eaT(const scalar p, const scalar TTR) const
{
    return this->EaT(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::g(const scalar p, const scalar TTR) const
{
    return this->G(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::a(const scalar p, const scalar TTR) const
{
    return this->A(p, TTR)*this->W_s();
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::K(const scalar p, const scalar TTR) const
{
    scalar arg = -this->Y()*this->G(Pstd, TTR)/(RR*TTR);

    if (arg < 600)
    {
        return exp(arg);
    }
    else
    {
        return VGREAT;
    }
}


template<class Thermo2T, template<class> class Type>
```

```
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::Kp(const scalar p, const scalar TTR) const
{
    return K(p, TTR);
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::Kc(const scalar p, const scalar TTR) const
{
    const scalar nm = this->Y()/this->W_s();

    if (equal(nm, SMALL))
    {
        return Kp(p, TTR);
    }
    else
    {
        return Kp(p, TTR)*pow(Pstd/(RR*TTR), nm);
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::Kx
(
    const scalar p,
    const scalar TTR
) const
{
    const scalar nm = this->Y()/this->W_s();

    if (equal(nm, SMALL))
    {
        return Kp(p, TTR);
    }
    else
    {
        return Kp(p, TTR)*pow(Pstd/p, nm);
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::Kn
(
    const scalar p,
    const scalar TTR,
    const scalar n
) const
{
    const scalar nm = this->Y()/this->W_s();

    if (equal(nm, SMALL))
    {
        return Kp(p, TTR);
    }
    else
    {
        return Kp(p, TTR)*pow(n*Pstd/p, nm);
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::THE_TR
(
    const scalar he,
    const scalar p,
    const scalar TTR0
) const
{
    return Type<thermo2T<Thermo2T, Type»::THE_TR(*this, he, p, TTR0);
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEH_Vib
(
    const scalar esVib,
    const scalar p,
    const scalar TTR0,
    const scalar TVib0
) const
{
    return Type<thermo2T<Thermo2T, Type»::TEH_Vib(*this, esVib, p, TTR0, TVib0);
}
```

```
template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::THs_TR
(
    const scalar hs,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        hs,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::Hs,
        &thermo2T<Thermo2T, Type>::CpTR,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::THa_TR
(
    const scalar ha,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        ha,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::Ha,
        &thermo2T<Thermo2T, Type>::CpTR,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEs_TR
(
    const scalar es,
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        es,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::EsT,
        &thermo2T<Thermo2T, Type>::CvT,
        &thermo2T<Thermo2T, Type>::limit
    );
}

template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEs_Vib
(
    const scalar esVib,
    const scalar p,
    const scalar TVib0,
    const scalar TTR0
) const
{
    return TVib
    (
        esVib,
        p,
        TTR0,
        TVib0,
        &thermo2T<Thermo2T, Type>::EsVib,
        &thermo2T<Thermo2T, Type>::CvVib,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar Foam::species::thermo2T<Thermo2T, Type>::TEa_TR
(thermo2T_H
    const scalar ea,
```

```
    const scalar p,
    const scalar TTR0
) const
{
    return T
    (
        ea,
        p,
        TTR0,
        &thermo2T<Thermo2T, Type>::EaT,
        &thermo2T<Thermo2T, Type>::CvT,
        &thermo2T<Thermo2T, Type>::limit
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::dKcdTbyKc
(
    const scalar p,
    const scalar TTR
) const
{
    const scalar dKcdTbyKc =
        (this->S(Pstd, TTR) + this->Gstd(TTR)/TTR)*this->Y()/(RR*TTR);

    const scalar nm = this->Y()/this->W_s();
    if (equal(nm, SMALL))
    {
        return dKcdTbyKc;
    }
    else
    {
        return dKcdTbyKc - nm/TTR;
    }
}


template<class Thermo2T, template<class> class Type>
inline Foam::scalar
Foam::species::thermo2T<Thermo2T, Type>::dcpdT(const scalar p, const scalar TTR) const
{
    return this->dCpdT(p, TTR)*this->W_s();
}

// * * * * * * * * * * * * * * Member Operators  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline void Foam::species::thermo2T<Thermo2T, Type>::operator+=
(
    const thermo2T<Thermo2T, Type>& st
)
{
    Thermo2T::operator+=(st);
}


template<class Thermo2T, template<class> class Type>
inline void Foam::species::thermo2T<Thermo2T, Type>::operator*=(const scalar s)
{
    Thermo2T::operator*=(s);
}


// * * * * * * * * * * * * * * Friend Operators  * * * * * * * * * * * * * //

template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type> Foam::species::operator+
(
    const thermo2T<Thermo2T, Type>& st1,
    const thermo2T<Thermo2T, Type>& st2
)
{
    return thermo2T<Thermo2T, Type>
    (
        static_cast<const Thermo2T&>(st1) + static_cast<const Thermo2T&>(st2)
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type> Foam::species::operator*
(
    const scalar s,
    const thermo2T<Thermo2T, Type>& st
)
```

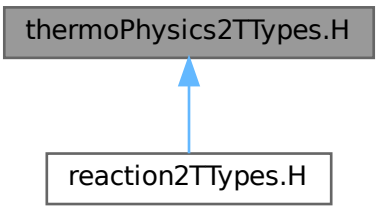
```
{
    return thermo2T<Thermo2T, Type>
    (
        s*static_cast<const Thermo2T&>(st)
    );
}


template<class Thermo2T, template<class> class Type>
inline Foam::species::thermo2T<Thermo2T, Type> Foam::species::operator==
(
    const thermo2T<Thermo2T, Type>& st1,
    const thermo2T<Thermo2T, Type>& st2
)
{
    return thermo2T<Thermo2T, Type>
    (
        static_cast<const Thermo2T&>(st1) == static_cast<const Thermo2T&>(st2)
    );
}


// ************************************************************************* //
```

7.109 include/thermoPhysics2TTypes.H File Reference

```
#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "incompressiblePerfectGas.H"
#include "rPolynomial.H"
#include "perfectFluid.H"
#include "adiabaticPerfectFluid.H"
#include "rhoConst.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "thermo2T.H"
#include "sensibleEnthalpy2T.H"
#include "blottnerEucken2TTransport.H"
```

Include dependency graph for include/thermoPhysics2TTypes.H:

This graph shows which files directly or indirectly include this file:



Namespaces

- namespace Foam

Typedefs

- typedef blottnerEucken2TTransport< species::thermo2T< janaf2TThermo< perfectGas< specie > >, sensibleEnthalpy2T > > Foam::blottnerEucken2TGasHThermoPhysics

7.110 include/thermoPhysics2TTypes.H

Go to the documentation of this file.

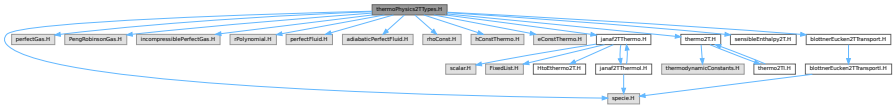
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2018-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Typedefs
    Foam::thermoPhysicsTypes

Description
    Type definitions for thermo2T-physics models

\*-------------------------------------------------------------------------*/

#ifndef thermoPhysics2TTypes_H
#define thermoPhysics2TTypes_H

#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "incompressiblePerfectGas.H"
#include "rPolynomial.H"
#include "perfectFluid.H"
#include "adiabaticPerfectFluid.H"
#include "rhoConst.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"

#include "thermo2T.H"
#include "sensibleEnthalpy2T.H"


#include "blottnerEucken2TTransport.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    // thermo2T physics types based on sensibleEnthalpy
    typedef
    blottnerEucken2TTransport
    <
        species::thermo2T
```

```
        <
            janaf2TThermo
            <
                perfectGas<specie>
            >,
            sensibleEnthalpy2T
        >
    > blottnerEucken2TGasHThermoPhysics;
}

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

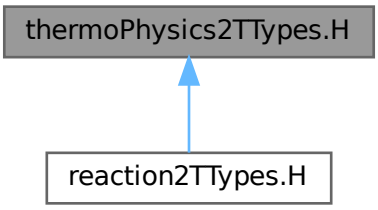
7.111 InInclude/thermoPhysics2TTypes.H File Reference

```
#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "incompressiblePerfectGas.H"
#include "rPolynomial.H"
#include "perfectFluid.H"
#include "adiabaticPerfectFluid.H"
#include "rhoConst.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"
#include "thermo2T.H"
#include "sensibleEnthalpy2T.H"
#include "blottnerEucken2TTransport.H"
```

Include dependency graph for InInclude/thermoPhysics2TTypes.H:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace Foam

7.112 lnInclude/thermoPhysics2TTypes.H

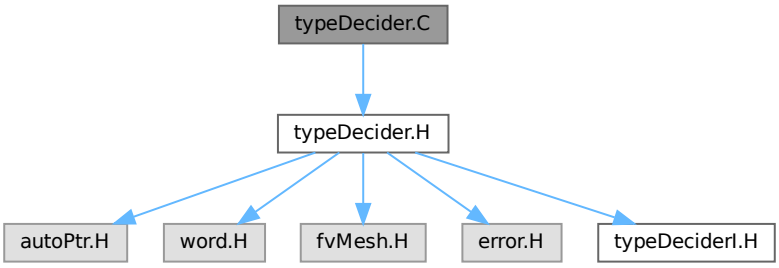
Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
    Copyright (C) 2011-2017 OpenFOAM Foundation
    Copyright (C) 2018-2023 OpenCFD Ltd.
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Typedefs
    Foam::thermoPhysicsTypes

Description
    Type definitions for thermo2T-physics models

\*-------------------------------------------------------------------------*/

#ifndef thermoPhysics2TTypes_H
#define thermoPhysics2TTypes_H

#include "specie.H"
#include "perfectGas.H"
#include "PengRobinsonGas.H"
#include "incompressiblePerfectGas.H"
#include "rPolynomial.H"
#include "perfectFluid.H"
#include "adiabaticPerfectFluid.H"
#include "rhoConst.H"
#include "hConstThermo.H"
#include "eConstThermo.H"
#include "janaf2TThermo.H"

#include "thermo2T.H"
#include "sensibleEnthalpy2T.H"


#include "blottnerEucken2TTransport.H"

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

namespace Foam
{
    // thermo2T physics types based on sensibleEnthalpy
    typedef
    blottnerEucken2TTransport
    <
        species::thermo2T
        <
            janaf2TThermo
            <
                perfectGas<specie>
            >,
            sensibleEnthalpy2T
        >
    > blottnerEucken2TGasHThermoPhysics;
}

// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

#endif

// ************************************************************************* //
```

7.113 typeDecider.C File Reference

`#include "typeDecider.H"`

Include dependency graph for typeDecider.C:



7.114 typeDecider.C

Go to the documentation of this file.

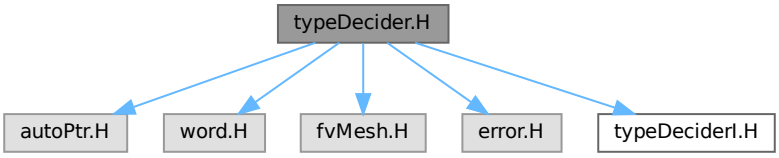
```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*---------------------------------------------------------------------------*/
#include "typeDecider.H"


// * * * * * * * * * * * * * * * * * Constructors  * * * * * * * * * * * * * * //
template<class OneT, class TwoT>
Foam::typeDecider<OneT, TwoT>::typeDecider
(
    const fvMesh& mesh
)
:
    thermoDict
    (
        IOobject
        (
            word("thermophysicalProperties"),
            mesh.time().constant(),    // path: constant/
            mesh,
            IOobject::MUST_READ,       // fail loudly if missing
            IOobject::NO_WRITE,
            false
        )
    ),
    tt
    (
```

```
        thermoDict.subDict("thermoType")
    ),
    tempModel
    (
        tt.get<word>("type")
    )
{

}
```

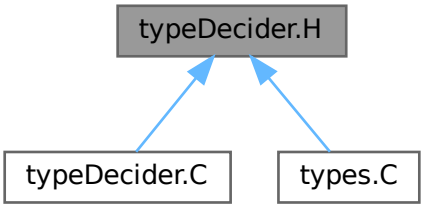
7.115 typeDecider.H File Reference

```
#include "autoPtr.H"
#include "word.H"
#include "fvMesh.H"
#include "error.H"
#include "typeDeciderI.H"
```

Include dependency graph for typeDecider.H:



This graph shows which files directly or indirectly include this file:



Classes

- class Foam::typeDecider< OneT, TwoT >

Namespaces

- namespace Foam

7.116 typeDecider.H

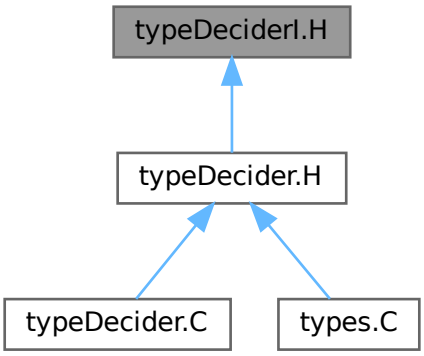
Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

Class
    Foam::typeDecider

Group
    grp

Description
    Decide which temperature model (1T or 2T) to use.

    Template params:
    - neTThermo : the 1-temperature thermo class (e.g. neTThermo)
    - ne2TThermo : the 2-temperature thermo class (e.g. ne2TThermo)
SourceFiles
    typeDeciderI.H
    typeDecider.C

\*-------------------------------------------------------------------------*/

#ifndef typeDecider_H
#define typeDecider_H

#include "autoPtr.H"
#include "word.H"
#include "fvMesh.H"
#include "error.H"

namespace Foam
{

template<class OneT, class TwoT>
class typeDecider
{
    protected:

        IOdictionary thermoDict;
        const dictionary tt;
        const word tempModel;

    public:

    // Constructor
        //- Construct from mesh and phase name
        typeDecider
        (
            const fvMesh&
        );

    // Member Functions

        // - Construct exactly one of the thermos based on `type'
        bool construct
        (
            autoPtr<OneT>& p1T,
            autoPtr<TwoT>& p2T,
            const fvMesh& mesh
        );
};

} // namespace Foam
```

```
#include "typeDeciderI.H"

#ifdef NoRepository
#   include "typeDecider.C"
#endif

#endif
```

7.117 typeDeciderI.H File Reference

This graph shows which files directly or indirectly include this file:



7.118 typeDeciderI.H

Go to the documentation of this file.

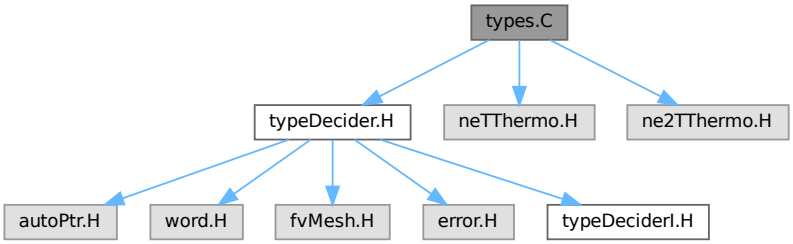
```
#ifndef typeDeciderI_H
#define typeDeciderI_H

template<class OneT, class TwoT>
inline bool Foam::typeDecider<OneT,TwoT>::construct
(
    autoPtr<OneT>& p1T,
    autoPtr<TwoT>& p2T,
    const fvMesh& mesh
)
{
    bool check2T = (tempModel == word("heNe2TThermo"));
    bool checkT = (tempModel == word("heNeTThermo"));

    if (check2T)
    {
        p2T = TwoT::New(mesh);
        Info « "Selecting two-temperature model " « nl « endl;
        return true;
    }
    else if (checkT)
    {
        p1T = OneT::New(mesh);
        Info « "Selecting one-temperature model " « nl « endl;
        return false;
    }
    else
    {
        Info
            « "thermoType.type = " « tempModel
```

```
            « " should be " « checkT « " or " « check2T
            « endl;
        return false;
    }
}

#endif
```

7.119 types.C File Reference

```
#include "typeDecider.H"
#include "neTThermo.H"
#include "ne2TThermo.H"
```

Include dependency graph for types.C:



Namespaces

- namespace Foam

7.120 types.C

Go to the documentation of this file.

```
/*-------------------------------------------------------------------------*\
  =========                 |
  \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox
   \\    /   O peration     |
    \\  /    A nd           | www.openfoam.com
     \\/     M anipulation  |
-------------------------------------------------------------------------------
License
    This file is part of OpenFOAM.

    OpenFOAM is free software: you can redistribute it and/or modify it
    under the terms of the GNU General Public License as published by
    the Free Software Foundation, either version 3 of the License, or
    (at your option) any later version.

    OpenFOAM is distributed in the hope that it will be useful, but WITHOUT
    ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or
    FITNESS FOR A PARTICULAR PURPOSE.  See the GNU General Public License
    for more details.

    You should have received a copy of the GNU General Public License
    along with OpenFOAM.  If not, see <http://www.gnu.org/licenses/>.

\*-------------------------------------------------------------------------*/

#include "typeDecider.H"

```

```
#include "neTThermo.H"    // 1-temperature model
#include "ne2TThermo.H"   // 2-temperature model

namespace Foam
{
    template class typeDecider<neTThermo, ne2TThermo>;
}
```

Index

~basic2TThermo
Foam::basic2TThermo, 24, 25
~fluid2TThermo
Foam::fluid2TThermo, 68, 69
~he2TThermo
Foam::he2TThermo< Basic2TThermo, MixtureType >, 78, 79
~heNe2TThermo
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >, 101, 102
~ne2TThermo
Foam::ne2TThermo, 122, 123

A
Foam::species::thermo2T< Thermo2T, Type >, 137
a
Foam::species::thermo2T< Thermo2T, Type >, 137
addThermoPhysicsThermo
Foam, 11, 12
make2TThermo.H, 261, 266
alpha
Foam::basic2TThermo, 25
alpha_
Foam::basic2TThermo, 48
alphaEff
Foam::basic2TThermo, 26
Foam::he2TThermo< Basic2TThermo, MixtureType >, 79
alphah
Foam::blottnerEucken2TTransport< Thermo2T >, 55
alphaheTR
Foam::basic2TThermo, 26
Foam::he2TThermo< Basic2TThermo, MixtureType >, 80

basic2TThermo
Foam::basic2TThermo, 24, 25
basic2TThermo.C, 159, 160, 167, 168
basic2TThermo.H, 176, 177, 185, 186
basic2TThermoTemplates.C, 194, 197
blottnerEucken2TGasHThermoPhysics
Foam, 10
blottnerEucken2TGasHThermoPhysicsHReaction
Foam, 10
blottnerEucken2TTransport
Foam::blottnerEucken2TTransport< Thermo2T >, 53, 54
blottnerEucken2TTransport.C, 290–293
blottnerEucken2TTransport.H, 295, 296, 299, 300
blottnerEucken2TTransportI.H, 303, 306, 307
blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGass
Foam, 11

clone
Foam::blottnerEucken2TTransport< Thermo2T >, 55, 56
coeffArray
Foam::janaf2TThermo< EquationOfState >, 106
composition
Foam::he2TThermo< Basic2TThermo, MixtureType >, 80, 81
construct
Foam::typeDecider< OneT, TwoT >, 157
correct
Foam::basic2TThermo, 26, 27
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >, 102
correctRho
Foam::fluid2TThermo, 69
Foam::ne2TThermo, 123, 124
CpByCpvTR
Foam::basic2TThermo, 27
Foam::he2TThermo< Basic2TThermo, MixtureType >, 81
Foam::sensibleEnthalpy2T< Thermo2T >, 129
Foam::species::thermo2T< Thermo2T, Type >, 137, 138
CpTR
EtoHthermo2T.H, 310, 312
Foam::basic2TThermo, 27, 28
Foam::he2TThermo< Basic2TThermo, MixtureType >, 81, 82
Foam::janaf2TThermo< EquationOfState >, 107
cpTR
Foam::species::thermo2T< Thermo2T, Type >, 138
CpvTR
Foam::basic2TThermo, 28
Foam::he2TThermo< Basic2TThermo, MixtureType >, 82, 83
Foam::sensibleEnthalpy2T< Thermo2T >, 129, 130
Foam::species::thermo2T< Thermo2T, Type >, 138, 139
CvR
Foam::basic2TThermo, 28, 29

Foam::he2TThermo< Basic2TThermo, MixtureType >, 83, 84
HtoEthermo2T.H, 314, 320
cvR
Foam::species::thermo2T< Thermo2T, Type >, 139
CvT
Foam::basic2TThermo, 29
Foam::he2TThermo< Basic2TThermo, MixtureType >, 84, 85
HtoEthermo2T.H, 315, 320
cvT
Foam::species::thermo2T< Thermo2T, Type >, 139, 140
CvVib
Foam::basic2TThermo, 29, 30
Foam::he2TThermo< Basic2TThermo, MixtureType >, 85, 86
HtoEthermo2T.H, 316, 321
cvVib
Foam::species::thermo2T< Thermo2T, Type >, 140

dCpdT
Foam::janaf2TThermo< EquationOfState >, 108
dcpdT
Foam::species::thermo2T< Thermo2T, Type >, 141
declareRunTimeSelectionTable
Foam::basic2TThermo, 30
Foam::fluid2TThermo, 69, 70
Foam::ne2TThermo, 124
defineRunTimeSelectionTable
Foam, 12, 13
defineTemplateTypeNameAndDebugWithName
Foam, 13
defineThermoPhysicsThermo
make2TThermo.H, 261, 267
dictName
Foam::basic2TThermo, 48
dKcdTbyKc
Foam::species::thermo2T< Thermo2T, Type >, 141
dpdt
Foam::basic2TThermo, 31
dpdt_
Foam::basic2TThermo, 48

EaT
HtoEthermo2T.H, 316, 321
eaT
Foam::species::thermo2T< Thermo2T, Type >, 141, 142
energyName
Foam::sensibleEnthalpy2T< Thermo2T >, 130
EsR
HtoEthermo2T.H, 317, 321
esR
Foam::species::thermo2T< Thermo2T, Type >, 142
EsT
HtoEthermo2T.H, 317, 321
esT
Foam::species::thermo2T< Thermo2T, Type >, 143
EsVib
HtoEthermo2T.H, 318, 321
esVib
Foam::species::thermo2T< Thermo2T, Type >, 143, 144
EtoHthermo2T.H, 310, 312, 313
CpTR, 310, 312
HaTR, 311, 313
HsTR, 311, 313

fluid2TThermo
Foam::fluid2TThermo, 68
fluid2TThermo.C, 200, 202
fluid2TThermo.H, 204, 205, 207, 208
Foam, 9
addThermoPhysicsThermo, 11, 12
blottnerEucken2TGasHThermoPhysics, 10
blottnerEucken2TGasHThermoPhysicsHReaction, 10
blottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfect 11
defineRunTimeSelectionTable, 12, 13
defineTemplateTypeNameAndDebugWithName, 13
heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEntha 11
heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleE 11
makeThermos, 13
operator<<, 14
operator+, 14
operator==, 15
operator∗, 13, 14
Foam::basic2TThermo, 17
∼basic2TThermo, 24, 25
alpha, 25
alpha_, 48
alphaEff, 26
alphaheTR, 26
basic2TThermo, 24, 25
correct, 26, 27
CpByCpvTR, 27
CpTR, 27, 28
CpvTR, 28
CvR, 28, 29
CvT, 29
CvVib, 29, 30
declareRunTimeSelectionTable, 30
dictName, 48
dpdt, 31
dpdt_, 48
gamma, 31

hc, 31, 32
he, 32, 33
heBoundaryBaseTypes, 33
heBoundaryTypes, 33
incompressible, 33
isochoric, 34
kappaEff, 34
kappaTR, 34, 35
kappaVib, 35
lookupThermo, 35
New, 35–38
p, 38
p_, 49
phaseName_, 49
phasePropertyName, 38–40
phaseScopedName, 40
pOwner_, 49
printThermoNames, 41
read, 41, 42
rho, 42
rhoEoS, 42
splitThermoName, 43
TEH_Vib, 43, 44
THE_TR, 44
thermoName, 44, 45
TOwnerTR_, 49
TOwnerVib_, 49
TTR, 45
TTR_, 49
TVib, 45, 46
TVib_, 50
updateTTR, 46
updateTVib, 46
validate, 46–48
W, 48
Foam::blottnerEucken2TTransport< Thermo2T >, 50
alphah, 55
blottnerEucken2TTransport, 53, 54
clone, 55, 56
kappaTR, 56
kappaVib, 56, 57
mu, 57
New, 57
operator<<, 59, 60
operator+, 59
operator+=, 58
operator∗, 59
operator∗=, 58
typeName, 58
write, 58, 59
Foam::fluid2TThermo, 60
∼fluid2TThermo, 68, 69
correctRho, 69
declareRunTimeSelectionTable, 69, 70
fluid2TThermo, 68
mu, 70, 71
New, 71, 72
nu, 72
psi, 72, 73
Foam::he2TThermo< Basic2TThermo, MixtureType >, 73
∼he2TThermo, 78, 79
alphaEff, 79
alphaheTR, 80
composition, 80, 81
CpByCpvTR, 81
CpTR, 81, 82
CpvTR, 82, 83
CvR, 83, 84
CvT, 84, 85
CvVib, 85, 86
gamma, 86
hc, 87
he, 87–89
he2TThermo, 78, 79
he_, 96
heBoundaryCorrection, 89
incompressible, 89
isochoric, 89, 90
kappaEff, 90, 91
kappaTR, 91, 92
kappaVib, 92, 93
read, 93
rhoEoS, 93, 94
TEH_Vib, 94
THE_TR, 95
thermoName, 95, 96
W, 96
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >, 97
∼heNe2TThermo, 101, 102
correct, 102
heNe2TThermo, 101, 102
Foam::janaf2TThermo< EquationOfState >, 103
coeffArray, 106
CpTR, 107
dCpdT, 108
Gstd, 108
Ha, 108
Hc, 109
highCpCoeffs, 109
Hs, 109
janaf2TThermo, 106, 107
limit, 109, 110
lowCpCoeffs, 110
nCoeffs_, 114
operator<<, 113
operator+, 113
operator+=, 110
operator==, 113
operator∗, 112
S, 110
Tcommon, 111
ThetaVib, 111
Thigh, 111
Tlow, 111, 112

typeName, 112
write, 112
Foam::ne2TThermo, 114
∼ne2TThermo, 122, 123
correctRho, 123, 124
declareRunTimeSelectionTable, 124
mu, 124, 125
mu_, 127
ne2TThermo, 122, 123
New, 125, 126
psi, 126
psi_, 127
rho, 126, 127
Foam::sensibleEnthalpy2T< Thermo2T >, 128
CpByCpvTR, 129
CpvTR, 129, 130
energyName, 130
HE, 130
sensibleEnthalpy2T, 129
TEH_Vib, 130, 131
THE_TR, 131
typeName, 131
Foam::species, 15
operator<<, 15
operator+, 15
operator==, 16
operator∗, 15
Foam::species::thermo2T< Thermo2T, Type >, 132
A, 137
a, 137
CpByCpvTR, 137, 138
cpTR, 138
CpvTR, 138, 139
cvR, 139
cvT, 139, 140
cvVib, 140
dcpdT, 141
dKcdTbyKc, 141
eaT, 141, 142
esR, 142
esT, 143
esVib, 143, 144
G, 144
g, 144
gamma, 145
ha, 145
hc, 145, 146
HE, 146
he, 146
heName, 146
hs, 147
K, 147
Kc, 147
Kn, 148
Kp, 148
Kx, 148
operator, 154
operator+, 154, 155
operator+=, 149
operator==, 155
operator∗, 154
operator∗=, 149
s, 149
TEa_TR, 149, 150
TEH_Vib, 150
TEs_TR, 150
TEs_Vib, 151
THa_TR, 151
THE_TR, 151, 152
thermo2T, 136, 137
thermoType, 136
THs_TR, 152
typeName, 152, 153
write, 153, 154
Foam::typeDecider< OneT, TwoT >, 155
construct, 157
tempModel, 157
thermoDict, 157
tt, 157
typeDecider, 156

G
Foam::species::thermo2T< Thermo2T, Type >, 144
g
Foam::species::thermo2T< Thermo2T, Type >, 144
gamma
Foam::basic2TThermo, 31
Foam::he2TThermo< Basic2TThermo, MixtureType >, 86
Foam::species::thermo2T< Thermo2T, Type >, 145
Gstd
Foam::janaf2TThermo< EquationOfState >, 108

Ha
Foam::janaf2TThermo< EquationOfState >, 108
ha
Foam::species::thermo2T< Thermo2T, Type >, 145
HaTR
EtoHthermo2T.H, 311, 313
Hc
Foam::janaf2TThermo< EquationOfState >, 109
hc
Foam::basic2TThermo, 31, 32
Foam::he2TThermo< Basic2TThermo, MixtureType >, 87
Foam::species::thermo2T< Thermo2T, Type >, 145, 146
HE
Foam::sensibleEnthalpy2T< Thermo2T >, 130
Foam::species::thermo2T< Thermo2T, Type >, 146
he
Foam::basic2TThermo, 32, 33

Foam::he2TThermo< Basic2TThermo, MixtureType >, 87–89
Foam::species::thermo2T< Thermo2T, Type >, 146
he2TThermo
Foam::he2TThermo< Basic2TThermo, MixtureType >, 78, 79
he2TThermo.C, 210, 223, 224
he2TThermo.H, 237, 243
he_
Foam::he2TThermo< Basic2TThermo, MixtureType >, 96
heBoundaryBaseTypes
Foam::basic2TThermo, 33
heBoundaryCorrection
Foam::he2TThermo< Basic2TThermo, MixtureType >, 89
heBoundaryTypes
Foam::basic2TThermo, 33
heName
Foam::species::thermo2T< Thermo2T, Type >, 146
heNe2TThermo
Foam::heNe2TThermo< BasicNe2TThermo, MixtureType >, 101, 102
heNe2TThermo.C, 249, 252
heNe2TThermo.H, 255, 256, 258, 259
heNe2TThermopureMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
Foam, 11
heNe2TThermopureZoneMixtureblottnerEucken2TTransportsensibleEnthalpy2Tjanaf2TThermoperfectGasspecie
Foam, 11
highCpCoeffs
Foam::janaf2TThermo< EquationOfState >, 109
Hs
Foam::janaf2TThermo< EquationOfState >, 109
hs
Foam::species::thermo2T< Thermo2T, Type >, 147
HsTR
EtoHthermo2T.H, 311, 313
HtoEthermo2T.H, 314, 319, 320, 322
CvR, 314, 320
CvT, 315, 320
CvVib, 316, 321
EaT, 316, 321
EsR, 317, 321
EsT, 317, 321
EsVib, 318, 321

incompressible
Foam::basic2TThermo, 33
Foam::he2TThermo< Basic2TThermo, MixtureType >, 89
isochoric
Foam::basic2TThermo, 34
Foam::he2TThermo< Basic2TThermo, MixtureType >, 89, 90

janaf2TThermo
Foam::janaf2TThermo< EquationOfState >, 106, 107
janaf2TThermo.C, 323–326
janaf2TThermo.H, 327, 328, 332, 333
janaf2TThermoI.H, 336, 337, 343, 344

K
Foam::species::thermo2T< Thermo2T, Type >, 147
kappaEff
Foam::basic2TThermo, 34
Foam::he2TThermo< Basic2TThermo, MixtureType >, 90, 91
kappaTR
Foam::basic2TThermo, 34, 35
Foam::blottnerEucken2TTransport< Thermo2T >, 56
Foam::he2TThermo< Basic2TThermo, MixtureType >, 91, 92
kappaVib
Foam::basic2TThermo, 35
Foam::blottnerEucken2TTransport< Thermo2T >, 56, 57
Foam::he2TThermo< Basic2TThermo, MixtureType >, 92, 93
Kc
Foam::species::thermo2T< Thermo2T, Type >, 147
Kn
Foam::species::thermo2T< Thermo2T, Type >, 148
Kp
Foam::species::thermo2T< Thermo2T, Type >, 148
Kx
Foam::species::thermo2T< Thermo2T, Type >, 148

limit
Foam::janaf2TThermo< EquationOfState >, 109, 110
lookupThermo
Foam::basic2TThermo, 35
lowCpCoeffs
Foam::janaf2TThermo< EquationOfState >, 110

make2TThermo.H, 261, 264, 266, 269
addThermoPhysicsThermo, 261, 266
defineThermoPhysicsThermo, 261, 267
makeThermo, 262, 267
makeThermoPhysicsThermo, 262, 267
makeThermoPhysicsThermos, 262, 268
makeThermos, 263, 268
typedefThermoPhysics, 263, 268
makeThermo
make2TThermo.H, 262, 267
makeThermoPhysicsThermo
make2TThermo.H, 262, 267
makeThermoPhysicsThermos

make2TThermo.H, 262, 268
makeThermos
Foam, 13
make2TThermo.H, 263, 268
mu
Foam::blottnerEucken2TTransport< Thermo2T >, 57
Foam::fluid2TThermo, 70, 71
Foam::ne2TThermo, 124, 125
mu_
Foam::ne2TThermo, 127

nCoeffs_
Foam::janaf2TThermo< EquationOfState >, 114
ne2TThermo
Foam::ne2TThermo, 122, 123
ne2TThermo.C, 271, 274
ne2TThermo.H, 277, 278, 280, 281
ne2TThermos.C, 283–286
ne2TZoneThermos.C, 287–289
New
Foam::basic2TThermo, 35–38
Foam::blottnerEucken2TTransport< Thermo2T >, 57
Foam::fluid2TThermo, 71, 72
Foam::ne2TThermo, 125, 126
nu
Foam::fluid2TThermo, 72

operator
Foam::species::thermo2T< Thermo2T, Type >, 154
operator<<
Foam, 14
Foam::blottnerEucken2TTransport< Thermo2T >, 59, 60
Foam::janaf2TThermo< EquationOfState >, 113
Foam::species, 15
operator+
Foam, 14
Foam::blottnerEucken2TTransport< Thermo2T >, 59
Foam::janaf2TThermo< EquationOfState >, 113
Foam::species, 15
Foam::species::thermo2T< Thermo2T, Type >, 154, 155
operator+=
Foam::blottnerEucken2TTransport< Thermo2T >, 58
Foam::janaf2TThermo< EquationOfState >, 110
Foam::species::thermo2T< Thermo2T, Type >, 149
operator==
Foam, 15
Foam::janaf2TThermo< EquationOfState >, 113
Foam::species, 16
Foam::species::thermo2T< Thermo2T, Type >, 155
operator*
Foam, 13, 14
Foam::blottnerEucken2TTransport< Thermo2T >, 59
Foam::janaf2TThermo< EquationOfState >, 112
Foam::species, 15
Foam::species::thermo2T< Thermo2T, Type >, 154
operator*=
Foam::blottnerEucken2TTransport< Thermo2T >, 58
Foam::species::thermo2T< Thermo2T, Type >, 149

p
Foam::basic2TThermo, 38
p_
Foam::basic2TThermo, 49
phaseName_
Foam::basic2TThermo, 49
phasePropertyName
Foam::basic2TThermo, 38–40
phaseScopedName
Foam::basic2TThermo, 40
pOwner_
Foam::basic2TThermo, 49
printThermoNames
Foam::basic2TThermo, 41
psi
Foam::fluid2TThermo, 72, 73
Foam::ne2TThermo, 126
psi_
Foam::ne2TThermo, 127

reaction2TTypes.H, 349–351
read
Foam::basic2TThermo, 41, 42
Foam::he2TThermo< Basic2TThermo, MixtureType >, 93
rho
Foam::basic2TThermo, 42
Foam::ne2TThermo, 126, 127
rhoEoS
Foam::basic2TThermo, 42
Foam::he2TThermo< Basic2TThermo, MixtureType >, 93, 94

S
Foam::janaf2TThermo< EquationOfState >, 110
s
Foam::species::thermo2T< Thermo2T, Type >, 149
sensibleEnthalpy2T
Foam::sensibleEnthalpy2T< Thermo2T >, 129
sensibleEnthalpy2T.H, 352, 354, 355
splitThermoName
Foam::basic2TThermo, 43

Tcommon
Foam::janaf2TThermo< EquationOfState >, 111

TEa_TR
Foam::species::thermo2T< Thermo2T, Type >, 149, 150
TEH_Vib
Foam::basic2TThermo, 43, 44
Foam::he2TThermo< Basic2TThermo, MixtureType >, 94
Foam::sensibleEnthalpy2T< Thermo2T >, 130, 131
Foam::species::thermo2T< Thermo2T, Type >, 150
tempModel
Foam::typeDecider< OneT, TwoT >, 157
TEs_TR
Foam::species::thermo2T< Thermo2T, Type >, 150
TEs_Vib
Foam::species::thermo2T< Thermo2T, Type >, 151
THa_TR
Foam::species::thermo2T< Thermo2T, Type >, 151
THE_TR
Foam::basic2TThermo, 44
Foam::he2TThermo< Basic2TThermo, MixtureType >, 95
Foam::sensibleEnthalpy2T< Thermo2T >, 131
Foam::species::thermo2T< Thermo2T, Type >, 151, 152
thermo2T
Foam::species::thermo2T< Thermo2T, Type >, 136, 137
thermo2T.C, 357, 358
thermo2T.H, 359, 361, 366, 367
thermo2TI.H, 372, 373, 381
thermoDict
Foam::typeDecider< OneT, TwoT >, 157
thermoName
Foam::basic2TThermo, 44, 45
Foam::he2TThermo< Basic2TThermo, MixtureType >, 95, 96
thermoPhysics2TTypes.H, 389–392
thermoType
Foam::species::thermo2T< Thermo2T, Type >, 136
ThetaVib
Foam::janaf2TThermo< EquationOfState >, 111
Thigh
Foam::janaf2TThermo< EquationOfState >, 111
THs_TR
Foam::species::thermo2T< Thermo2T, Type >, 152
Tlow
Foam::janaf2TThermo< EquationOfState >, 111, 112
TOwnerTR_
Foam::basic2TThermo, 49
TOwnerVib_
Foam::basic2TThermo, 49
tt
Foam::typeDecider< OneT, TwoT >, 157
TTR
Foam::basic2TThermo, 45
TTR_
Foam::basic2TThermo, 49
TVib
Foam::basic2TThermo, 45, 46
TVib_
Foam::basic2TThermo, 50
typeDecider
Foam::typeDecider< OneT, TwoT >, 156
typeDecider.C, 393
typeDecider.H, 394, 395
typeDeciderI.H, 396
typedefThermoPhysics
make2TThermo.H, 263, 268
typeName
Foam::blottnerEucken2TTransport< Thermo2T >, 58
Foam::janaf2TThermo< EquationOfState >, 112
Foam::sensibleEnthalpy2T< Thermo2T >, 131
Foam::species::thermo2T< Thermo2T, Type >, 152, 153
types.C, 397

updateTTR
Foam::basic2TThermo, 46
updateTVib
Foam::basic2TThermo, 46

validate
Foam::basic2TThermo, 46–48

W
Foam::basic2TThermo, 48
Foam::he2TThermo< Basic2TThermo, MixtureType >, 96
write
Foam::blottnerEucken2TTransport< Thermo2T >, 58, 59
Foam::janaf2TThermo< EquationOfState >, 112
Foam::species::thermo2T< Thermo2T, Type >, 153, 154